



Ain shams University  
Faculty of Engineering  
Cairo - Egypt

## **Electronics and Electrical Communication Engineering Department**

---

# **BLE Digital Transmitter**

---

Prepared By:

**Ahmed Alaa Mohamed Elsodany 2101434**

**Sarah Hamed Mahmoud Al Sayed 2101518**

**Habiba Essam Eldin Abdelhamid Foad 2100321**

**Hanin Ahmed Abdelfattah Ahmed 2100661**

**Mina Ayad Younan Bebawi 2100546**

Sponsored By: **Si-Vision**

Supervised By: **Prof. Hani Fikry Ragaai**

## Acknowledgment

We would like to express our sincere gratitude to our supervisor, **Prof. Hani Fekry**, for his guidance and support. Dr. Hani's inspiring scientific career has served as a constant source of motivation and aspiration for us.

We would also like to express our deep gratitude for the continued support and encouragement provided by our mentors **Eng. Ahmed Hany**, **Eng. Moaz Khaled** and **Eng. Abdelrahman** throughout the course of this work. Their valuable guidance, constructive feedback, and technical perspectives were instrumental in refining our understanding and overcoming the challenges encountered during the most demanding phases of the project. This support greatly contributed to the successful completion of the work and to our overall learning experience.

## Abstract

Bluetooth Low Energy (BLE) has become a key wireless communication standard for low-power and Internet-of-Things (IoT) applications due to its energy efficiency and operation in the 2.4 GHz ISM band. The physical layer (PHY) of BLE transmitter plays a critical role in determining system performance, spectral efficiency, and regulatory compliance. It is responsible for converting digital data bits into a spectrally efficient radio frequency (RF) signal that meets strict power and regulatory constraints. In this work we will discuss in detail every block in the architecture designed for the BLE transmitter starting at the baseband level, where input data bits are mapped to frequency deviations and shaped using a Gaussian filter, enabling Gaussian Frequency Shift Keying (GFSK) modulation with controlled bandwidth and reduced spectral spreading. The filtered modulation signal is then applied to an All-Digital Phase-Locked Loop (ADPLL), which performs frequency synthesis and RF carrier generation. ADPLL is chosen over analog PLL due to its flexibility, configurability, small area and easy portability. The ADPLL architecture and its role in achieving frequency accuracy and low power operation are examined, along with associated non-idealities such as phase noise, reference spurs, and spurious tones. The complete RTL source code and simulation scripts accompanying this work are publicly available at [this repository](#).

# Contents

|                                                               |           |
|---------------------------------------------------------------|-----------|
| <b>Acknowledgment</b>                                         | <b>1</b>  |
| <b>List of Figures</b>                                        | <b>6</b>  |
| <b>List of Tables</b>                                         | <b>12</b> |
| <b>Introduction</b>                                           | <b>13</b> |
| <b>BLE Overview</b>                                           | <b>13</b> |
| Modulation Scheme . . . . .                                   | 14        |
| Overview . . . . .                                            | 14        |
| GFSK Modulation . . . . .                                     | 14        |
| BLE 5.4 Specifications . . . . .                              | 15        |
| Modulation characteristics . . . . .                          | 15        |
| Spurious Emissions . . . . .                                  | 16        |
| <b>System Architecture</b>                                    | <b>18</b> |
| System Block diagram . . . . .                                | 18        |
| Usage of ADPLL and Limitation of PLL . . . . .                | 19        |
| Comparison of Different ADPLL Architectures . . . . .         | 19        |
| Divider-based ADPLLs . . . . .                                | 20        |
| Divider-less ADPLLs . . . . .                                 | 21        |
| Comparison Summary and Chosen Architecture . . . . .          | 22        |
| Linearized Model of ADPLL . . . . .                           | 23        |
| Converting Frequency to Phase . . . . .                       | 23        |
| Phase Detector . . . . .                                      | 23        |
| Loop Filter . . . . .                                         | 24        |
| DCO . . . . .                                                 | 24        |
| Time to Digital Converter . . . . .                           | 24        |
| Type II ADPLL Parameters . . . . .                            | 26        |
| Noise Transfer Functions . . . . .                            | 27        |
| s-Domain Approximation . . . . .                              | 27        |
| ADPLL Noise Analysis . . . . .                                | 30        |
| Phase Noise . . . . .                                         | 30        |
| Noise Sources in ADPLL . . . . .                              | 31        |
| DCO Dithering Effect on Phase Noise . . . . .                 | 31        |
| Total output phase noise . . . . .                            | 32        |
| MATLAB Simulation Results . . . . .                           | 32        |
| Comparison of One-point vs Two Point Modulation . . . . .     | 35        |
| 1 Point Modulation Analysis . . . . .                         | 35        |
| 2 Point Modulation Analysis . . . . .                         | 40        |
| Gaussian Filter . . . . .                                     | 44        |
| FIR Filter Fundamentals and Linear-Phase Properties . . . . . | 44        |
| FIR Filter Implementations . . . . .                          | 44        |
| Symmetry in FIR Filters and Computational Reduction . . . . . | 45        |
| Summary of FIR Realizations and Selection Criteria . . . . .  | 47        |

|                                                             |     |
|-------------------------------------------------------------|-----|
| Proposed Architecture (Multiplier-Free FIR Architecture)    | 47  |
| Baseline FIR and Symmetric Pair Reduction                   | 47  |
| Odd-Length Symmetric FIR (Type-I Linear Phase)              | 47  |
| Even-Length Symmetric FIR (Type-II Linear Phase)            | 48  |
| Center Tap Handling for Odd-Length FIR                      | 50  |
| Gaussian Filter Parameterization and Design Tradeoffs       | 51  |
| Discrete-Time Parameters: Span and Samples-Per-Symbol (SPS) | 51  |
| Configuration Used for Primary Design and Verification      | 53  |
| RTL implementation                                          | 53  |
| Radix-4 Booth Recoding Multiplier                           | 64  |
| Abstraction                                                 | 64  |
| Architecture and Operation                                  | 64  |
| Full Multiplier block diagram                               | 69  |
| Time to Digital Converter                                   | 70  |
| TDC Phase Domain Operation                                  | 70  |
| TDC System Block Diagram                                    | 72  |
| Different TDC Core Architectures                            | 76  |
| TDC Core Block Diagram                                      | 77  |
| TDC Core Operation                                          | 78  |
| TDC Core Individual Block Implementations                   | 80  |
| Error Correction in TDC                                     | 81  |
| Random Phase Start                                          | 88  |
| RTL Implementation Block Diagrams                           | 89  |
| Sub-Blocks Implementation                                   | 91  |
| RTL Parameters                                              | 93  |
| RTL Interface                                               | 94  |
| Loop Filter                                                 | 95  |
| Type I ADPLL                                                | 95  |
| Type II ADPLL                                               | 96  |
| ADPLL Modes of Operation                                    | 97  |
| Proposed Hitless Mode transition Scheme                     | 98  |
| Type I ADPLL frequency response                             | 98  |
| Type II ADPLL frequency response                            | 99  |
| RTL Block Diagram                                           | 101 |
| RTL Interface                                               | 102 |
| Sigma Delta                                                 | 102 |
| Types of Sigma-Delta Modulators                             | 102 |
| MASH Architecture                                           | 102 |
| First-Order MASH Sigma-Delta Analysis                       | 103 |
| Higher Order Sigma Delta Modulators                         | 104 |
| Normalized ( $M = 1$ ) Linear Model of the First-Order EFM  | 105 |
| Modified Sigma-Delta Modulator                              | 107 |
| Linear Model of the proposed Sigma delta                    | 108 |
| Implementation of the proposed Sigma delta                  | 110 |
| DCO and Gain Estimation                                     | 114 |
| Introduction                                                | 114 |
| Frequency Control of Digital Oscillators                    | 115 |
| DCO Operational Modes                                       | 115 |

|                                               |            |
|-----------------------------------------------|------------|
| PVT Calibration Mode . . . . .                | 116        |
| Acquisition Mode . . . . .                    | 116        |
| Tracking Mode . . . . .                       | 116        |
| DCO Architecture and Implementation . . . . . | 117        |
| Normalized DCO (nDCO) . . . . .               | 117        |
| DCO Interface . . . . .                       | 118        |
| Gain Calibration Using LMS . . . . .          | 119        |
| Sign-Sign LMS (SS-LMS) . . . . .              | 120        |
| RTL Implementation . . . . .                  | 121        |
| IIR Filter . . . . .                          | 126        |
| Introduction . . . . .                        | 126        |
| IIR Filter Analysis . . . . .                 | 126        |
| First IIR Filter . . . . .                    | 127        |
| Second IIR Filter . . . . .                   | 128        |
| Final Bandpath Response . . . . .             | 129        |
| RTL Block Diagram . . . . .                   | 131        |
| RTL Interface . . . . .                       | 132        |
| Control Unit . . . . .                        | 132        |
| Control Unit Architecture . . . . .           | 133        |
| CU FSM . . . . .                              | 134        |
| Time Management Unit (TMU) . . . . .          | 137        |
| TMU flow diagram . . . . .                    | 138        |
| Interfaces . . . . .                          | 140        |
| Link Layer Interface . . . . .                | 140        |
| Analog Interface . . . . .                    | 145        |
| Power Management Unit . . . . .               | 148        |
| Introduction . . . . .                        | 148        |
| Power Management . . . . .                    | 151        |
| Configuration Register File . . . . .         | 152        |
| Register Map . . . . .                        | 153        |
| Time and Power Optimizations . . . . .        | 155        |
| <b>Simulation Results</b>                     | <b>155</b> |
| Gaussian Filter . . . . .                     | 155        |
| MATLAB Modeling . . . . .                     | 155        |
| RTL Verification . . . . .                    | 157        |
| Time to Digital Converter . . . . .           | 159        |
| MATLAB Modeling . . . . .                     | 159        |
| RTL Verification . . . . .                    | 160        |
| Loop Filter . . . . .                         | 165        |
| MATLAB Modeling . . . . .                     | 165        |
| RTL Verification . . . . .                    | 168        |
| Sigma Delta . . . . .                         | 169        |
| MATLAB Simulation . . . . .                   | 169        |
| RTL Verification . . . . .                    | 173        |
| IIR Filter . . . . .                          | 176        |
| RTL Verification . . . . .                    | 176        |
| System Integration . . . . .                  | 177        |

|                                            |            |
|--------------------------------------------|------------|
| MATLAB Simulations . . . . .               | 177        |
| RTL vs MATLAB . . . . .                    | 184        |
| <b>ASIC Implementation</b>                 | <b>188</b> |
| Synthesis Summary . . . . .                | 188        |
| Quality of Results (QOR) Report . . . . .  | 188        |
| Area Report Summary . . . . .              | 188        |
| Clock Domain Crossing (CDC) . . . . .      | 189        |
| Equivalence Checking (Formality) . . . . . | 189        |
| <b>Specification validation</b>            | <b>190</b> |
| <b>Conclusion and Future Work</b>          | <b>193</b> |
| Summary . . . . .                          | 193        |
| Future Work . . . . .                      | 193        |
| <b>References</b>                          | <b>193</b> |

## List of Figures

|    |                                                                                                                                                                  |    |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1  | Radio connection between BLE peripheral and central devices . . . . .                                                                                            | 13 |
| 2  | Frequency Shift Keying . . . . .                                                                                                                                 | 14 |
| 3  | BLE modulation characteristics and frequency deviation behavior . . . . .                                                                                        | 16 |
| 4  | Inband spurious emission specifications . . . . .                                                                                                                | 17 |
| 5  | Transmit spectrum mask and spurious emission limits for BLE transmission at 1 Msym/s . . . . .                                                                   | 17 |
| 6  | System-level block diagram of the proposed architecture . . . . .                                                                                                | 18 |
| 7  | Divider based ADPLL . . . . .                                                                                                                                    | 20 |
| 8  | Divider-less ADPLL . . . . .                                                                                                                                     | 21 |
| 9  | $z$ -domain model of ADPLL [5] . . . . .                                                                                                                         | 25 |
| 10 | $s$ -domain model of the ADPLL . . . . .                                                                                                                         | 28 |
| 11 | Illustration of phase noise . . . . .                                                                                                                            | 30 |
| 12 | Phase noise plot . . . . .                                                                                                                                       | 32 |
| 13 | ADPLL design space: stability and BLE adjacent-channel power(ACP) combined                                                                                       | 33 |
| 14 | ADPLL Output Phase Noise . . . . .                                                                                                                               | 33 |
| 15 | DCO Frequency vs Time . . . . .                                                                                                                                  | 34 |
| 16 | ADPLL with one point modulation at phase detector input . . . . .                                                                                                | 36 |
| 17 | Magnitude response of ADPLL one point modulation at reference accumulator input . . . . .                                                                        | 37 |
| 18 | ADPLL with one point modulation at DCO input . . . . .                                                                                                           | 38 |
| 19 | Magnitude response of ADPLL one point modulation at DCO input . . . . .                                                                                          | 39 |
| 20 | ADPLL with two point modulation . . . . .                                                                                                                        | 41 |
| 21 | Magnitude response of ADPLL 2 point modulation with gain overestimation ( $\hat{K}_{DCO} = 2K_{DCO}$ and $\hat{K}_{DCO} = 10K_{DCO}$ respectively) . . . . .     | 42 |
| 22 | Magnitude response of ADPLL 2 point modulation with gain underestimation ( $1.5\hat{K}_{DCO} = K_{DCO}$ and $1.9\hat{K}_{DCO} = K_{DCO}$ respectively) . . . . . | 43 |
| 23 | Direct-form FIR filter realization . . . . .                                                                                                                     | 44 |
| 24 | Transposed-form FIR filter realization . . . . .                                                                                                                 | 45 |
| 25 | Exploiting coefficient symmetry for an odd number of FIR filter taps . . . . .                                                                                   | 46 |
| 26 | Exploiting coefficient symmetry for an even number of FIR filter taps . . . . .                                                                                  | 46 |
| 27 | Odd-length symmetric FIR filter (Type-I linear phase) showing symmetric tap pairing and center coefficient . . . . .                                             | 48 |
| 28 | Even-length symmetric FIR filter (Type-II linear phase) showing symmetric tap pairing without a center coefficient . . . . .                                     | 48 |
| 29 | 3:1 multiplexer for selecting $\{+2h(k), 0, -2h(k)\}$ based on $s_k$ . . . . .                                                                                   | 49 |
| 30 | Modified even-length symmetric FIR structure exploiting coefficient symmetry                                                                                     | 50 |
| 31 | Power spectral density of the Gaussian pulse for different span and SPS values .                                                                                 | 52 |
| 32 | Time-domain Gaussian pulse amplitude for different span and SPS values . . .                                                                                     | 53 |
| 33 | NRZ Block Diagram . . . . .                                                                                                                                      | 53 |
| 34 | Upsampling Block Diagram . . . . .                                                                                                                               | 55 |
| 35 | Gaussian Filter Block Diagram . . . . .                                                                                                                          | 57 |
| 36 | Internal architecture of the Gaussian filter — symmetric FIR structure with 64-sample delay line. . . . .                                                        | 60 |
| 37 | Symmetric sample pair selection and multiply-accumulate unit. . . . .                                                                                            | 61 |
| 38 | Outer system architecture implementing BLE two-point modulation. . . . .                                                                                         | 62 |
| 39 | Interface abstraction of the radix-4 Booth recoding multiplier. . . . .                                                                                          | 64 |

|    |                                                                                                                                                                             |    |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 40 | Architecture of the radix-4 Booth recoding multiplier. . . . .                                                                                                              | 65 |
| 41 | Example of modified radix-4 Booth recoding showing group formation, Booth digit generation, partial-product computation, and the corresponding recoding table.[6] . . . . . | 67 |
| 42 | Step-by-step execution table of the radix-4 Booth multiplication algorithm for $a = 0110_2$ and $x = 1010_2$ . . . . .                                                      | 68 |
| 43 | Dot diagram representing the partial product generation, shifting, and accumulation groupings. . . . .                                                                      | 68 |
| 44 | block diagram . . . . .                                                                                                                                                     | 69 |
| 45 | TDC System Block Diagram [13] . . . . .                                                                                                                                     | 72 |
| 46 | Reference clock re-timing [13] . . . . .                                                                                                                                    | 73 |
| 47 | ADPLL diagram with TDC [13] . . . . .                                                                                                                                       | 73 |
| 48 | Reference clock re-timing for multiple reference cycles [2] . . . . .                                                                                                       | 74 |
| 49 | Phase values of expected phase and phase due to re-timing of reference clock [13] . . . . .                                                                                 | 74 |
| 50 | Flash TDC [14] . . . . .                                                                                                                                                    | 76 |
| 51 | Vernier TDC [14] . . . . .                                                                                                                                                  | 76 |
| 52 | TDC Core Block Diagram [13] . . . . .                                                                                                                                       | 77 |
| 53 | TDC Core Timing Diagram [13] . . . . .                                                                                                                                      | 78 |
| 54 | Timing diagram showing DCDO rising edge leading FREF rising edge [13] . . . . .                                                                                             | 79 |
| 55 | Timing diagram showing DCDO rising edge lagging FREF rising edge [13] . . . . .                                                                                             | 79 |
| 56 | Timing diagram showing FREF rising edge lagging the DCO rising edge with small time difference . . . . .                                                                    | 81 |
| 57 | Timing diagram showing FREF rising edge leading the DCO rising edge with small time difference . . . . .                                                                    | 82 |
| 58 | Timing diagram showing capturing done with CKR2 for case FREF lead DCO and CKR settles to 1 . . . . .                                                                       | 83 |
| 59 | Timing diagram showing capturing done with CKR2 for case FREF lead DCO and CKR settles to 0 . . . . .                                                                       | 84 |
| 60 | Timing diagram showing capturing done with CKR2 for case where both FREF and FREF2 are away from the metastability window . . . . .                                         | 85 |
| 61 | Timing diagram showing capturing done with CKR2 for case FREF lags DCO and CKR settles to 1 . . . . .                                                                       | 86 |
| 62 | Timing diagram showing capturing done with CKR2 for case FREF lags DCO and CKR settles to 0 . . . . .                                                                       | 86 |
| 63 | Timing diagram showing capturing done with CKR2 for case where FREF2 edge enters the metastability window . . . . .                                                         | 87 |
| 64 | Timing diagram to demonstrate random phase start correction . . . . .                                                                                                       | 88 |
| 65 | TDC system block diagram . . . . .                                                                                                                                          | 89 |
| 66 | TDC core top module block diagram . . . . .                                                                                                                                 | 90 |
| 67 | TDC Core block diagram . . . . .                                                                                                                                            | 90 |
| 68 | 64-bit Priority Encoder . . . . .                                                                                                                                           | 91 |
| 69 | Priority encoder index decoding logic . . . . .                                                                                                                             | 92 |
| 70 | ADPLL with Type I loop filter . . . . .                                                                                                                                     | 95 |
| 71 | ADPLL with Type II loop filter . . . . .                                                                                                                                    | 96 |
| 72 | ADPLL different modes of operation . . . . .                                                                                                                                | 97 |
| 73 | ADPLL type I frequency response . . . . .                                                                                                                                   | 98 |
| 74 | Closed loop response of ADPLL for different $\zeta$ values . . . . .                                                                                                        | 99 |

|     |                                                                                                                                                         |     |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 75  | Closed loop response of ADPLL for different $\alpha$ values, $\zeta = 0.25$ . . . . .                                                                   | 100 |
| 76  | Digital loop filter abstract diagram . . . . .                                                                                                          | 101 |
| 77  | Digital loop filter block diagram . . . . .                                                                                                             | 101 |
| 78  | Block-level model of a MASH sigma–delta modulator form [17]. . . . .                                                                                    | 102 |
| 79  | First-order MASH sigma–delta model and its digital implementation. . . . .                                                                              | 103 |
| 80  | Linear model of a first-order digital EFM from [16] . . . . .                                                                                           | 105 |
| 81  | Linear noise cancellation mechanism in an $L$ th-order MASH modulator from [16] . . . . .                                                               | 106 |
| 82  | Noise shaping behavior of the implemented modified sigma–delta modulator from [15]. . . . .                                                             | 107 |
| 83  | linear model of the modified dither MASH 1–1–1 sigma–delta modulator. . . . .                                                                           | 108 |
| 84  | Block diagram . . . . .                                                                                                                                 | 113 |
| 85  | Drive strength control and shunt capacitance controlled standard cells used in DCO . . . . .                                                            | 114 |
| 86  | LC tank-based oscillator with switchable capacitors . . . . .                                                                                           | 114 |
| 87  | Modeling a binary-weighted switchable capacitor of index $k$ . . . . .                                                                                  | 115 |
| 88  | Normalized DCO hardware abstraction layer . . . . .                                                                                                     | 117 |
| 89  | Control circuit of oscillator acquisition bits (OA) . . . . .                                                                                           | 118 |
| 90  | DCO gain paths: implementation block diagram . . . . .                                                                                                  | 119 |
| 91  | Sign-sign LMS adaptation algorithm for DCO gain calibration . . . . .                                                                                   | 120 |
| 92  | High level DCO overview . . . . .                                                                                                                       | 121 |
| 93  | DCO Interface . . . . .                                                                                                                                 | 121 |
| 94  | Gain normalization block diagram . . . . .                                                                                                              | 123 |
| 95  | LMS high level interaction . . . . .                                                                                                                    | 124 |
| 96  | LMS Block diagram . . . . .                                                                                                                             | 124 |
| 97  | LMS Block diagram . . . . .                                                                                                                             | 124 |
| 98  | First filter response . . . . .                                                                                                                         | 127 |
| 99  | second filter response . . . . .                                                                                                                        | 128 |
| 100 | band pass response . . . . .                                                                                                                            | 129 |
| 101 | Cross- correlation between modulation data and phase error at $\alpha_1 = 2^{-3}$ , $\alpha_2 = 2^{-4}$ . . . . .                                       | 130 |
| 102 | Cross-correlation between modulation data and phase error at $\alpha_1 = 2^{-3}$ , $\alpha_2 = 2^{-6}$ . . . . .                                        | 130 |
| 103 | IIR filter abstract diagram . . . . .                                                                                                                   | 131 |
| 104 | IIR filter block diagram . . . . .                                                                                                                      | 131 |
| 105 | Overall control architecture of the proposed BLE transmitter showing the interaction between the crystal clock domain and the CKR clock domain. . . . . | 132 |
| 106 | Main control unit Architecture . . . . .                                                                                                                | 133 |
| 107 | Control Unit FSM . . . . .                                                                                                                              | 134 |
| 108 | Control Unit FSM . . . . .                                                                                                                              | 138 |
| 109 | Link Layer interface block diagram showing input and output signals. . . . .                                                                            | 141 |
| 110 | Analog RF interface block diagram showing input and output signals. . . . .                                                                             | 146 |
| 111 | Small sensor battery . . . . .                                                                                                                          | 149 |
| 112 | Power domain in system. Yellow indicates the switchable power domain, while blue indicates the always-on power domain. . . . .                          | 150 |
| 113 | PMU fsm . . . . .                                                                                                                                       | 151 |
| 114 | APB v.2 slave FSM [19] . . . . .                                                                                                                        | 152 |

|     |                                                                                                                                              |     |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 115 | Time-aligned NRZ input and Gaussian filter output after group-delay compensation . . . . .                                                   | 155 |
| 116 | Amplitude comparison between reference FIR and multiplier-free symmetric FIR implementations . . . . .                                       | 156 |
| 117 | Verification flow comparing the MATLAB bit-accurate model with the HDL model. . . . .                                                        | 157 |
| 118 | Gaussian filter output: MATLAB bit-accurate model vs. RTL simulation (no flush). . . . .                                                     | 157 |
| 119 | Maximum TDC error against DCO clock period . . . . .                                                                                         | 159 |
| 120 | Output waveforms for DCO clock frequency = 2.5GHz . . . . .                                                                                  | 161 |
| 121 | Output waveforms for DCO clock frequency = 4GHz . . . . .                                                                                    | 162 |
| 122 | Output waveforms for DCO clock period = 364 ps . . . . .                                                                                     | 163 |
| 123 | Toggle coverage holes in TDC testbench . . . . .                                                                                             | 163 |
| 124 | Coverage report summary for TDC testbench . . . . .                                                                                          | 164 |
| 125 | Fractional parts covered in TDC testbench . . . . .                                                                                          | 164 |
| 126 | Transient response for $\alpha = 2^{-2}$ , $\rho = 2^{-5}$ (critically damped response) . . . . .                                            | 165 |
| 127 | Transient response for $\alpha = 2^{-3}$ , $\rho = 2^{-7}$ (critically damped response) . . . . .                                            | 166 |
| 128 | Transient response for $\alpha = 2^{-3}$ , $\rho = 2^{-5}$ (under-damped response) . . . . .                                                 | 166 |
| 129 | Digital loop filter RTL output vs MATLAB . . . . .                                                                                           | 167 |
| 130 | Digital loop filter RTL output vs MATLAB . . . . .                                                                                           | 168 |
| 131 | Extended output spectrum of the sample-based MASH 1-1-1 $\Sigma\Delta$ modulator for input 0.25 . . . . .                                    | 169 |
| 132 | Extended output spectrum of the sample-based MASH 1-1-1 $\Sigma\Delta$ modulator for an irrational input ( $X_{in} = \sqrt{2}/10$ ). . . . . | 170 |
| 133 | Extended output spectrum of the sample-based MASH 1-1-1 $\Sigma\Delta$ modulator for input 0.75 . . . . .                                    | 170 |
| 134 | Extended output spectrum of the sample-based MASH 1-1-1 $\Sigma\Delta$ modulator for input 0.5. . . . .                                      | 171 |
| 135 | Extended output spectrum of the sample-based MASH 1-1-1 $\Sigma\Delta$ modulator for aninput 0.000052. . . . .                               | 171 |
| 136 | Simulation results for input 0.25 . . . . .                                                                                                  | 173 |
| 137 | Simulation results for 0.5 input . . . . .                                                                                                   | 173 |
| 138 | Block diagram of Verification . . . . .                                                                                                      | 174 |
| 139 | Comparison of matlab and HDL model outputs . . . . .                                                                                         | 174 |
| 140 | phase error path RTL output vs MATLAB . . . . .                                                                                              | 176 |
| 141 | Modulation data path RTL output vs MATLAB . . . . .                                                                                          | 176 |
| 142 | Instantaneous Eye Diagram . . . . .                                                                                                          | 177 |
| 143 | Modulation signal (Instantaneous over filtered) . . . . .                                                                                    | 178 |
| 144 | Filtered Eye Diagram . . . . .                                                                                                               | 178 |
| 145 | Modulated signal & training sequence . . . . .                                                                                               | 179 |
| 146 | SSLMS coverage . . . . .                                                                                                                     | 179 |
| 147 | BLE channel lock . . . . .                                                                                                                   | 180 |
| 148 | ADPLL lock on channel through three modes . . . . .                                                                                          | 181 |
| 149 | ACQ mode (Zoomed) . . . . .                                                                                                                  | 182 |
| 150 | Tracking mode (finest mode) . . . . .                                                                                                        | 183 |
| 151 | Frequency looked at different channel (Start = 2.580GHz, Target = 2.464GHz) . . . . .                                                        | 183 |
| 152 | Sending many packets during one connection interval . . . . .                                                                                | 184 |
| 153 | DCO frequency vs time during the startup sequence. . . . .                                                                                   | 184 |

|     |                                                                                                                                                                                                               |     |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 154 | DCO instantaneous frequency across a full multi-packet transmission sequence.                                                                                                                                 | 185 |
| 155 | Extracted filtered modulation deviation comparing MATLAB and RTL across multiple packets. . . . .                                                                                                             | 185 |
| 156 | Zoomed view of the filtered modulation deviation during the modulation phase.                                                                                                                                 | 186 |
| 157 | Eye diagram of the RTL extracted baseband frequency deviation. . . . .                                                                                                                                        | 187 |
| 158 | RTL internal LMS gain error trajectory. . . . .                                                                                                                                                               | 187 |
| 159 | Data Qualifier Synchronizer . . . . .                                                                                                                                                                         | 189 |
| 160 | Eye diagram of the officially extracted baseband frequency from RTL simulation, showing the spread of zero crossings around the ideal transition point. . .                                                   | 190 |
| 161 | Full-scale eye diagram of the extracted baseband frequency deviation from RTL simulation, showing the upper and lower deviation bands of the two-point modulation. . . . .                                    | 191 |
| 162 | Extracted frequency deviation versus time for the input pattern 0000111101010101, showing the settled plateau deviation from sustained bit runs and the reduced peak deviation from alternating bits. . . . . | 192 |

## List of Tables

|    |                                                                                                                                     |     |
|----|-------------------------------------------------------------------------------------------------------------------------------------|-----|
| 1  | Comparison of Divider-based vs. Divider-less Architectures . . . . .                                                                | 22  |
| 2  | FIR Implementation Comparison . . . . .                                                                                             | 47  |
| 3  | NRZ Block Port List . . . . .                                                                                                       | 54  |
| 4  | Upsampling Block Port List . . . . .                                                                                                | 56  |
| 5  | Gaussian Filter Block Port List . . . . .                                                                                           | 58  |
| 6  | Radix-4 Booth Recoding Multiplier Interface . . . . .                                                                               | 65  |
| 7  | Truth table to find SUB signal . . . . .                                                                                            | 87  |
| 8  | TDC Engine Parameters Description . . . . .                                                                                         | 93  |
| 9  | TDC Engine Ports Description . . . . .                                                                                              | 94  |
| 10 | Digital loop filter port list . . . . .                                                                                             | 102 |
| 11 | Sigma-Delta Modulator (SD_top) Interface Signals . . . . .                                                                          | 111 |
| 12 | DCO Interface Ports Description . . . . .                                                                                           | 122 |
| 13 | Gain Normalization Block Signal Details . . . . .                                                                                   | 123 |
| 14 | SS-LMS Signal List . . . . .                                                                                                        | 125 |
| 15 | IIR Filter ports description . . . . .                                                                                              | 132 |
| 16 | Default values of the TMU timing parameters . . . . .                                                                               | 139 |
| 17 | TMU output signals . . . . .                                                                                                        | 140 |
| 18 | Extended clock-cycle behavior of a MASH 1–1–1 sigma–delta modulator for $X_{in} = 0.25$ . . . . .                                   | 169 |
| 19 | Average output comparison under no frequency-step mismatch. The error magnitude is defined as $ \text{Average} - X_{in} $ . . . . . | 172 |
| 20 | Timing Path Group Summary . . . . .                                                                                                 | 188 |
| 21 | Formality Compare Points . . . . .                                                                                                  | 189 |

## Introduction

Bluetooth Low Energy (BLE) is designed as a complementary technology to classic Bluetooth, but it follows a fundamentally different approach. Instead of targeting high data rates, BLE is optimized for ultra-low power consumption and very low implementation cost, making it suitable for battery-powered and resource-constrained devices. BLE operates in the globally available 2.4 GHz ISM band and uses narrowband channels with well-defined frequency spacing and hopping to ensure robust communication and coexistence with other wireless systems. By prioritizing energy efficiency, stable frequency operation, and simple modulation schemes, BLE achieves long battery lifetime while maintaining reliable short-range wireless connectivity.

## BLE Overview

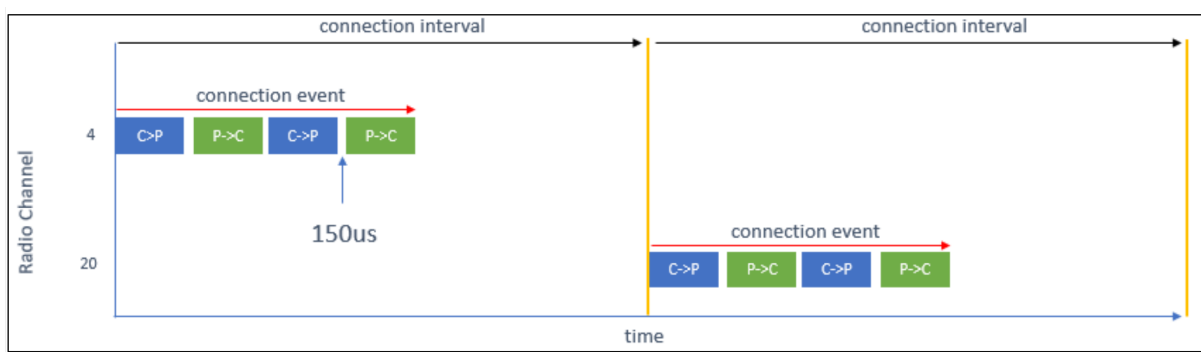


Figure 1: Radio connection between BLE peripheral and central devices

- Bluetooth Low Energy operates over 40 radio channels in the 2.4 GHz ISM band, with each channel occupying a bandwidth of 2 MHz.
- Communication between a peripheral device and a central device is established using configurable connection intervals. Several parameters are defined to minimize power consumption, including:
  - Connection interval duration
  - Peripheral latency
  - Channel hopping sequence
- Device discovery and connection establishment are performed using advertisement packets transmitted on dedicated channels 37, 38, and 39.

## Modulation Scheme

### Overview

BLE 5.4 standard employs frequency shift keying (FSK) and specifically the Gaussian Frequency Shift Keying (GFSK) as its modulation scheme. The use of frequency shift keying enables use of power-efficient nonlinear amplifiers which are more suitable for BLE as it is a low power device. This section reviews the theory and key properties of FSK and GFSK. The diagram of binary frequency shift keying (BFSK) is illustrated in figure 2 below.

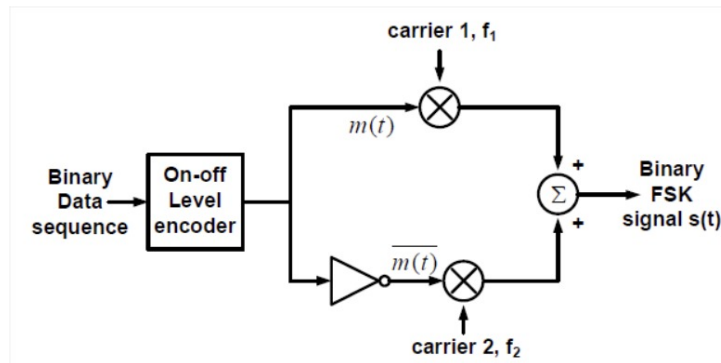


Figure 2: Frequency Shift Keying

In BFSK, independent oscillators generate separate in-phase and quadrature components. These oscillators switch at the start of each symbol period to create the carrier frequency for modulation.

However, due to varying amplitudes and phases of the oscillators, abrupt frequency changes occur for every bit transition. The modulated FSK signal has a wide spectrum (out of band energy) with non-negligible side-lobes.

### GFSK Modulation

GFSK uses a Gaussian low pass filter before modulating the baseband waveform signal (the data signal). The Gaussian filter smooths out the abrupt frequency changes in FSK (called pulse shaping).

The filter's impulse response follows a Gaussian function, which narrows the frequency response. As a result, any sudden frequency shifts are filtered out, making transitions smoother. The spectral width of the GFSK signal is reduced compared to unfiltered FSK.

The main advantages and tradeoffs of GFSK are explained below:

- **Reduced side lobes:** GFSK minimizes sideband power and out-of-band spectrum.
- **Better spectral efficiency:** By shaping the pulses, GFSK achieves improved spectral utilization.
- **Constant Envelope:** GFSK/FSK modulates only the frequency and not the amplitude, the transmitted signal has a constant envelope. This allows the use of nonlinear power amplifiers (such as Class C or E), which are more energy-efficient — an essential feature for battery-powered devices.
- While GFSK reduces spectral width, it increases the likelihood of inter-symbol interference (ISI) due to spreading in the time domain.

## BLE 5.4 Specifications

### Modulation characteristics

**Modulation Rate and Bit Rate in BLE:** Bluetooth Low Energy (BLE) defines two modulation schemes at the physical layer. The mandatory modulation scheme, referred to as the *1 Msym/s modulation*, employs shaped binary frequency modulation to minimize transceiver complexity. In this scheme, the symbol rate is fixed at 1 Msym/s and it supports two different PHY modes. The *LE 1M PHY* transmits uncoded data at a rate of 1 Mb/s, where the data rate is equal to the symbol rate. In contrast, the *LE Coded PHY* also transmits symbols at 1 Msym/s, but the effective data rate is lower than the symbol rate due to the insertion of redundancy bits. This redundancy enables improved receiver sensitivity and extended communication range, making the LE Coded PHY particularly suitable for long-range and low-power IoT applications. In this context, the term *coded* refers to the use of forward error correction (FEC) and symbol-level coding at the physical layer, where multiple symbols are transmitted per information bit to enhance robustness against noise, fading, and interference.

An optional modulation scheme, known as the *2 Msym/s modulation*, operates at a higher symbol rate of 2 Msym/s and supports a single PHY mode, namely the *LE 2M PHY*. This PHY transmits uncoded data at a rate of 2 Mb/s, enabling faster data transmission and reduced on-air time. However, the increased symbol rate results in reduced robustness to noise and a shorter communication range compared to the LE Coded PHY. According to the BLE specification, support for the LE 1M PHY is mandatory for all devices, while support for the LE Coded PHY is optional.

**GFSK Modulation Index:** The modulation index in Gaussian Frequency Shift Keying (GFSK) describes the frequency separation between the two modulation tones relative to the symbol rate and plays a critical role in controlling intersymbol interference (ISI). It is mathematically defined as:

$$h = \frac{\Delta f}{R_s}$$

where  $\Delta f$  is the peak frequency deviation and  $R_s$  is the symbol rate. For the standard BLE 1M PHY, the Bluetooth specification mandates a tight modulation index range:

$$0.45 \leq h \leq 0.55$$

Low modulation indices (e.g.,  $h < 0.3$ ) result in a very narrow occupied bandwidth and high spectral efficiency, but significantly degrade demodulation performance because the receiver struggles to distinguish between the frequency tones. Conversely, high modulation indices (e.g.,  $h > 1$ ) improve noise immunity but expand the signal spectrum, increasing adjacent-channel leakage and reducing spectral efficiency. A modulation index of approximately  $h = 0.5$  represents an optimal tradeoff for BLE, achieving strong noise immunity while maintaining tight spectral containment, which aligns with regulatory requirements and enables efficient Gaussian pulse shaping.

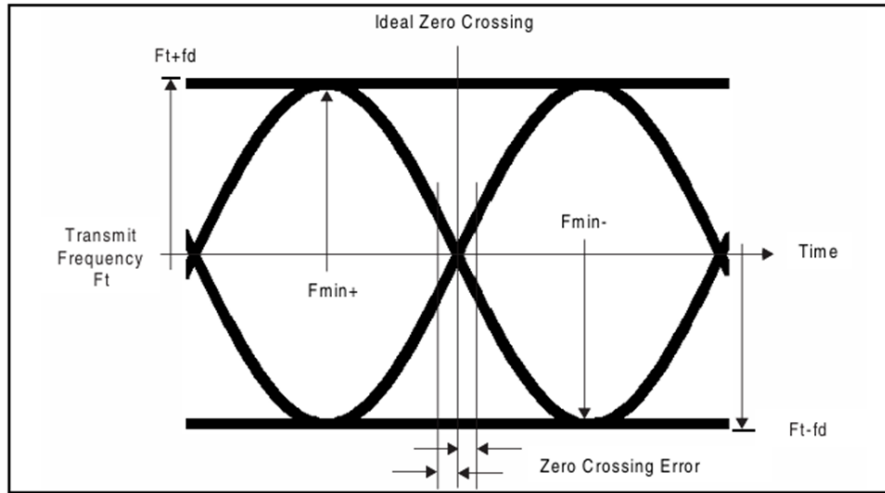


Figure 3: BLE modulation characteristics and frequency deviation behavior

For each transmission, the minimum frequency deviation is defined as:

$$F_{\min} = \min \{ |F_{\min}^+|, |F_{\min}^-| \}$$

This corresponds to a 1010 bit sequence and shall be no smaller than  $\pm 80\%$  of the frequency deviation with respect to the transmit frequency, which corresponds to a 00001111 sequence.

**Minimum Frequency Deviation Requirements:** The minimum frequency deviation shall never be less than **185 kHz** when transmitting at a symbol rate of **1 Msym/s**. The minimum frequency deviation shall never be less than **370 kHz** when transmitting at a symbol rate of **2 Msym/s**.

**Symbol Timing and Zero-Crossing Error:** The symbol timing accuracy shall be better than  $\pm 50$  ppm. The zero-crossing error, defined as the time difference between the ideal symbol period and the measured crossing time, shall be less than  $\pm \frac{1}{8}$  of a symbol period.

### Spurious Emissions

Spurious emissions are unintentional and unwanted radio-frequency spectral components that appear outside the necessary signal bandwidth, effectively contributing to radio-frequency noise pollution. These emissions can degrade coexistence with neighboring wireless systems and must be carefully controlled to meet regulatory requirements.

Spurious emissions are generally classified into *in-band* and *out-of-band* components. In-band spurious emissions occur within the BLE operating band of **2.400–2.4835 GHz** and are often caused by signal spectral spreading due to modulation effects, particularly sharp transitions in the transmitted waveform. Another significant contributor is phase noise from the phase-locked loop (PLL), which can introduce unwanted frequency components both inside and near the band of interest. The in-band spurious emission is presented in the standard as shown in Figure 4 below.

| Frequency offset                    | Spurious Power |
|-------------------------------------|----------------|
| 2 MHz ( $ M-N  = 2$ )               | -20 dBm        |
| 3 MHz or greater ( $ M-N  \geq 3$ ) | -30 dBm        |

Figure 4: Inband spurious emission specifications

Out-of-band spurious emissions refer to unwanted spectral components that fall outside the allocated BLE frequency band. These emissions may originate from several sources within the transmitter chain, including clock-related harmonics, power amplifier (PA) nonlinearities that generate harmonic distortion, and leakage from the frequency synthesizer. Additional contributors include reference spurs and fractional- $N$  PLL spurs, which can appear at deterministic frequency offsets from the carrier.

Compliance with out-of-band spurious emission limits in the ISM band is the responsibility of the equipment manufacturer and must satisfy the regulatory requirements of the intended countries of sale. According to the standard spurious emission specifications, the tolerable spur level is limited to approximately **-20 dBm** at a **2 MHz** offset and **-30 dBm** at a **3 MHz** offset from the carrier frequency.

Figure 5 illustrates the transmit spectrum mask for BLE transmission using the 1 Msym/s modulation, highlighting the allowed spectral envelope and the corresponding spurious emission constraints.

| Parameter                       | Frequency Drift |
|---------------------------------|-----------------|
| Maximum drift                   | $\pm 50$ kHz    |
| Maximum drift rate <sup>1</sup> | 400 Hz/ $\mu$ s |

Figure 5: Transmit spectrum mask and spurious emission limits for BLE transmission at 1 Msym/s

# System Architecture

## System Block diagram

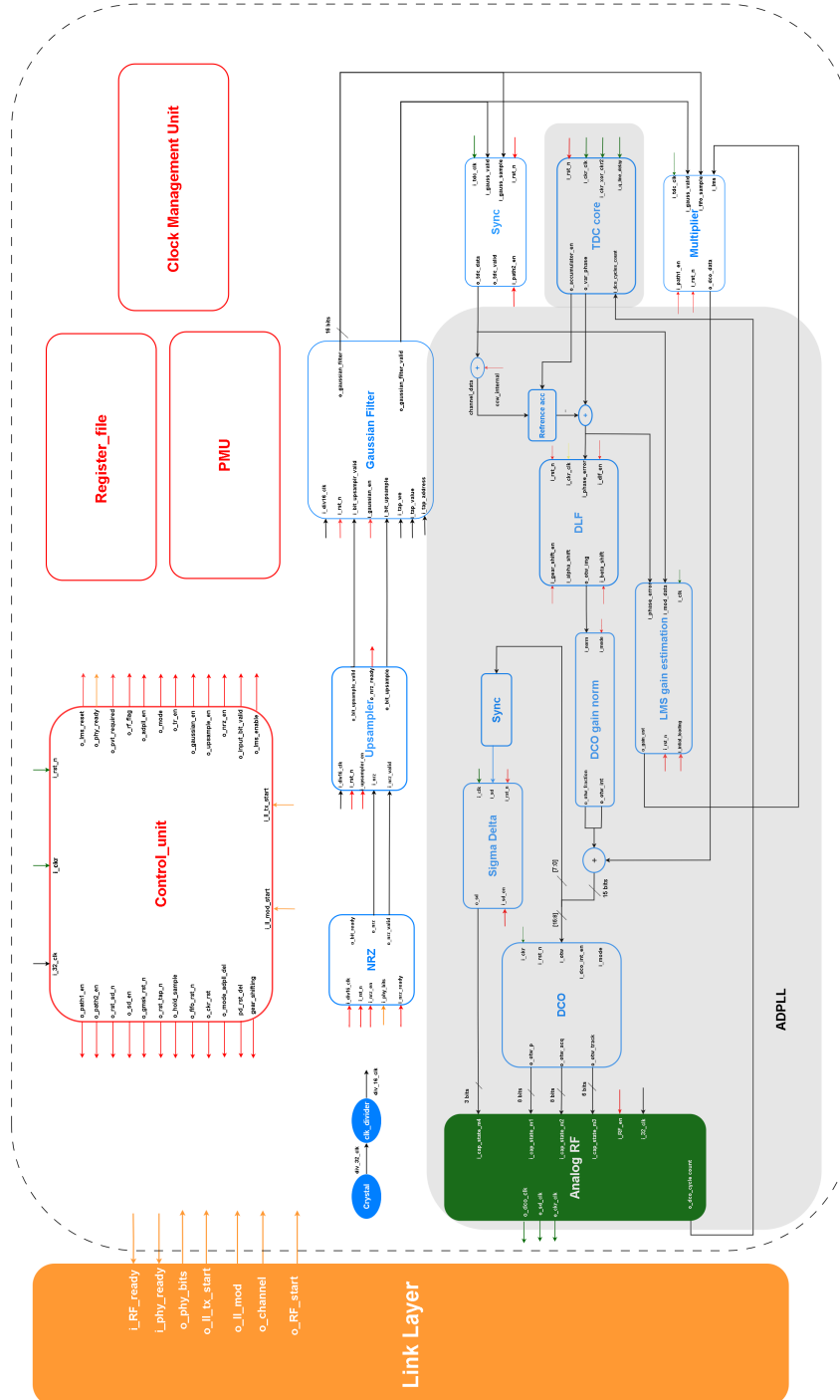


Figure 6: System-level block diagram of the proposed architecture

## Usage of ADPLL and Limitation of PLL

Analog PLLs suffer from limited scalability and robustness as CMOS technology advances. The reliance on analog components such as charge pumps, passive loop filters, and voltage-controlled oscillators makes their performance highly sensitive to process, voltage, and temperature (PVT) variations. In addition, external loop filters increase system complexity, occupy larger PCB area, and introduce additional noise sources, which are undesirable for highly integrated and low-power wireless systems.

All-Digital Phase-Locked Loops (ADPLLs) overcome these limitations by replacing analog blocks with digital counterparts. Digital signal processing is inherently easier to design, verify, and scale than analog signal processing, as it relies on well-defined logic operations rather than continuous-time behavior. Digital blocks benefit from repeatability, portability across technology nodes, and straightforward functional verification using simulation and formal methods.

In an ADPLL, phase detection, loop filtering, and frequency control are implemented in the digital domain, enabling full CMOS compatibility, improved scalability with technology scaling, and easier portability across process nodes. Digital implementation also allows precise programmability, calibration, and adaptive control, which are difficult to achieve reliably in analog PLLs.

For Bluetooth Low Energy (BLE) applications, ADPLLs are particularly attractive due to their low power consumption, fast frequency hopping capability, and robustness against PVT variations. The use of a Digitally Controlled Oscillator (DCO) eliminates the need for an analog control voltage and enables fine frequency resolution through digital tuning and noise-shaping techniques. Furthermore, digital loop filters and calibration algorithms, such as LMS-based gain normalization, can be efficiently integrated to enhance modulation accuracy and spectral compliance.

In summary, the inherent limitations of conventional analog PLLs, combined with the advantages of digital signal processing, motivate the transition toward ADPLLs as a more scalable, flexible, and robust frequency synthesis solution for modern low-power wireless transceivers.

## Comparison of Different ADPLL Architectures

This section compares the two principal ADPLL topologies used to sense and correct the DCO: the conventional divider-based architecture and the counter-based (dividerless) architecture. Both achieve closed-loop frequency and phase control, but they differ fundamentally in how the phase/frequency error is formed. The divider-based design inserts a frequency divider in the DCO feedback path so that a low-rate digital phase/frequency detector or accumulator can compare a divided-down DCO to the reference. The counter-based (dividerless) approach eliminates the feedback divider and instead measures the ratio of DCO cycles to reference cycles directly using high-speed counters and/or a time-to-digital converter (TDC), operating on the DCO timebase.

### Divider-based ADPLLs

The divider-based ADPLL architecture is presented in figure 7 below:

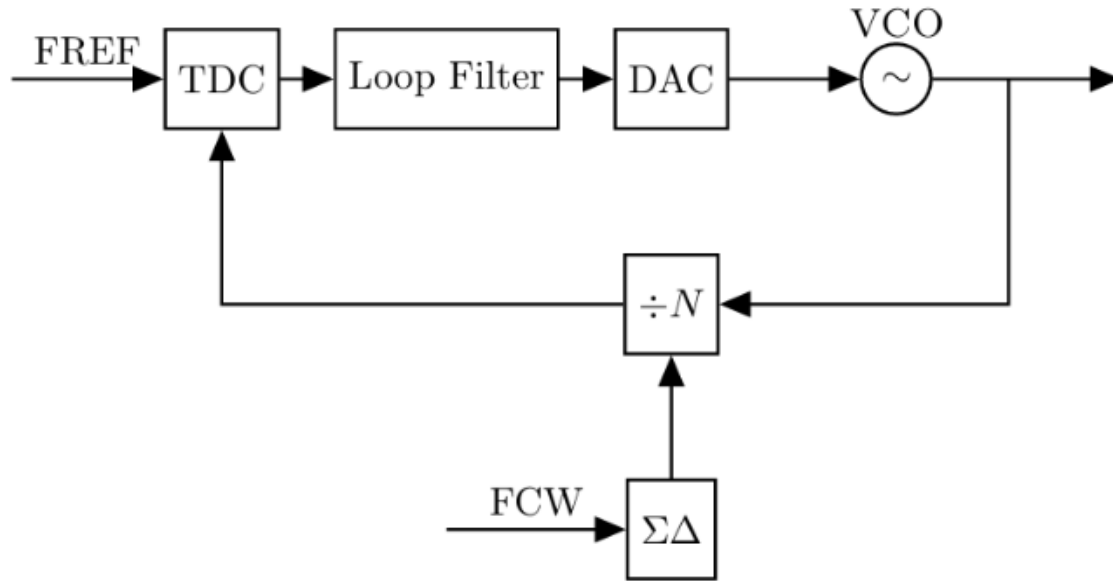


Figure 7: Divider based ADPLL

The first stage in feedback loop is the multi modulus divider (MMD) that performs integer division, while fractional division is achieved through sigma delta modulation of the divider control word (FCW). The TDC acts as a phase detector that produces quantized phase difference between DCO scaled output and the reference. Then the phase error is fed to digital loop filter (DLF) which produces an oscillator tuning word (OTW) to drive the DCO toward the target frequency.

This ADPLL commonly employs two sigma-delta modulators (SDM). One is used by DCO to produce a fine average frequency from DCO by SDM dithering and enables quantization noise shaping. While the other is used to get the fractional division operation in MMD.

The TDC and SDMs contribute to in-band phase noise due to quantization errors and fractional spurs. This mandates the use of high resolution TDC and SDMs to reduce the quantization error of both which significantly increases power consumption. Moreover, a wide range TDC is needed to cover the entire reference clock period.

### Divider-less ADPLLs

The divider-less ADPLL architecture is presented in figure 8 below:

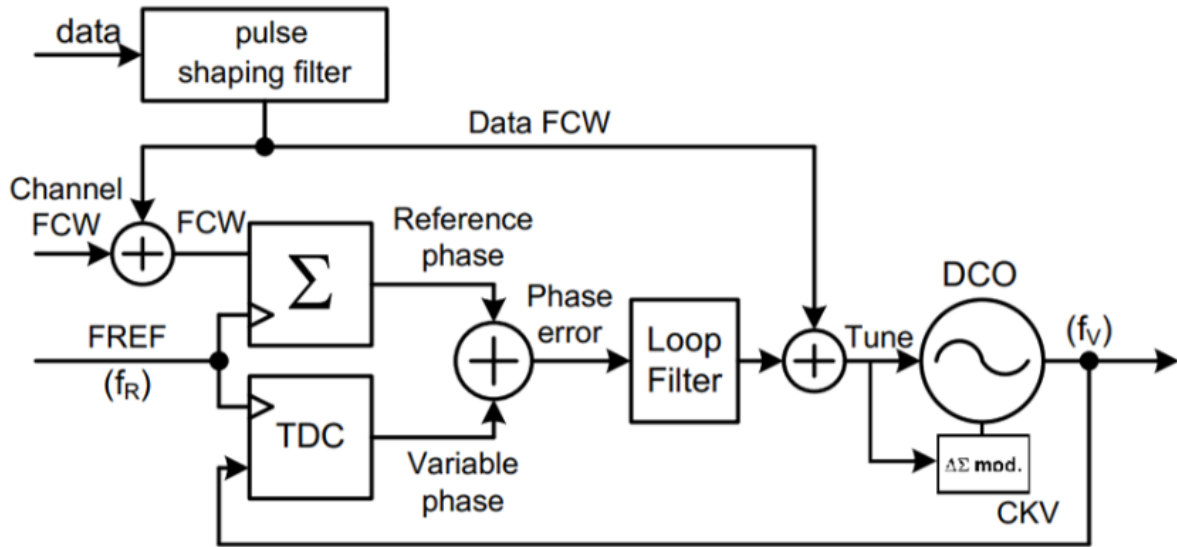


Figure 8: Divider-less ADPLL

This architecture operates in purely digitized phase domain which enables a fully digital phase-domain representation and simplifies modeling. It eliminates the need for divider (MMD). Divider-less operation has a simpler architecture which can achieve better in-band phase noise and requires only limited-range TDC.

A counter is usually used to calculate the integer part of phase error by counting the number of output clock cycles, while TDC handles the fractional remainder part, Thus the required operation range for the TDC is only one output frequency period and the counter could be turned off in lock in state (The integer error is zero) that greatly saves power. However, proper clock gating should be done to prevent high frequency counters from continuously running.

A major challenge in this architecture is metastability arising from clock domain crossing, Phase reference accumulator and DCO phase accumulator use different clocks, so synchronization is essential. To mitigate the issue of metastability, a re-timing circuit is used to sample the reference phase accumulator at the DCO cycle.

### Comparison Summary and Chosen Architecture

The main differences between the two architectures in terms of power consumption, hardware complexity and phase noise are summarized in table 1 below:

|                            | <b>Divider-based</b>                          | <b>Divider-less</b>                               |
|----------------------------|-----------------------------------------------|---------------------------------------------------|
| <b>TDC range</b>           | Wide range ( $T_{REF}$ )                      | Limited range ( $T_{DCO}$ )                       |
| <b>Power consumption</b>   | High                                          | Lower                                             |
| <b>Hardware complexity</b> | More complex<br>(Requires Fast MMD, Two SDMs) | Less complex<br>(Digital Logic Dominant, One SDM) |
| <b>Phase noise</b>         | High in-band phase noise                      | Lower in-band phase noise                         |
| <b>Major challenge</b>     | HW complexity                                 | Metastability                                     |

Table 1: Comparison of Divider-based vs. Divider-less Architectures

The counter-based architecture was selected for this project primarily due to its power efficiency and reduced hardware complexity, which aligns with the constraints of BLE applications.

## Linearized Model of ADPLL

In this section we will go through the  $z$ -domain model of the all digital phase lock loop (ADPLL) for type II. From this  $z$ -domain model we analyze one and two point modulation of ADPLL. Note that this analysis assumes that the signal transfer function of the sigma delta is one to simplify the analysis.

For the type II ADPLL architecture chosen in this thesis, its  $z$  domain model is shown in figure 9.

### Converting Frequency to Phase

The input  $N_r$  is the desired channel frequency multiplication ratio. So the desired output frequency  $f_v^{\text{desired}} = N_r \cdot f_{\text{ref}}$ . This desired frequency is also referred to as the steady state output frequency or  $f_v^{ss}$  for short.

To convert the desired frequency ratio to frequency, the ratio  $N_r$  is multiplied by  $f_{\text{ref}}$ . The frequency to phase converter (FPC) takes desired frequency  $f_v^{ss}$  and converts it to phase  $\varphi_r[n]$  which is evaluated at discrete time stamps  $t = nT_{\text{ref}}$ . The reference phase  $\varphi_r[n]$  is given by:

$$\varphi_r(t) = \varphi_0 + \int_0^t f(\tau) d\tau$$

As  $f(t)$  is constant for one reference clock period the reference phase at time  $t = nT_{\text{ref}}$  could be expressed as:

$$\begin{aligned} \varphi_r[n] &= \varphi_0 + \sum_{k=0}^n f[k]T_{\text{ref}} = f[n]T_{\text{ref}} + \varphi_0 + \sum_{k=0}^{n-1} f[k]T_{\text{ref}} \\ \therefore \varphi_r[n] &= f[n]T_{\text{ref}} + \varphi_r[n-1] \end{aligned}$$

So converting this equation to  $z$ -domain we get:

$$\begin{aligned} \varphi(z) &= f(z)T_{\text{ref}} + z^{-1}\varphi(z) \\ \therefore \frac{\varphi(z)}{f(z)} &= H_{\text{FPC}}(z) = \frac{T_{\text{ref}}}{1 - z^{-1}} \end{aligned}$$

Note that at as the accumulator takes one reference cycle latency its output at time  $t = nT_{\text{ref}}$  is  $\varphi[n-1]$  instead of  $\varphi[n]$  which introduces a cycle delay element  $z^{-1}$  which is multiplied by the transfer function of the frequency to phase converter. The same phenomena happens in the feedback path with the frequency to phase converter used to convert DCO output frequency to phase.

### Phase Detector

The frequency to phase converter is used to produce the reference phase  $\varphi_r[n-1]$  and the variable phase  $\varphi_v[n-1]$ . As both phases are sampled at the same instant at time  $t = (n-1)T_{\text{ref}}$  the two phase values can be subtracted directly to produce the phase error in the phase detector which is given by:

$$\varphi_e[n] = \varphi_r[n-1] - \varphi_v[n-1]$$

This phase error  $\varphi_e[n]$  is input to the loop filter to get a tune word used to control the frequency of the DCO.

### Loop Filter

The loop filter for type II ADPLL is a PI controller with proportional and integral paths. The integral path is added to reduce steady state errors.

The output tune word of the loop filter  $\bar{N}_v(z)$  is given by:

$$\bar{N}_v(z) = \left( \alpha + \frac{z^{-1}\rho}{1 - z^{-1}} \right) \varphi_e(z) = L_{II}(z)\varphi_e(z)$$

The output tune word of the loop filter is a dimensionless tune ratio that is used to control the DCO frequency. The LSB of the integer part of the output tune word  $\bar{N}_v(z)$  corresponds to change of DCO frequency by  $K_{\text{DCO}}f_{\text{ref}}$  so to convert the output tune word of the loop filter to the digital word that corresponds to desired DCO frequency, the output tune word is multiplied by  $f_{\text{ref}}$ .

### DCO

The DCO converts a digital input word  $d[n]$  into a signal with frequency  $f_v[n]$  where  $f_v[n]$  is directly proportional to  $d[n]$  with a proportionality constant given by  $K_{\text{DCO}}$  of units Hz/LSB. Thus the transfer function of the DCO could be given by:

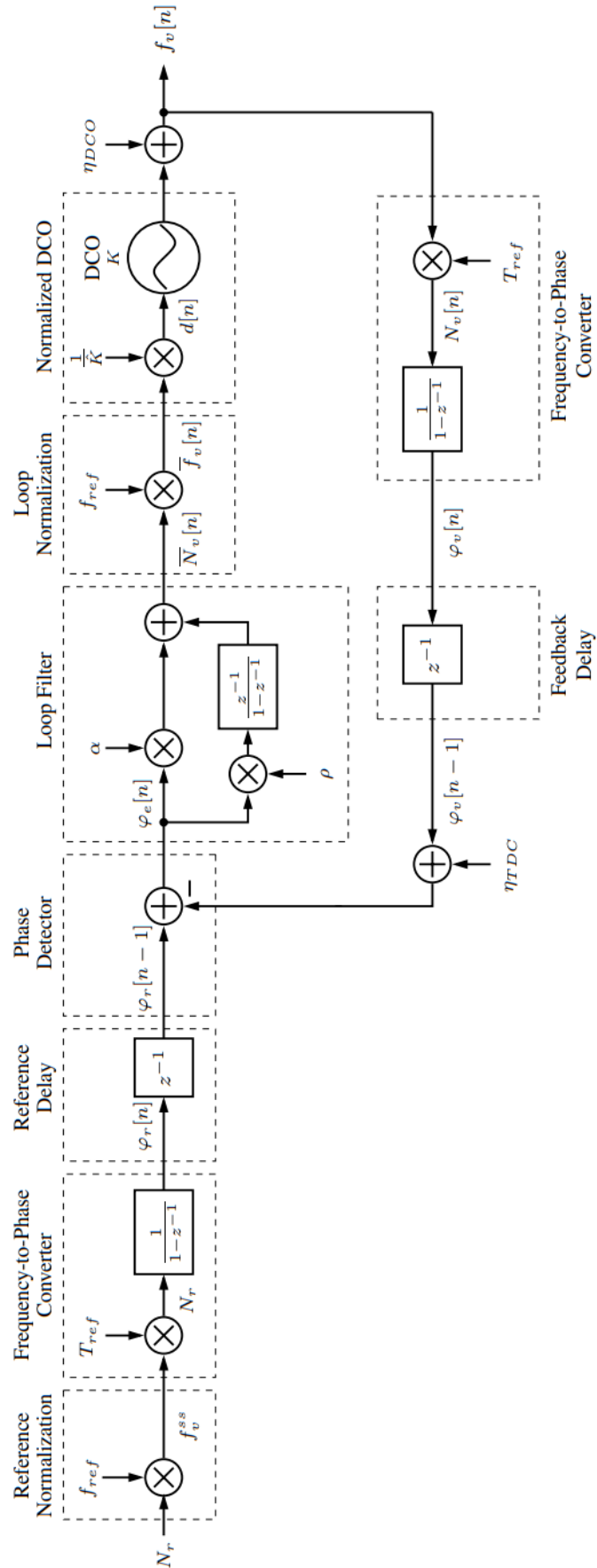
$$\frac{f_v(z)}{d(z)} = H_{\text{DCO}}(z) = K_{\text{DCO}}$$

Note that DCO is an analog component therefore its performance is dependent on supply, process and temperature variations (PVT variations). This then implies that  $K_{\text{DCO}}$  is not a constant and is dependent on PVT variations.

To ensure proper two point modulation (which will be discussed below) and to ensure that DCO performance remains robust under PVT variations, the gain of the DCO is normalized by multiplying it by  $1/\hat{K}_{\text{DCO}}$  where  $\hat{K}_{\text{DCO}}$  is an estimated value of  $K_{\text{DCO}}$  to cancel the effects of PVT variations that alter the value of  $K_{\text{DCO}}$ .

### Time to Digital Converter

In the feedback path, a time to digital converter (TDC) is used to obtain the variable or output phase of the DCO. It is modeled here as a frequency to phase converter identical to frequency to phase converter used to convert reference frequency to reference phase and it is used to give DCO output phase.

Figure 9:  $z$ -domain model of ADPLL [5]

## Type II ADPLL Parameters

### Closed Loop Response

The closed loop transfer function is given by:

$$H_{cl}(z) = \frac{f_v(z)}{N_r(z)} = \frac{z^{-1}}{1 - z^{-1}} \left( \frac{G(z)}{1 + G(z)H(z)} \right)$$

Where  $G(z) = (K_{DCO}f_{ref}/\hat{K}_{DCO}) \cdot (\alpha + \rho/(z - 1))$  and  $H(z) = T_{ref} \cdot (1/(z - 1))$

Setting  $r = K_{DCO}/\hat{K}_{DCO}$  we get:

$$H_{cl}(z) = rf_{ref} \left( \frac{\alpha(z - 1) + \rho}{(z - 1)^2 + r\alpha(z - 1) + r\rho} \right)$$

**Stability Criteria** To ensure the stability of the closed loop response of the ADPLL, the poles must lie within the unit circle  $|z|=1$ . The characteristic equation is given by:

$$((z_p - 1) + 0.5r\alpha)^2 + r\rho - 0.25r^2\alpha^2 = 0$$

$$\therefore z_p = 1 - 0.5r\alpha \pm \sqrt{0.25r^2\alpha^2 - r\rho}$$

To ensure stability of the closed loop response:

$$|1 - 0.5r\alpha \pm \sqrt{0.25r^2\alpha^2 - r\rho}| < 1$$

Using this stability criteria we find that  $\alpha > \rho$ . To avoid saturation or overflow at the DCO input the phase error is scaled by factor  $\alpha$  with  $0 < \alpha < 1$ . Because  $\alpha$  and  $\rho$  are multiplied in the signal path, they are chosen to be powers of two to reduce hardware complexity and replace their multipliers with bit shifters.

For type I ADPLL  $\rho = 0$  the stability criteria reduces to:

$$|1 - r\alpha| < 1$$

$$\therefore 0 < \alpha < \frac{2}{r} \quad (1)$$

## Noise Transfer Functions

The main noise sources of the ADPLL are the quantization noise due to limited time resolution in TDC, the finite bit-width used to express the tune word for the DCO and oscillator noise in the DCO. By analyzing each type of noise in ADPLL, the effects of ADPLL design parameters on output phase noise and based on specifications put on in-band spurious emissions the design parameters are set.

### TDC Noise Transfer Functions

The TDC transfer function is given by:

$$f_v(z) = -\eta_{\text{TDC}} \left( \frac{\alpha(z-1) + \rho}{z-1} \right) f_{\text{ref}} \cdot r - f_v(z) \left( \frac{\alpha(z-1) + \rho}{(z-1)^2} \right) r$$

$$\therefore f_v(z) \left( \frac{(z-1)^2 + (\alpha(z-1) + \rho)r}{(z-1)^2} \right) = -\eta_{\text{TDC}} \left( \frac{\alpha(z-1) + \rho}{z-1} \right) f_{\text{ref}} \cdot r$$

$$\frac{f_v(z)}{\eta_{\text{TDC}}(z)} = -f_{\text{ref}} \cdot r \left( \frac{\alpha(z-1)^2 + \rho(z-1)}{(z-1)^2 + (\alpha(z-1) + \rho)r} \right)$$

The TDC noise transfer function in-terms of the variable phase  $\varphi_v(z)$  is given by:

$$\Phi_{\text{TDC}} = \frac{\varphi_v(z)z^{-1}}{\eta_{\text{TDC}}} = \frac{f_v(z)}{\eta_{\text{TDC}}(z)} \frac{T_{\text{ref}}}{z-1} = -r \left( \frac{\alpha(z-1) + \rho}{(z-1)^2 + (\alpha(z-1) + \rho)r} \right)$$

The TDC noise transfer function is low pass in nature and thus contribute to the in-band phase noise.

### DCO Noise Transfer Functions

The DCO transfer function is given by:

$$f_v(z) = \eta_{\text{DCO}} - f_v(z) \left( \frac{\alpha(z-1) + \rho}{(z-1)^2} \right) r$$

$$\therefore f_v(z) \left( \frac{(z-1)^2 + (\alpha(z-1) + \rho)r}{(z-1)^2} \right) = \eta_{\text{DCO}}$$

$$\frac{f_v(z)}{\eta_{\text{DCO}}(z)} = \frac{(z-1)^2}{(z-1)^2 + \alpha \cdot r(z-1) + \rho \cdot r}$$

The transfer function of DCO to the output phase of the DCO is given by:

$$\Phi_{\text{DCO}}(z) = \frac{\varphi_v(z)z^{-1}}{\eta_{\text{DCO}}(z)} = \frac{(z-1)T_{\text{ref}}}{(z-1)^2 + \alpha \cdot r(z-1) + \rho \cdot r}$$

The DCO noise transfer function has a double zero at  $\omega = 0$  so it has a high pass characteristic and thus contribute to the out of band and less to the in-band phase noise.

## s-Domain Approximation

To extract more characteristics of the closed loop response like the bandwidth and lock time the  $s$ -domain approximation of the ADPLL closed loop response is used. The approximation used is as follows:

$$z = e^{sT_{\text{ref}}} \approx 1 + sT_{\text{ref}}$$

This approximation holds when the sampling frequency  $f_{\text{ref}}$  is at least time times the band-width of the ADPLL.

The  $s$ -domain model of the ADPLL is shown in figure 10 below.

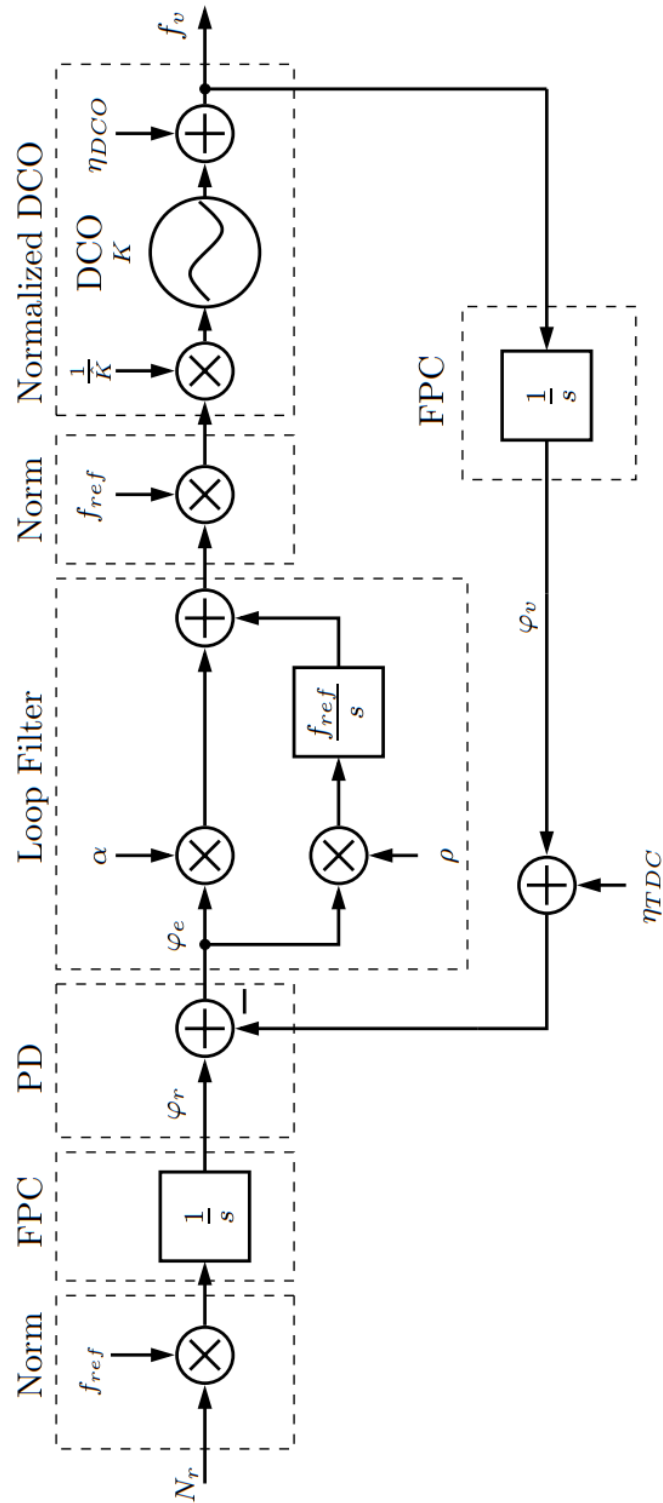


Figure 10:  $s$ -domain model of the ADPLL

Using the  $s$ -domain approximation the closed loop response of the ADPLL for the channel control word is given by:

$$H_{cl}(s) = \frac{f_v(s)}{N_r(s)} = r f_{ref} \left( \frac{\alpha s T_{ref} + \rho}{(s T_{ref})^2 + r \alpha s T_{ref} + r \rho} \right)$$

So the closed loop response is a second order system which can be rewritten in the form:

$$H_{cl}(s) = r f_{ref} \left( \frac{\alpha s f_{ref} + \rho (f_{ref})^2}{s^2 + r \alpha s f_{ref} + r \rho (f_{ref})^2} \right)$$

Thus the damping ratio  $\zeta$  and natural frequency  $\omega_n$  are given by:

$$\omega_n = f_{ref} \sqrt{r \rho}$$

$$\zeta = \frac{r \alpha f_{ref}}{2 \omega_n} = \frac{\alpha}{2} \sqrt{\frac{r}{\rho}}$$

Using the definition of the damping ratio and damping frequency the lock time which is defined as the 2% settling time is equal to:

$$T_s = \frac{4}{\omega_n \zeta} = \frac{8 T_{ref}}{r \alpha}$$

It can be seen that the lock time is dependent only the proportional coefficient  $\alpha$ . The bandwidth of the ADPLL is given by:

$$\omega_{BW} = f_{ref} \sqrt{\frac{-B + \sqrt{B^2 - 4C}}{2}}$$

where:

$$B = r^2 \alpha^2 - 2 r \rho - 2 \alpha^2$$

$$C = (r \rho)^2 - 2 \rho^2$$

The TDC noise transfer function is given by in the  $s$ -domain is given by:

$$\frac{f_v(s)}{\eta_{TDC}(s)} = -f_{ref} \cdot r \left( \frac{\alpha (s T_{ref})^2 + \rho s T_{ref}}{(s T_{ref})^2 + (\alpha s T_{ref} + \rho) r} \right)$$

$$\Phi_{TDC}(s) = -r \left( \frac{\alpha (s T_{ref}) + \rho}{(s T_{ref})^2 + (\alpha s T_{ref} + \rho) r} \right)$$

The DCO noise transfer function is given by:

$$\frac{f_v(s)}{\eta_{DCO}(s)} = -\frac{(s T_{ref})^2}{(s T_{ref})^2 + \alpha \cdot r s T_{ref} + \rho \cdot r}$$

$$\Phi_{DCO}(s) = \frac{s (T_{ref})^2}{(s T_{ref})^2 + \alpha \cdot r (s T_{ref}) + \rho \cdot r}$$

## ADPLL Noise Analysis

### Phase Noise

An oscillator ideally should concentrate all its power at a single frequency, its oscillating frequency  $\omega_c$ . In a practical oscillator, the spectrum spreads into nearby frequencies around  $\omega_c$  as is the following figure. This spreading is called phase noise which causes interference to adjacent channels. This is illustrated in Figure 11

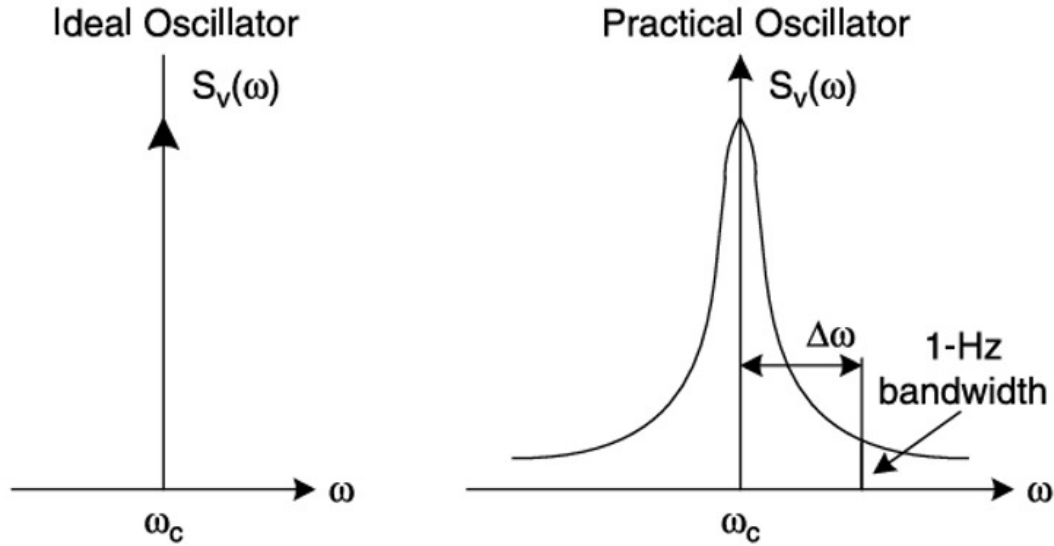


Figure 11: Illustration of phase noise

For an ideal oscillator operating at frequency  $\omega_c$ , the voltage output can be expressed as:

$$v(t) = A \cos(\omega_c t + \varphi)$$

its power spectrum is:

$$S_v(\omega) = \frac{A^2}{2} \delta(\omega - \omega_c)$$

This phase noise ( $\mathcal{L}(\Delta\omega)$ ) can be quantified by considering a 1-Hz unit bandwidth at an offset of  $\Delta\omega$  from the carrier, calculating the noise power in the band, and dividing this result by the carrier power gives the single-sided spectral noise density in units of decibel carrier per hertz (dBc/Hz). The phase noise equation is shown below:

$$\mathcal{L}(\Delta\omega) = 10 \log_{10} \left[ \frac{\text{noise power in a 1-Hz bandwidth at frequency } \omega_c + \Delta\omega}{\text{carrier power}} \right]$$

It is equal to half of the phase-noise spectrum, which contains both upper- and lower-frequency components as shown in the equation below.

$$\mathcal{L}(\Delta\omega) = 10 \log_{10} \left[ \frac{S_\phi(\Delta\omega)}{2} \right]$$

Where  $S_\phi(\Delta\omega)$  is given by:

$$S_\phi(\Delta\omega) = \frac{S_v(\Delta\omega)}{\text{carrier power}}$$

### Noise Sources in ADPLL

In the ADPLL due to its digital nature, there are only two places where phase noise could be injected at the DCO and TDC as indicated by  $\eta_{\text{DCO}}$  and  $\eta_{\text{TDC}}$  respectively in Figure 9. The rest of the system is completely immune from time or amplitude-domain perturbations. The main two noise sources are due to quantization errors in TDC and DCO.

#### TDC Resolution Effect on Phase Noise

TDC single side phase-noise spectrum is represented by the following equation:

$$\mathcal{L} = \frac{(2\pi)^2}{12} \left( \frac{\Delta t_{\text{res}}}{T_V} \right)^2 \frac{1}{f_R}$$

Where  $\Delta t_{\text{res}}$  is TDC resolution (buffer delay). TDC quantization noise can be modeled as white noise. The TDC noise transfer function in-terms of the variable phase  $\varphi_v(z)$ :

$$\Phi_{\text{TDC}} = \frac{\varphi_v(z)z^{-1}}{\eta_{\text{TDC}}} = \frac{f_v(z)}{\eta_{\text{TDC}}(z)} \frac{T_{\text{ref}}}{z-1} = -r \left( \frac{\alpha(z-1) + \rho}{(z-1)^2 + (\alpha(z-1) + \rho)r} \right)$$

As derived earlier in 27.

#### DCO Quantization Effect on Phase Noise

DCO has a frequency-resolution limit that creates noise, The DCO frequency quantization error is modeled as an additive uniformly distributed random variable  $\Delta f_{n,v}$  (frequency error) with white noise spectral characteristic. DCO quantization noise single-sided power spectral density at the output is: (by introducing the term  $1/\Delta f^2$  into the spectrum to turn the frequency noise to phase noise)

$$\mathcal{L}\{\Delta f\} = \frac{1}{12} \left( \frac{\Delta f_{\text{res}}}{\Delta f} \right)^2 \frac{1}{f_R}$$

Since DCO input samples are not impulse, we need to multiply by the sinc function.

$$\mathcal{L}\{\Delta f\} = \frac{1}{12} \left( \frac{\Delta f_{\text{res}}}{\Delta f} \right)^2 \frac{1}{f_R} \left( \text{sinc} \left( \frac{\Delta f}{f_R} \right) \right)^2$$

This equation will introduce large phase noise in addition to thermal noise, so without dithering, the resulting phase noise will normally be high for wireless applications.

#### DCO Dithering Effect on Phase Noise

The following equation models the phase noise contribution originating from finite DCO frequency resolution when  $\Delta\Sigma$  dithering is applied (noise-shaped DCO dithering):

$$\mathcal{L}\{\Delta f\} = \frac{1}{12} \left( \frac{\Delta f_{\text{res}}}{\Delta f} \right)^2 \frac{1}{f_{\text{dth}}} \left( 2 \sin \left( \frac{\pi \Delta f}{f_{\text{dth}}} \right) \right)^{2n}$$

The closed-loop transfer function of the DCO noise:

$$\frac{f_v(z)}{\eta_{\text{DCO}}(z)} = \frac{(z-1)^2}{(z-1)^2 + \alpha \cdot r(z-1) + \rho \cdot r}$$

### Total output phase noise

From the previous analysis, the total output ADPLL power spectral density is given by the following equation:

$$S_{\text{out}}(f) = \sum S_i(f) \cdot |H_i(z)|^2$$

Adjacent channel power is then calculated by integrating the resulting output PSD over the frequency range corresponding to the adjacent channel.

$$P_{\text{ACP}} = \int_{f_L}^{f_H} S_{\text{out}}(f) df$$

where  $f_L$  and  $f_H$  define the lower and upper limits of the adjacent channel bandwidth.

### MATLAB Simulation Results

Figure 12 presents a plot showing each phase noise of TDC, DCO and their total. This plot was obtained for  $\Delta t_{\text{res}} = 30$  ps,  $r = 1$ ,  $\alpha = 2^{-3}$ ,  $\rho = 2^{-13}$  and  $K_{\text{DCO}} = 25$  KHz

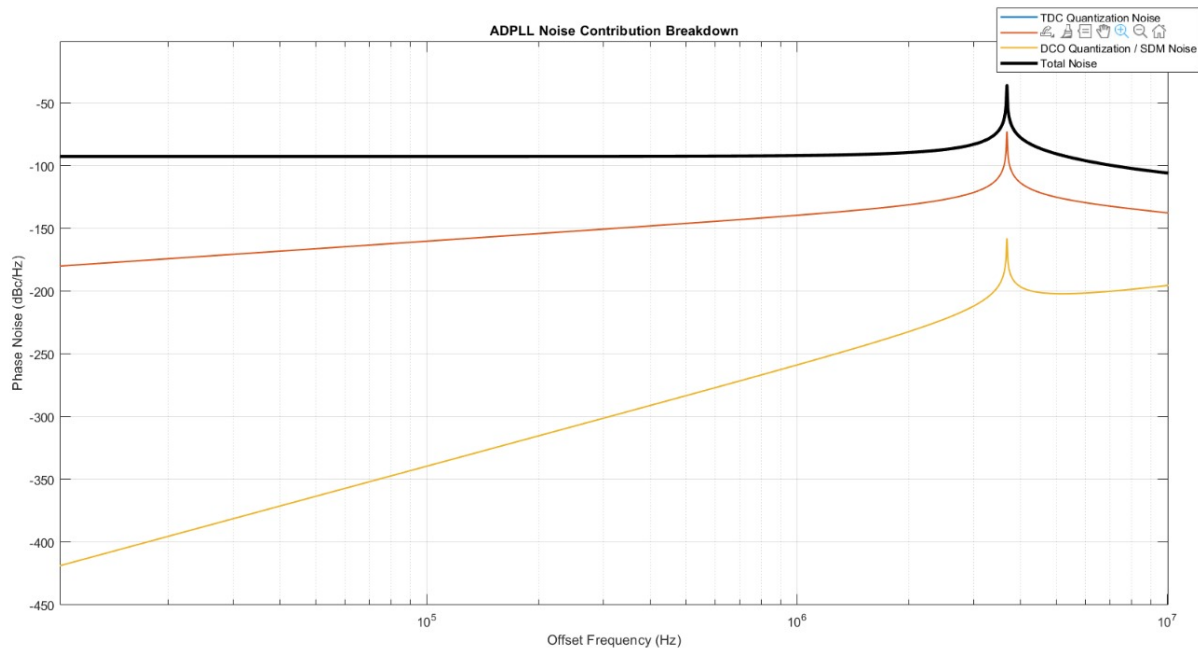


Figure 12: Phase noise plot

## Type II ADPLL

Applying stability condition check (discussed in 26) and adjacent channel power constraints on the loop filter coefficients. Assuming carrier power = 0dB, perfect DCO gain estimation ( $r = 1$ ) we get the design space shown in Figure 13

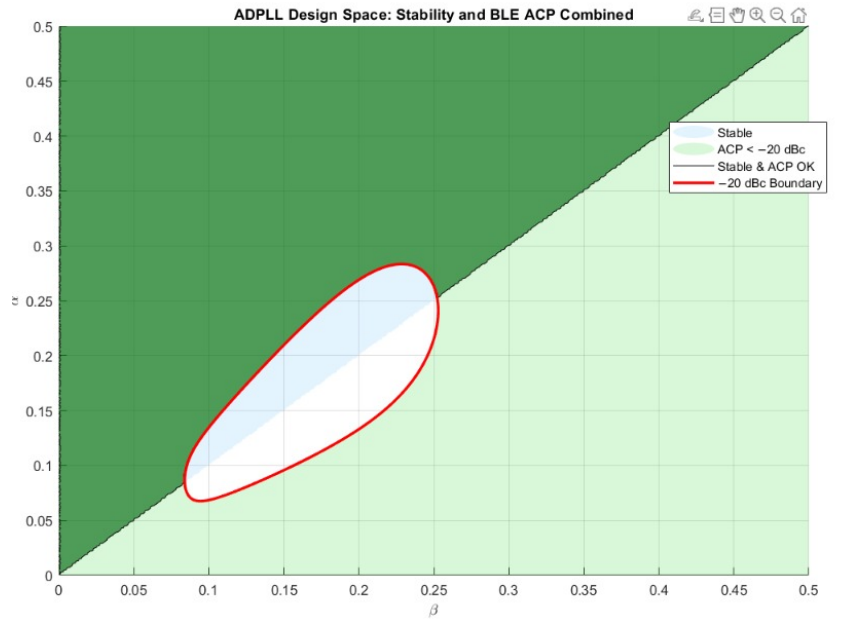


Figure 13: ADPLL design space: stability and BLE adjacent-channel power(ACP) combined

According to the results,  $\alpha$  and  $\beta$  were chosen to be  $2^{-6}$  and  $2^{-13}$  respectively. The values will give critically damped system with damping coefficient  $\zeta = 12$  and settling time  $t_s = 16 \mu s$ . ACP < -20 dBc, as well as bandwidth  $\approx 115$  KHz which is a good compromise for filtering TDC and DCO noise.

The total output phase noise from ADPLL at the chosen parameters is shown in Figure 14 :

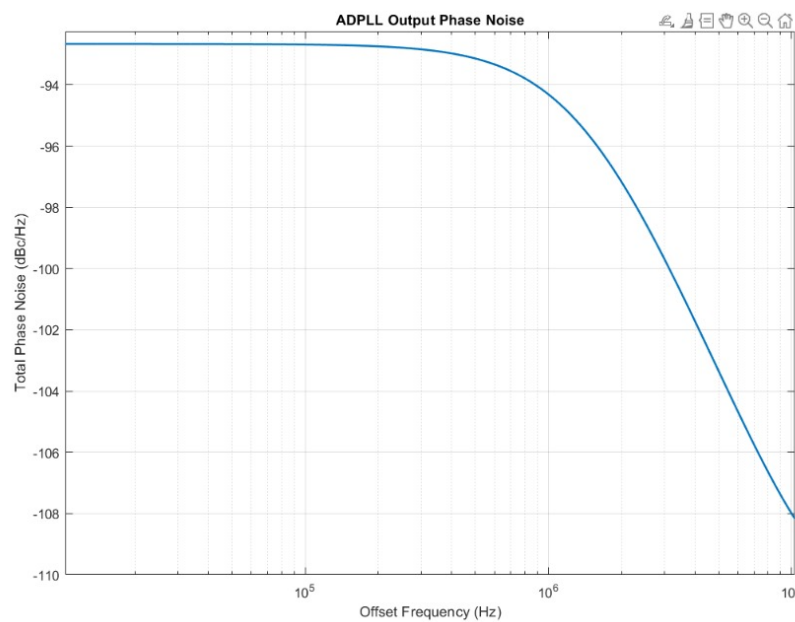


Figure 14: ADPLL Output Phase Noise

**Type II Closed loop response**

For DCO central frequency = 2.35 GHz, target frequency = 2.432 GHz, the closed loop response is shown in Figure 15.

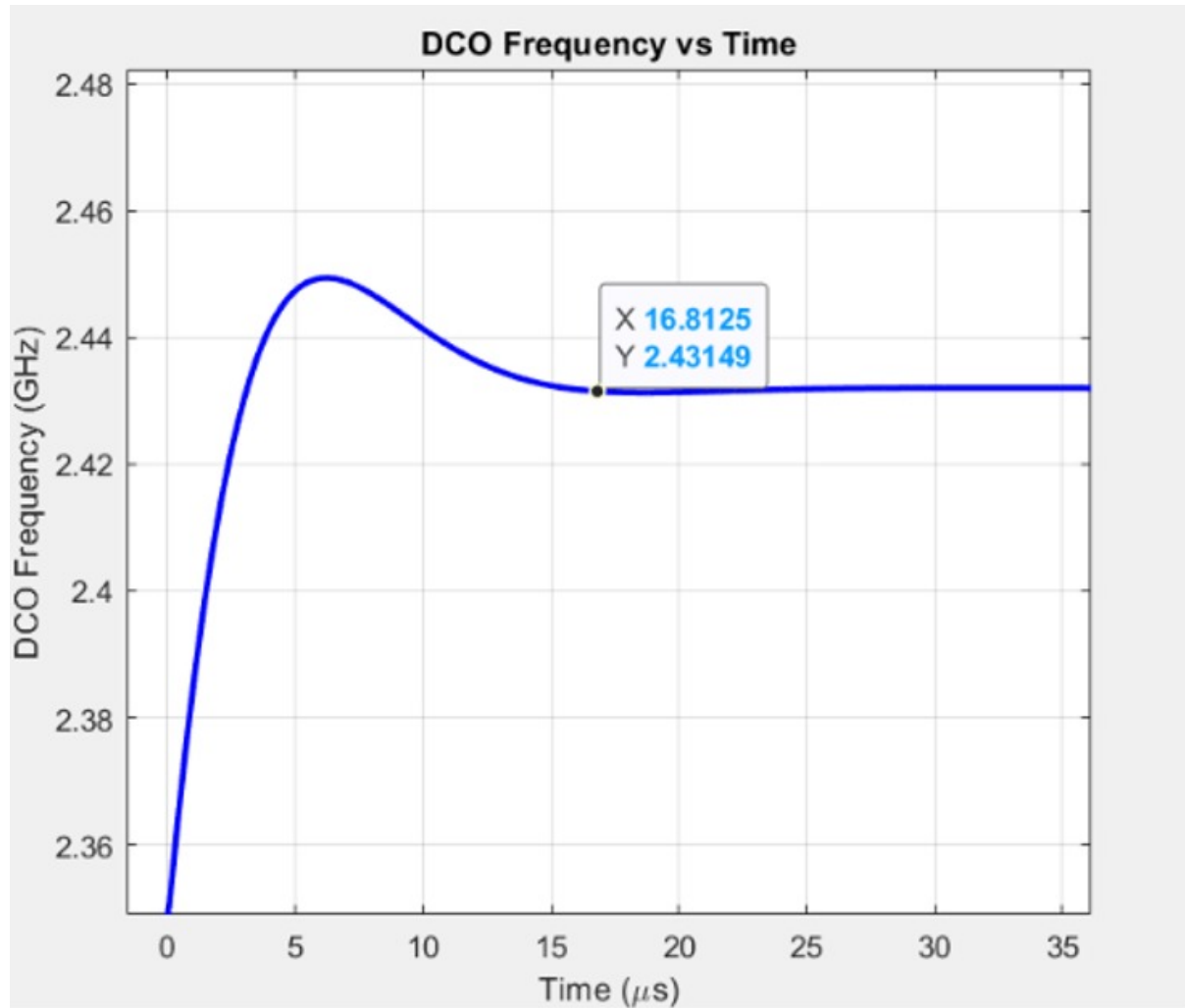


Figure 15: DCO Frequency vs Time

**Type I ADPLL**

For PVT calibration and acquisition modes. The frequency resolutions are larger of approximately 2 MHz and 0.5 MHz, respectively. The main objective is fast frequency convergence within stable system. Based on stability analysis, Type-I proportional gain coefficient was selected as  $\alpha = 2^{-3}$  results in a closed-loop bandwidth of approximately 636 kHz and a settling time of about  $1.15 \mu s$ . The bandwidth is sufficiently large to correct DCO frequency error while avoiding excessive noise transfer which may lead to loss of stability.

## Comparison of One-point vs Two Point Modulation

### 1 Point Modulation for type II ADPLL Analysis

The modulating data can either be inserted directly to the input of the DCO tune word or is combined with the channel frequency control word. Modulation at both points analyzed in detail below.

#### Modulation as Input to TDC

When carrying out one point modulation by applying the modulation data  $y[n]$  as an reference frequency control word to the input of the reference accumulator, it is combined with the channel frequency control word  $N_r$  as shown in figure 16.

The response of the modulating data to the output DCO frequency is obtained by setting the channel frequency control word  $N_r = 0$ ,  $\eta_{TDC} = 0$  and  $\eta_{DCO} = 0$  and using superposition to find:

$$H(z) = r f_{\text{ref}} \left( \frac{\alpha(z-1) + \rho}{(z-1)^2 + r\alpha(z-1) + r\rho} \right)$$

The magnitude response is plotted against frequency for  $\alpha, \rho$  values. The plot is shown below in Figure 17 for  $\alpha = 2^{-6}, \rho = 2^{-9}$  and  $\alpha = 2^{-8}, \rho = 2^{-15}$  respectively.

As shown in figure 17, the response resembles a low pass filter which indicates that high frequency components of the modulating data will be attenuated by the loop dynamics when the modulating data is input to the reference accumulator.

#### Modulation Directly at DCO Tune-Word Input

When applying the modulating data  $y[n]$  directly to the DCO input, it is combined with the loop filter digital word and used as an input to the DCO as shown in figure 18.

The response of the modulating data to the output DCO frequency is obtained by setting the channel frequency control word  $N_r = 0$ ,  $\eta_{TDC} = 0$  and  $\eta_{DCO} = 0$  and using superposition to find:

$$f_v(z) = r f_{\text{ref}} \left( y(z) - f_v(z) \left( \frac{T_{\text{ref}}}{z-1} \right) \left( \frac{\alpha z + (\rho - \alpha)}{z-1} \right) \right)$$

$$\therefore \frac{f_v(z)}{y(z)} = \frac{r f_{\text{ref}} (z-1)^2}{(z-1)^2 + r\alpha(z-1) + r\rho}$$

The transfer function has a double zero at  $z = 1$  so for  $z = e^{j\omega}$  the pole lies at  $\omega = 0$ . This makes the response act like a high pass filter and attenuates lower frequency components.

The magnitude response is plotted against frequency for  $\alpha, \rho$  values. The plot is shown below in Figure 19 for  $\alpha = 2^{-6}, \rho = 2^{-9}$  and  $\alpha = 2^{-8}, \rho = 2^{-15}$  respectively.

As shown in figure 19, the response resembles a high pass filter which indicates that low frequency components of the modulating data will be attenuated when the modulating data is input directly to the input of the DCO.

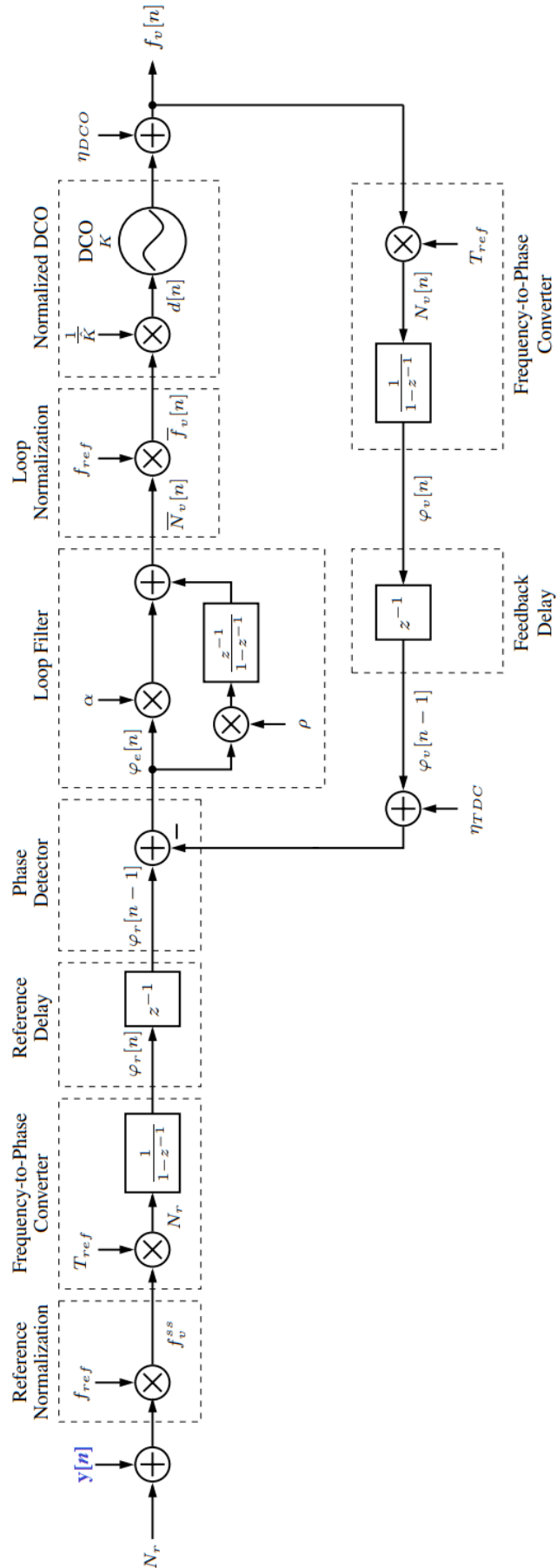


Figure 16: ADPLL with one point modulation at phase detector input

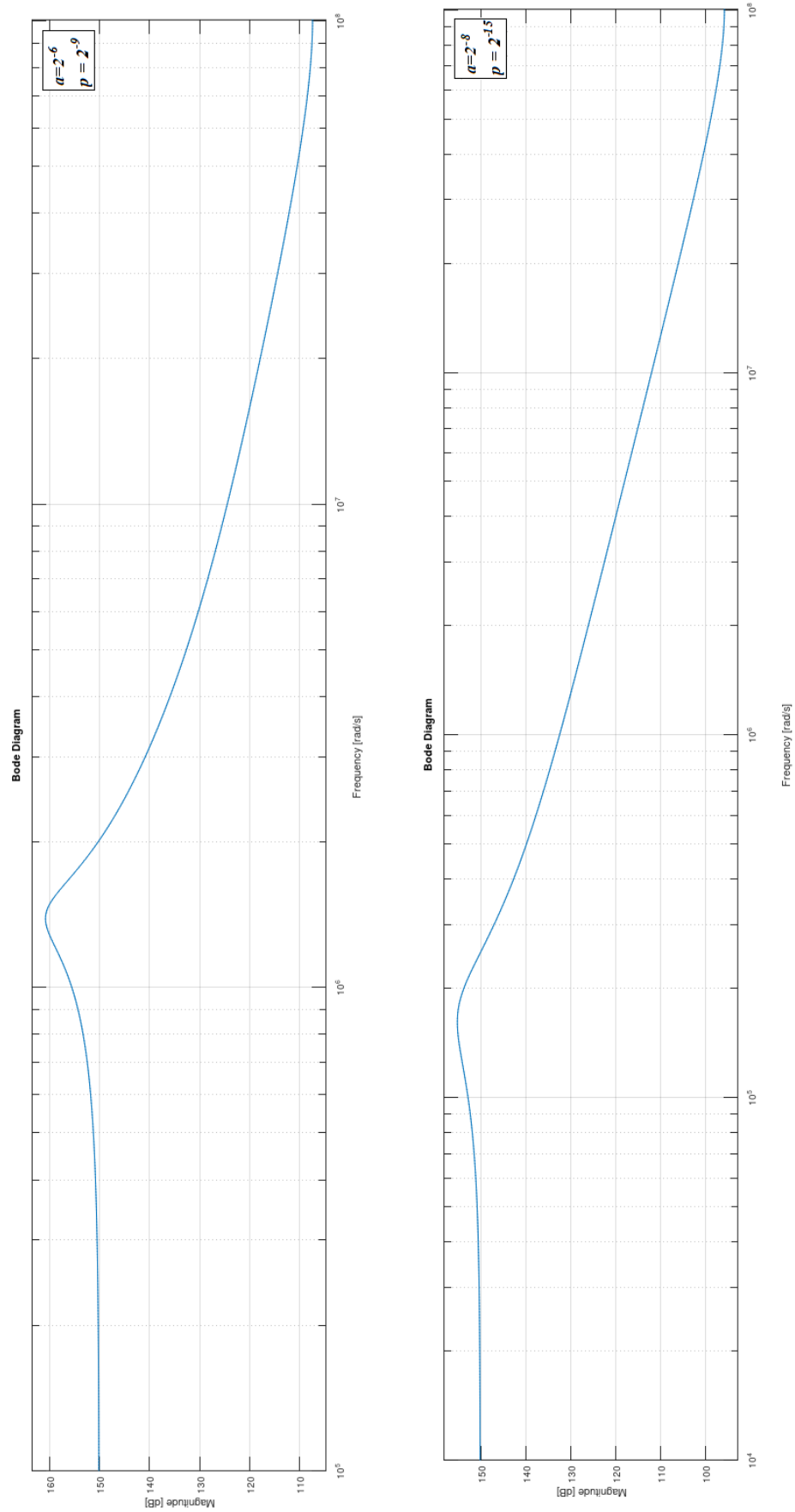
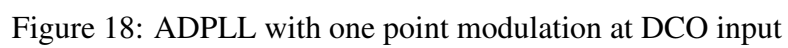


Figure 17: Magnitude response of ADPLL one point modulation at reference accumulator input



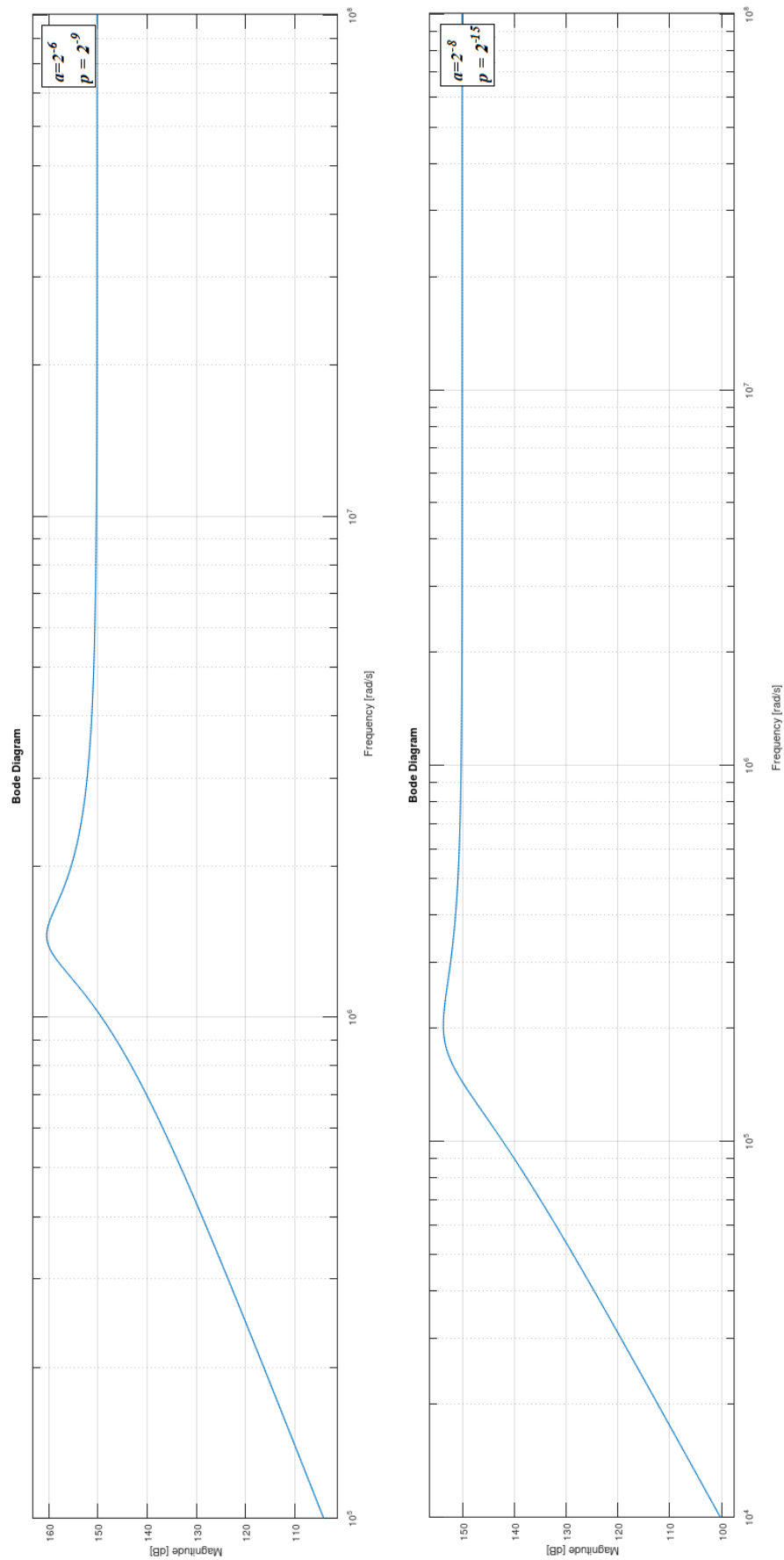


Figure 19: Magnitude response of ADPLL one point modulation at DCO input

## 2 Point Modulation Analysis

In two point modulation, the modulating data is applied as input to the TDC and also as direct input to the DCO. Two point modulation is illustrated in figure 20.

The transfer function of the modulating data in this case is a superposition of transfer function of one point modulation at DCO input and one point modulation at the input to the phase detector. Thus the transfer function is given by:

$$\frac{f_v(z)}{y(z)} = \frac{r f_{\text{ref}}(z-1)^2}{(z-1)^2 + r\alpha(z-1) + r\rho} + r f_{\text{ref}} \left( \frac{\alpha(z-1) + \rho}{(z-1)^2 + r\alpha(z-1) + r\rho} \right)$$

$$\therefore \frac{f_v(z)}{y(z)} = r f_{\text{ref}} \left( \frac{(z-1)^2 + \alpha(z-1) + \rho}{(z-1)^2 + r\alpha(z-1) + r\rho} \right)$$

For perfect gain estimation, where  $\hat{K}_{\text{DCO}} = K_{\text{DCO}}$  we have  $r = 1$ , thus the transfer function of the modulating data is given by:

$$\frac{f_v(z)}{y(z)} = f_{\text{ref}} \left( \frac{(z-1)^2 + \alpha(z-1) + \rho}{(z-1)^2 + \alpha(z-1) + \rho} \right) = f_{\text{ref}}$$

Which implies that when the DCO gain is fully normalized, two-point modulation reduces to a constant-factor transfer function, so the output frequency deviation is directly proportional to the modulating signal.

If the gain is overestimated meaning  $\hat{K}_{\text{DCO}} > K_{\text{DCO}}$  meaning  $r < 1$  we get the magnitude response shown in figure 21.

As shown in figure 21, the gain overestimation causes the modulation response to split around a certain frequency where for  $r = 0.5$  the response is split around frequency  $f \approx 1\text{MHz}$  where for frequency less than 1MHz the magnitude of the response is 150dB, near 1 MHz a resonant peak appears and the response then falls, settling around 144dB for higher frequencies. For  $r = 0.1$  the response is also split around 1MHz, however, peaking magnitudes and the high-frequency drop becomes more significant: the low-frequency level remains 150dB but the higher frequencies settle at 130dB, producing a substantially larger split.

If the gain is underestimated meaning  $r > 1$  we get the magnitude response shown in figure 22. As shown in figure 22, gain underestimation produces a response that is essentially the mirror image of the gain-overestimation case. The modulation response again splits around frequency  $f \approx 1.5\text{MHz}$ , but with inverted behavior: the low-frequency magnitude decreases slightly as the frequency approaches 1.5MHz, followed by a resonant peak at approximately 1.5MHz. Beyond this magnitude peak, the response settles to a higher plateau where, the magnitude rises from roughly 142dB at low frequencies to about 144dB at high frequencies for  $r = 1.5$  and from 150dB to approximately 156dB for  $r = 1.9$ .

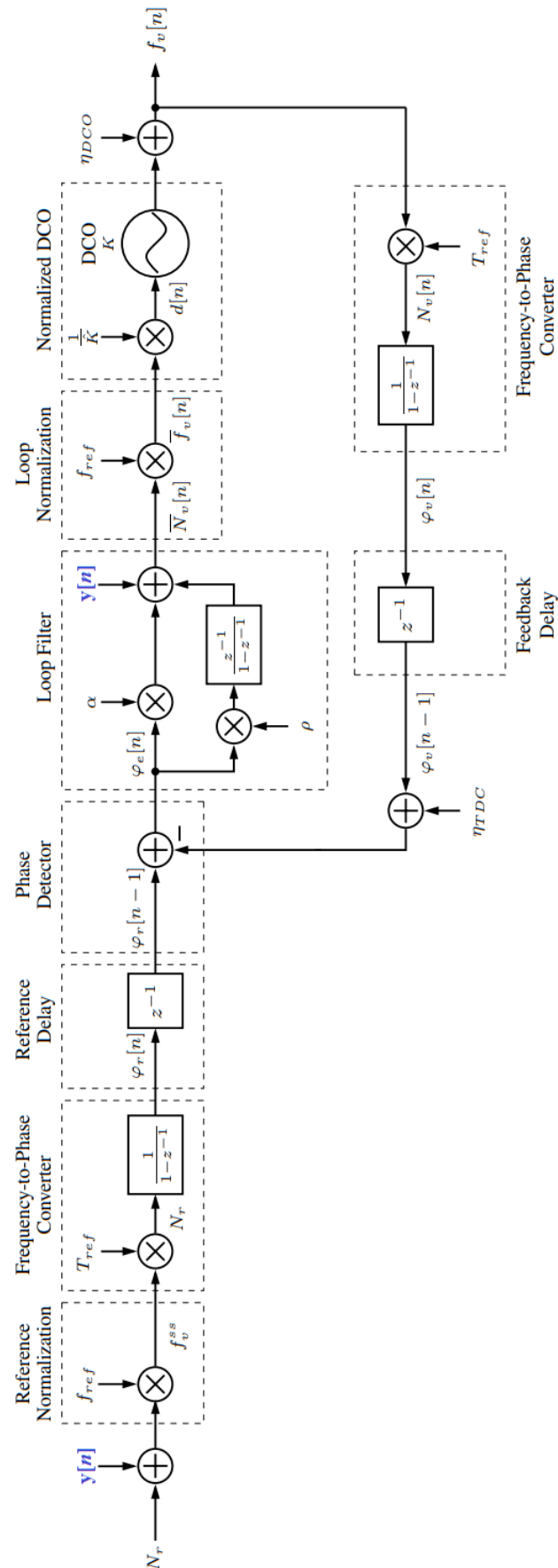


Figure 20: ADPLL with two point modulation

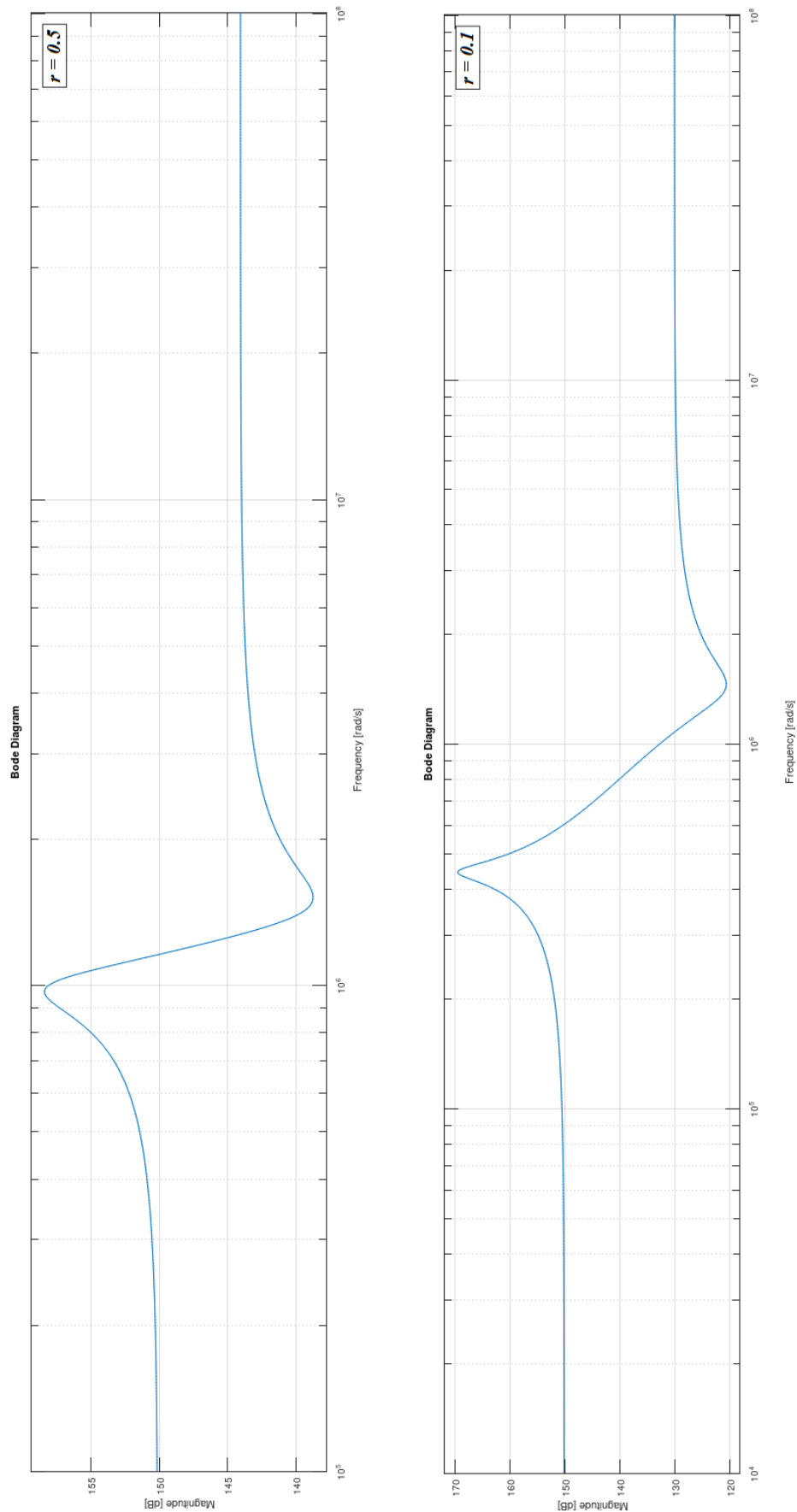


Figure 21: Magnitude response of ADPLL 2 point modulation with gain overestimation ( $\hat{K}_{DCO} = 2K_{DCO}$  and  $\hat{K}_{DCO} = 10K_{DCO}$  respectively)

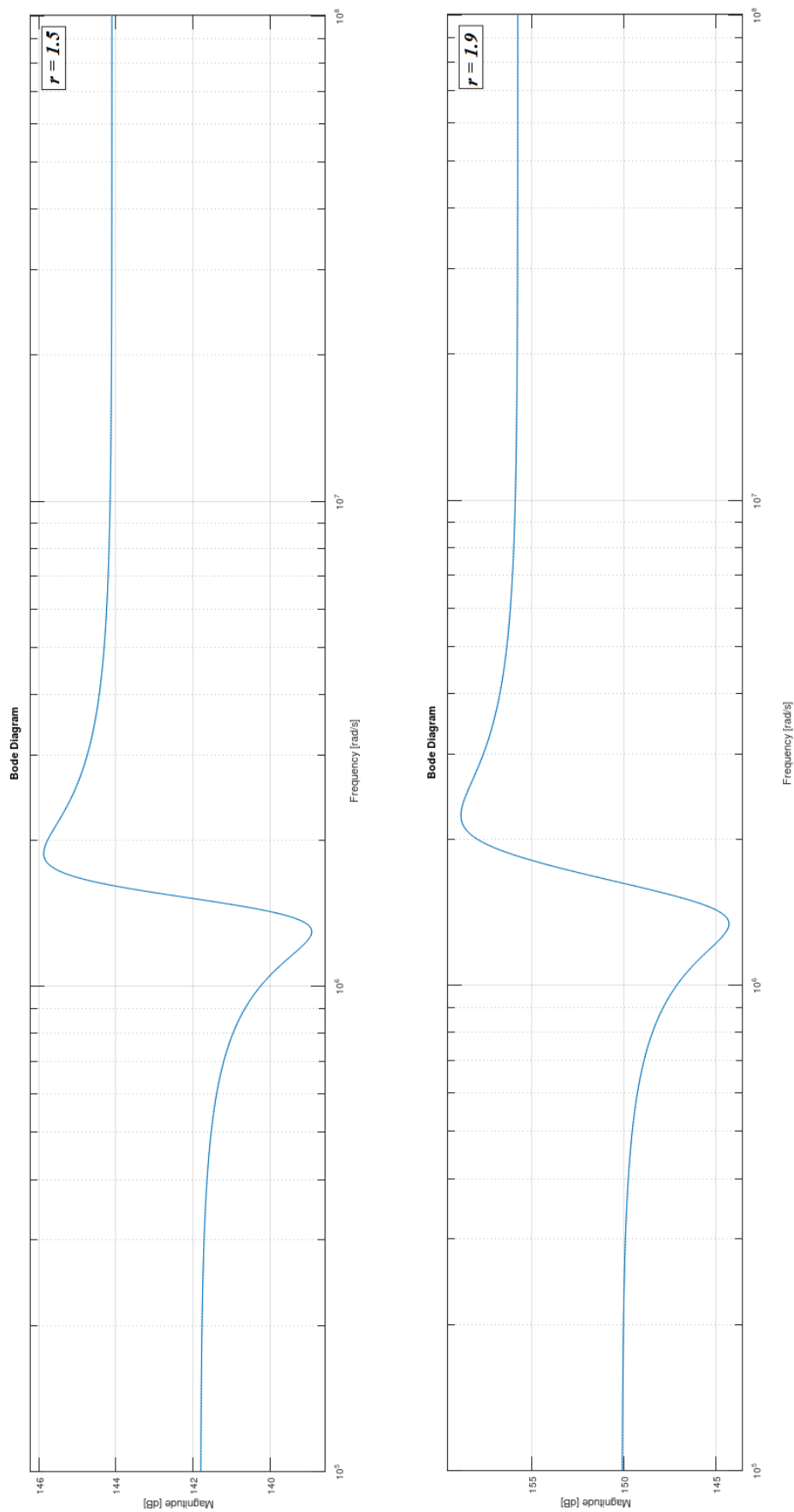


Figure 22: Magnitude response of ADPLL 23 point modulation with gain underestimation ( $1.5\hat{K}_{DCO} = K_{DCO}$  and  $1.9\hat{K}_{DCO} = K_{DCO}$  respectively)

## Gaussian Filter

For the digital implementation, the Gaussian pulse-shaping response is realized as an FIR approximation. Two parameters govern the discrete-time implementation quality and complexity: the filter **span** (in symbols) and the **samples per symbol (SPS)**. Together, they determine the tap length and time resolution of the pulse-shaping waveform. This thesis later evaluates span and SPS as design parameters because they directly impact both spectral performance and hardware complexity.

## FIR Filter Fundamentals and Linear-Phase Properties

A finite impulse response (FIR) filter produces an output sequence  $y[n]$  by a weighted sum of current and past input samples:

$$y[n] = \sum_{k=0}^{N-1} h[k]x[n-k]$$

where  $h[k]$  is the impulse response (tap coefficients) and  $N$  is the number of taps. FIR filters are widely used in digital communication transmitters because they can be designed with an exactly linear-phase response, ensuring predictable time-domain behavior and avoiding phase distortion of the transmitted waveform. For an FIR filter with symmetric coefficients, the filter exhibits linear phase and a constant group delay. For an odd-length filter, the group delay is:

$$\tau_g = \frac{N-1}{2}$$

This group delay is important in pulse-shaping applications because it determines the time alignment between the input bitstream and the shaped output waveform, and it must be accounted for in simulation and verification.

## FIR Filter Implementations

**Direct Form FIR** The direct-form realization implements the FIR convolution by first creating delayed versions of the input and then weighting each delayed sample by its corresponding coefficient. A tap-delay line generates  $x[n], x[n-1], \dots, x[n-(N-1)]$ . Each tap is multiplied by  $b_k$ , and the products are summed (typically through an adder chain) to produce:

$$y[n] = \sum_{k=0}^{N-1} b_k x[n-k]. \quad (2)$$

This structure closely matches the mathematical FIR definition, is straightforward to understand, and is commonly used as a baseline implementation. Its hardware cost is dominated by the delay-line registers,  $N$  multipliers, and the accumulation network.

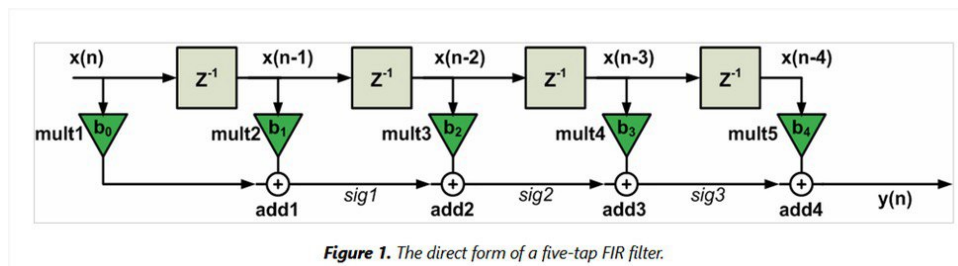


Figure 23: Direct-form FIR filter realization

**Transposed-form FIR** The transposed-form realization is obtained using the transposition (flow-graph reversal) theorem. In this structure, the input sample  $x[n]$  is applied simultaneously to all multiplier blocks (input broadcast). The multiplier outputs are then accumulated through a cascade of adders with registers placed between stages, such that intermediate partial sums are stored and forwarded until the final output  $y[n]$  is produced. Although the coefficient order appears reversed compared to the direct form, the realized transfer function is identical, and the same FIR relationship holds:

$$y[n] = \sum_{k=0}^{N-1} b_k x[n - k]. \quad (3)$$

Because registers naturally lie on the accumulation path, the transposed form more readily supports high-throughput operation than the basic direct-form adder chain.

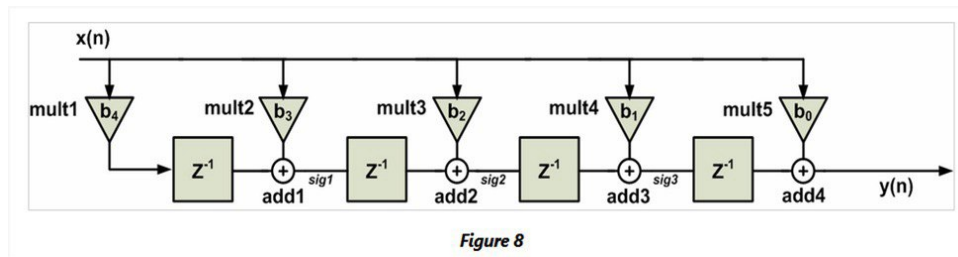


Figure 24: Transposed-form FIR filter realization

**Difference between Direct and Transposed Forms:** Both architectures implement the same FIR filter, but they differ in how delays and arithmetic operations are distributed. In the direct form, delays are placed on the input path: each input sample is shifted through a delay line and then multiplied by the filter coefficients before being summed. Consequently, a single input sample reaches different multipliers at different time instants, and the output is obtained by accumulating all weighted samples.

In contrast, the transposed form relocates the delays to the accumulation path. The current input sample is multiplied by all coefficients simultaneously, and the resulting products are accumulated through a cascade of adders and registers. This structure requires broadcasting the input to all multipliers, which increases input fanout and routing activity as the number of taps grows.

From an implementation perspective, the direct form is often preferred for its routing simplicity and controlled fanout, whereas the transposed form is attractive for high-throughput designs due to its inherently pipelined nature. In this thesis, the direct-form structure is adopted as the starting point and is subsequently optimized using coefficient symmetry to reduce arithmetic complexity and enable a multiplier-free realization.

### Symmetry in FIR Filters and Computational Reduction

This section reviews how coefficient symmetry in FIR filters can be exploited to reduce arithmetic complexity and hardware cost. The impulse response of many practical FIR filters, including Gaussian pulse-shaping FIR approximations, exhibits significant symmetry. When the impulse response is even-symmetric (linear phase), the filter coefficients satisfy:

$$h[k] = h[N - 1 - k]$$

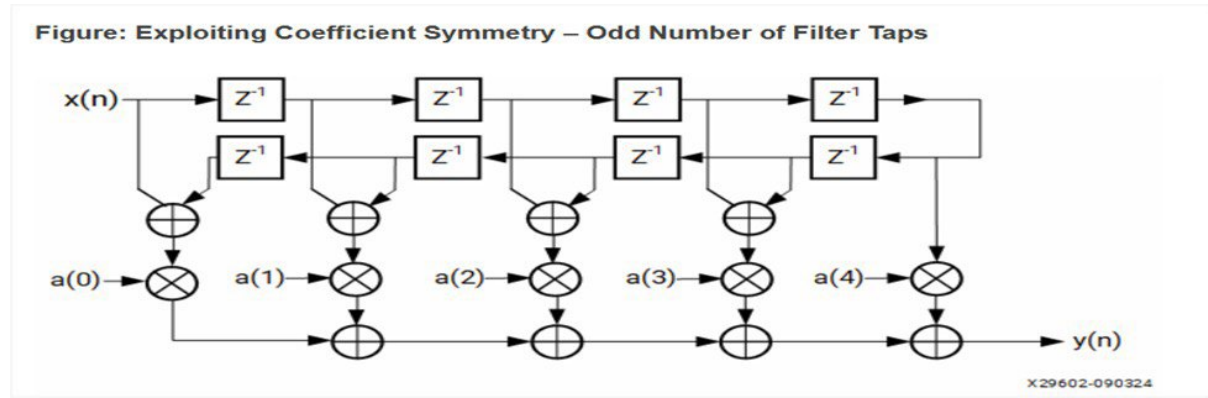


Figure 25: Exploiting coefficient symmetry for an odd number of FIR filter taps

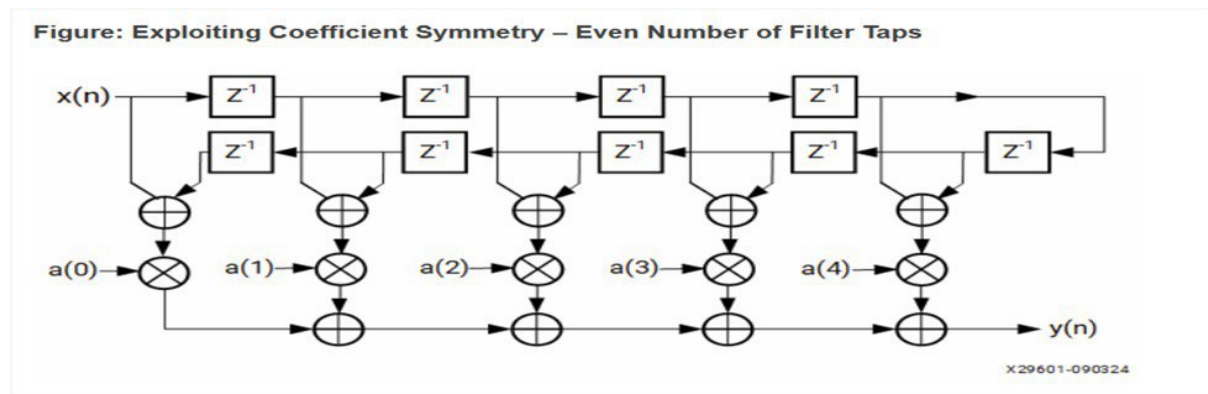


Figure 26: Exploiting coefficient symmetry for an even number of FIR filter taps

A conventional  $N$ -tap FIR filter computes:

$$y[n] = \sum_{k=0}^{N-1} h[k] x[n - k]$$

which requires  $N$  multiplications and  $(N - 1)$  additions in the straightforward direct-form implementation. By exploiting coefficient symmetry, tap pairs that share the same coefficient can be grouped together using a pre-adder. For an odd-length FIR filter, which is common in practice, the output expression can be rewritten as:

$$y[n] = \sum_{k=0}^{\frac{N-3}{2}} h[k] (x[n - k] + x[n - (N - 1 - k)]) + h\left(\frac{N-1}{2}\right) x\left[n - \frac{N-1}{2}\right]$$

In this symmetric form, two multiplications associated with each symmetric tap pair are replaced by a single multiplication, preceded by a pre-adder that computes  $(x[n - k] + x[n - (N - 1 - k)])$ . As a result, the number of multipliers is reduced from  $N$  to approximately  $\lceil N/2 \rceil$ , while the number of additions increases modestly due to the additional pre-add operations. Overall, this approach achieves a significant reduction in computational workload and typically leads to more area- and power-efficient hardware realizations.

This symmetry-based reduction provides the baseline architecture used in the next chapter. Chapter 3 builds on this structure by exploiting NRZ input properties to further eliminate the remaining multiplications through sign-select logic (multiplexers) and add/subtract operations.

## Summary of FIR Realizations and Selection Criteria

Table 2: FIR Implementation Comparison

| FIR Form                                   | Multipliers          | Adders                       | Registers         | Critical Path                       | Fanout       | Latency         |
|--------------------------------------------|----------------------|------------------------------|-------------------|-------------------------------------|--------------|-----------------|
| Direct Form                                | High ( $\approx N$ ) | Moderate ( $\approx N - 1$ ) | Low               | Long (adder chain + multiplier)     | Low-Moderate | Moderate        |
| Transposed Form                            | High ( $\approx N$ ) | Moderate                     | Often lower       | Short; naturally pipelined          | High         | Moderate-Higher |
| Symmetric (Linear Phase)                   | $\approx N/2$        | Higher (pre-adders)          | Similar to direct | Depends on adder organization       | Moderate     | Same            |
| Symmetric + Multiplier-Free (NRZ $\pm 1$ ) | Zero                 | Higher (pre-add + add/sub)   | Similar + control | Often shorter than multiplier-based | Moderate     | Same            |

### Proposed Architecture (Multiplier-Free FIR Architecture)

The objective of the proposed design is to implement a BLE Gaussian pulse-shaping FIR filter with reduced hardware complexity, targeting lower area and power consumption while preserving the intended pulse-shaping behavior. The architecture assumes fixed Gaussian coefficients determined by the selected bandwidth–time (BT) product, filter span, and samples per symbol (SPS). Consequently, the coefficients can be stored in hardware using register file.

The filter is designed to exhibit linear-phase behavior with even symmetry, enabling symmetric-pair computation. The input stream is modeled as an oversampled non-return-to-zero (NRZ) sequence with discrete amplitude levels:

$$x[n] \in \{-1, +1\}$$

This assumption is consistent with typical digital baseband processing, where binary symbols are mapped to  $\pm 1$  prior to pulse shaping.

### Baseline FIR and Symmetric Pair Reduction

A conventional  $N$ -tap FIR filter is defined by:

$$y[n] = \sum_{k=0}^{N-1} h[k] x[n - k]$$

For linear-phase filters with even symmetry, the coefficients satisfy:

$$h[k] = h[N - 1 - k]$$

Under this condition, the computation can be reduced by pairing terms that share the same coefficient. This symmetry-based formulation forms the baseline architecture upon which the proposed multiplier-free implementation is developed.

### Odd-Length Symmetric FIR (Type-I Linear Phase)

When the number of taps is odd ( $N$  odd), the symmetry center lies on a sample and a center coefficient exists. In this case, the FIR output expression can be rewritten as:

$$y[n] = \sum_{k=0}^{\frac{N-3}{2}} h[k] (x[n - k] + x[n - (N - 1 - k)]) + h\left(\frac{N-1}{2}\right) x\left[n - \frac{N-1}{2}\right]$$

Define the symmetric pre-adder output as:

$$s_k[n] = x[n - k] + x[n - (N - 1 - k)]$$

This formulation reduces each symmetric coefficient pair to a single coefficient operation driven by  $s_k[n]$ , in addition to a center-tap contribution. This structure significantly reduces the number of arithmetic operations and serves as the foundation for the proposed multiplier-free architecture.

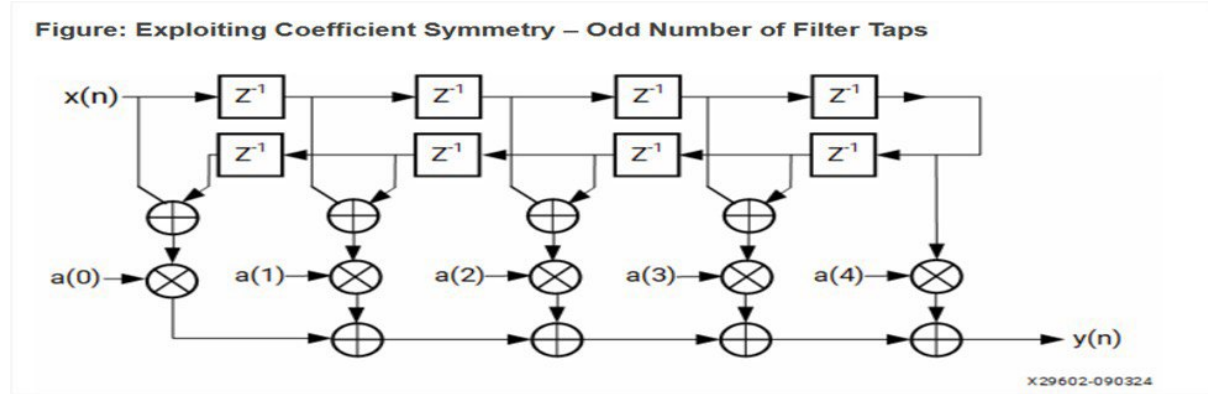


Figure 27: Odd-length symmetric FIR filter (Type-I linear phase) showing symmetric tap pairing and center coefficient

### Even-Length Symmetric FIR (Type-II Linear Phase)

When the number of taps is even ( $N$  even), the symmetry center lies between samples and no center tap exists. In this case, all coefficients are handled as symmetric pairs, and the FIR output can be expressed as:

$$y[n] = \sum_{k=0}^{\frac{N}{2}-1} h[k](x[n-k] + x[n-(N-1-k)])$$

The resulting architecture therefore consists solely of repeated symmetric-pair processing blocks followed by accumulation, without a dedicated center-tap path. This structure further simplifies the hardware implementation compared to the odd-length case and is well suited for multiplier-free realizations based on symmetric pre-adder logic.

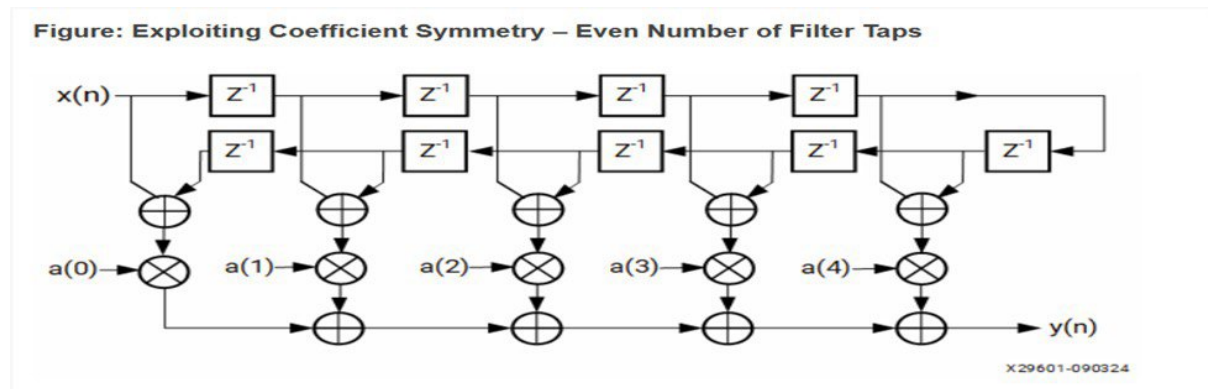


Figure 28: Even-length symmetric FIR filter (Type-II linear phase) showing symmetric tap pairing without a center coefficient

**Implementation note:** Both odd- and even-length symmetric FIR realizations are valid for Gaussian pulse shaping. Many coefficient-generation methods (e.g., common Gaussian design

utilities) produce an odd-length impulse response (Type-I), while even-length filters (Type-II) require half-sample centered coefficients. The remainder of this chapter presents multiplier-free optimization for both cases.

### Multiplier Elimination Using NRZ Input Properties

The key observation enabling multiplier-free implementation is the discrete nature of the NRZ input. For either odd- or even-length symmetric FIR, each symmetric pair uses the pre-adder output:

$$s_k[n] = x[n - k] + x[n - (N - 1 - k)]$$

Since  $x[n] \in \{-1, +1\}$ , the pre-adder output is restricted to three values:

$$s_k[n] \in \{-2, 0, +2\}$$

Therefore, the symmetric pair product becomes:

$$h[k] \cdot s_k[n] \in \{-2h[k], 0, +2h[k]\}$$

This replaces a general multiplication with a three-level selection controlled by  $s_k[n]$ , enabling an implementation using only shifts, sign selection, and addition/subtraction.

### Sign-Select Micro-Architecture/MUX for One Symmetric Pair

For each symmetric coefficient  $h[k]$ , the pair contribution can be generated by producing  $\pm 2h[k]$  and selecting the appropriate value based on  $s_k[n]$ . In hardware, the ternary control value  $s_k[n]$  is converted into a binary select signal using a small decoder:

$$\text{decode}(s_k) \rightarrow \text{sel}[1 : 0]$$

A left shift implements scaling by two:

$$+2h[k] \equiv h[k] \ll 1$$

and two's complement provides the negative value  $-2h[k]$ . A 3:1 multiplexer then selects:

$$\text{term}_k[n] \in \{+2h[k], 0, -2h[k]\}$$

The selected term is accumulated into the running sum, producing the final filter output after all pairs are processed.

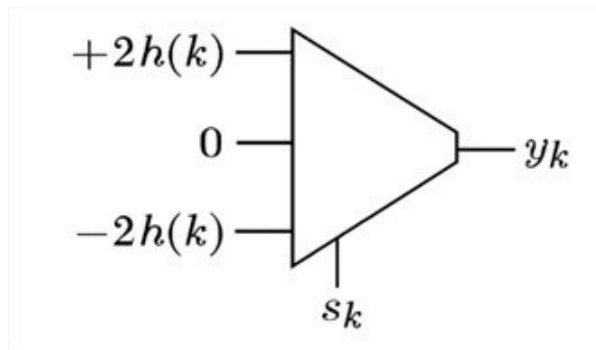


Figure 29: 3:1 multiplexer for selecting  $\{+2h(k), 0, -2h(k)\}$  based on  $s_k$

### Center Tap Handling for Odd-Length FIR

In the odd-length (Type-I) case, the center coefficient contributes:

$$h\left(\frac{N-1}{2}\right) x\left[n - \frac{N-1}{2}\right]$$

Since  $x[\cdot] \in \{-1, +1\}$ , this operation does not require a general multiplier:

$$h_{\text{mid}} \cdot x_{\text{mid}} \in \{-h_{\text{mid}}, +h_{\text{mid}}\}$$

Accordingly, the center tap is implemented using a simple sign select (2:1 selection between  $+h_{\text{mid}}$  and  $-h_{\text{mid}}$ ) followed by accumulation. The even-length (Type-II) case omits this path entirely.

### Complete Multiplier-Free Gaussian FIR Architecture

The complete architecture is composed of the following blocks:

- **Delay line (shift register):** stores the most recent  $N$  input samples, updated once per sample clock.
- **Symmetric pre-adders:** compute  $s_k[n] = x[n - k] + x[n - (N - 1 - k)]$  for each symmetric pair.
- **Coefficient storage:** stores Gaussian coefficients  $h[k]$  (or a subset due to symmetry) in ROM/LUT.
- **Sign-select pair blocks / MUX:** for each  $k$ , generate  $\{+2h[k], 0, -2h[k]\}$  using shift/negate and select using  $\text{decode}(s_k) \rightarrow \text{sel}[1:0]$ .
- **Accumulator:** sums all selected pair contributions; in the odd-length case, adds the center-tap sign-selected term.
- **Output register:** registers the final accumulated sum to produce  $y[n]$ .

Figure: Exploiting Coefficient Symmetry – Even Number of Filter Taps

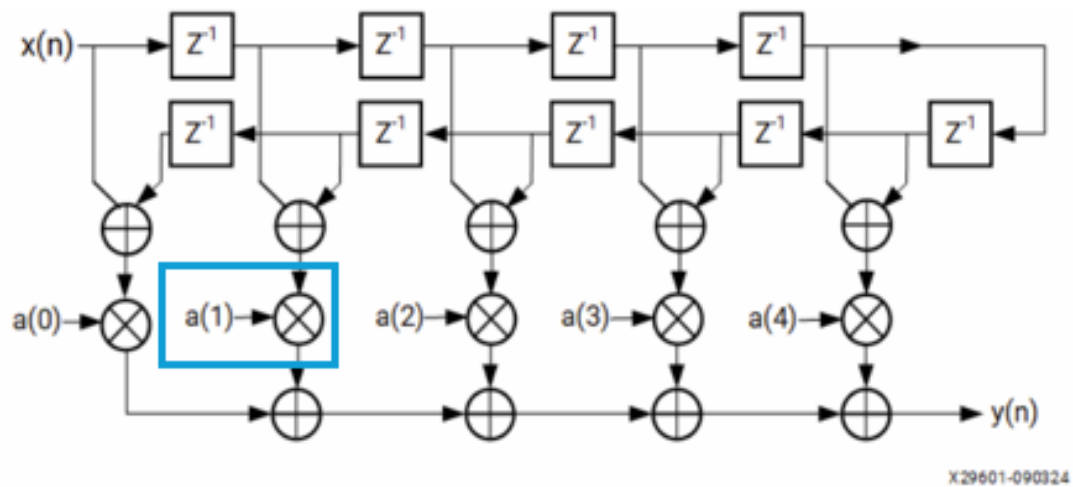
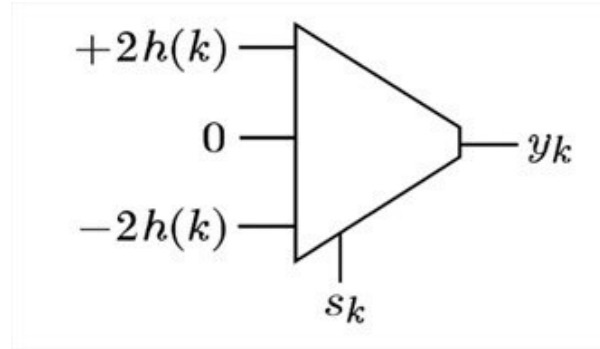


Figure 30: Modified even-length symmetric FIR structure exploiting coefficient symmetry



## Gaussian Filter Parameterization and Design Tradeoffs

### Gaussian Pulse Shaping and the BT Product

Gaussian pulse shaping is used in BLE transmitters to smooth NRZ symbol transitions before frequency modulation, reducing spectral sidelobes and improving spectral containment. In Gaussian-filtered FSK (GFSK), the amount of smoothing is commonly specified by the bandwidth–time product:

$$BT = B \cdot T$$

where  $B$  is the Gaussian filter's 3-dB bandwidth and  $T$  is the symbol period. A smaller  $BT$  corresponds to stronger smoothing (narrower spectrum but potentially more intersymbol influence), while a larger  $BT$  produces weaker smoothing (wider spectrum and sharper transitions). BLE specifies  $BT = 0.5$  as a standardized tradeoff between spectral containment and implementability.

## Discrete-Time Parameters: Span and Samples-Per-Symbol (SPS)

### Samples per Symbol (SPS)

SPS is the oversampling factor, defined as the number of discrete-time samples representing one symbol period. For a symbol rate  $R_s$ , the sampling rate is:

$$f_s = \text{SPS} \cdot R_s$$

Increasing SPS improves time resolution and yields a smoother discrete-time representation of the Gaussian pulse, but increases filter length and implementation cost.

### Span

Span specifies the duration of the impulse response in units of symbols, indicating how many symbol intervals the Gaussian response covers. Increasing the span captures more of the Gaussian tails, reducing truncation effects and improving approximation quality, at the cost of more taps and increased group delay.

### Odd- vs. Even-Length Gaussian FIR Realizations

Gaussian pulse shaping can be realized using either an odd-length (Type-I) or even-length (Type-II) linear-phase FIR filter. Both structures benefit from coefficient symmetry, but they differ in whether a center tap exists and where the symmetry center lies.

### Odd-Length (Type-I) Realization — Center Tap Present

In many design flows (e.g., common coefficient generators), the FIR length is odd:

$$N_{\text{odd}} = \text{span} \cdot \text{SPS} + 1$$

This structure has a symmetry center exactly on a sample and includes a center coefficient. The group delay is:

$$\tau_g = \frac{N_{\text{odd}} - 1}{2}$$

In the multiplier-free symmetric implementation, the center tap becomes a simple sign-select term ( $\pm h_{\text{mid}}$ ) because the NRZ input is  $\pm 1$ .

### Even-Length (Type-II) Realization — No Center Tap

An alternative realization uses an even-length FIR filter:

$$N_{\text{even}} = \text{span} \cdot \text{SPS}$$

This structure has its symmetry center between samples; therefore, no center tap exists, and all coefficients are processed as symmetric pairs. The group delay is:

$$\tau_g = \frac{N_{\text{even}} - 1}{2}$$

This corresponds to an effective half-sample-centered symmetry in interpretation. In practice, this can appear as half-sample timing behavior in time-domain inspection; however, verification remains straightforward when the RTL and MATLAB reference models use the same coefficient set and tap length.

### Span/SPS Sweep Interpretation and Parameter Selection

To guide design selection, span and SPS were swept across representative values (span = 3 vs. 4, SPS = 8 vs. 16). The sweep results exhibit consistent trends in both the frequency and time domains:

- Increasing span (3  $\rightarrow$  4) extends the impulse-response support, capturing more of the Gaussian tails and reducing truncation effects. This generally improves spectral containment and yields a closer approximation to the intended Gaussian shaping.
- Increasing SPS (8  $\rightarrow$  16) increases time resolution, producing a smoother discrete-time waveform representation and improved numerical fidelity of the Gaussian pulse shape, at the cost of a higher tap count.

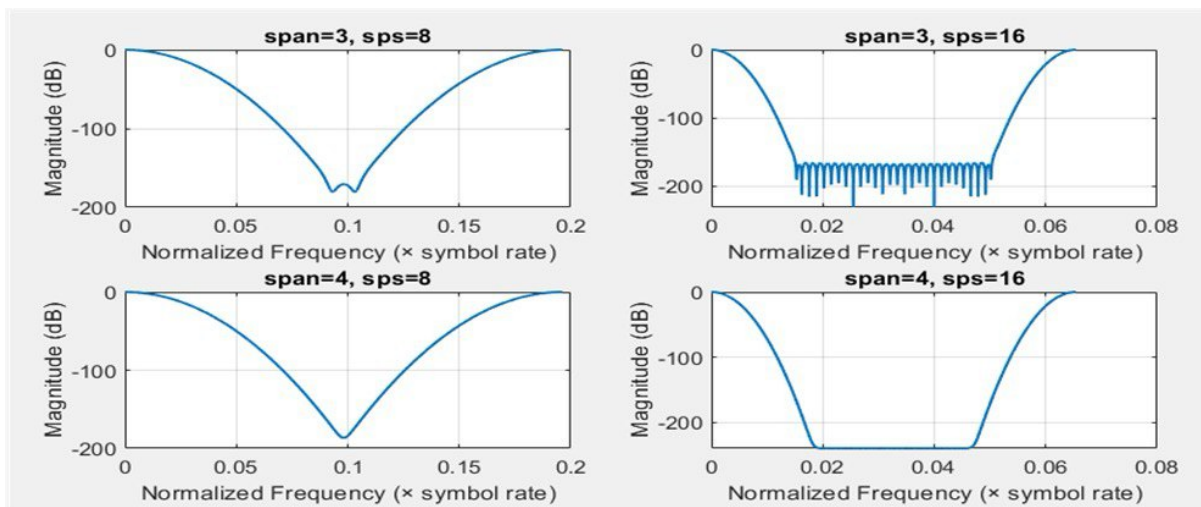


Figure 31: Power spectral density of the Gaussian pulse for different span and SPS values

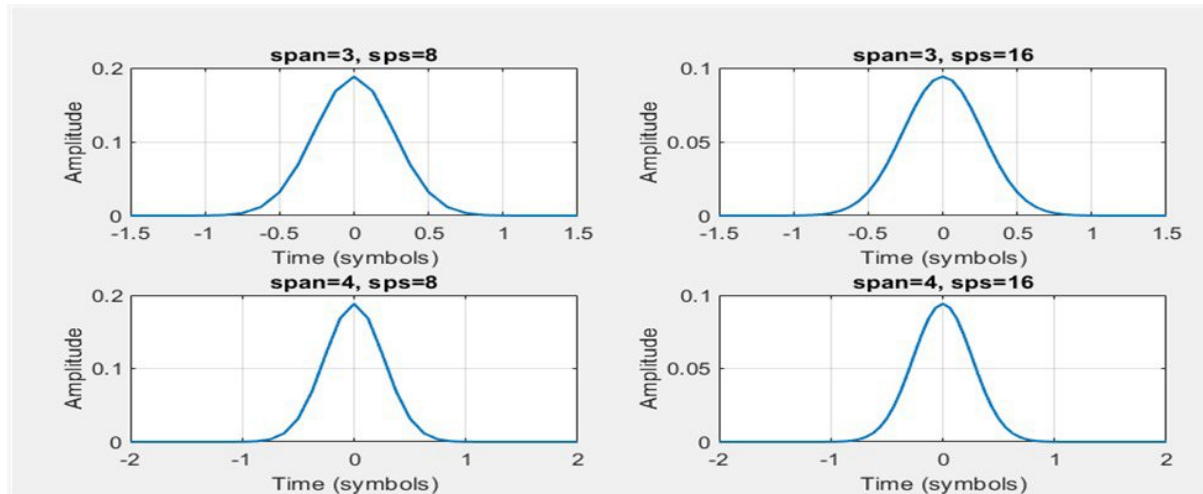


Figure 32: Time-domain Gaussian pulse amplitude for different span and SPS values

### Configuration Used for Primary Design and Verification

Based on the sweep results and the need for a robust verification baseline, this thesis uses:

$$\text{span} = 4, \text{ SPS} = 16$$

This configuration provides sufficient time support ( $\text{span} = 4$ ) and adequate time resolution ( $\text{SPS} = 16$ ) for clear pulse-shaping behavior and reliable comparison between the reference FIR and the multiplier-free symmetric realization.

## RTL implementation

### NRZ Block

#### Abstraction

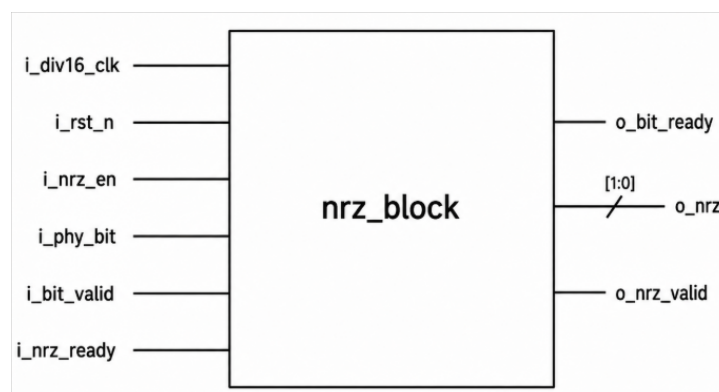


Figure 33: NRZ Block Diagram

## Port Description

Table 3: NRZ Block Port List

| Port Name   | Direction | Width / Type       | Description                                                                                                  |
|-------------|-----------|--------------------|--------------------------------------------------------------------------------------------------------------|
| i_div16_clk | Input     | logic              | Main clock for the NRZ block. The NRZ conversion is performed on the rising edge of this clock.              |
| i_rst_n     | Input     | logic              | Active-low asynchronous reset. Resets the stored NRZ output and valid flag.                                  |
| i_nrz_en    | Input     | logic              | Enable signal for the NRZ block. When low, the output data and valid signal are cleared.                     |
| i_phy_bit   | Input     | logic              | Input binary PHY bit to be converted into NRZ format. A value of 1 is mapped to +1, while 0 is mapped to -1. |
| i_bit_valid | Input     | logic              | Indicates that the input bit i_phy_bit is valid and can be accepted when the block is ready.                 |
| o_bit_ready | Output    | logic              | Ready signal sent to the previous block. Indicates that the NRZ block can accept a new input bit.            |
| i_nrz_ready | Input     | logic              | Ready signal from the next block. Indicates that the next block can accept the NRZ output sample.            |
| o_nrz       | Output    | logic signed [1:0] | Signed NRZ output sample. The binary input bit is mapped to either +1 or -1.                                 |
| o_nrz_valid | Output    | logic              | Indicates that the NRZ output sample o_nrz is valid.                                                         |

## Functional Description

The NRZ block is the first digital processing stage in the BLE transmitter baseband chain. Its primary function is to convert the incoming binary PHY bit stream into a signed non-return-to-zero (NRZ) representation suitable for further processing by the pulse-shaping chain. A binary value of 1 is mapped to +1, while a binary value of 0 is mapped to -1, producing a bipolar data stream required before upsampling and Gaussian filtering.

The block operates on the i\_div16\_clk clock, which runs at 16 MHz in the overall transmitter chain. Since BLE operates at a symbol rate of 1 Msymbol/s and the design uses 16 samples per symbol, the 16 MHz processing clock allows one symbol to be represented by 16 samples while maintaining the required 1 MS/s symbol rate.

The NRZ block incorporates a ready/valid handshaking mechanism. An input bit is accepted only when i\_bit\_valid is asserted and the block is ready to receive a new bit. The output signal o\_bit\_ready indicates to the previous stage that the NRZ block can accept a new input bit, while o\_nrz\_valid indicates that the signed NRZ output sample is valid and ready to be

consumed by the next stage.

Internally, the converted NRZ value is stored in a signed 2-bit register. The 2-bit signed representation is sufficient because the output only needs to encode the values  $+1$  and  $-1$ . When reset is asserted or the enable signal `i_nrz_en` is deasserted, the stored NRZ output and valid flag are cleared, preventing invalid samples from propagating through the transmitter chain during reset or disabled operation.

In summary, the NRZ stage prepares the binary input stream for digital modulation by converting the one-bit data representation into a signed amplitude representation suitable for pulse shaping.

## Upsampling Block

### Abstraction

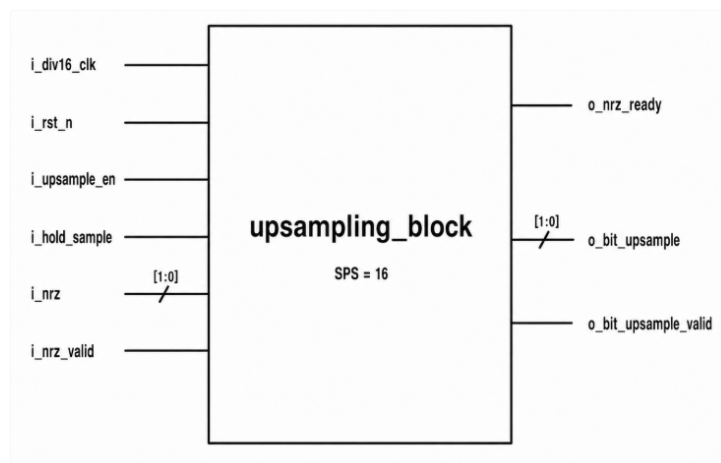


Figure 34: Upsampling Block Diagram

## Port Description

Table 4: Upsampling Block Port List

| Port Name            | Dir. | Width / Type       | Description                                                                     |
|----------------------|------|--------------------|---------------------------------------------------------------------------------|
| i_div16_clk          | In   | logic              | Main clock. Repeated samples generated on rising edge.                          |
| i_rst_n              | In   | logic              | Active-low async reset. Clears stored sample, counter, flag, output, and valid. |
| i_upsample_en        | In   | logic              | Enable signal. When low, internal state and outputs are cleared.                |
| i_hold_sample        | In   | logic              | Hold control for pipeline flushing. Holds previous sample when active.          |
| i_nrz                | In   | logic signed [1:0] | Signed NRZ input sample from the NRZ block.                                     |
| i_nrz_valid          | In   | logic              | Indicates i_nrz is valid.                                                       |
| o_nrz_ready          | Out  | logic              | Ready signal to NRZ block. High when block can accept a new sample.             |
| o_bit_upsample       | Out  | logic signed [1:0] | Upsampled output. Each NRZ symbol repeated for SPS cycles.                      |
| o_bit_upsample_valid | Out  | logic              | Indicates o_bit_upsample is valid.                                              |

## Functional Description

The upsampling block receives the signed NRZ output samples and increases the sample rate by repeating each NRZ symbol for a fixed number of clock cycles. In this design, the upsampling factor is  $SPS = 16$ , meaning each NRZ input symbol is repeated for 16 clock cycles. This provides the time resolution required by the subsequent Gaussian pulse-shaping filter.

The block operates on the same 16 MHz datapath clock. Since the BLE transmitter targets a symbol rate of 1 MS/s, using 16 samples per symbol requires a sample clock of:

$$16 \text{ samples/symbol} \times 1 \text{ Msymbol/s} = 16 \text{ MS/s}$$

Therefore, the 16 MHz clock allows the digital transmitter to generate 16 samples per BLE symbol while preserving the required data rate.

The upsampler uses internal control logic to repeat each received NRZ sample. When a valid NRZ sample is received and the block is not currently repeating a previous sample, the input is stored in an internal register. The block then outputs this stored value repeatedly until 16 total samples have been generated. A repeat counter tracks the remaining repetitions: the counter is loaded with  $SPS - 1 = 15$  after the first sample is output, then decremented on each subsequent clock cycle.

A repeating flag indicates whether the block is currently generating repeated samples for the current NRZ symbol. When the repeat counter reaches the final repetition, the repeating state is cleared and the block becomes ready to accept the next NRZ symbol.

The ready signal `o_nrz_ready` provides flow control between the NRZ block and the upsampler. It is asserted when the block is idle or on the final repetition cycle of the current sample, ensuring that a new NRZ symbol is only accepted when the upsampler is ready to begin a new 16-sample sequence.

The `i_hold_sample` signal supports pipeline flushing. When active, the block holds the previous stored sample instead of loading a new one, maintaining a controlled output during end-of-packet behavior or flushing operations.

In summary, the upsampling block converts the 1 MS/s signed NRZ symbol stream into a 16 MS/s sample stream, which is then passed to the Gaussian filter for pulse shaping.

## Gaussian Filter Block

### Abstraction

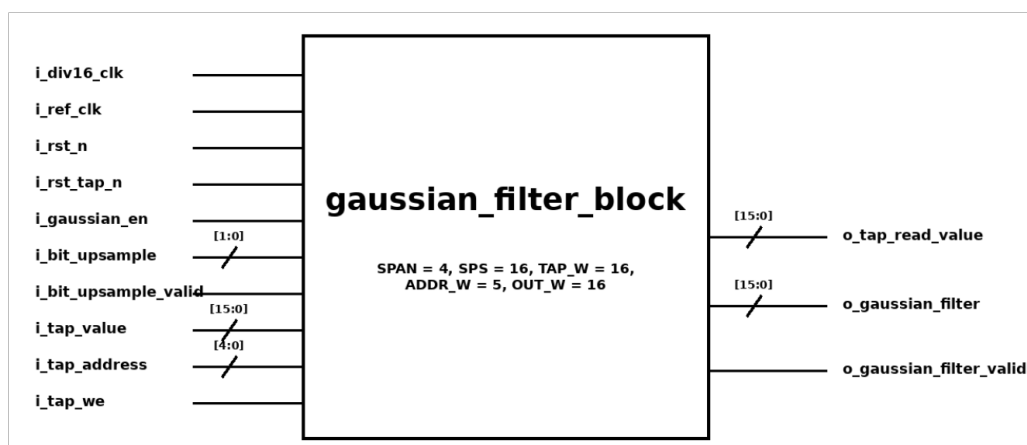


Figure 35: Gaussian Filter Block Diagram

**Port Description**

Table 5: Gaussian Filter Block Port List

| Port Name               | Dir. | Width / Type             | Description                                                                                       |
|-------------------------|------|--------------------------|---------------------------------------------------------------------------------------------------|
| i_div16_clk             | In   | logic                    | Main datapath clock for filtering, delay-line shifting, output registering, and valid generation. |
| i_ref_clk               | In   | logic                    | Reference clock for loading tap coefficients into tap_mem.                                        |
| i_rst_n                 | In   | logic                    | Active-low async reset. Clears Gaussian output, valid signal, and delay line.                     |
| i_rst_tap_n             | In   | logic                    | Active-low reset for tap memory. Clears all stored coefficients when low.                         |
| i_gaussian_en           | In   | logic                    | Enable for the filter datapath. When asserted, valid upsampled samples are processed.             |
| i_bit_upsample          | In   | logic signed [1:0]       | Signed 2-bit upsampled NRZ input from the upsampling block.                                       |
| i_bit_upsample_valid    | In   | logic                    | Indicates i_bit_upsample is valid. Triggers delay-line shift and output update.                   |
| i_tap_value             | In   | logic signed [TAP_W-1:0] | Signed tap coefficient to write into tap memory (16-bit with TAP_W = 16).                         |
| i_tap_address           | In   | logic [ADDR_W-1:0]       | Tap memory address for write and readback (5-bit, addresses 32 half-taps).                        |
| i_tap_we                | In   | logic                    | Tap write enable. When asserted on i_ref_clk, writes i_tap_value into tap_mem.                    |
| o_tap_read_value        | Out  | logic signed [TAP_W-1:0] | Readback of tap_mem[i_tap_address], allowing coefficient verification.                            |
| o_gaussian_filter       | Out  | logic signed [OUT_W-1:0] | Signed 16-bit saturated Gaussian filter output (OUT_W = 16).                                      |
| o_gaussian_filter_valid | Out  | logic                    | Indicates that o_gaussian_filter is valid.                                                        |

## Functional Description

The Gaussian filter block performs the pulse-shaping operation required for the BLE transmitter. After NRZ conversion and upsampling, the signal consists of repeated rectangular samples. If transmitted directly, this rectangular waveform would produce sharp transitions and a wide frequency spectrum. The Gaussian filter smooths these transitions and limits the occupied bandwidth, as required for Gaussian Frequency Shift Keying (GFSK) modulation used in BLE. The filter is implemented as a Finite Impulse Response (FIR) filter with a Gaussian impulse response. The design uses the following parameters:

- Bandwidth-time product:  $BT = 0.5$
- Filter span:  $SPAN = 4$  symbol periods
- Samples per symbol:  $SPS = 16$

The total filter length is therefore:

$$L = SPAN \times SPS = 4 \times 16 = 64 \text{ taps}$$

Because the Gaussian impulse response is symmetric, only half of the coefficients need to be stored. The design stores  $L/2 = 32$  coefficients and reuses them via symmetric sample pairs from the delay line, reducing storage requirements and simplifying the hardware implementation.

## Clock Domains

The Gaussian filter operates across two clock domains:

- **Filtering path** (`i_div16_clk`, 16 MHz): governs input sample processing, delay-line shifting, and output register updates.
- **Coefficient-loading path** (`i_ref_clk`): used to write Gaussian tap coefficients into the tap memory via `i_tap_value`, `i_tap_address`, and `i_tap_we`.

## Fixed-Point Representation

The tap coefficients are stored in signed Q1.15 fixed-point format, using 16-bit signed integers to represent the fractional Gaussian coefficients. This avoids floating-point arithmetic while maintaining sufficient precision for pulse shaping. The filter output is also represented as a signed 16-bit Q1.15 value, generated by accumulating the weighted contributions of symmetric sample pairs and saturating the final result to the signed 16-bit output range.

## Block Diagram

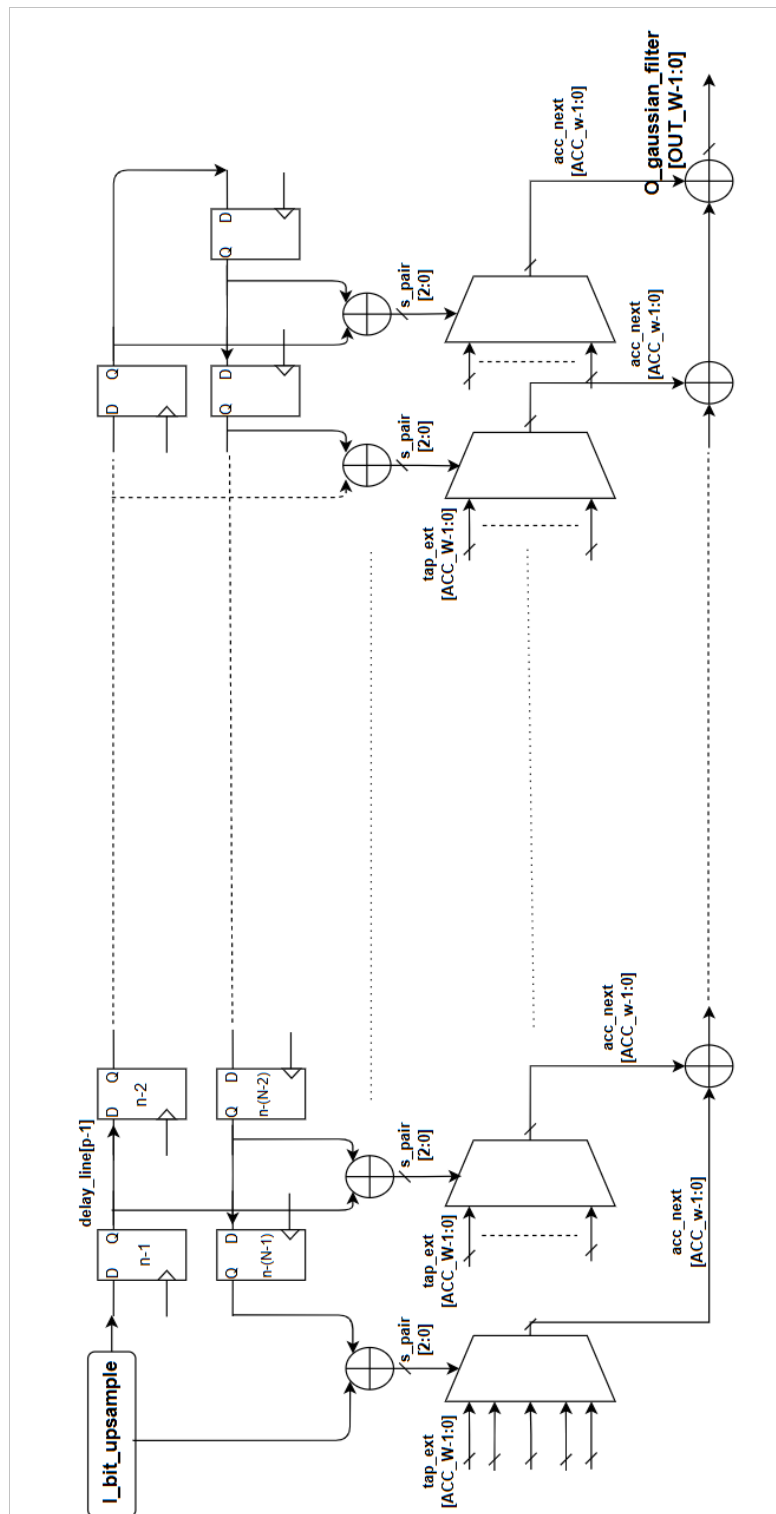


Figure 36: Internal architecture of the Gaussian filter — symmetric FIR structure with 64-sample delay line.

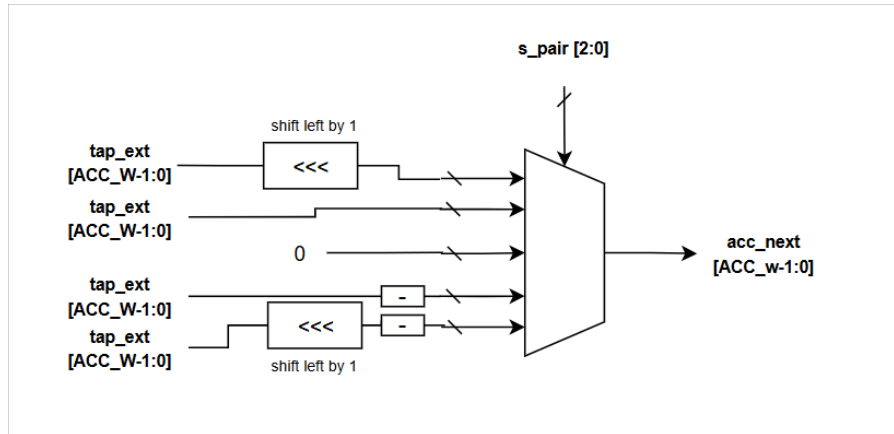


Figure 37: Symmetric sample pair selection and multiply-accumulate unit.

The internal Gaussian filter architecture is based on a symmetric FIR structure. The input upsampled data is shifted through a 64-sample delay line. For each output sample, symmetric samples from the delay line are paired together. Instead of multiplying every individual sample by a separate coefficient, the filter adds each symmetric sample pair first and then multiplies the pair by one stored coefficient.

### Symmetric Pair Multiplication

The symmetric pair sum is defined as:

$$s\_pair = x[k] + x[L - 1 - k]$$

Since the input samples are signed NRZ values, each sample can be  $-1$ ,  $0$ , or  $+1$ . Therefore, the pair sum takes one of five possible values:  $\{-2, -1, 0, +1, +2\}$ .

This allows the multiplication by the Gaussian coefficient to be simplified. Instead of a full multiplier, the design selects one of the following operations based on  $s\_pair$ :

- $s\_pair = +2$ : add  $2 \times tap$
- $s\_pair = +1$ : add  $tap$
- $s\_pair = 0$ : add  $0$
- $s\_pair = -1$ : subtract  $tap$
- $s\_pair = -2$ : subtract  $2 \times tap$

Multiplication by 2 is implemented as a left shift by one bit, reducing hardware complexity to add, subtract, and shift operations only — eliminating the need for a full signed multiplier at every tap.

The accumulator width is larger than the output width to prevent overflow during summation. After accumulation, the result is saturated to the signed 16-bit output range, ensuring the output remains within the Q1.15 format used throughout the transmitter chain.

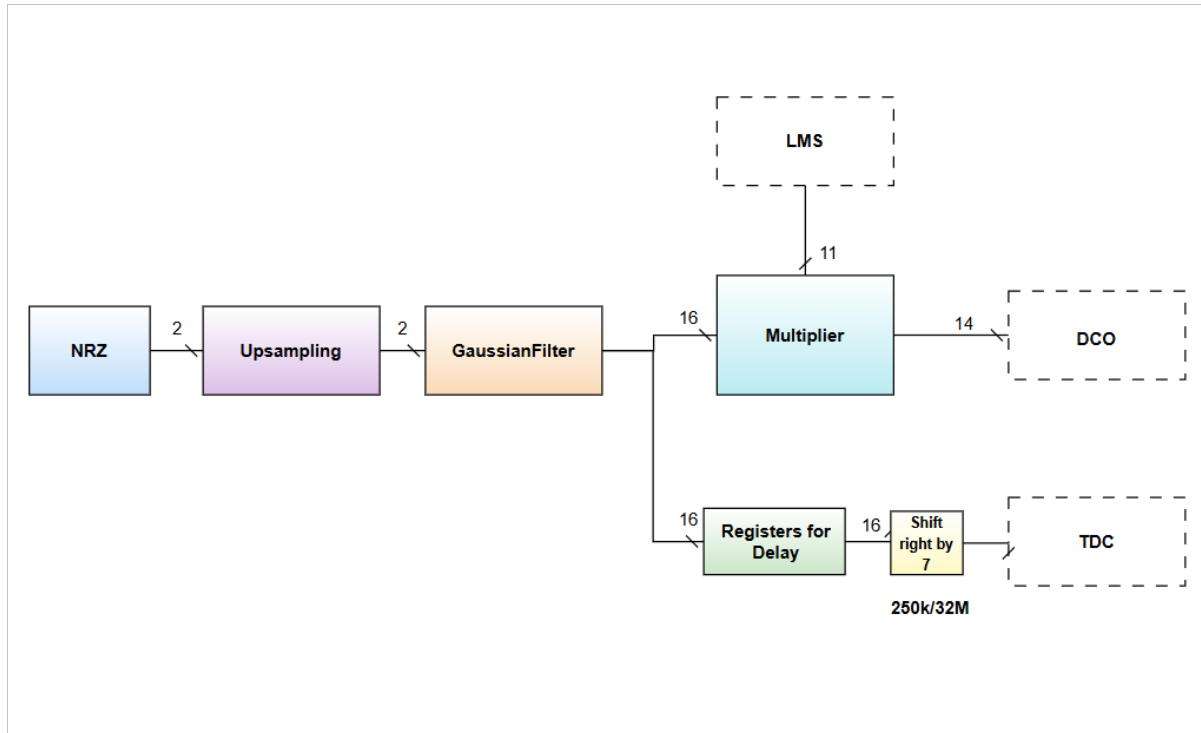


Figure 38: Outer system architecture implementing BLE two-point modulation.

### Gaussian Filter Coefficient Generation

The Gaussian filter coefficients were generated in MATLAB using the following BLE pulse-shaping parameters:  $BT = 0.5$ ,  $SPAN = 4$ , and  $SPS = 16$ . These produce a 64-tap Gaussian impulse response. The coefficient vector is generated using a half-sample-centered time axis and normalized so that the sum of all coefficients equals one.

Since the impulse response is symmetric, only  $L/2 = 32$  half-tap coefficients are stored in the RTL design. The floating-point coefficients are quantized to signed Q1.15 fixed-point format by multiplying each value by  $2^{15}$ , rounding to the nearest integer, and saturating to the signed 16-bit range. The quantized coefficients are written to `gauss_half.mem` as 16-bit hexadecimal values:

```
0000, 0000, 0001, 0002, 0004, 0009, 0013, 0028,
004E, 0090, 00FB, 019D, 0285, 03B8, 0531, 06DC,
0891, 0A1F, 0B50, 0BF6
```

These coefficients are loaded into the RTL coefficient register file through the tap-loading interface. The control unit writes each coefficient using `i_tap_value`, selects the storage location using `i_tap_address`, and asserts `i_tap_we` during the coefficient-loading phase. Once all coefficients are loaded, the Gaussian filter begins processing the upsampled NRZ stream.

This coefficient-generation flow ensures that the MATLAB model and RTL implementation use identical Q1.15 coefficient values, which is necessary for bit-accurate verification.

### Outer System and Two-Point Modulation

After the Gaussian filter, the shaped modulation signal is divided into two separate paths to implement two-point modulation, namely the DCO modulation path and the TDC (phase-detector)

modulation path. This architecture is adopted because the closed-loop response of the ADPLL can distort the modulation when the data is injected at only one point. By applying the modulation signal simultaneously to both paths, the loop dynamics are compensated and the desired GFSK frequency deviation is preserved.

### DCO Modulation Path

In the DCO path, the output of the Gaussian filter is multiplied by the LMS gain estimate using a radix-4 Booth multiplier. The Gaussian filter output corresponds to the shaped modulation data, whereas the LMS output represents the estimated correction factor for the DCO gain.

The DCO gain determines the frequency change produced by a given digital tuning word. Since the actual DCO gain varies with process, voltage, temperature, and operating conditions, gain estimation is necessary to maintain matching between the two modulation paths.

The multiplication operation produces the gain-normalized DCO modulation word, which is applied directly to the DCO input. Consequently, the oscillator generates the required BLE frequency deviation while compensating for variations in the DCO gain. Without this gain normalization, the overall two-point modulation transfer function becomes non-flat, leading to modulation distortion.

### TDC Modulation Path

The second modulation path applies the Gaussian-filtered data to the TDC (phase-detector) side of the ADPLL. In this path, the Gaussian filter output is first delayed using registers and then scaled by shifting the value right by seven bits.

The scaling factor is derived from the ratio between the nominal BLE frequency deviation and the reference clock frequency:

$$\frac{250 \text{ kHz}}{32 \text{ MHz}} = \frac{250\,000}{32\,000\,000} = \frac{1}{128} = 2^{-7}. \quad (4)$$

Therefore, multiplication by

$$\frac{250 \text{ kHz}}{32 \text{ MHz}} \quad (5)$$

can be efficiently implemented in fixed-point hardware using a right shift of seven bits. Here, 250 kHz corresponds to the nominal BLE frequency deviation, while 32 MHz represents the reference clock frequency. This operation converts the Gaussian modulation data into the normalized quantity required by the phase-detector side of the ADPLL.

### Timing Alignment

Delay registers are inserted in the TDC path to compensate for the latency introduced by the multiplier and output registers present in the DCO path. Since both paths exhibit different processing delays, timing alignment is required to ensure that corresponding modulation samples reach their destinations with the intended relationship.

In the implemented architecture, the modulation sample arrives at the DCO path one clock cycle earlier than the corresponding sample applied to the TDC path. This timing arrangement enables proper two-point modulation operation and minimizes phase mismatch between the two injected components.

## Summary

Overall, the outer system uses the Gaussian filter output in two different ways. In the DCO path, the signal is multiplied by the LMS gain estimate to generate a gain-calibrated DCO modulation word. In the TDC path, the signal is delayed and scaled by

$$\frac{250 \text{ kHz}}{32 \text{ MHz}} = 2^{-7}, \quad (6)$$

which is implemented as a seven-bit right shift. Together, these two paths allow the ADPLL to generate the required BLE GFSK frequency deviation while compensating for both loop dynamics and DCO gain variations.

## Radix-4 Booth Recoding Multiplier

### Abstraction

Figure 39 illustrates the interface of the implemented radix-4 Booth recoding multiplier. The module receives two signed operands and generates their product after Booth recoding, partial-product generation, and accumulation. The output product and valid signal are registered on the rising edge of the clock.

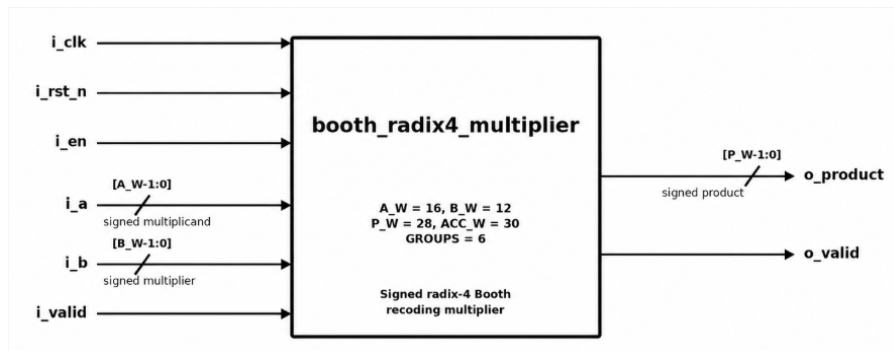


Figure 39: Interface abstraction of the radix-4 Booth recoding multiplier.

Table 6 summarizes the interface of the radix-4 Booth recoding multiplier used in the BLE transmitter. The multiplier is employed to multiply the Gaussian filter output by the LMS gain estimate in order to generate the gain-calibrated DCO modulation word.

### Architecture and Operation

Figure 40 illustrates the architecture of the implemented radix-4 Booth recoding multiplier. The multiplier is used in the DCO modulation path to multiply the Gaussian-filtered modulation sample by the LMS gain estimate.

Table 6: Radix-4 Booth Recoding Multiplier Interface

| Port Name | Direction | Width / Type           | Description                                                                                                                                                                                                     |
|-----------|-----------|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| i_clk     | Input     | logic                  | Clock signal used to register the multiplier output and valid signal. Booth recoding and partial-product accumulation are combinational, while the final product is stored on the rising edge of this clock.    |
| i_rst_n   | Input     | logic                  | Active-low asynchronous reset. When asserted low, the registered product output and valid signal are cleared.                                                                                                   |
| i_en      | Input     | logic                  | Enable signal for the multiplier. When deasserted, the output product and valid signal are cleared.                                                                                                             |
| i_a       | Input     | logic signed [A_W-1:0] | Signed multiplicand input. In the BLE transmitter, this input corresponds to the Gaussian filter output. It is sign-extended internally before partial-product generation.                                      |
| i_b       | Input     | logic signed [B_W-1:0] | Signed multiplier input. In the BLE transmitter, this input represents the LMS gain estimate. A zero is appended to the least significant side to form the extended multiplier used for radix-4 Booth recoding. |
| i_valid   | Input     | logic                  | Indicates that the multiplier inputs are valid. When asserted, the computed product is registered at the output.                                                                                                |
| o_product | Output    | logic signed [P_W-1:0] | Registered multiplication result corresponding to $i_a \times i_b$ after Booth recoding, partial-product shifting, and accumulation.                                                                            |

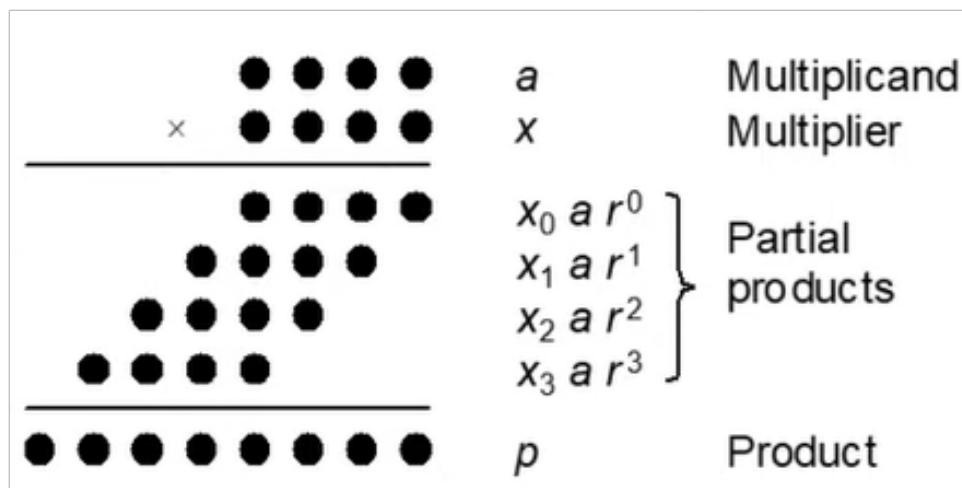


Figure 40: Architecture of the radix-4 Booth recoding multiplier.

A conventional binary multiplier generates one partial product for each multiplier bit. Therefore, for a multiplier width of  $B_W$ , a radix-2 multiplier produces approximately  $B_W$  partial products. These partial products must then be shifted and accumulated, increasing the number of adders, switching activity, silicon area, and dynamic power consumption.

Radix-4 Booth recoding reduces this complexity by processing two multiplier bits at a time. Instead of generating one partial product for every multiplier bit, the multiplier operand is divided into overlapping three-bit groups of the form

$$\{x_{2i+1}, x_{2i}, x_{2i-1}\}, \quad (7)$$

where  $x_{2i+1}$  and  $x_{2i}$  are the current multiplier bits and  $x_{2i-1}$  is the previous lower-order bit. The least significant previous bit is initialized to zero by appending a zero to the multiplier operand. In the RTL implementation, this is expressed as

$$b_{\text{ext}} = \{i_b, 1'b0\}. \quad (8)$$

Each three-bit group is then converted into a Booth digit  $z_i$  according to

$$z_i = -2x_{2i+1} + x_{2i} + x_{2i-1}. \quad (9)$$

The resulting Booth digit can assume only five possible values:

$$z_i \in \{-2, -1, 0, +1, +2\}. \quad (10)$$

Consequently, each Booth group selects one of the following partial products:

$$0, +A, -A, +2A, -2A, \quad (11)$$

where  $A$  denotes the multiplicand. In this design,  $A$  corresponds to the Gaussian filter output, while the multiplier operand corresponds to the LMS gain estimate.

The RTL implementation realizes this recoding using a `case` statement. Specifically, Booth groups 000 and 111 generate zero, 001 and 010 generate  $+A$ , 011 generates  $+2A$ , 100 generates  $-2A$ , and 101 and 110 generate  $-A$ .

After generation, each partial product is shifted left by  $2i$ , where  $i$  denotes the Booth group index. Since each radix-4 Booth group represents two multiplier bits, the final product is obtained by accumulating all shifted partial products:

$$\text{Product} = \sum_i z_i A 2^{2i}. \quad (12)$$

Because two multiplier bits are processed simultaneously, the number of generated partial products is approximately reduced by half compared with a conventional radix-2 multiplier.

For the implemented design, the LMS gain estimate has a width of

$$B_W = 12. \quad (13)$$

Hence, the number of Booth groups is

$$\text{GROUPS} = \left\lceil \frac{B_W}{2} \right\rceil = \left\lceil \frac{12}{2} \right\rceil = 6. \quad (14)$$

Therefore, the multiplier generates only six Booth partial products instead of twelve radix-2 partial products.

Signed multiplication is naturally supported by the Booth algorithm because negative Booth digits directly generate the partial products  $-A$  and  $-2A$ . Consequently, no separate sign-correction stage is required. In the RTL implementation, the multiplicand `i_a` is sign-extended to the accumulator width prior to partial-product generation, while negative partial products are produced using two's-complement operations.

The Booth recoding, partial-product generation, shifting, and accumulation stages are implemented combinatorially. The final accumulated result is registered at the output when `i_valid` is asserted. Therefore, the multiplier does not employ internal pipelining; however, it includes a registered output stage that stores `o_product` and aligns it with `o_valid`.

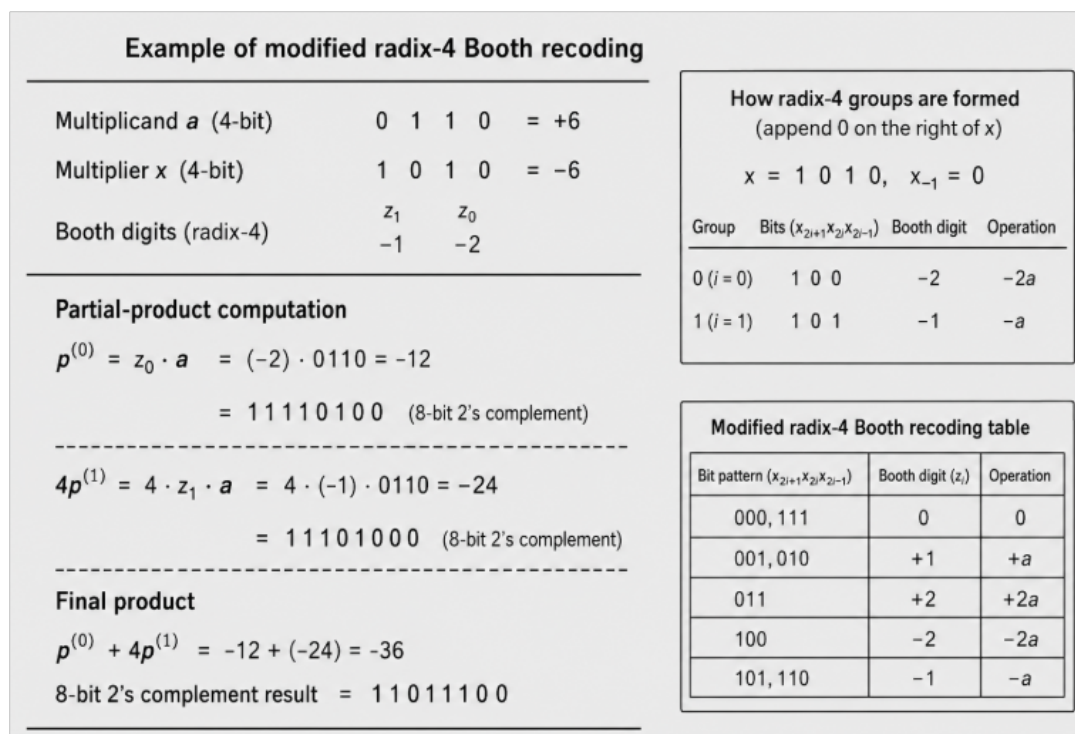


Figure 41: Example of modified radix-4 Booth recoding showing group formation, Booth digit generation, partial-product computation, and the corresponding recoding table.[6]

The worked example illustrates how the radix-4 Booth recoding table is applied to a 2's-complement multiplier. First, a zero is appended to the least significant side of the multiplier to create the initial previous bit ( $x_{-1} = 0$ ). The multiplier bits are then grouped into overlapping 3-bit Booth groups. Each group is converted into a Booth digit using the radix-4 recoding equation:

$$z_i = -2x_{2i+1} + x_{2i} + x_{2i-1} \quad (15)$$

For example, a group of 100 gives:

$$z_i = -2(1) + 0 + 0 = -2 \quad (16)$$

so the selected partial product is  $(-2A)$ . A group of 101 gives:

$$z_i = -2(1) + 0 + 1 = -1 \quad (17)$$

so the selected partial product is  $(-A)$ .

|            |  |  |  |    |    |   |   |         |   |
|------------|--|--|--|----|----|---|---|---------|---|
| =====      |  |  |  |    |    |   |   |         |   |
| $a$        |  |  |  | 0  | 1  | 1 | 0 |         |   |
| $x$        |  |  |  | 1  | 0  | 1 | 0 |         |   |
| $z$        |  |  |  | -1 | -2 |   |   | Radix-4 |   |
| =====      |  |  |  |    |    |   |   |         |   |
| $p^{(0)}$  |  |  |  | 0  | 0  | 0 | 0 | 0       | 0 |
| $+z_0a$    |  |  |  | 1  | 1  | 0 | 1 | 0       | 0 |
| <hr/>      |  |  |  |    |    |   |   |         |   |
| $4p^{(1)}$ |  |  |  | 1  | 1  | 0 | 1 | 0       | 0 |
| $p^{(1)}$  |  |  |  | 1  | 1  | 1 | 1 | 0       | 1 |
| $+z_1a$    |  |  |  | 1  | 1  | 1 | 0 | 1       | 0 |
| <hr/>      |  |  |  |    |    |   |   |         |   |
| $4p^{(2)}$ |  |  |  | 1  | 1  | 0 | 1 | 1       | 1 |
| $p^{(2)}$  |  |  |  |    |    | 1 | 1 | 0       | 1 |
| =====      |  |  |  |    |    |   |   |         |   |

Figure 42: Step-by-step execution table of the radix-4 Booth multiplication algorithm for  $a = 0110_2$  and  $x = 1010_2$ .

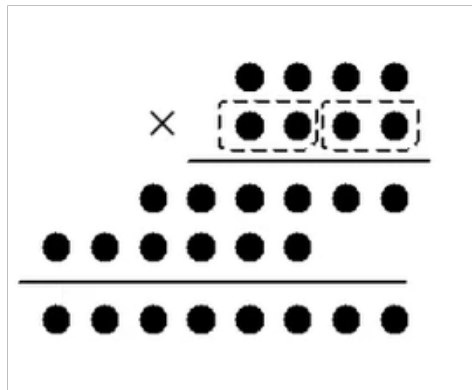


Figure 43: Dot diagram representing the partial product generation, shifting, and accumulation groupings.

Each Booth digit is then multiplied by the multiplicand and shifted according to the group index. Since radix-4 processes two bits per group, the second group is shifted by two bits, which is equivalent to multiplying by 4. The final product is obtained by adding all shifted partial products. This is the same method used in the RTL multiplier, where each 3-bit Booth group generates a signed partial product, shifts it by  $2i$ , and accumulates it into the final product.

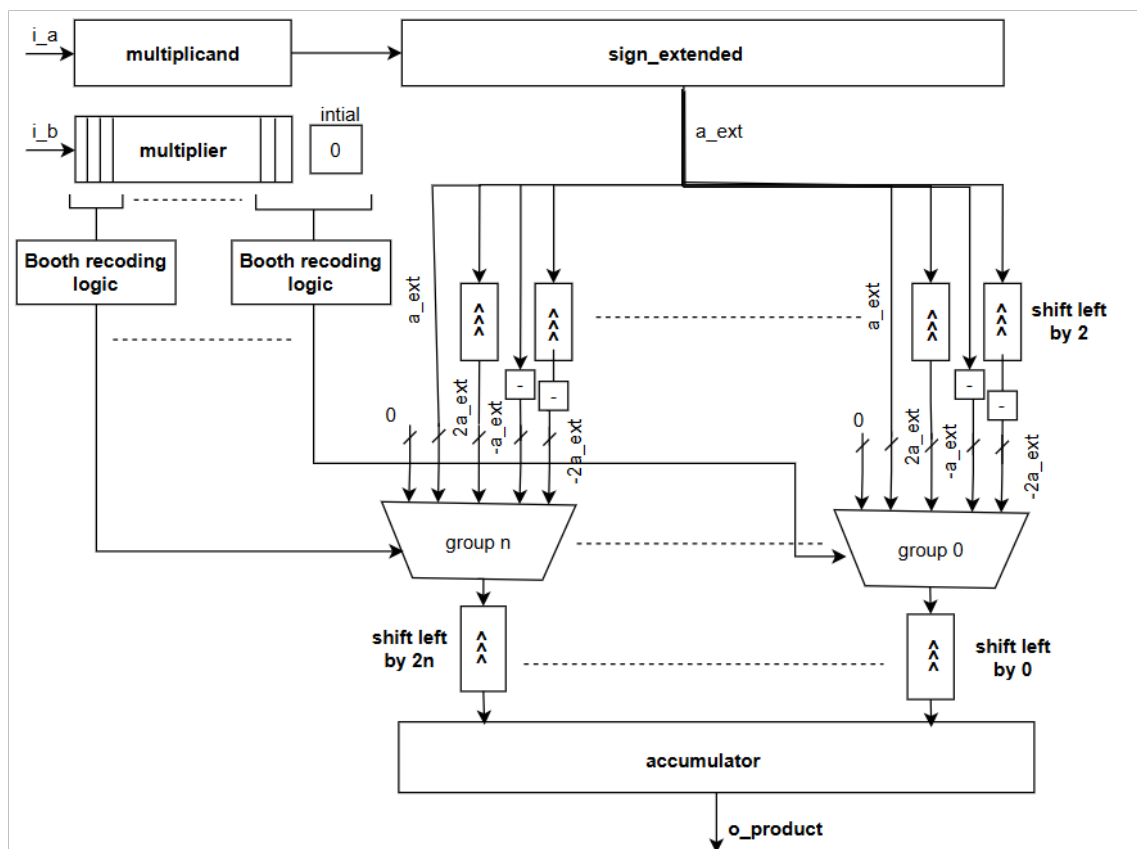
**Full Multiplier block diagram**

Figure 44: block diagram

## Time to Digital Converter

The Time-to-Digital Converter (TDC) in the ADPLL converts timing differences between the DCO and the reference clock into a digital phase value. This digital phase value represents the accumulation of the frequency ratio  $f_{\text{DCO}}/f_{\text{ref}}$  over multiple reference cycles, and therefore gives the measured phase of the reference clock expressed in units of the DCO clock period. The ADPLL then compares this measured phase to the expected reference phase which is the accumulation of the desired frequency ratio  $f_{\text{DCO}}^{\text{desired}}/f_{\text{ref}}$  and the difference between these two quantities is interpreted as the phase/frequency error used to correct the DCO.

The TDC output is typically composed of two complementary words: a fractional (sub-cycle) value that captures the time from the nearest DCO edge to the reference edge, and an integer count of whole DCO cycles that occur between consecutive reference edges. The fractional value is obtained by the quantization of the time difference between the rising edge of the reference clock to the nearest rising edge of the DCO clock using a tapped delay line (chain of identical buffers). The resulting sampled tap pattern encodes the fractional (sub-cycle) portion of the measured reference phase.

To obtain the integer part of the measured reference phase, an integer counter driven by the DCO which counts the number DCO cycles between reference edges provides the whole-number count of DCO cycles that occur during one reference period. Adding the integer count and the fractional TDC word gives the full digital estimate of the measured reference phase (running sum of  $f_{\text{DCO}}/f_{\text{ref}}$ ) which the loop compares to the desired reference phase.

## TDC Phase Domain Operation

We begin with formal definitions that describe what the TDC measures and how the integer and fractional outputs combine to form a discrete-time phase estimate.

Let  $T_{\text{ref}} = 1/f_{\text{ref}}$  be the reference period and  $T_v = 1/f_{\text{DCO}}$  by the DCO (CKV) clock period.

The measured phase (in units of seconds) of the reference clock after  $k$  reference clock rising edges  $t_R$  is defined by :

$$t_V[k] = kT_{\text{ref}} + t_0$$

where  $t_0$  is the initial phase of the reference clock. For simplicity  $t_0$  is assumed to be zero.

So  $t_V$  can be rewritten as:

$$t_V[k] = \sum_{n=1}^k T_{\text{ref}}$$

The measured phase of the reference clock  $t_V[k]$  could be expressed in units of DCO clock period  $\theta_V[k]$  which yields the dimensionless quantity given by:

$$\theta_V[k] = \sum_{n=1}^k \frac{T_{\text{ref}}}{T_v[n]} = \sum_{n=1}^k \frac{f_{\text{DCO}}[n]}{f_{\text{ref}}}$$

So  $\theta_V[k]$  is the running sum of the frequency ratio of the DCO to the reference clock over multiple reference clock periods.

$\theta_V[k]$  can be composed into integer and fractional parts, so it can be written as:

$$\theta_V[k] = R_{v,i}[k] + R_{v,f}[k]$$

where  $R_{v,i}[k]$  denotes integer part and  $R_{v,f}[k]$  denotes fractional part. To get an estimate of the integer and fractional parts of  $\theta_V[k]$ , the variable phase could be rewritten as:

$$\theta_V[k] = \sum_{n=1}^{k-1} \frac{T_{\text{ref}}}{T_v[n]} + \frac{T_{\text{ref}}}{T_v[k]}$$

The reference to DCO clock period could be expressed as:

$$\frac{T_{\text{ref}}}{T_v[n]} = a_i[n] + a_f[n]$$

where  $a_i[n]$  is the integer part of the ratio and  $a_f[n]$  is the fractional part. In BLE application  $T_{\text{ref}} \gg T_v[n]$  as the crystal clock frequency is in the order of MHz (in this project it is 32MHz) and the DCO clock frequency is in the order of GHz (ranges from 2.4GHz to 2.48GHz in the ISM band). Thus  $a_i[n] \gg a_f[n]$  which means the  $a_f[n]$  can be neglected relative to  $a_i[n]$  to get:

$$\therefore \theta_V[k] \approx \sum_{n=1}^{k-1} \frac{a_i[n]T_v[n]}{T_v[n]} + \frac{a_i[k]T_v[k] + a_f[k]T_v[k]}{T_v[k]} = \left( \sum_{n=1}^k a_i[n] \right) + a_f[k]$$

From this equation we find that :

$$R_{v,i}[k] \approx \sum_{n=1}^k a_i[n]$$

where the sum of  $a_i[n]$  is the number of complete DCO cycles that occurred at  $t_V$  which can be measured by DCO driven counters. The fractional part of  $R_{v,f}[k]$  is estimated by:

$$R_{v,f}[k] \approx \frac{a_f[k]T_v[k]}{T_v[k]} = \frac{\Delta t_r}{T_v[k]}$$

Where  $\Delta t_r$  is the time difference between the nearest DCO clock rising edge and  $t_V$ . This time difference can be measured using the TDC to get an estimate of the sub-DCO cycle that is included in  $t_V$ . Each reference cycle the desired DCO frequency is set by a frequency control word FCW which expressed the desired ratio of  $f_{\text{DCO}}/f_{\text{ref}}$ . Thus the desired phase of the reference clock normalized with DCO clock period  $R_R[k]$  is given by:

$$R_R[k] = \sum_{n=1}^k \text{FCW}[n] = \sum_{n=1}^k \frac{f_{\text{DCO}}^{\text{desired}}[n]}{f_{\text{ref}}}$$

The variable phase  $\theta_V[k]$  is then compared to the reference phase  $R_R[k]$  and if  $\theta_V[k] > R_R[k]$  this implies the DCO clock frequency is greater than the desired frequency so to minimize the phase error  $\theta_V[k] - R_R[k]$  a control word is sent to the DCO to reduce its frequency. Otherwise, if  $\theta_V[k] < R_R[k]$  to reduce the phase error the control word is sent to increase the DCO frequency.

### TDC System Block Diagram

The block diagram of the TDC system is shown in Figure 45 below:

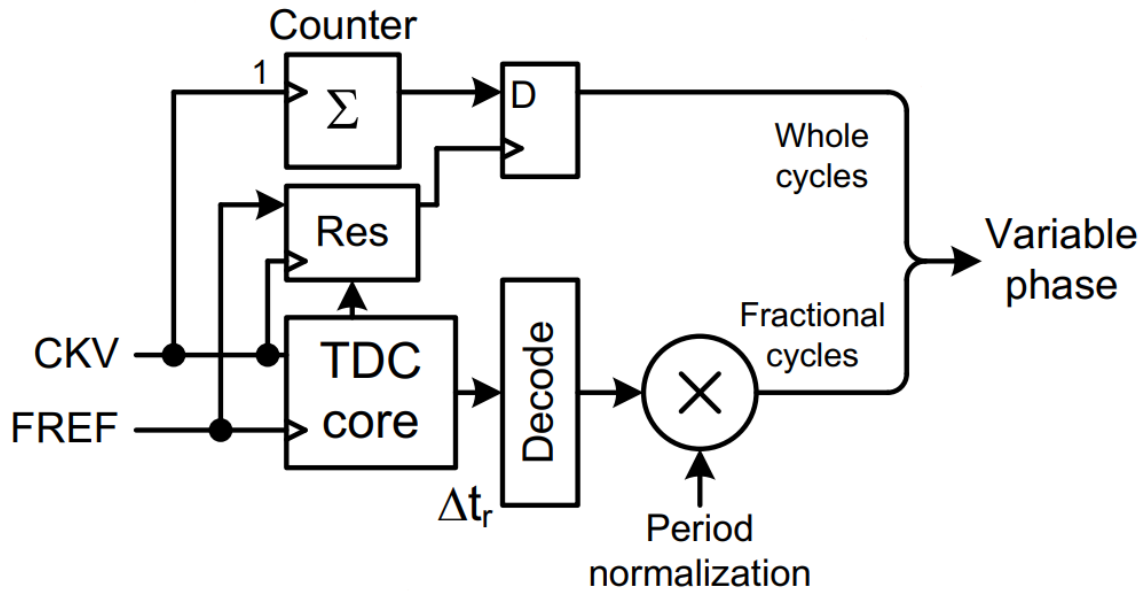


Figure 45: TDC System Block Diagram [13]

The TDC core produces a pseudothermometer code of the sampled tap outputs that contains the time difference information between the rising edge of the DCO (CKV) and reference (FREF) clocks. A pseudothermometer edge detector (denoted by decode in figure 45) finds the number of buffer delays between the nearest rising edge of DCO reference clocks  $\Delta t_r$ . Then period normalization is performed where  $\Delta t_r$  is multiplied by the inverse of the DCO clock period to give the fractional part of the measured reference phase (which is denoted by variable phase in this figure).

The integer part is obtained by counting CKV rising edges with a counter clocked by CKV then the count is sampled on each rising edge of FREF using an edge-triggered D-flip flop. However, FREF and CKV clocks are inherently asynchronous as during frequency acquisition their phase relation is not known and during phase lock their relative phase is not fixed as the ratio of their frequencies is not necessarily an integer. Consequently, direct sampling of the counter output from the CKV domain to the reference clock domain could lead to metastability or incorrect multi-bit captures.

To prevent metastability when capturing data from the DCO clock domain to the reference clock domain, re-timing of the reference clock domain is done where the reference clock is sampled by the DCO clock producing a new clock CKR which is synchronous to the DCO clock domain. Therefore the retimed clock CKR can be used to capture values from the DCO domain. This resampling of the reference clock is done by the block denoted by Res in figure 45.

The retiming of the reference clock using the DCO clock is shown in Figure 46 below.

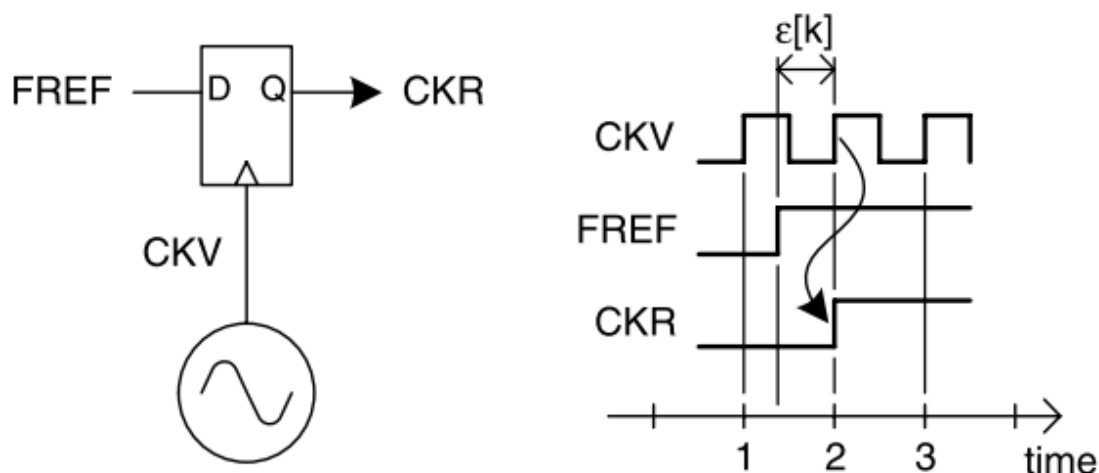


Figure 46: Reference clock re-timing [13]

The ADPLL diagram with the TDC is shown in Figure 47 below.

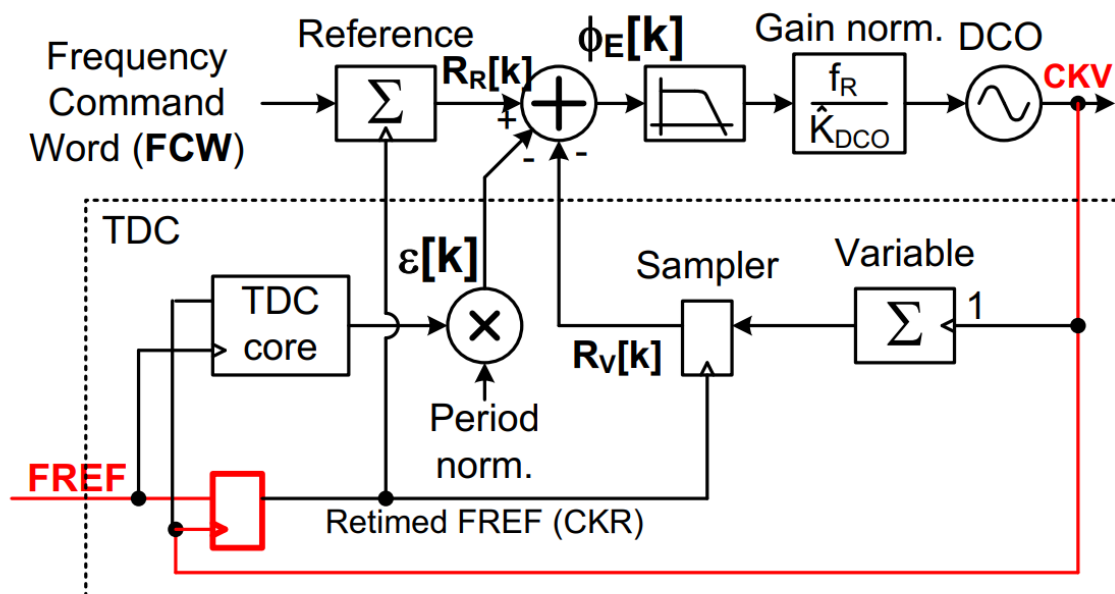


Figure 47: ADPLL diagram with TDC [13]

Once the reference clock FREF has been sampled by the DCO clock to produce the reference-derived, DCO-synchronous clock CKR, CKR is used to capture the CKV-driven counter and thus to produce the measured (or variable) phase on every rising edge of CKR. To ensure a stable handoff between the counter and the CKR sampler, the DCO-domain counter is updated on the DCO falling edge rather than on the rising edge. Because CKR is aligned to the DCO rising edge, updating the counter on the falling edge provides a half-cycle margin so that the counter output is stable by the subsequent CKR rising edge, avoiding setup/hold violations at the capture flip-flops.

The variable phase is compared to the expected phase, which is the accumulation of the frequency control word FCW, and this comparison is done by subtracting the two phase values to get the phase error. Mathematically, both the measured and expected phases are discrete-time signals, so a correct point-wise comparison requires that they be defined at the same sampling instants. Because the measured phase is sampled on CKR rising edges, the FCW must be accumulated on the CKR timebase as well; accumulating the FCW using CKR yields an expected-phase sequence aligned to the same CKR rising edge and therefore permits a direct subtraction (measured minus expected) without interpolation or additional timing alignment. Figure 48 shows the effect of reference clock re-timing for multiple reference cycles where the ratio of the DCO clock frequency to the reference frequency is 2.25.

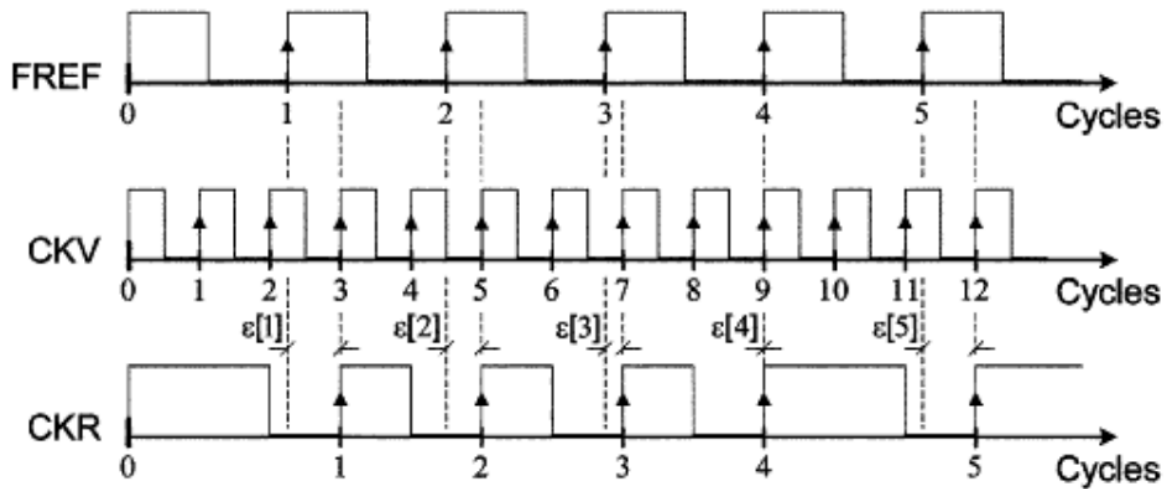


Figure 48: Reference clock re-timing for multiple reference cycles [2]

Figure 49 shows the phase values of the expected or reference phase, DCO phase and phase captured by the retimed reference clock CKR all normalized with the DCO clock period.

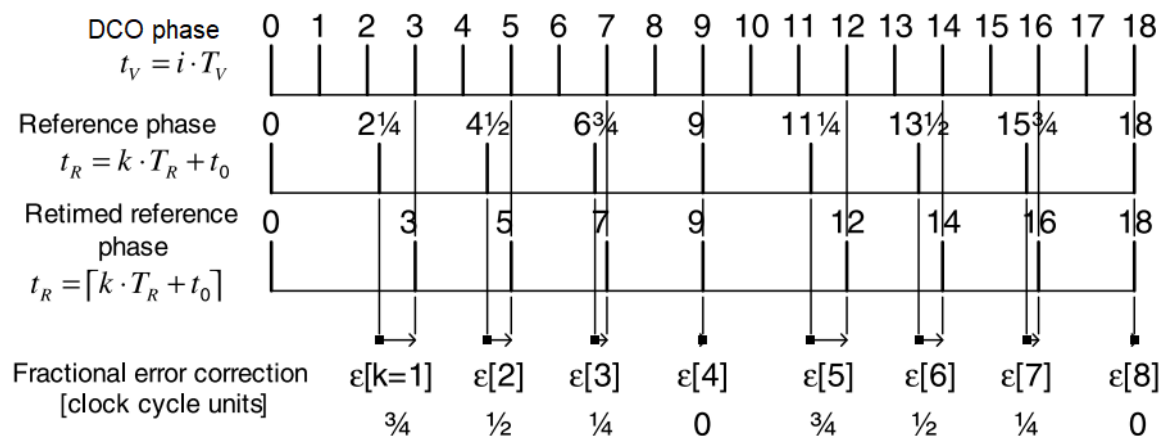


Figure 49: Phase values of expected phase and phase due to re-timing of reference clock [13]

As shown in figure 49 due to re-timing a ceiling operation is done where the reference phase is rounded to the nearest integer. This ceiling introduces an error in the fractional part which can be corrected by adding fractional error correction term  $\epsilon[k]$ . The measured reference phase is given by:

$$R_v[k] = R_{v,i}[k] + R_{v,f}[k]$$

where  $R_{v,i}[k]$  is the integer part of  $R_v[k]$  and  $R_{v,f}[k]$  is the fractional part. The retimed reference phase is given by:

$$R'_v[k] = \lceil R_v[k] \rceil$$

Thus for  $R_{v,f}[k] > 0$ .

$$R'_v[k] = R_{v,i}[k] + 1$$

$$\therefore R_v[k] = R'_v[k] - (1 - R_{v,f}[k]) = R'_v[k] - \epsilon[k]$$

The fractional part  $R_{v,f}[k]$  is obtained from the TDC core shown in figure 45. So taking the 1s complement of the fractional part we can obtain the error correction term to be subtracted from the retimed reference phase to obtain the correct reference phase. For the case where  $R_{v,f}[k] = 0$  or where the DCO rising edge is close to the reference edge, further error correction procedures are required to minimize the error in the phase estimate which are discussed further in the Error correction in TDC section.

In practice, to ensure CKR is a clean, metastability-free clock when FREF is sampled by the DCO (CKV), a multi-stage synchronizer is used: the FREF-derived signal is sampled by two or more cascaded CKV-clocked flip-flops so that any metastable event has an extra CKV cycle to resolve and CKR reliably settles to a logic 0 or 1. Because this synchronization introduces a fixed multi-CKV-cycle latency, the DCO-domain counters are gated (or only enabled) after those CKV cycles; doing so absorbs the synchronizer delay and preserves the integer (ceiling) operation described above. In short, the multi-stage synchronizer provides metastability protection without changing the functional ceiling/counting behavior, it only imposes a deterministic multi-cycle capture latency.

## Different TDC Core Architectures

The TDC core may be implemented using one of two common architectures: a flash TDC or a vernier TDC. Both architectures convert the time interval between a start and stop edge into a digital code, but they differ in resolution, complexity, area, and power consumption. The flash TDC is shown in Figure 50

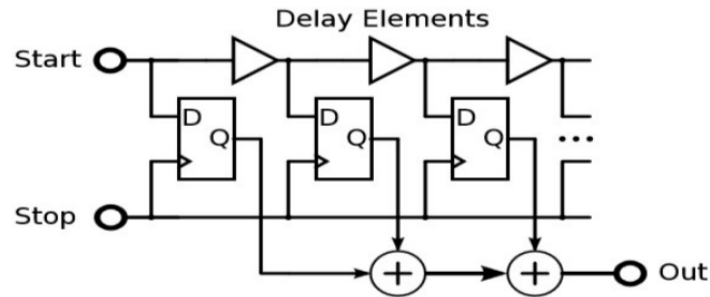


Figure 50: Flash TDC [14]

In this architecture a start edge (here the DCO clock, CKV) propagates down a chain of delay cells while the stop edge (the reference clock, FREF) samples the outputs of every tap with a bank of flip-flops. The sampled outputs form a pseudo-thermometer code whose detected transition location indicates the time difference between the start and stop edges. The flash TDC is simple and fast, but its single-tap resolution is limited by the intrinsic gate delay. That coarse granularity increases fractional measurement error and can therefore contribute more phase noise compared with higher-resolution architectures.

The vernier TDC is shown in figure 51 below.

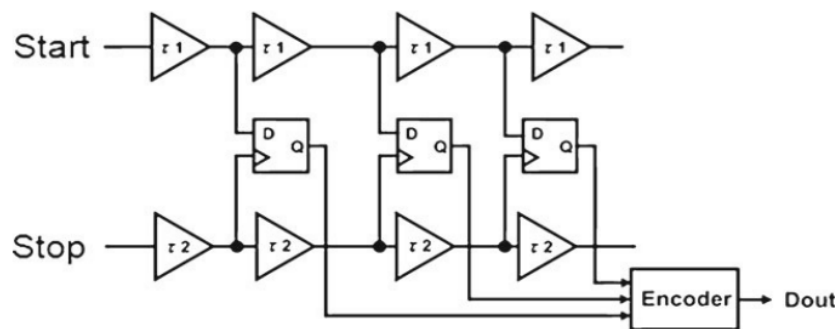


Figure 51: Vernier TDC [14]

Here both the start and stop signals are propagated through separate delay lines whose element delays are intentionally offset: the start delay line uses elements of delay  $\tau_1$  and the stop delay line uses elements of delay  $\tau_2$  where  $\tau_1 > \tau_2$ . This then makes the delay tap be given by  $\tau_1 - \tau_2$  which thus makes the vernier TDC achieve sub-gate resolution. The vernier therefore attains much finer timing granularity than the flash approach, but at the cost of additional delay elements and flip-flops to cover the same interval (one DCO period), which therefore increase its power and area consumption and it also typically requires more careful matching.

For this project—where transmitter power and area are primary constraints—the flash TDC was selected. Its relative simplicity and lower static power and area make it a better choice despite the lower time resolution.

### TDC Core Block Diagram

The block diagram of the TDC core block is shown below:

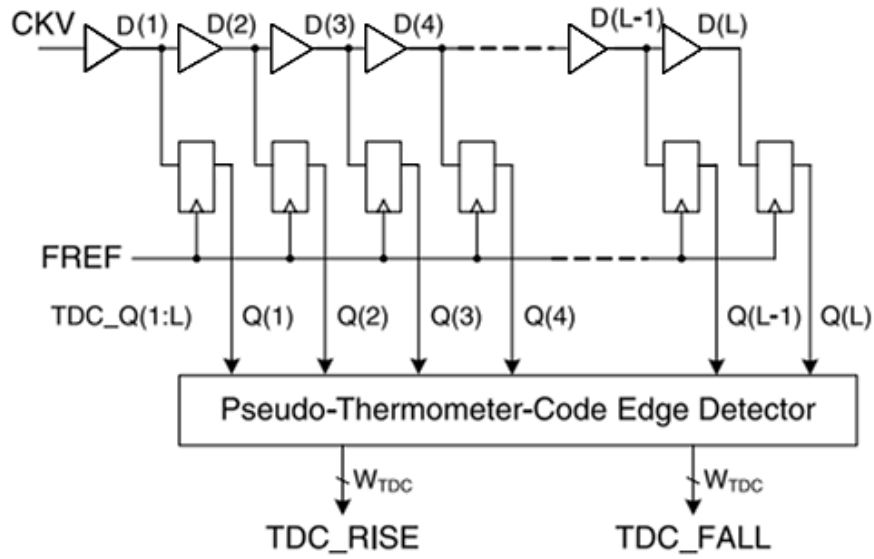


Figure 52: TDC Core Block Diagram [13]

The number of buffer elements used in delay line  $L$  should cover a whole DCO clock cycle to ensure it covers both the rising and falling edges of CKV. So  $L$  is given by the following inequality:

$$L \geq \frac{\min(T_{CKV})}{\max(t_{buf})}$$

If too many inverters are used then circuit consumes more unnecessary area and power. However, some margin for the number of buffers has to be kept to ensure the full DCO cycle is covered under fast corners and slow DCO clock frequencies.

### TDC Core Operation

An example timing diagram of the TDC core is shown below:

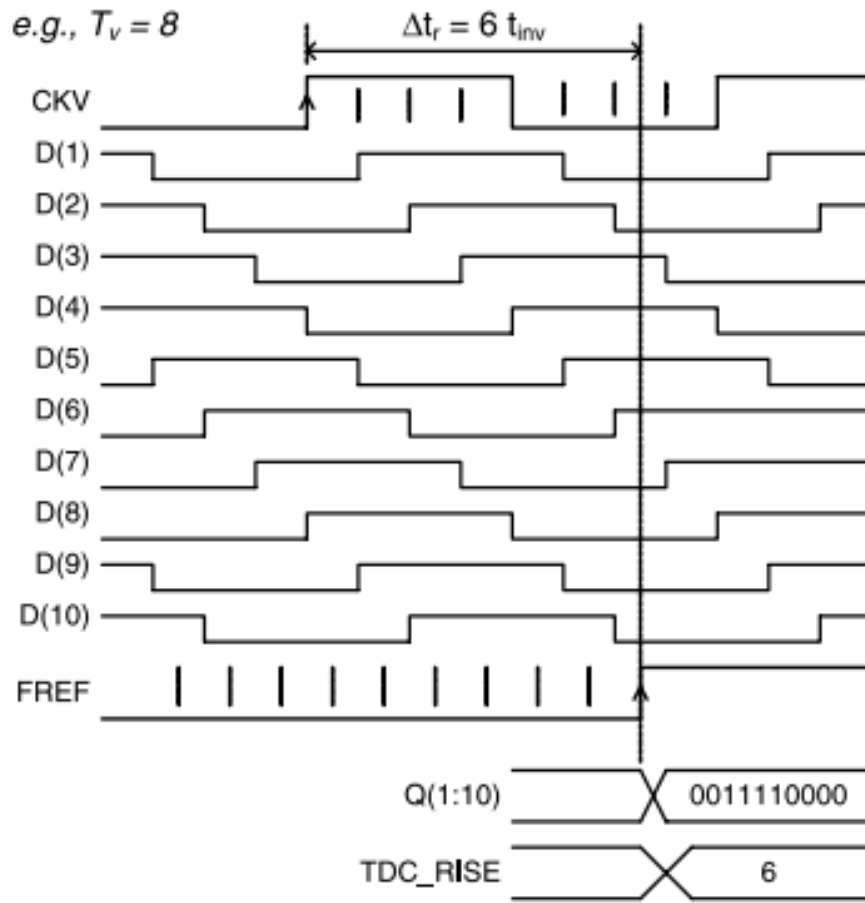


Figure 53: TDC Core Timing Diagram [13]

For every rising edge of  $f_{ref}$  the DCO output clock CKV is sampled at equally spaced time intervals where this time interval is given by the buffer delay  $\Delta t$ . The output word TDC\_Q is composed of series of consecutive 1s followed by consecutive 0s which makes it a pseudo-thermometer code. And as output  $Q(L)$  is delayed by  $L$  buffers while  $Q(1)$  is delayed by 1 buffer then  $Q(1)$  is more recent sample in CKV compared to  $Q(L)$ . So the number of buffer delays from the most recent rising edge of CKV to the rising edge of Fref which is given by  $\Delta t_r$  is given by the position of the transition from 1 to 0 in the Q word. Similarly, the position of the transition from 0 to 1 will give the number of buffer delays between the rising edge of the reference clock FREF to the most recent falling edge of the DCO clock ( $\Delta t_f$ ).

In the example shown in figure 53 we see that  $\Delta t_f$  is given by 2 buffer delays while  $\Delta t_r$  is given 6 buffer delays.

To find the fractional value we consider two cases where the rising edge of the CKV is leading FREF and when the rising edge of DCO is lagging FREF.

### 1) CKV is leading FREF

An example of the CKV rising edge leading FREF rising edge is shown in figure 54 below.

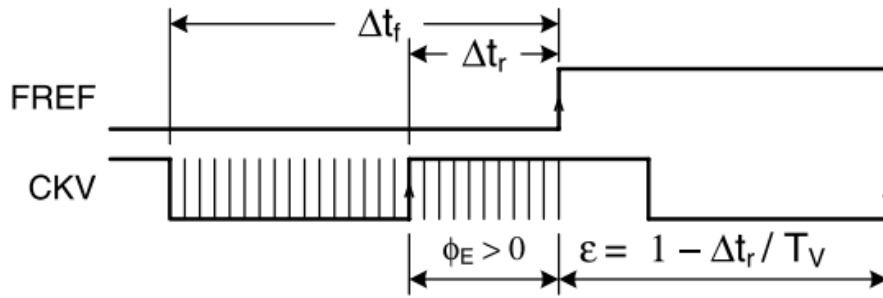


Figure 54: Timing diagram showing DCDO rising edge leading FREF rising edge [13]

The value  $\Delta t_r$  is the number of buffer delays between rising edge of DCO output and reference clock.

To obtain the number of buffer delays in the half a cycle is given by:

$$\Delta t_f - \Delta t_r$$

Thus the number of buffer delays in the whole cycle is given by:

$$2(\Delta t_f - \Delta t_r) = T_v$$

So to get the fractional cycle of DCO clock within the reference clock period we divide  $\Delta t_r$  by  $T_v$ . This cancels the delay of the buffer and thus ensures this ratio is more immune to PVT variations.

### 2) CKV is lagging FREF

An example of the CKV rising edge lagging FREF rising edge is shown in figure 55 below.

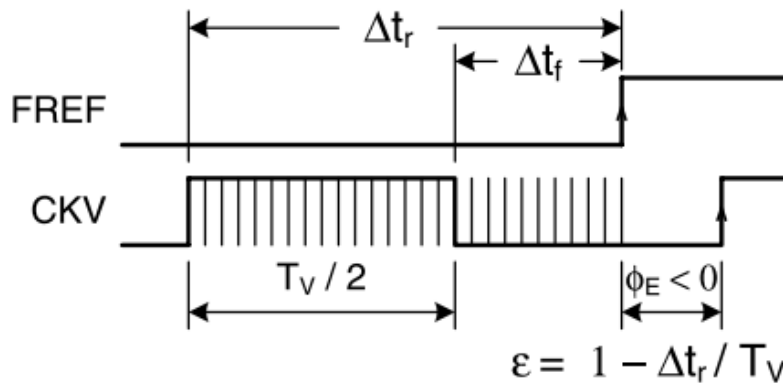


Figure 55: Timing diagram showing DCDO rising edge lagging FREF rising edge [13]

In this case, the number of buffer delays in the half a cycle is given by:

$$\Delta t_r - \Delta t_f$$

The fractional cycle of DCO clock within the reference clock period is also given by  $\Delta t_r / T_v$ .

## TDC Core Sub-Blocks

### Pseudo-Thermometer Code Edge Detector

The pseudo-thermometer code edge detector works in a priority based scheme where it outputs the first position of a transition from 1 to 0 to get  $t_r$  and the first position of a transition from 0 to 1 to get  $t_f$ .

### Divider

The role of the divider is to compute the ratio:

$$\epsilon' = \frac{t_r}{2|t_f - t_r|} = \frac{t_r}{T_v}$$

As  $t_r < T_v$  then  $\epsilon$  is represented by entirely fractional bits in a Q1. $W_f$  format where  $W_f$  is the fractional word length. So  $\epsilon'$  could be expressed as:

$$q = \frac{k}{2^{W_f}}$$

Where  $k$  is an integer

$$\begin{aligned} \therefore \frac{k}{2^{W_f}} &= \frac{t_r}{T_v} \Rightarrow k = \frac{2^{W_f} \cdot t_r}{T_v} \\ \therefore k &= \frac{2^{W_f} \cdot t_r}{T_v} \end{aligned}$$

So  $k$  can be obtained by dividing  $2^{W_f} \cdot t_r$  by  $T_v$  using an integer divider as  $T_v$  and  $t_r$  are integers. To find the fractional correction term to the error caused by re-timing of the reference the 1's complement of  $\epsilon'$  is taken. Thus :

$$\epsilon = 1 - \epsilon' = \frac{k'}{2^{W_f}}$$

Where  $k'$  is the 2's complement of  $k$ .

## Error Correction in TDC

In the TDC, two sources of error arise that can introduce error in the phase of up to  $\pm 1$  DCO cycle. The first source of error is due to the finite resolution of the TDC in measuring the time difference between the DCO and reference rising edges and second source of error is metastability that can cause the captured clock CKR to settle to an unknown logic state which then can introduce an off by one error in the integer part of the measured phase.

The limited resolution of the TDC core introduces an error for the case where the time difference between the DCO and reference rising edges is less than the tap delay of the TDC as shown in figure 56:

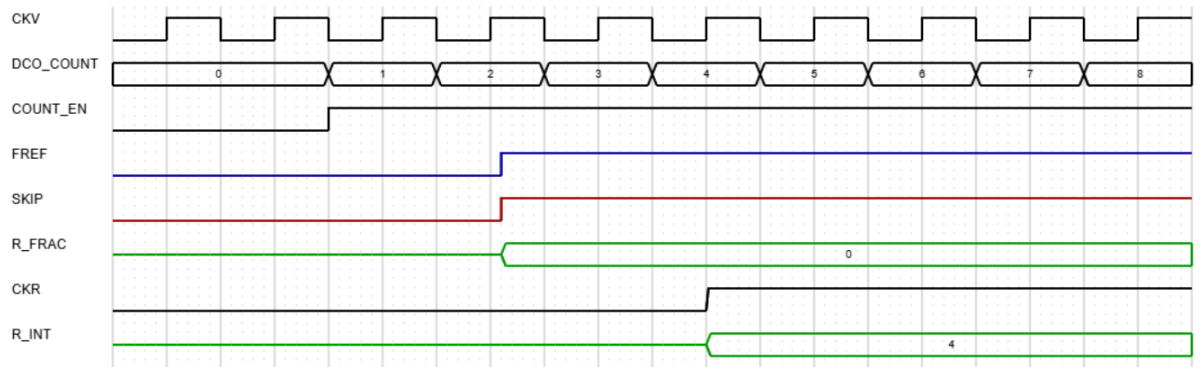


Figure 56: Timing diagram showing FREF rising edge lagging the DCO rising edge with small time difference

In this case the measured phase  $\theta_V$  is given by:

$$\theta_V = R_{V,i} + \epsilon'$$

where  $R_{V,i}$  is the integer number of whole DCO clock cycles counted at the reference rising edge and  $\epsilon'$  is the fractional offset (in units of a DCO period). As the value of  $\epsilon'$  is small:

$$\theta_V \approx R_{V,i}$$

In this case, the pseudo-thermometer code will not capture the most recent rising edge correctly and will instead indicate that the recent rising edge occurred 1 DCO clock period away from the reference clock rising edge and thus report a large fractional value  $\epsilon'$ . With the ceiling operation of the re-timing of the reference clock this introduces an error in the integer LSB value as due to re-timing the integer part captured will be the ceiling of  $\theta_V$  so the measured reference phase after re-timing will be:

$$\theta'_V = R_{V,i} + 1 - \epsilon$$

where  $\epsilon$  the fractional correction term. When  $\epsilon'$  is so small that it doesn't get captured correctly by the TDC and instead is reported to be 1 then the fractional correction term  $\epsilon = 1 - \epsilon' = 0$ . Thus the measured phase after re-timing will be given by:

$$\theta'_V \approx R_{V,i} + 1$$

which introduces an error of up to 1 in the integer part.

To detect and correct this wrapped fractional case we introduce a SKIP signal. The SKIP indicator is asserted when the measured tap count implies that the most recent DCO rising edge occurred within a small margin of the end of the DCO period — equivalently, when the number of buffer delays between that DCO edge and the reference edge is within a threshold of the full-period tap count. For example, if the number of buffer delays spanning 1 DCO clock period is 8 and the rising edge of the DCO occurred 6 buffer delays or more earlier than the rising edge of the reference clock then the SKIP signal will be asserted to 1 to flag that the DCO edge was close enough to the previous period boundary that a whole DCO cycle may have been missed in the re-timing. When SKIP=1, the integer part of the measured phase is decremented by one to compensate for the fractional wrap, restoring the correct integer value.

However, this SKIP correction can itself produce a false correction in the complementary case: when the reference edge actually arrives slightly earlier than the DCO edge by less than one tap delay (as shown in figure 57), the fractional measurement is correct (near one) and no wrap has occurred. If SKIP is asserted in this case, subtracting one from the integer part yields an erroneous -1 integer error. Both of the problematic scenarios occur when FREF transitions very close to the DCO edge: the reference transition then lies within the setup/hold window of the flip-flop that samples FREF to produce CKR, so the sampled CKR pulse can resolve either to 1 (producing the intended capture) or to 0 (effectively skipping a whole DCO period). In other words, the combination of coarse fractional resolution and the retiming uncertainty of CKR leads to two symmetric  $\pm 1$ -cycle error modes — one corrected by SKIP, the other induced as a false positive by the same SKIP logic.

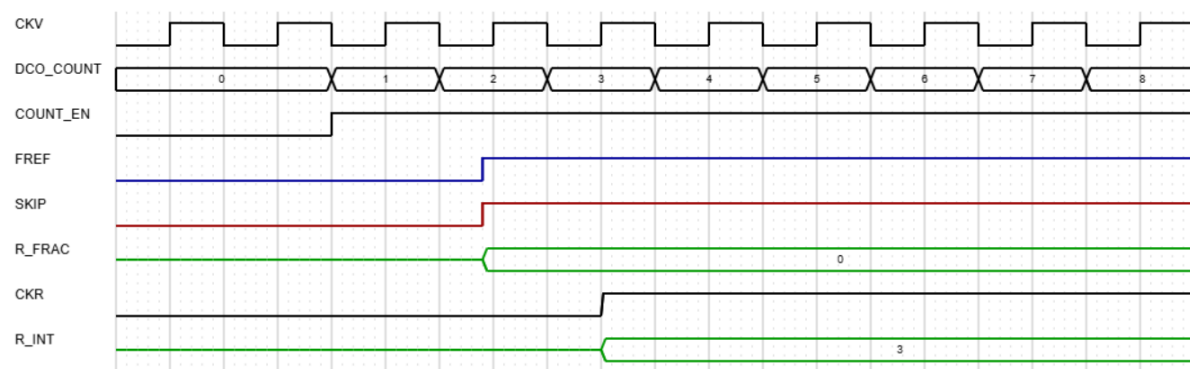


Figure 57: Timing diagram showing FREF rising edge leading the DCO rising edge with small time difference

Thus, to find a more robust error correction scheme against the  $\pm 1$  cycle error described above, FREF is duplicated and delayed by 5 buffer delays to produce a second clock FREF2. The delay is selected so that it is substantially smaller than one DCO period yet larger than the metastability window of the sampling flip-flops. With this choice, at most one of the two reference edges (FREF or FREF2) will fall inside the metastability window of the DCO-domain sampler. FREF2 is then retimed to produce clock CKR2 which is synchronous to the DCO domain and CKR2 is used to form a second measurement of the number of DCO cycles within the measured reference phase.

For the scenario shown in figure 57, because FREF2 is delayed by 5 taps, its rising edge occurs after the nearest DCO rising edge with a comfortable margin. So when FREF2 is sampled and produces CKR2, the resulting CKR2 capture will be offset by one DCO cycle relative to a CKR capture that settled to 1. This offset effectively adds an additional DCO cycle to the second capture done by CKR2 which can then be compensated by the SKIP subtraction logic; the net effect is that the wrapped-fraction case is corrected without producing a false -1 correction in the complementary scenario. This also makes the case of the reference edge arriving slightly before the DCO clock (CKV) rising edge behave in the same way as when the rising edge of FREF comes slightly after the rising edge of CKV which is expected as in this case the measured phase is approximately the same for both cases.

The sampling by CKR2 is shown in figure 58 for the case where the rising edge of FREF comes slightly before the rising edge of CKV and CKR settles to 1.

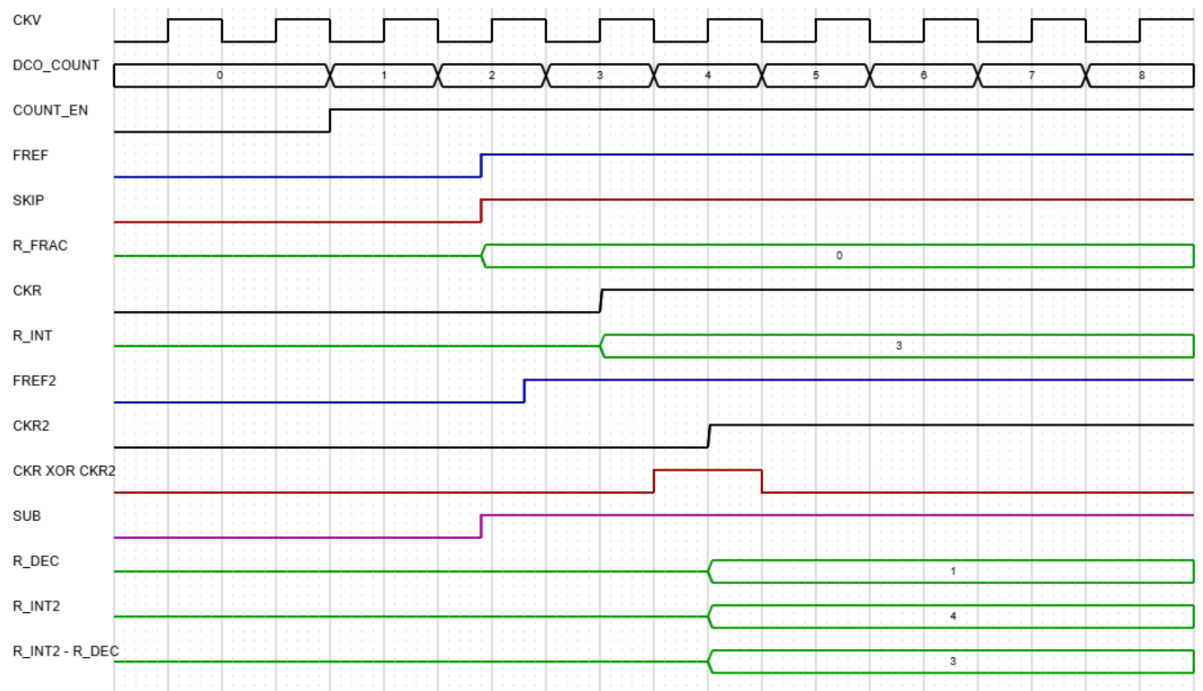


Figure 58: Timing diagram showing capturing done with CKR2 for case FREF lead DCO and CKR settles to 1

As shown in figure 58 SUB is a control signal that indicates if subtraction by 1 is required to correct for the integer error due to metastability or fractional part wrapping in TDC. R\_DEC is the value that is deduced from the integer part (R\_INT2) of the phase captured by the new clock CKR2. The measured phase is supposed to be around 3 DCO cycles in this case and this value is obtained correctly as the measured phase  $\theta_V$  is obtained by:

$$\theta_V = R\_INT2 - R\_DEC - R\_FRAC$$

Thus the measured phase is 3 DCO cycles which matches expected value.

The sampling by CKR2 for the case where the rising edge of FREF comes slightly before the rising edge of CKV and CKR settles to 0 is shown in figure 59.

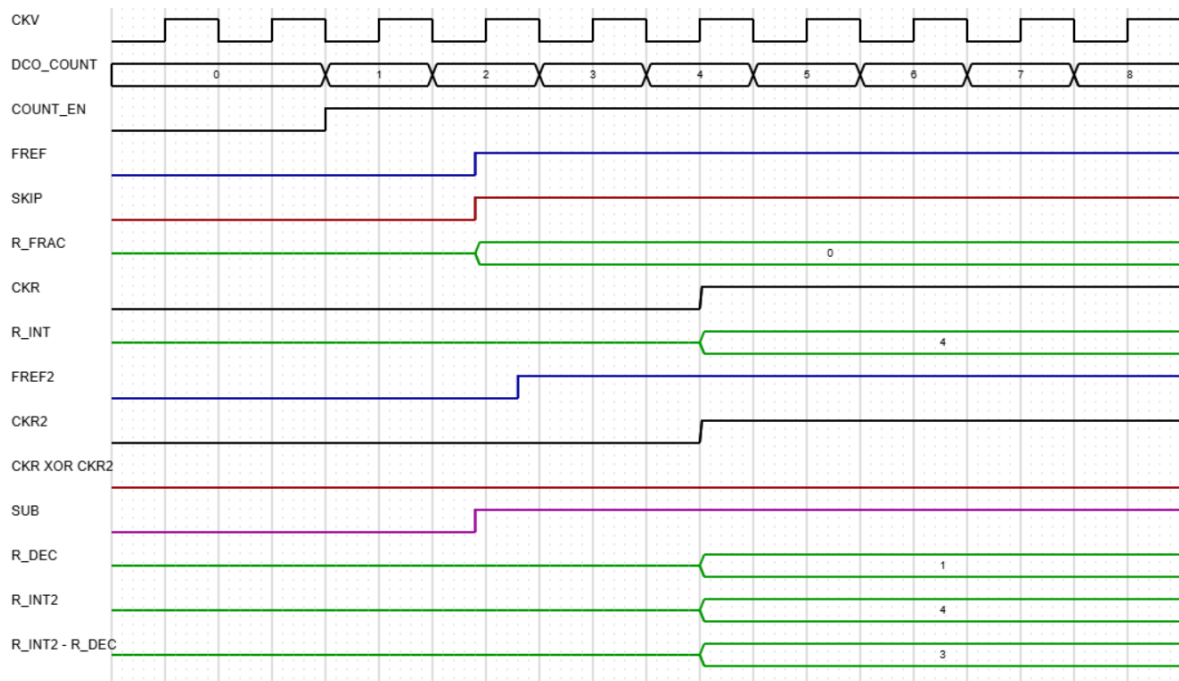


Figure 59: Timing diagram showing capturing done with CKR2 for case FREF lead DCO and CKR settles to 0

As shown in figure 59 The measured phase is supposed to be around 3 DCO cycles in this case and this value is obtained correctly for R\_INT2 - R\_DEC - R\_FRAC.

This solution is shown for multiple cases in the below figures to ensure that it gives valid results for all cases and not introduce additional errors.

Figure 60 shows this method for the case where both FREF and FREF2 are away from the metastability window:

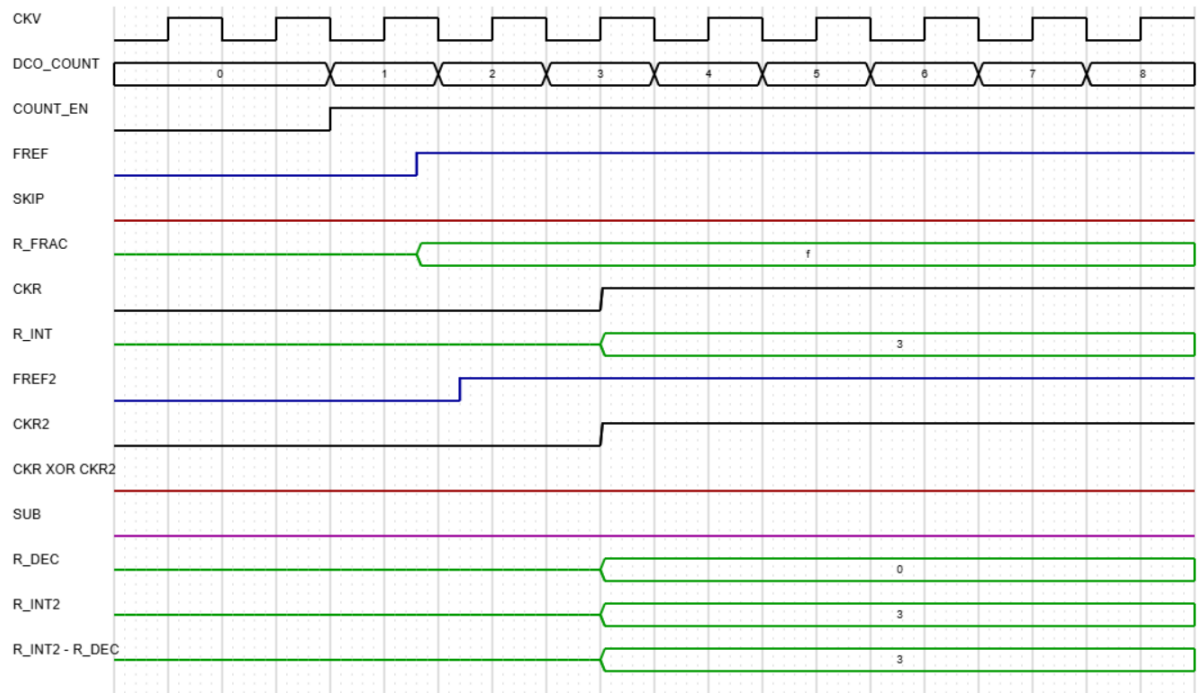


Figure 60: Timing diagram showing capturing done with CKR2 for case where both FREF and FREF2 are away from the metastability window

In this case the measured phase is supposed to be:

$$\theta_V = 3 - f$$

where  $f$  is the fractional error correction term. The measured phase is :

$$\theta_V = R\_INT2 - R\_DEC - R\_FRAC = 3 - f$$

which matches expected.

Figures 61 and 62 illustrate the capturing done by CKR2 in case where FREF rising edge arrives slightly the DCO rising edge leading to hold violation that causes CKR to settle to 1 and 0 respectively. As shown in both figures the measured phase obtained from the integer part captured by CKR2 matches expected phase.

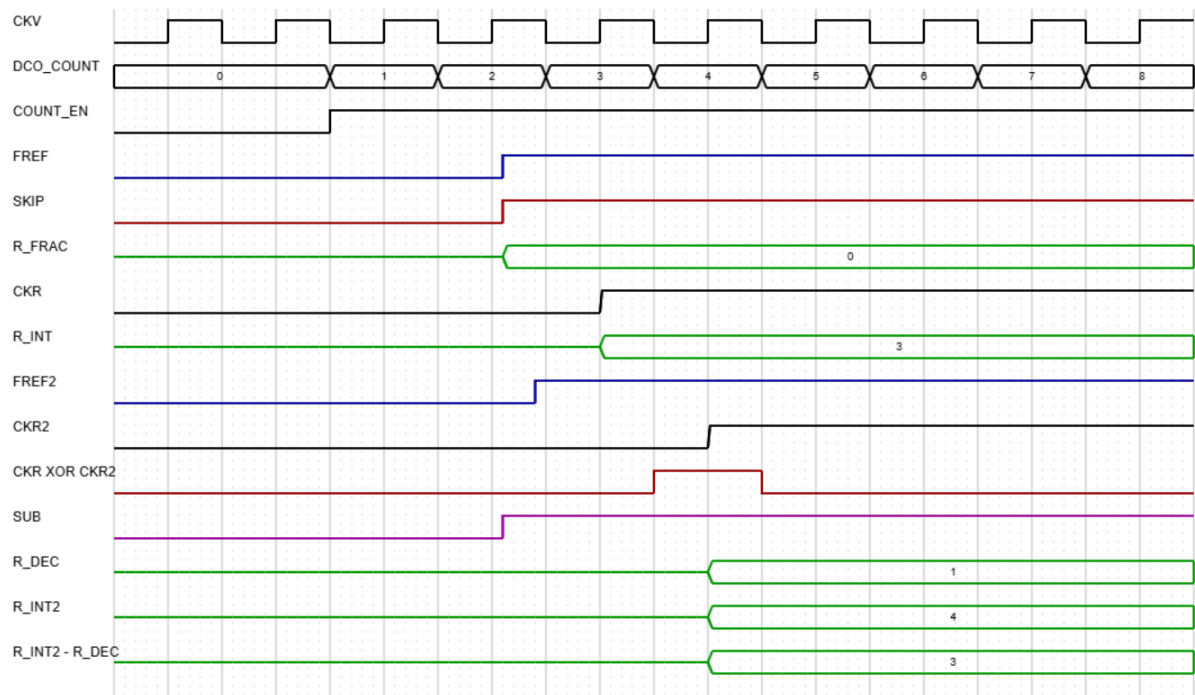


Figure 61: Timing diagram showing capturing done with CKR2 for case FREF lags DCO and CKR settles to 1

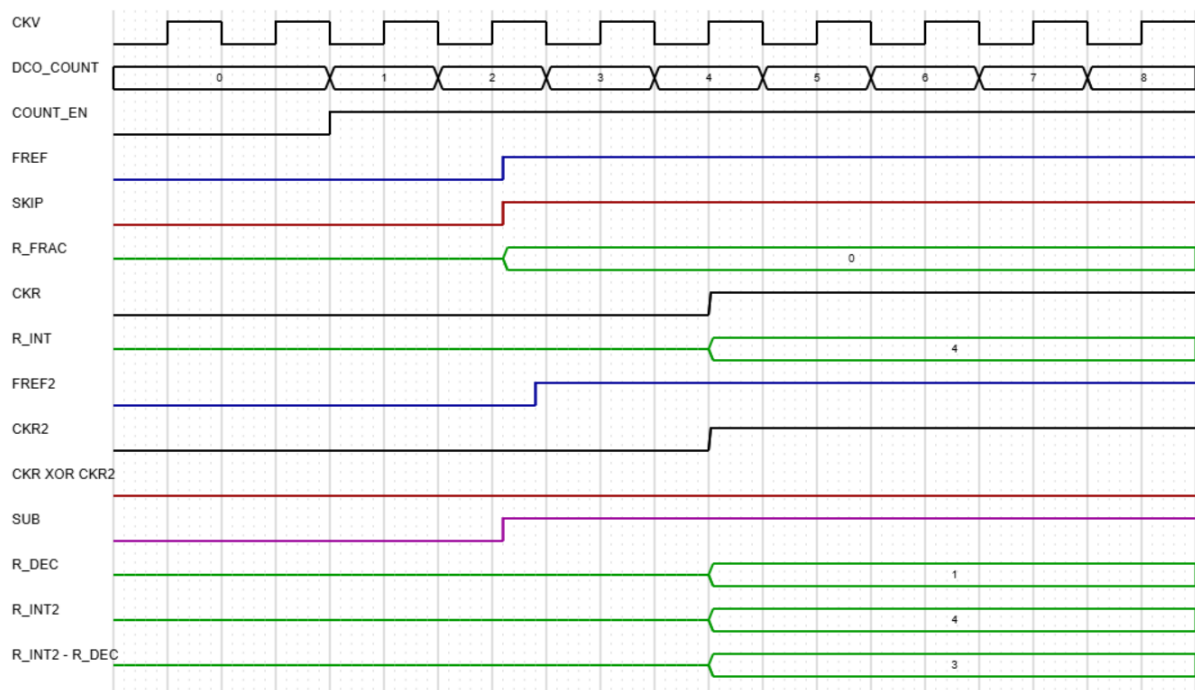


Figure 62: Timing diagram showing capturing done with CKR2 for case FREF lags DCO and CKR settles to 0

A problem that can arise in this solution, however, is if FREF2 is within the metastability window instead of FREF. In this case CKR2 might settle to 0 instead of 1 leading to a complete DCO cycle being skipped and thus can introduce an error of +1 to the integer LSB of the measured phase. This is illustrated in figure 63 below.

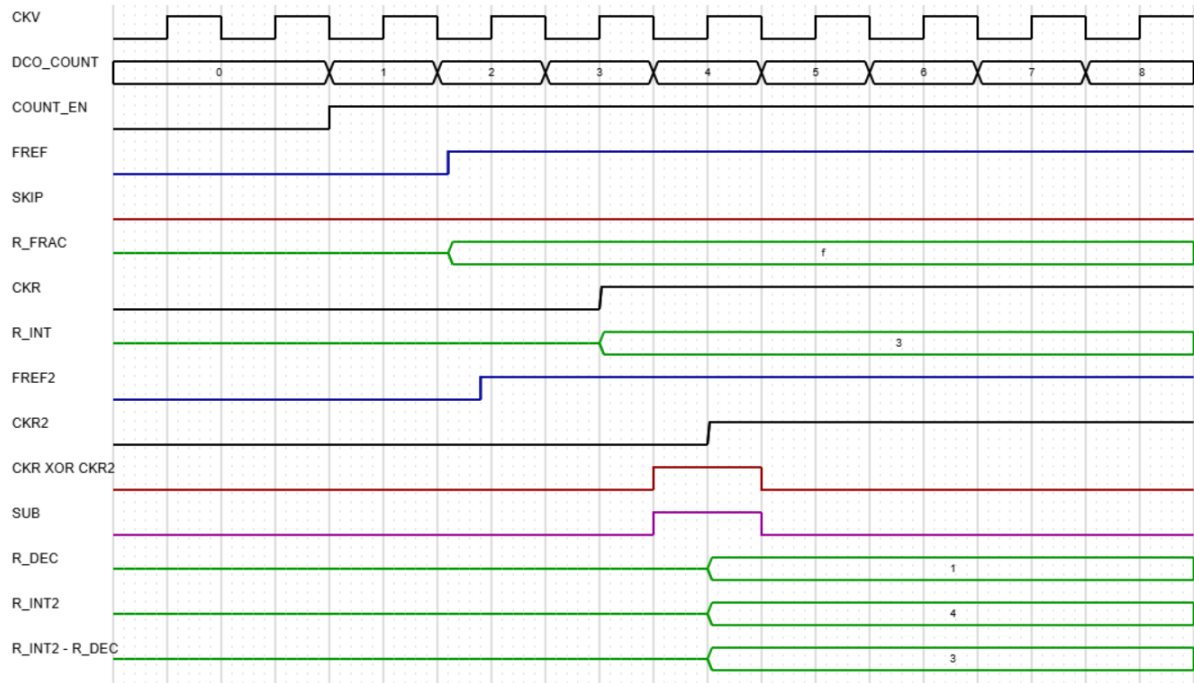


Figure 63: Timing diagram showing capturing done with CKR2 for case where FREF2 edge enters the metastability window

To correct for this error the CKR\_XOR\_CKR2 signal is used to indicate if CKR2 differs from CKR by one DCO clock cycle. CKR\_XOR\_CKR2 is measured synchronously with the DCO falling edge to ensure that its value is stable before being captured by CKR2 and then used to make the decision of subtracting 1 from the measured phase to correct for the cycle skipped. So by investigating the cases illustrated above a truth table is constructed to obtain SUB from SKIP and CKR\_XOR\_CKR2 signals. This truth table is presented in table 7 below:

| SKIP | (CKR_XOR_CKR2) | SUB |
|------|----------------|-----|
| 0    | 0              | 0   |
| 1    | 0              | 1   |
| 0    | 1              | 1   |
| 1    | 1              | 1   |

Table 7: Truth table to find SUB signal

By inspecting the truth table we find that :

$$SUB = SKIP | (CKR\_XOR\_CKR2)$$

where  $|$  denotes the logical OR.

With this error correction scheme CKR2 becomes the ADPLL system clock and is used with loop-filter, DCO and TDC system to calculate the variable phase.

## Random Phase Start

The analysis above assumes that the DCO clock (CKV) and the reference clock (FREF) start in phase at reset de-assertion. In a digital implementation, however, CKV and FREF are asynchronous thus, after reset their relative phase at startup is unknown. As a result, the first measured TDC value contains an arbitrary initial offset that must be removed before the phase sequence can be interpreted as correct.

To establish a consistent reference, the zero-phase point is chosen as the CKV rising edge closest to the first rising edge of FREF after reset de-assertion. The first CKR2 capture then measures an initial phase offset, which we denote by  $\theta_{V,0}$ . This initial capture contains the random startup phase as a fractional value as well as the usual ceiling and correction terms introduced by re-timing. Let the initial measured value be:

$$\theta_{V,0} = R_0 - (1 - \epsilon_0) - r\_dec_0$$

where  $R_0$  is the ceil of the measured phase,  $\epsilon_0$  is the fractional part, and  $r\_dec_0$  is the integer-cycle correction term.

For any subsequent CKR2 capture, the measured phase is

$$\theta_{V,k} = R_k - (1 - \epsilon_k) - r\_dec_k$$

Since the startup phase is arbitrary, the physically meaningful phase evolution is obtained by subtracting the initial offset from all later measurements. Therefore, the corrected variable phase is

$$\theta'_{V,k} = \theta_{V,k} - \theta_{V,0}$$

which gives

$$\theta'_{V,k} = (R_k - R_0) + (\epsilon_k - \epsilon_0) + (r\_dec_0 - r\_dec_k)$$

Hence, the first CKR2 measurement after reset is used as the baseline, and all subsequent CKR2 phase values are corrected relative to this initial capture. This removes the unknown random startup phase and ensures that the reported phase evolution reflects only the true phase change of the reference clock in terms of DCO clock cycles.

The correction for the random phase start can be illustrated more clearly using the waveform shown in Figure 64 below.

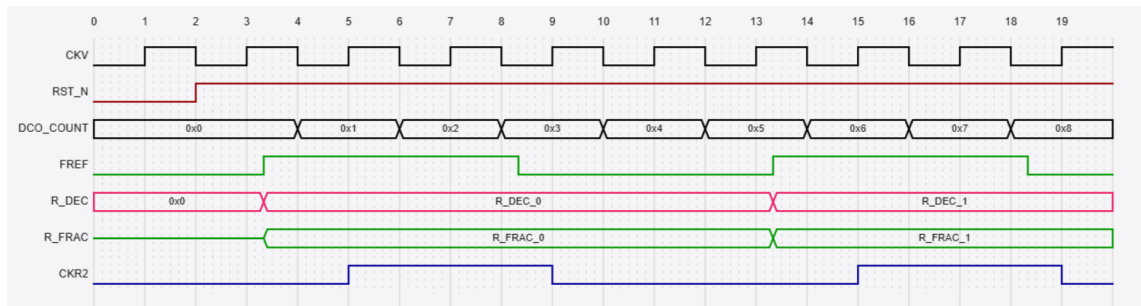


Figure 64: Timing diagram to demonstrate random phase start correction

In Figure 64, the CKV rising edge at time 3 is chosen as the phase reference point immediately after reset de-assertion. The first CKR2 rising edge at time 5 captures the initial startup offset, which is stored as the baseline measurement. The CKR2 rising edge at time 15 then captures the accumulated phase relative to this baseline. Therefore, the corrected phase is obtained by subtracting the initial measurement from the later CKR2 phase value.

## RTL Implementation Block Diagrams

The overall block diagram of the TDC is shown in Figure 65 below:

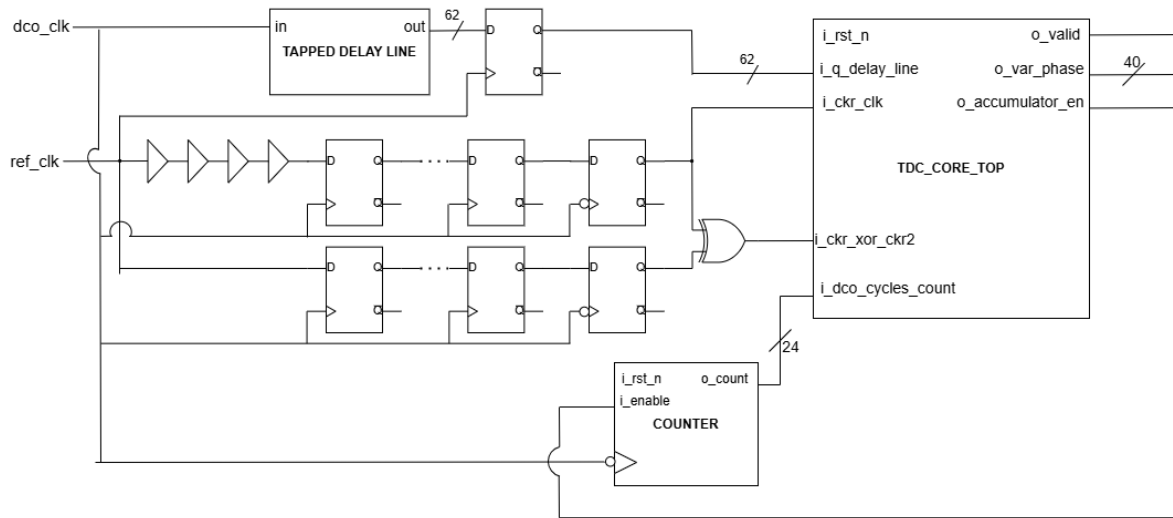


Figure 65: TDC system block diagram

As shown in Figure 65, the TDC is divided into three main sections according to clock domain. The first section is the tapped delay line, which generates the pseudo-thermometer code sampled by the FREF-domain registers. In SystemVerilog, the delay line is modeled using fixed delay values under the assumption of equal buffer delays. Since the capture register must meet very tight setup and hold constraints to minimize bubbles and metastability, it is assumed to be implemented in full-custom logic and is therefore modeled only at RTL.

The second section delays FREF and uses an 12-stage CKV-domain synchronizer to generate CKR and CKR2 for the error-correction logic described earlier. This synchronizer ensures reliable sampling across clock domains and establishes a safe phase relationship between FREF and CKR2. At 2.5 GHz, the 12 stage-synchronizer introduce about 4.8 ns of delay, which is sufficient for timing closure under slow corners while remaining much smaller than the 31.25 ns FREF period, so it does not add an extra  $z^{-1}$  term to the loop dynamics. This section also includes the DCO cycle counter and the XOR-based CKR/CKR2 correction signal.

The third section, TDC\_CORE\_TOP, is the main computation block that produces the variable phase. Its interface and functionality are described in more detail below.

The block diagram of the TDC core top module and TDC core are shown in Figures 66 and 67 below.

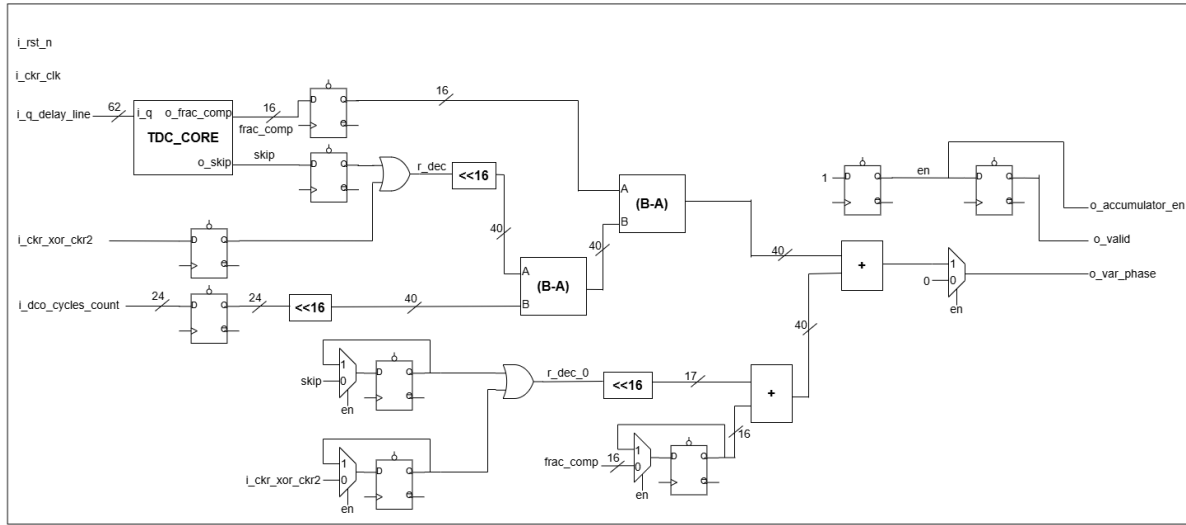


Figure 66: TDC core top module block diagram

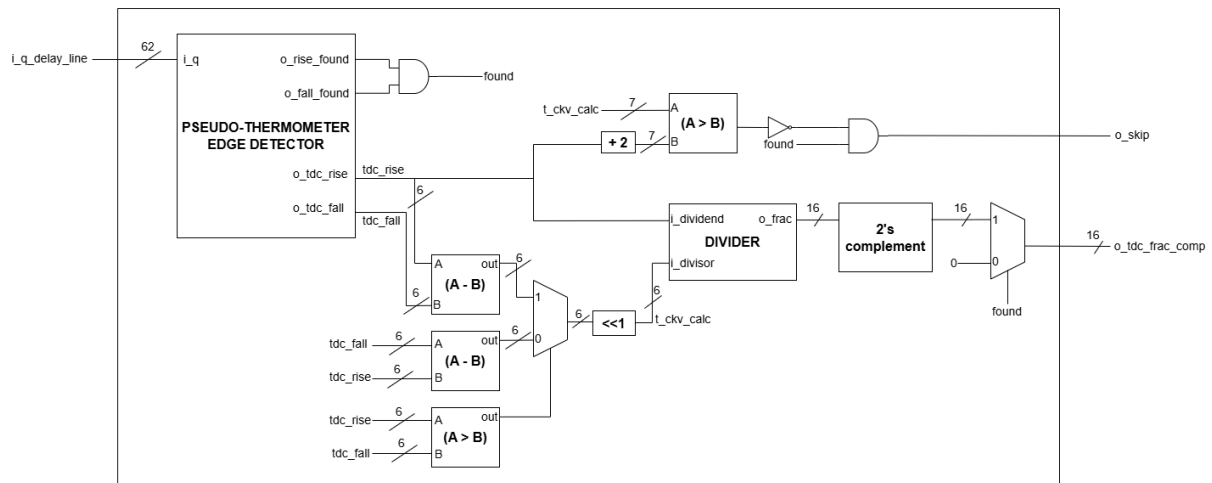


Figure 67: TDC Core block diagram

Note that in Figure 66, the reset input of all registers is connected to `i_rst_n` and the sampling clock is `i_ckt_clk`; these connections are omitted from the diagram for clarity. Likewise, in Figures 66 and 67, some wires are replaced by signal names to keep the diagrams readable and avoid clutter.

As shown in Figure 66, the inputs `i_dco_cycles_count` and `i_ckt_xor_ckt2` are registered to isolate the adder propagation delay from the path between the CKV falling edge, which samples both signals, and the CKR2 clock, which is synchronous with CKV and aligned with its rising edge. This provides sufficient settling time before the values are captured by CKR2 and helps ensure timing closure. In contrast, `i_q_delay_line` is not registered. Although registering it would decouple the TDC core from the FREF-to-CKR2 path and could reduce the required number of synchronizer stages, it would cause the fractional and skip signals to

be computed from the previous delay-line value rather than the current one. As a result, an additional CKR cycle would be needed to account for the initial phase, increasing latency. Achieving one-cycle capture of the initial phase would also require duplicating the TDC core and adding 62-bit registers to store the delay-line word, which would significantly increase area and leakage power compared with the chosen 12-stage synchronizer.

## Sub-Blocks Implementation

### Pseudo-Thermometer Code Edge Detector

The first step in the pseudo-thermometer-code edge detector is to form two transition masks. The signal  $q\_rise$  is obtained by ANDing each bit  $q[i]$  with the negation of its adjacent bit  $q[i + 1]$ , thereby identifying transition from 0 to 1 indicating rising edge in the pseudo-thermometer code. Similarly,  $q\_fall$  is formed as  $(\neg q[i]) \& q[i + 1]$  to identify the opposite transition from 1 to 0 and thus detect the falling edges.

To locate the first CKV rising and falling edges closest to FREF, a tree-structured priority encoder is used, as shown in Figure 68. In this architecture,  $q\_rise$  and  $q\_fall$  are zero-extended to 64 bits and processed in parallel by 4-bit priority encoders. The resulting 16 valid signals are then reduced by a second stage of 4-bit priority encoders, and finally by a top-level priority encoder that selects the first asserted input. The output index of the top-level priority encoder is connected to the index-decoding logic, which uses it to traverse the tree and locate the first set bit in the  $q\_rise$  or  $q\_fall$  word as illustrated in Figure 69. This tree-based structure reduces latency by exploiting parallelism and lowers dynamic power by shortening the combinational path, which also reduces glitching activity compared with a long ripple-style encoder.

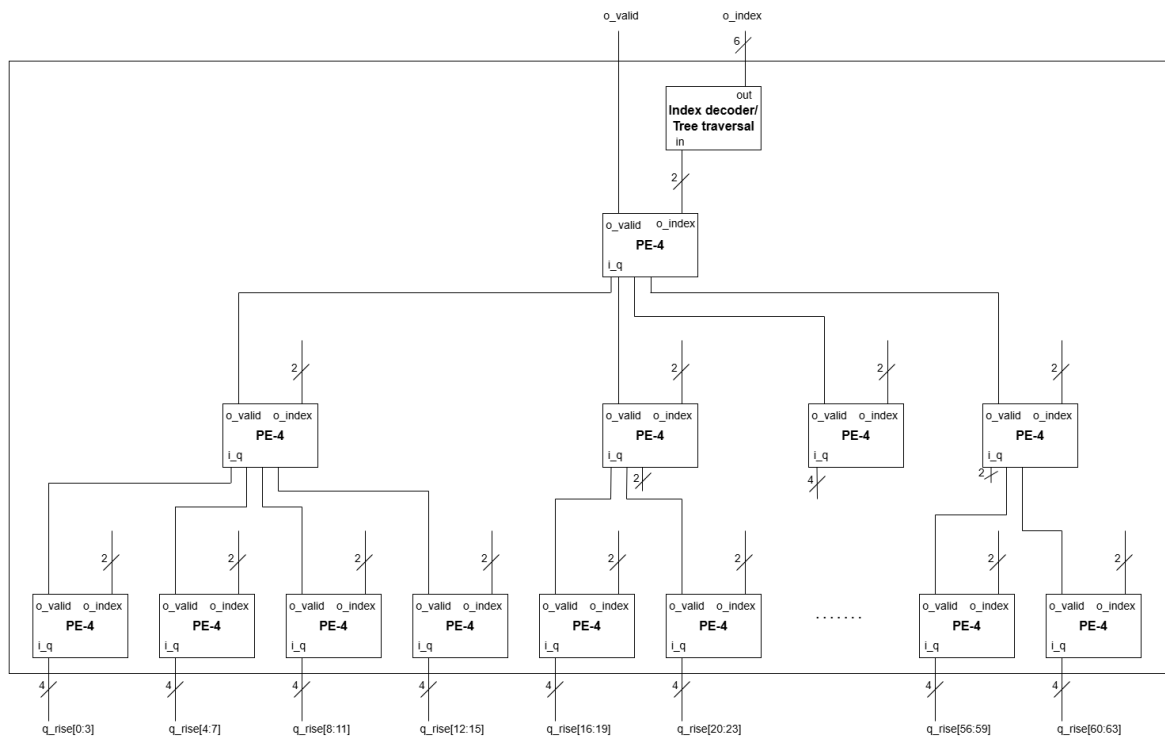


Figure 68: 64-bit Priority Encoder

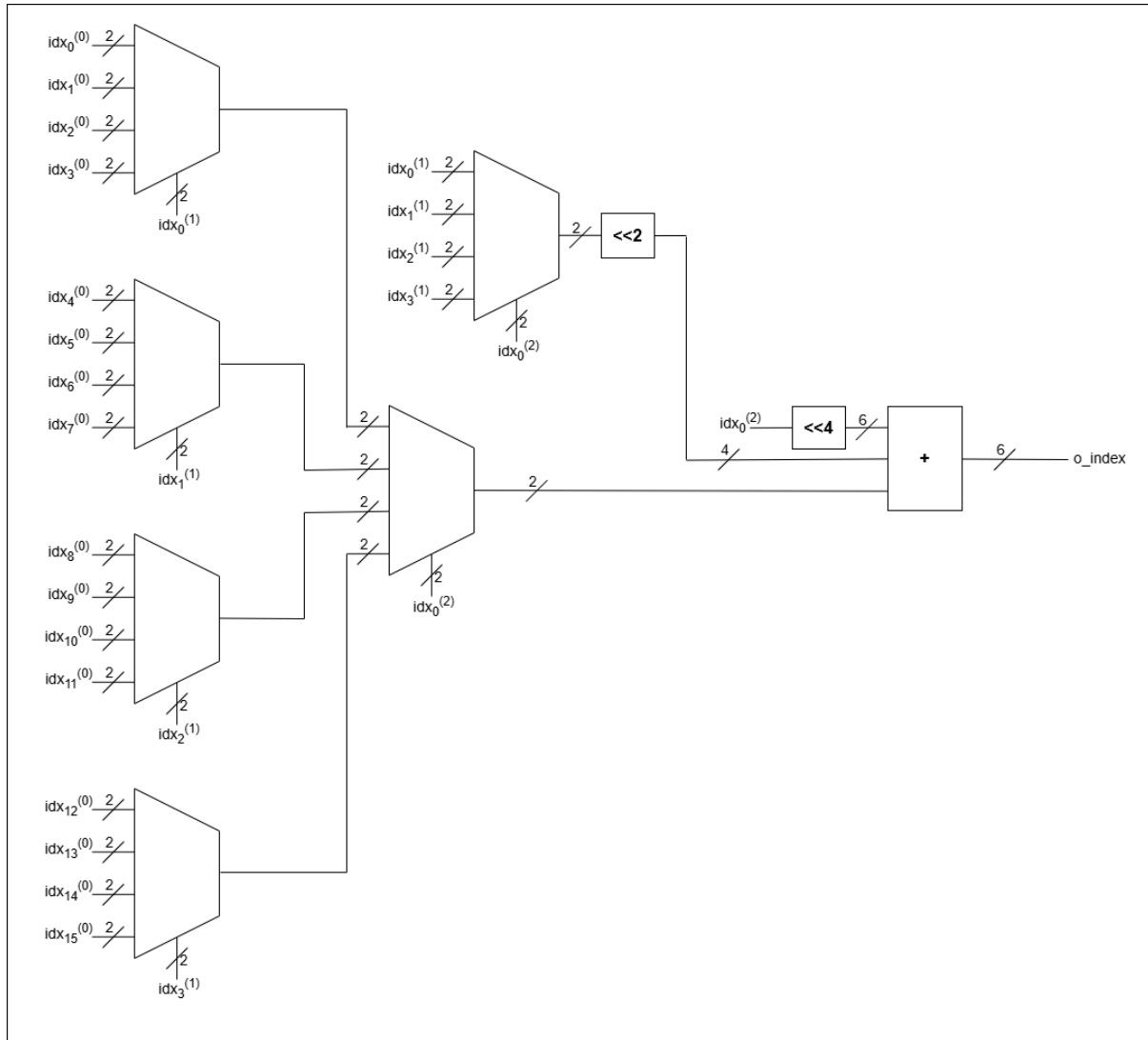


Figure 69: Priority encoder index decoding logic

Where  $idx$  denotes the output index ( $o\_index$ ) of each priority encoder in the tree. The superscript denotes the level of the priority encoder in the tree where 2 denotes the top priority encoder and 0 denotes the priority encoders at the bottom of the tree. The subscript indicates the number of the priority encoder in the level where bottom level has 16 priority encoders, second level has 4 priority encoders and top level has one.

### Divider

The divider maps two 6-bit unsigned operands to a Q0.16 quotient. Since the operand range is small, the quotient is precomputed offline and stored in a  $62 \times 62$  lookup table instead of being computed by a general-purpose 16-bit divider. This reduces the divider latency at the cost of a modest increase in storage. If the dividend is greater than the divisor, the output is forced to zero.

## RTL Parameters

The parameters of the TDC\_CORE\_TOP (TDC engine) module are listed in Table 8.

| Name                  | Description                                      | Value |
|-----------------------|--------------------------------------------------|-------|
| FRACTIONAL_BIT]_WIDTH | Number of bits in fractional part of o_var_phase | 16    |
| INTEGER_BIT_WIDTH     | Number of bits in integer part in o_var_phase    | 24    |
| TAP_COUNT             | Number of taps in tapped delay line              | 62    |
| DIVIDEND_SIZE         | Number of possible dividends in divider          | 62    |
| DIVISOR_SIZE          | Number of possible divisors in divider           | 62    |

Table 8: TDC Engine Parameters Description

The bit-width of the fractional part of the variable phase output was chosen to satisfy the minimum frequency deviation requirement. The minimum number of fractional bits necessary to achieve a frequency deviation of 185 kHz with a 32 MHz reference clock is calculated as:

$$\left\lceil \log_2 \left( \frac{32 \text{ MHz}}{185 \text{ kHz}} \right) \right\rceil = 8$$

However, as the value of the proportional path in the loop filter is equal to  $2^{-6}$  in tracking mode then 14 fractional bits will be required to avoid losing frequency information. To minimize quantization noise and ensure the DCO maintains a sufficient resolution for fine tuning, the fractional bit-width is extended to 16 bits.

The width of the integer portion of the variable phase is chosen to prevent overflow of the phase accumulator, as overflow would introduce nonlinearity into the ADPLL. The required width is determined from the maximum duration for which the TDC remains active during packet transmission. Based on the current system simulations, the TDC operates for up to  $1\mu\text{s}$  during PVT calibration,  $1\mu\text{s}$  during acquisition, and  $20\mu\text{s}$  during tracking, resulting in a maximum active interval of  $22\mu\text{s}$ . To account for frequency offsets and modulation, a conservative maximum DCO frequency of 2.49 GHz as well active interval of  $30\mu\text{s}$  is assumed. Therefore, the maximum accumulated phase is:

$$\frac{2.49 \times 10^9}{32 \times 10^6} \times \frac{150 \times 10^{-6}}{0.03125 \times 10^{-6}} = 74,700$$

which means the number of bits required is:

$$\lceil \log_2(74,700) \rceil = 17$$

To provide additional design margin and accommodate future modifications to the timing budget, the integer portion of the variable phase is implemented using 24 bits.

The number of delay-line taps is selected based on the maximum CKV period, which is assumed to be 520ps, corresponding to a frequency of 1.923GHz. Assuming an average tap delay of 10ps, at least 52 taps are required to span one CKV period. An additional 10 taps are included as margin to ensure full cycle coverage and to capture both the rising and falling edges, resulting in a total of 62 taps.

## RTL Interface

The port descriptions of the TDC\_CORE\_TOP (TDC engine) module are listed in Table 9.

| Name               | Direction | Width | Source                                       | Destination                    | Description                                                                                                   |
|--------------------|-----------|-------|----------------------------------------------|--------------------------------|---------------------------------------------------------------------------------------------------------------|
| i_rst_n            | Input     | 1     | Control unit / system controller             | TDC engine                     | Active-low asynchronous reset.                                                                                |
| i_ckr_clk          | Input     | 1     | Crystal oscillator and DCO running registers | TDC registers                  | CKR sampling clock.                                                                                           |
| i_q_delay_line     | Input     | 62    | Tapped delay line                            | Edge detector                  | Captured pseudo-thermometer code.                                                                             |
| i_ckr_xor_ckr2     | Input     | 1     | TDC system                                   | TDC engine                     | CKR/CKR2 XOR used for error correction.                                                                       |
| i_dco_cycles_count | Input     | 24    | DCO counters                                 | TDC engine                     | DCO cycle count for the integer phase.                                                                        |
| o_var_phase        | Output    | 40    | TDC engine                                   | Phase error calculator         | Variable phase in Q24.16 format.                                                                              |
| o_valid            | Output    | 1     | TDC engine                                   | Phase error calculator         | Indicates that o_var_phase is valid.                                                                          |
| o_accumulator_en   | Output    | 1     | TDC engine                                   | DCO counters / FCW accumulator | Enables counters after the first CKR2 edge to align FCW accumulation. Also used to capture the initial phase. |

Table 9: TDC Engine Ports Description

i\_rst\_n is the global system active low reset signal ANDed with a control unit phase detector reset signal. The phase detector reset signal is used to reset the counters in the TDC as well as o\_var\_phase between mode transitions.

o\_accumulator\_en is asserted after the first CKR2 edge to enable the DCO and FCW accumulators only after the initial phase has been captured. This avoids an explicit integer-phase subtraction, and thus avoids having counters running before first CKR2 rising edge which reduces dynamic power, and keeps the FCW reference aligned with the variable-phase accumulation.

## Loop Filter

Digital loop filter is a key functional block in ADPLL that processes the phase error signal obtained from TDC. It filters the phase error to suppress noise and generates the DCO tuning word required to achieve desired output frequency. The loop filter controls critical system parameters including loop bandwidth, stability, lock time, transient behavior and noise suppression. Replacing the analog loop filter used in traditional PLLs with digital loop filter provides significant advantages in the system, mainly programmability through selecting filter coefficients, compatibility with deep-submicron CMOS processes and reduced filter area. The structure of digital loop filter determines the number of integrators in the loop and therefore ADPLL type, it is classified as Type I and Type II.

### Type I ADPLL

A Type I ADPLL is first-order loop, defined as having only one integration pole, from DCO frequency-to-phase conversion, it uses only a proportional gain ( $\alpha$ ). Due to absence of integrator the loop provides fast dynamics and relatively wide bandwidth, adding more integrators increase loop order bringing the system closer to instability and limits the bandwidth to keep it stable.

Type I ADPLL is ideal for fast acquisition applications. It change frequency rapidly but at the cost of less filtering effect (20 dB/decade out-of-band attenuation). The fundamental limitation of the Type I ADPLL, however, is the presence of steady-state frequency error, when the loop reaches lock, the phase error becomes zero preventing the loop from maintaining the required DCO tuning correction and leaving a residual frequency offset.

Block diagram of ADPLL with type I loop filter is shown in figure 70

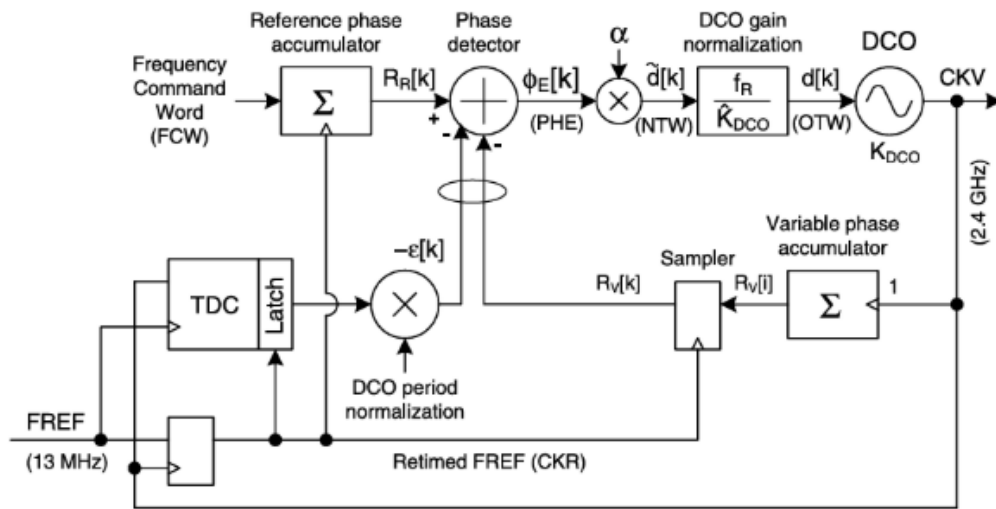


Figure 70: ADPLL with Type I loop filter

The control word of the type I loop filter is given by:

$$u[k] = \alpha \phi_E[k]$$

where  $\phi_E[k]$  is the phase error.

## Type II ADPLL

Type II ADPLL is a second order loop that includes an additional integrator in the system. This integrator is implemented in the digital loop filter which adopts a PI controller structure composed of proportional path and integral path.

The proportional path similar to that in Type-I ADPLL is responsible for fast loop dynamics and transient response. The integral path accumulates the phase error and scales it by integral loop gain ( $\rho$ ) to eliminate the steady state error that cannot be removed by proportional control alone, then proportional and integral paths are added.

Due to the presence of two integrators in the loop, the Type II ADPLL exhibits improved low-frequency noise suppression, providing up to 40 dB/decade attenuation and resulting in superior phase-noise performance compared to Type I architectures. For this reason, Type II ADPLLs are widely used in frequency-synthesizer applications where accurate frequency locking and low in-band phase noise are required.

Block diagram of ADPLL with type II loop filter is shown in figure 71

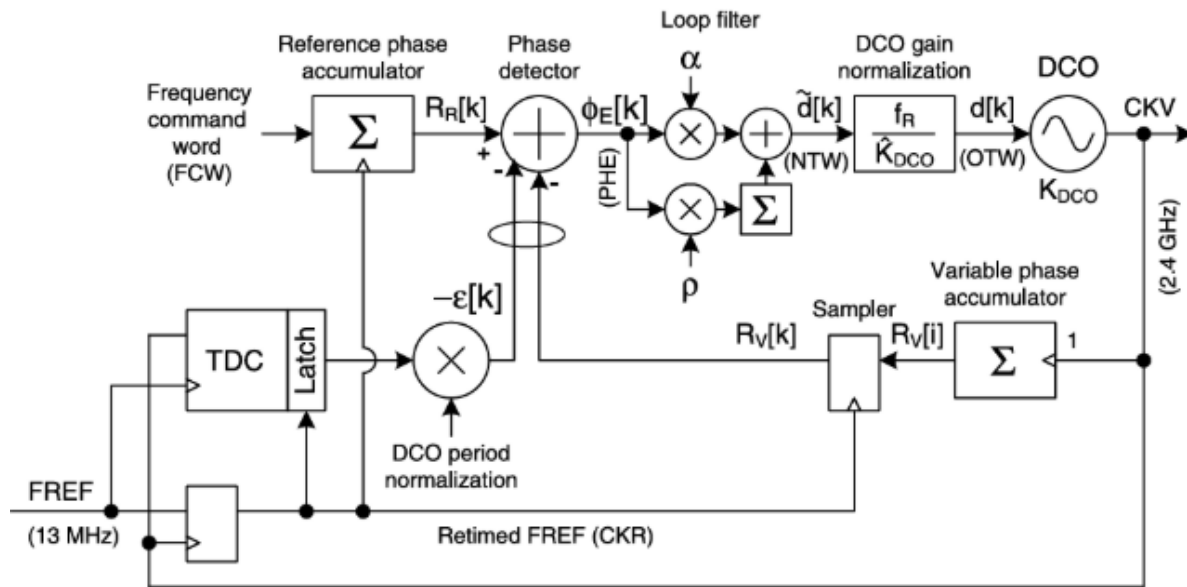


Figure 71: ADPLL with Type II loop filter

The control word of the type II loop filter is given by:

$$u[k] = \alpha \cdot \phi_E[k] + \rho \cdot \sum_{i=0}^{k-1} \phi_E[k]$$

where  $\phi_E[k]$  is the phase error.

## ADPLL Modes of Operation

The different operating modes of the ADPLL are shown in figure 72 below.

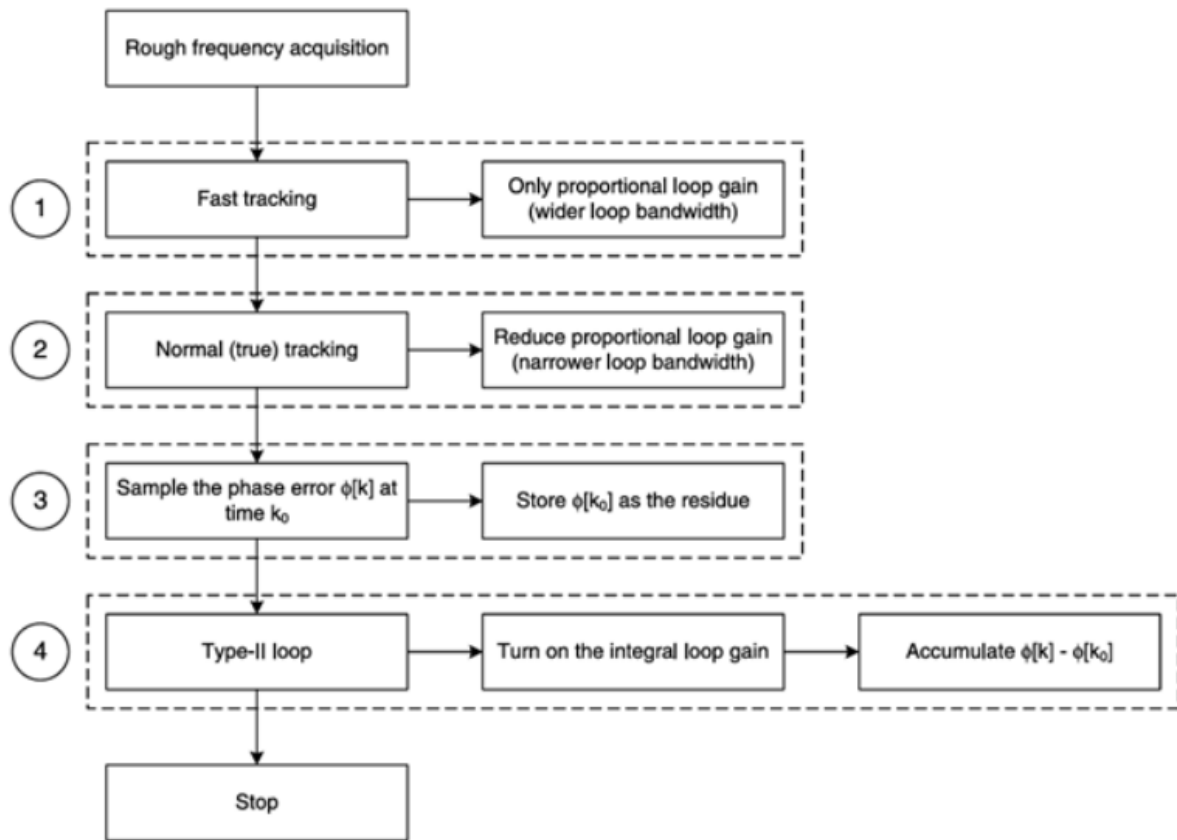


Figure 72: ADPLL different modes of operation

PI configuration introduces additional transient dynamics, in order not to degrade the switching time between different modes of ADPLL, the integral part is turned off during acquisition and fast tracking (acts as Type I) then turned back on in normal tracking mode. In this way a fast transient loop characteristic is used during acquisition, and a slower transient, but with a better filtering capability, is used during regular tracking.

The proportional gain is large in fast tracking mode for fast response and less settling time with wide bandwidth at the expense of enhanced phase noise and spurs. In normal tracking mode where actual transmitting and receiving is done, the loop bandwidth and error eliminating are now of much greater importance so loop bandwidth has to be kept lower to minimize phase noise and spurs, proportional gain is decreased, phase error is then sampled and stored, then the integral path is on with its internal accumulator set initially to zero.

The residual DC offset of the preceding mode would now be considered an undesired phase-error bias which might take a long time to work out. A solution is to subtract this bias from the phase error before the accumulation operation.

### Proposed Hitless Mode transition Scheme

While the conventional literature suggests sampling and storing the phase error residue  $\phi[k_0]$  to compute the offset  $\phi[k] - \phi[k_0]$  as shown in Figure 72, this requires dedicated storage registers and subtraction arithmetic. In this work, a more hardware-efficient approach is proposed. To mitigate transient instability and eliminate frequency overshoot at the mode boundary due to changing the filter coefficients, instead of storing and subtracting the residue, the control unit introduces a **single-cycle freeze state**. During this cycle, the Digitally Controlled Oscillator (DCO) preserves its previous frequency tuning word to prevent any frequency shoot, while the phase accumulators and loop filter integrator are reset to zero. Closed loop tracking then resumes under the updated proportional and integral loop gains, successfully preserving frequency continuity.

### Type I ADPLL frequency response

Since the loop filter is realized here as simple proportional gain, and the loop is first order. The proportional gain is the only control variable in the loop which controls the system bandwidth and settling time.

From the previous linear model analysis parameters in sub-section 26, we get that for  $r = 1$ , the loop is stable for alpha:

$$\alpha < 1$$

With magnitude response can be plotted with alpha variation as shown in Figure 73:

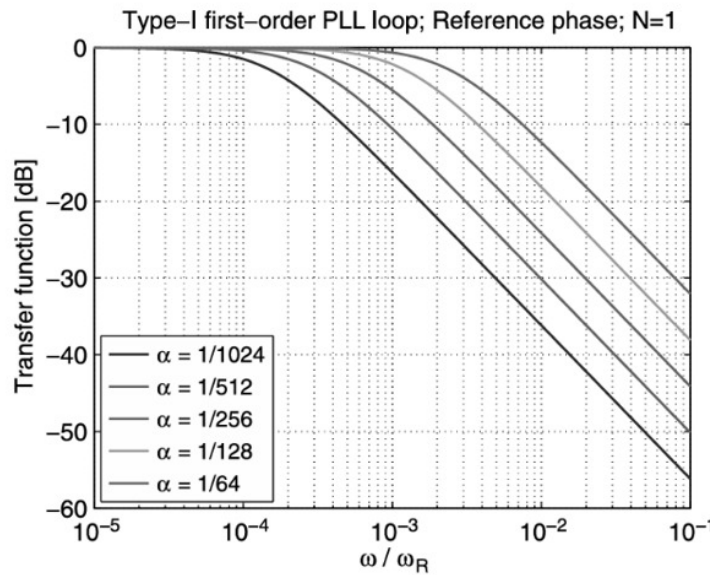


Figure 73: ADPLL type I frequency response

The primary objective of the Type-I loop is to perform coarse frequency acquisition and PVT calibration by reducing the large initial frequency offset between the dco output and the desired channel frequency.

The proportional gain alpha directly controls the convergence characteristics of this calibration process (stability, bandwidth, and settling behavior).

Therefor alpha must be chosen to control the trade-off between settling speed and stability, while ensuring reliable convergence of the ADPLL.

### Type II ADPLL frequency response

Type II ADPLL has a higher loop order due to additional integrator, therefore it requires careful selection of loop gain parameters to ensure stability.

From type II  $s$ -domain analysis discussed before in section (), we concluded that the damping coefficient ( $\zeta$ ) and natural frequency ( $\omega_n$ ) of the second order loop are:

Damping coefficient:

$$\zeta = \frac{1}{2} \left( \frac{\alpha}{\sqrt{\rho}} \right)$$

Natural frequency:

$$\omega_n = \sqrt{\rho} f_r$$

The following figure 74 shows the effect of damping coefficient  $\zeta$  on the loop:

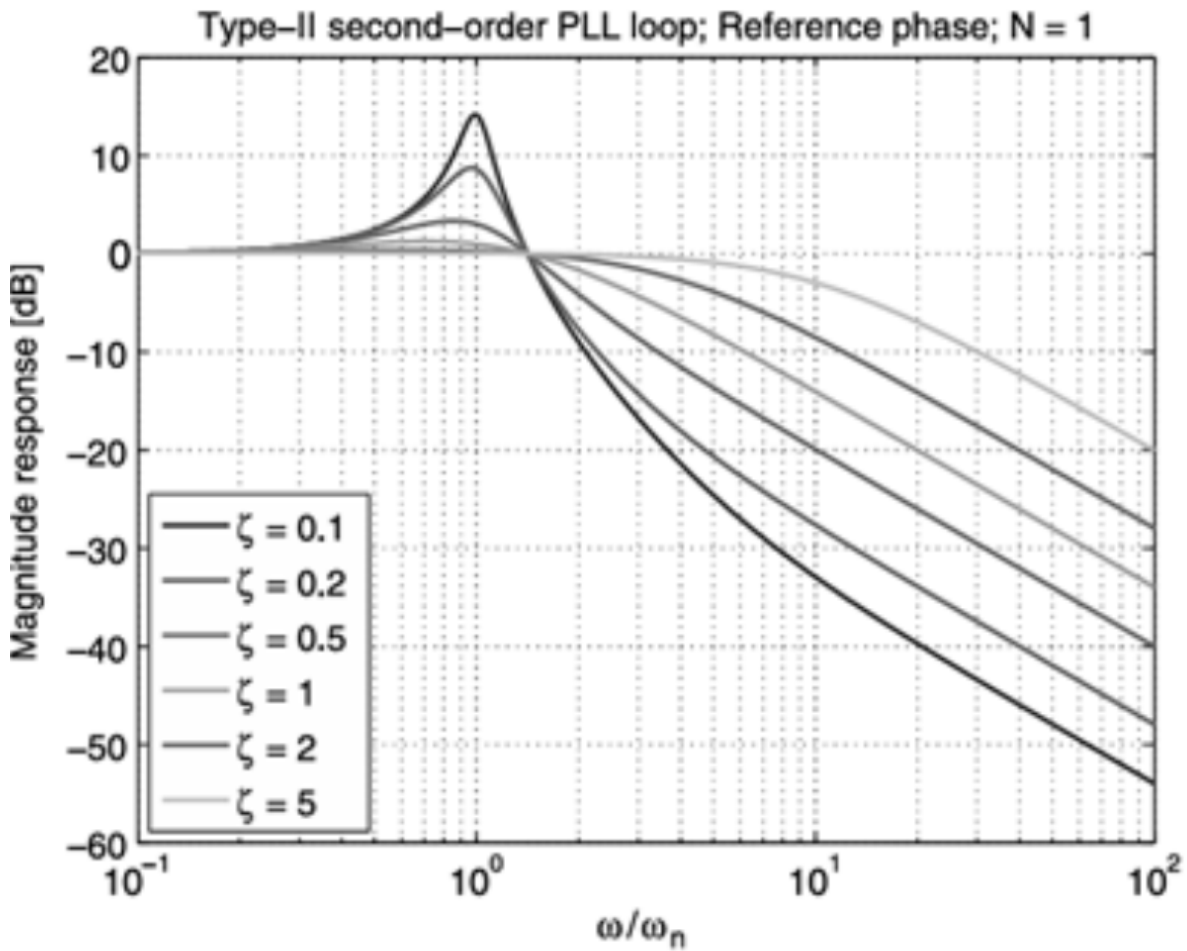


Figure 74: Closed loop response of ADPLL for different  $\zeta$  values

For system to be stable and exhibits acceptable peaking damping coefficient needs to be:

$$\zeta \geq \frac{1}{\sqrt{2}}$$

As damping coefficient increases the system becomes overdamped resulting in reduced peaking and slower transient response.

A lower damping factor leads to under damped response which introduces peaking in the frequency response.

Changing the loop proportional gain  $\alpha$  gives large flexibility in adjusting the loop noise performance, the following figure 75 illustrates the closed-loop transfer function for different values of  $\alpha$  with constant  $\zeta = 0.25$ .

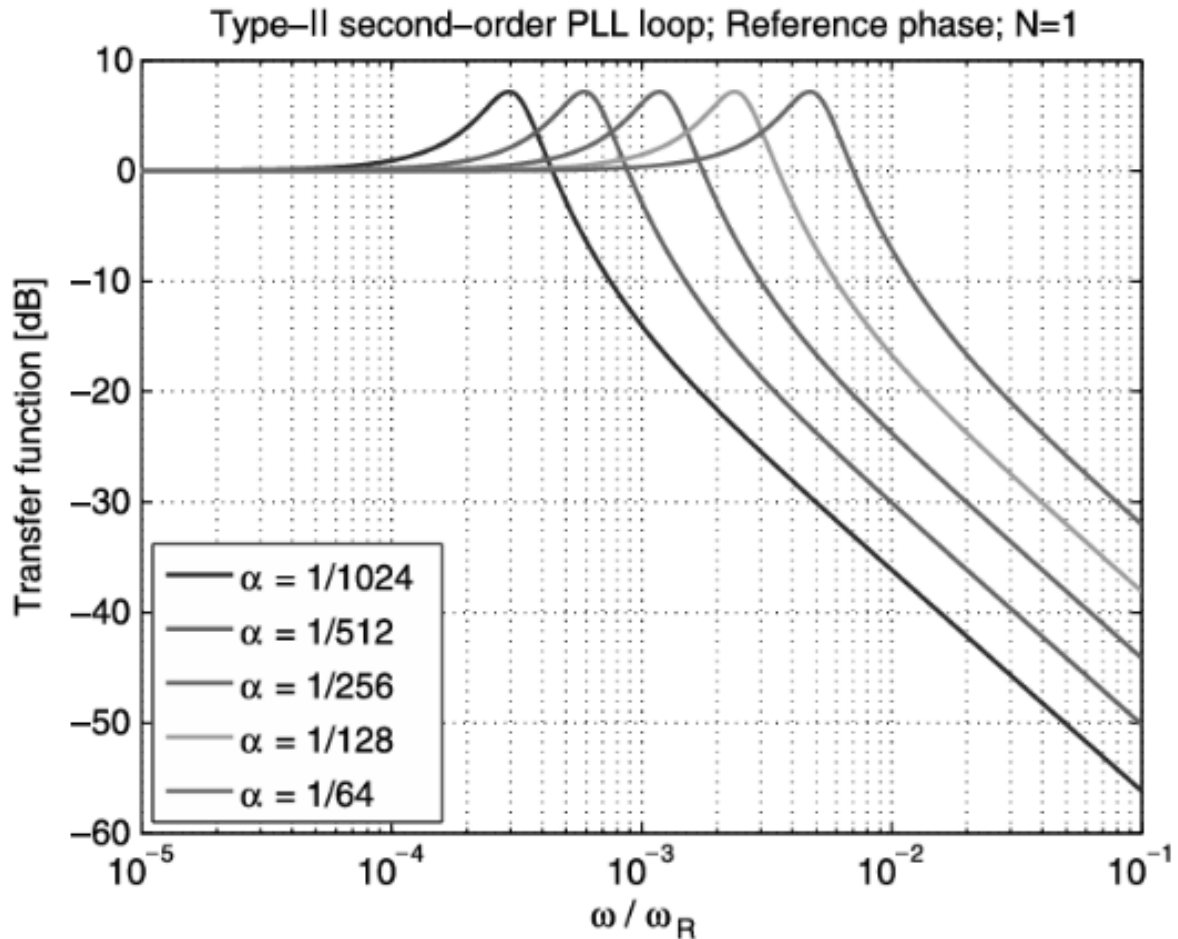


Figure 75: Closed loop response of ADPLL for different  $\alpha$  values,  $\zeta = 0.25$

From figure 75 we can see that  $\alpha$  has direct effect on loop bandwidth, this relationship highlights a fundamental tradeoff in the system:

A larger  $\alpha$  gives a wider bandwidth and faster transient response, allowing for quicker lock times. However, this comes at the cost of reduced noise suppression and a larger damping coefficient ( $\zeta$ ), potentially leading to an overdamped response. Conversely, a smaller  $\alpha$  narrows the loop bandwidth, which significantly improves the filtering of quantization noise and reference spurs, but results in a slower settling time and a potentially under-damped response. Therefore, proportional gain must be carefully selected to achieve frequency response that satisfies BLE specifications while providing an optimal tradeoff between fast settling behavior and effective noise filtering.

## RTL Block Diagram

The abstract block diagram of the digital loop filter is presented in Figure 76.

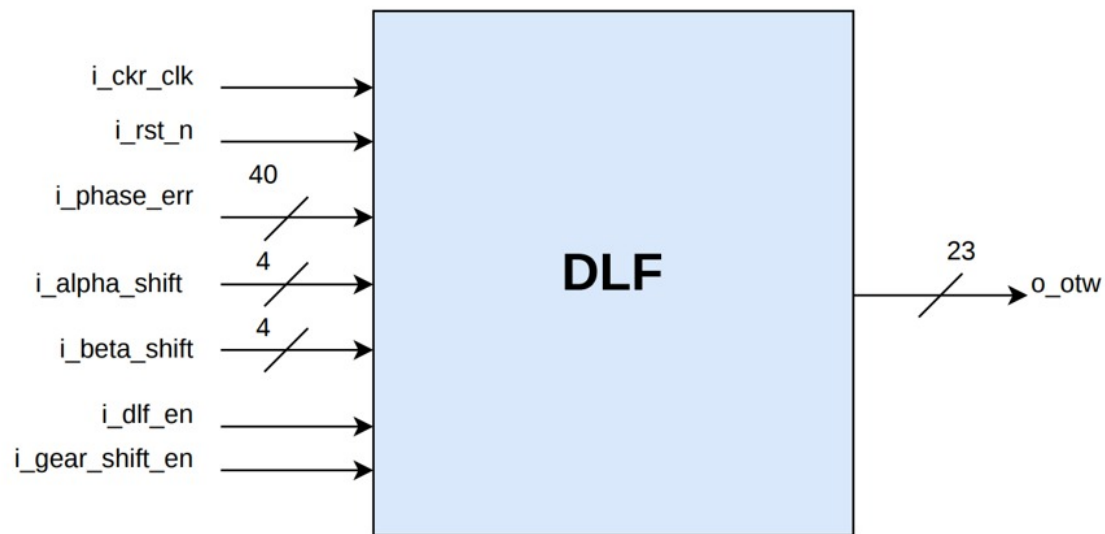


Figure 76: Digital loop filter abstract diagram

A more detailed gate level block diagram of the digital loop filter is shown in Figure 77.

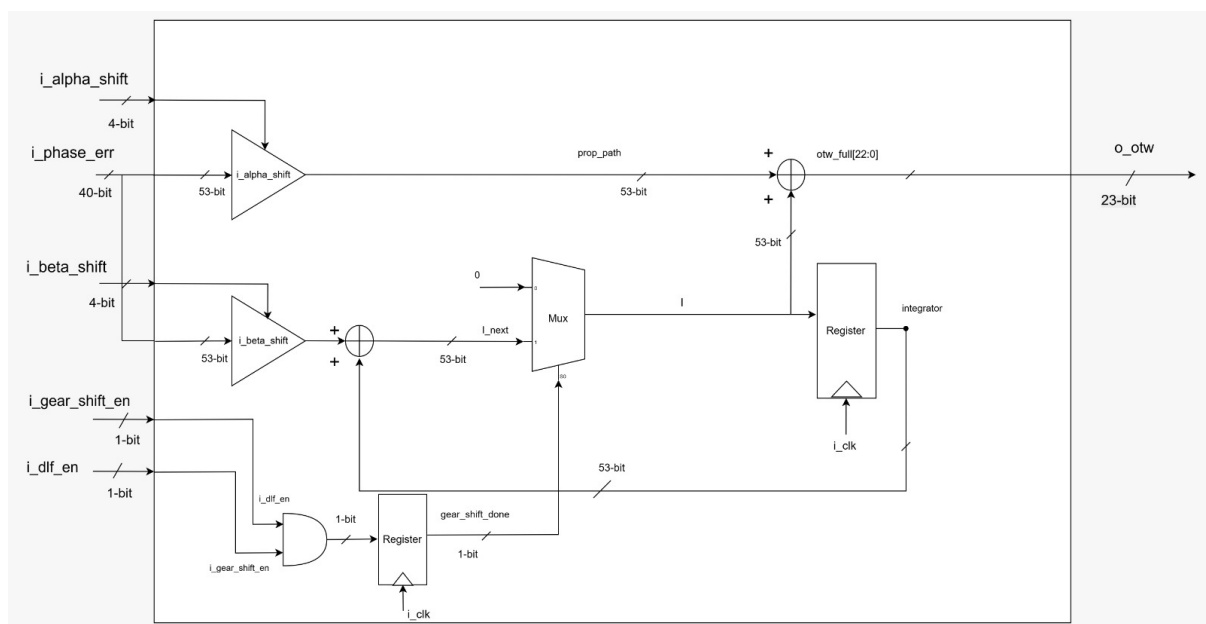


Figure 77: Digital loop filter block diagram

## RTL Interface

The port descriptions of the digital loop filter implementation are listed in Table 10.

| Name            | Direction | Width | Description                                                           |
|-----------------|-----------|-------|-----------------------------------------------------------------------|
| i_ckr_clk       | Input     | 1     | Input clock ckr (retimed reference clock).                            |
| i_rst_n         | Input     | 1     | Active low asynchronous reset.                                        |
| i_dlf_en        | Input     | 1     | Active high digital loop filter enable.                               |
| i_gear_shift_en | Input     | 1     | Gear shift control signal. High at the start of normal tracking mode. |
| i_alpha_shift   | Input     | 4     | Proportional gain as shift right value.                               |
| i_beta_shift    | Input     | 4     | Integral gain as shift right value.                                   |
| i_phase_err     | Input     | 40    | Input phase error of TDC.                                             |
| o_otw           | Output    | 23    | DCO tuning word.                                                      |

Table 10: Digital loop filter port list

## Sigma Delta

### Types of Sigma-Delta Modulators

Sigma-delta modulators can be implemented using several architectures, each offering different trade-offs between noise shaping performance, stability, spur behavior, and hardware complexity. The most common digital sigma-delta modulator architectures are MASH sigma delta and you can see some comparison between some of the sigma delta from [17].

### MASH Architecture

The MASH (Multi-Stage Noise Shaping) sigma-delta architecture cascades multiple first-order stages and uses digital noise cancellation to achieve high-order noise shaping with a unity signal transfer function. This structure ensures inherent stability, low hardware complexity, and wide adoption in digital frequency synthesis applications.

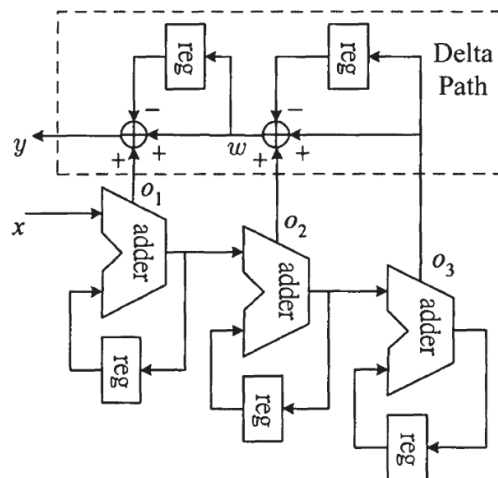


Figure 78: Block-level model of a MASH sigma-delta modulator form [17].

### First-Order MASH Sigma-Delta Analysis

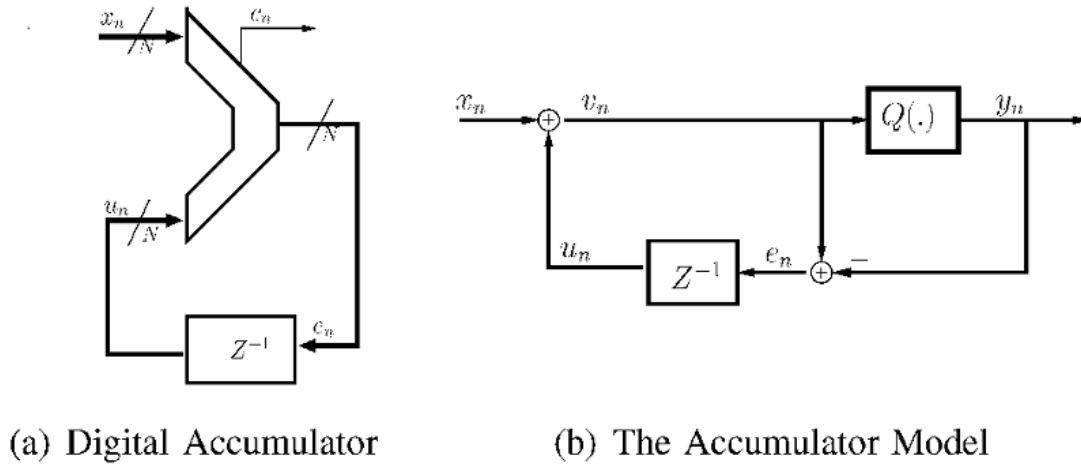


Figure 79: First-order MASH sigma-delta model and its digital implementation.

All this analysis are from [16] .

The first-order MASH sigma-delta modulator is equivalent to a digital Error-Feedback Modulator (EFM). The structure consists of a digital accumulator, comparator (quantizer), and a feedback loop.

The comparator output before normalization is

$$c_n = \begin{cases} 0, & x_n + u_n < 2^N, \\ 1, & x_n + u_n \geq 2^N. \end{cases} \quad (18)$$

After normalization, the quantizer input-output relationship becomes

$$y_n = \begin{cases} 0, & v_n < 1, \\ 1, & v_n \geq 1. \end{cases} \quad (19)$$

Before normalization,

$$x_n, u_n \in \{0, 1, 2, \dots, 2^N - 1\},$$

and after normalization,

$$x_n, u_n \in \left\{0, \frac{1}{2^N}, \frac{2}{2^N}, \dots, 1 - \frac{1}{2^N}\right\}.$$

The accumulator input satisfies

$$v_n \in \left\{0, \frac{1}{2^N}, \dots, 1, \dots, 2 - \frac{1}{2^{N-1}}\right\}.$$

The accumulator update is

$$u_n = \begin{cases} e_{n-1}, & n \geq 1, \\ u_0, & n = 0. \end{cases} \quad (20)$$

The quantization error is defined as

$$e_n = v_n - y_n.$$

Two cases arise:

**Case 1:** For  $v_n \geq 1$ ,

$$e_n = v_n - 1 = \langle v_n \rangle.$$

**Case 2:** For  $v_n < 1$ ,

$$e_n = v_n = \langle v_n \rangle.$$

Thus, for all  $n$ ,

$$e_n = \langle v_n \rangle. \quad (21)$$

Since  $v_n = x_n + u_n$ ,

$$e_n = \langle x_n + u_n \rangle. \quad (22)$$

Using the recursion  $u_n = e_{n-1}$ ,

$$e_n = \langle x_n + e_{n-1} \rangle. \quad (23)$$

- Expanding:

$$e_0 = \langle x_0 + u_0 \rangle, \quad (24)$$

$$e_1 = \langle x_1 + e_0 \rangle, \quad (25)$$

$$e_n = \langle x_n + e_{n-1} \rangle. \quad (26)$$

The closed-form expression is

$$e_n = \left\langle u_0 + \sum_{k=0}^n x_k \right\rangle. \quad (27)$$

**Special Case: Constant Input  $X$**

If  $x_n = X$ ,

$$e_n = \langle u_0 + (n + 1)X \rangle. \quad (28)$$

without the modulu the error will tend to be a straight line and this would not be effective but after the modulu this yields a modulo-1 ramp (sawtooth), characteristic of first-order noise shaping.

If there is a period of  $N_1$  for  $e$

$$e_n = e_{n+N_1} = \langle u_0 + (n + N_1 + 1)X \rangle = \langle u_0 + (n + 1)X + N_1X \rangle. \quad (29)$$

then if  $N_1 X$  is an integer the error will be periodic which will cause idle tones in the frequency domain and this affects the spectrum. The case  $Nx$  is a non-integer we will not have the periodic behavior (for example  $x = (\sqrt{2}/10)$ )

## Higher Order Sigma Delta Modulators

Higher-order MASH sigma-delta modulators are constructed by cascading first-order EFMs. Each stage shapes the quantization noise of the previous stage, resulting in high-order noise shaping.

**Stage 1:**

$$e_{1n} = \langle u_1 + (n + 1)X \rangle. \quad (30)$$

**Stage 2:**

$$e_{2n} = \left\langle u_2 + \sum_{k=0}^n e_{1k} \right\rangle. \quad (31)$$

**General  $L$ -th stage:**

$$e_{Ln} = \left\langle u_L + \sum_{k=0}^n e_{(L-1)k} \right\rangle. \quad (32)$$

**Numerical Example (2-Stage MASH,  $X = 0.25$ )**

**Normalized ( $M = 1$ ) Linear Model of the First-Order EFM**

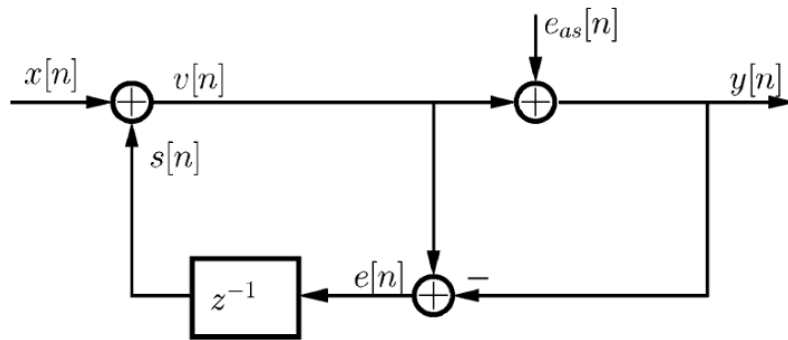


Figure 80: Linear model of a first-order digital EFM from [16]

A normalized ( $M = 1$ ) linear model is used to analyze the first-order error-feedback modulator (EFM). The quantizer is replaced by an additive quantization noise source  $E_{as}(z)$ , enabling linear  $z$ -domain analysis.

From the block diagram, the quantizer input is

$$V(z) = X(z) + z^{-1}E(z), \quad (33)$$

and the quantizer output is modeled as

$$Y(z) = V(z) + E_{as}(z). \quad (34)$$

The feedback error signal is therefore

$$E(z) = V(z) - Y(z) = -E_{as}(z). \quad (35)$$

Substituting into the input expression yields

$$V(z) = X(z) - z^{-1}E_{as}(z). \quad (36)$$

Consequently, the overall input–output relation becomes

$$\boxed{Y(z) = X(z) + (1 - z^{-1}) E_{as}(z)}. \quad (37)$$

Thus, the signal and noise transfer functions are

$$\text{STF}(z) = 1, \quad \text{NTF}(z) = 1 - z^{-1}, \quad (38)$$

indicating first-order noise shaping.

### Noise Cancellation in MASH Architectures

For a MASH modulator composed of cascaded first-order EFMs, the output of the  $k$ th stage is given by

$$Y_k(z) = X_k(z) + (1 - z^{-1}) E_k(z), \quad (39)$$

where the input to the next stage is

$$X_{k+1}(z) = -E_k(z). \quad (40)$$

For a third-order MASH (MASH 1-1-1), the stage outputs are

$$Y_1(z) = X(z) + (1 - z^{-1}) E_1(z), \quad (41)$$

$$Y_2(z) = -E_1(z) + (1 - z^{-1}) E_2(z), \quad (42)$$

$$Y_3(z) = -E_2(z) + (1 - z^{-1}) E_3(z). \quad (43)$$

The digital noise-cancellation network forms

$$Y(z) = Y_1(z) + (1 - z^{-1}) Y_2(z) + (1 - z^{-1})^2 Y_3(z). \quad (44)$$

Substitution shows exact cancellation of  $E_1(z)$  and  $E_2(z)$ , yielding

$$Y(z) = X(z) + (1 - z^{-1})^3 E_3(z). \quad (45)$$

More generally, for an  $L$ th-order MASH modulator,

$$Y(z) = X(z) + (1 - z^{-1})^L E_L(z), \quad (46)$$

demonstrating that all intermediate quantization noise terms are eliminated, while the final-stage noise is shaped to order  $L$ .

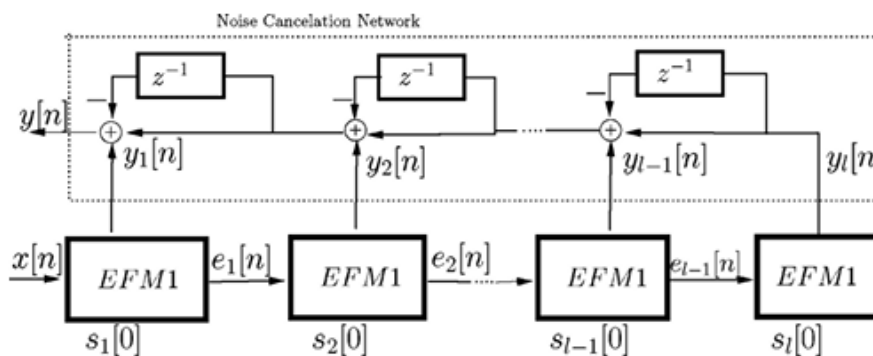


Figure 81: Linear noise cancellation mechanism in an  $L$ th-order MASH modulator from [16]

## Modified Sigma-Delta Modulator

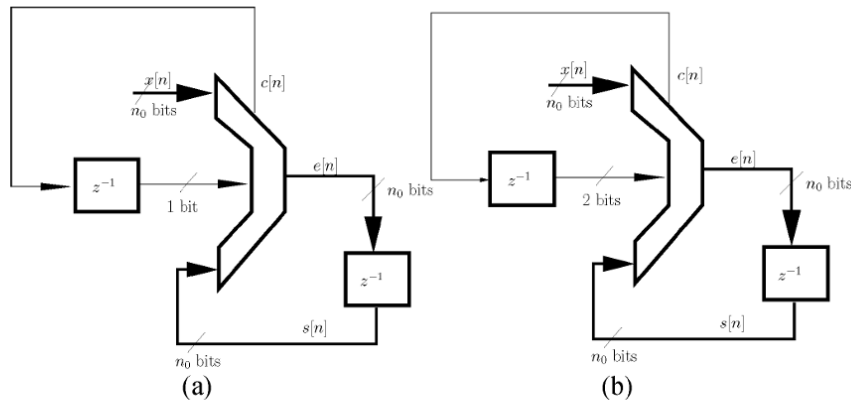


Fig. 11. Digital implementation of the modified EFM (a)  $a = 1$  and (b)  $a = 3$ .

Figure 82: Noise shaping behavior of the implemented modified sigma-delta modulator from [15].

The modified first-order error-feedback sigma-delta modulator forms the basic building block of the MASH 1-1-1 architecture. An additional feedback path scaled by the coefficient  $\alpha$  is introduced to guarantee maximum sequence-length operation for all constant inputs.

Using a linear additive-noise model for the quantizer, the  $z$ -domain output of the modulator is

$$Y(z) = \frac{1}{1 - \alpha z^{-1}} X(z) + \frac{1 - z^{-1}}{1 - \alpha z^{-1}} E_{as}(z). \quad (47)$$

The corresponding signal and noise transfer functions are

$$\text{STF}(z) = \frac{1}{1 - \alpha z^{-1}}, \quad \text{NTF}(z) = \frac{1 - z^{-1}}{1 - \alpha z^{-1}}. \quad (48)$$

The modified structure preserves first-order noise shaping while introducing a pole at  $z = \alpha$ , enabling its use in higher-order MASH configurations. The resulting sequence length is

$$N = 2^{n_0} - a = M - a, \quad (49)$$

which eliminates the need for dithering. A systematic output overestimation is observed and discussed in the simulation results. The mathematical proof and more details about modified sigma delta are found in [15].

### Linear Model of the proposed Sigma delta

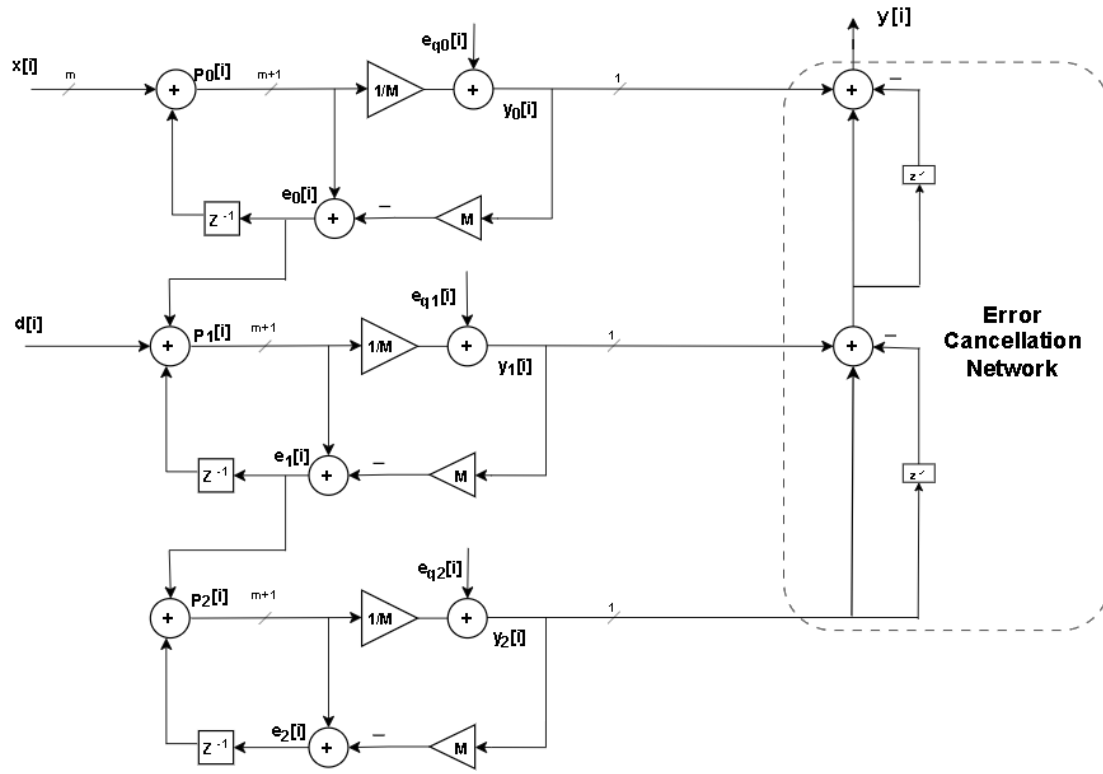


Figure 83: linear model of the modified dither MASH 1–1–1 sigma–delta modulator.

### Transfer Function Derivation of the Modified MASH Architecture

In this section, the transfer function of the proposed modified MASH architecture is derived using a linear model. The quantizer is modeled using the modulo relation

$$e_{qk}[i] = -e_k[i], \quad k \in \{0, 1, 2\}, \quad (50)$$

such that the quantizer output is expressed as

$$y_k[i] = p_k[i] - e_k[i]. \quad (51)$$

Only explicit delay elements  $z^{-1}$  introduce memory in the system.

#### Stage 0

The first stage is described by

$$\begin{aligned} p_0[i] &= x[i] + e_0[i-1], \\ y_0[i] &= \frac{1}{M}p_0[i] + e_{q0}[i]. \end{aligned} \quad (52)$$

Using the relation  $e_0[i] = -Me_{q0}[i]$ , the internal signal can be written as

$$p_0[i] = x[i] - Me_{q0}[i-1]. \quad (53)$$

Substituting into the output equation yields

$$y_0[i] = \frac{1}{M}x[i] - e_{q0}[i-1] + e_{q0}[i]. \quad (54)$$

Taking the  $z$ -transform gives

$$Y_0(z) = \frac{1}{M}X(z) + (1 - z^{-1})E_{q0}(z) \quad (55)$$

### Stage 1

The second stage includes additive dither and delayed feedback:

$$\begin{aligned} p_1[i] &= e_0[i] + d[i] + e_1[i-1], \\ y_1[i] &= \frac{1}{M}p_1[i] + e_{q1}[i]. \end{aligned} \quad (56)$$

Substituting  $e_0[i] = -Me_{q0}[i]$  and  $e_1[i-1] = -Me_{q1}[i-1]$  gives

$$p_1[i] = -Me_{q0}[i] + d[i] - Me_{q1}[i-1]. \quad (57)$$

Thus, the output of the second stage is

$$y_1[i] = -e_{q0}[i] + \frac{1}{M}d[i] - e_{q1}[i-1] + e_{q1}[i]. \quad (58)$$

In the  $z$ -domain,

$$Y_1(z) = -E_{q0}(z) + \frac{1}{M}D(z) + (1 - z^{-1})E_{q1}(z) \quad (59)$$

### Stage 2 (Modified Structure)

Unlike the previous architecture, the third stage no longer includes the feedforward path from  $y_0[i]$ . Its input consists only of the quantization error from the second stage and the delayed local feedback:

$$\begin{aligned} p_2[i] &= e_1[i] + e_2[i-1], \\ y_2[i] &= \frac{1}{M}p_2[i] + e_{q2}[i]. \end{aligned} \quad (60)$$

Using

$$e_1[i] = -Me_{q1}[i], \quad e_2[i-1] = -Me_{q2}[i-1], \quad (61)$$

gives

$$p_2[i] = -Me_{q1}[i] - Me_{q2}[i-1]. \quad (62)$$

Therefore,

$$y_2[i] = -e_{q1}[i] - e_{q2}[i-1] + e_{q2}[i]. \quad (63)$$

Taking the  $z$ -transform yields

$$Y_2(z) = -E_{q1}(z) + (1 - z^{-1})E_{q2}(z) \quad (64)$$

## Digital Noise Cancellation Network

To match the pipelined hardware implementation, two delay elements are inserted after the first-stage output and one delay element is inserted after the second-stage output. Consequently, the overall output of the modulator is obtained as

$$Y(z) = z^{-2}Y_0(z) + z^{-1}(1 - z^{-1})Y_1(z) + (1 - z^{-1})^2Y_2(z) \quad (65)$$

## Overall Transfer Function

Substituting the expressions of  $Y_0(z)$ ,  $Y_1(z)$ , and  $Y_2(z)$  into the modified cancellation network gives

$$\begin{aligned} Y(z) = & z^{-2} \left[ \frac{1}{M}X(z) + (1 - z^{-1})E_{q0}(z) \right] \\ & + z^{-1}(1 - z^{-1}) \left[ -E_{q0}(z) + \frac{1}{M}D(z) + (1 - z^{-1})E_{q1}(z) \right] \\ & + (1 - z^{-1})^2 \left[ -E_{q1}(z) + (1 - z^{-1})E_{q2}(z) \right]. \end{aligned} \quad (66)$$

Expanding the above expression yields

$$\begin{aligned} Y(z) = & \frac{1}{M}z^{-2}X(z) + \frac{1}{M}z^{-1}(1 - z^{-1})D(z) \\ & + [z^{-2}(1 - z^{-1}) - z^{-1}(1 - z^{-1})] E_{q0}(z) \\ & + [z^{-1}(1 - z^{-1})^2 - (1 - z^{-1})^2] E_{q1}(z) \\ & + (1 - z^{-1})^3 E_{q2}(z). \end{aligned} \quad (67)$$

Since the additional pipeline delays alter the exact timing alignment of the quantization noise components, perfect cancellation is no longer guaranteed. However, for analytical simplicity, the contributions associated with  $E_{q1}(z)$  and  $E_{q2}(z)$  are assumed to approximately cancel each other, resulting in an overall third-order noise shaping behavior. Under this approximation, the modulator output becomes

$$Y(z) \approx \frac{1}{M}z^{-2}X(z) + (1 - z^{-1})^3 E_q(z) \quad (68)$$

where  $E_q(z)$  represents the dominant residual quantization noise.

Therefore, the signal transfer function (STF) is

$$\text{STF}(z) = \frac{1}{M}z^{-2} \quad (69)$$

and the corresponding noise transfer function (NTF) is

$$\text{NTF}(z) = (1 - z^{-1})^3 \quad (70)$$

## Implementation of the proposed Sigma delta

### System Overview

The implemented system consists of an integrator, a quantizer, and a feedback loop. The modulator operates at an oversampling frequency  $f_s$  and produces a 3-bit output bitstream whose average value represents the input signal.

## Inputs and Outputs

### Input Signals

Table 11: Sigma-Delta Modulator (SD\_top) Interface Signals

| Name    | Direction | Bit Width | Source                  | Destination          | Description                                                                                                                      |
|---------|-----------|-----------|-------------------------|----------------------|----------------------------------------------------------------------------------------------------------------------------------|
| i_clk   | Input     | 1         | DCO Clock Divider (/4)  | SD_top               | Clock driving the sigma-delta modulator. The clock is generated from the DCO and divided by four before entering the module.     |
| i_rst_n | Input     | 1         | System Reset Controller | SD_top               | Active-low asynchronous reset used to initialize the internal registers and accumulator of the sigma-delta modulator.            |
| i_sd_en | Input     | 1         | Control Unit            | SD_top               | Enable signal for the sigma-delta modulator. The modulator operates only during the modulation phase (tracking mode of the DCO). |
| i_sd    | Input     | 9         | Digital Loop Filter     | SD_top               | Signed fixed-point input from the digital loop filter consisting of one sign bit and eight fractional bits.                      |
| o_sd    | Output    | 3         | SD_top                  | DCO Varactor Control | Signed 3-bit integer output used to control the DCO varactor bank for fractional frequency tuning.                               |

- **Fractional Input Signal  $X[n]$**

The input to the Sigma-Delta modulator is derived from the output of the loop filter. The loop filter output is decomposed into an integer part and a fractional part. The integer part is applied directly to the Digitally Controlled Oscillator (DCO), while the fractional part is processed by the Sigma-Delta modulator.

The fractional input signal  $X[n]$  represents a normalized value in the range:

$$0 \leq X[n] < 1$$

and is represented using a fixed-point format with a bit-width of  $K = 10$  bits.

- **Clock (CLK)**

System clock driving all synchronous elements of the Sigma-Delta modulator. The modulator operates on the rising edge of the clock.

- **Asynchronous Reset ( $\text{rst\_n}$ )**

Active-low asynchronous reset signal used to initialize the internal registers and integrator states of the modulator to a known condition.

### Output Signals

- **Sigma-Delta Output  $Y[n]$**

The Sigma-Delta modulator produces a 3-bit signed output representing the quantized fractional control signal. The output range is:

$$Y[n] \in [-4, 3]$$

The signal is encoded in signed binary format and is binary-weighted.

Since the Digitally Controlled Oscillator (DCO) capacitor bank is implemented using unit-weighted varactors, the binary-weighted output of the Sigma-Delta modulator cannot be directly applied to the capacitor array.

To enable proper control of the unit capacitor bank, a constant offset of +4 is added to the signed output:

$$Y_{\text{ctrl}}[n] = Y[n] + 4$$

This operation shifts the control word from the signed range  $[-4, 3]$  to the unsigned range:

$$Y_{\text{ctrl}}[n] \in [0, 7]$$

The resulting 3-bit unsigned control signal is then applied to the DCO varactor array, which is modeled as digitally controlled switches driving unit capacitors.

### Block Diagram

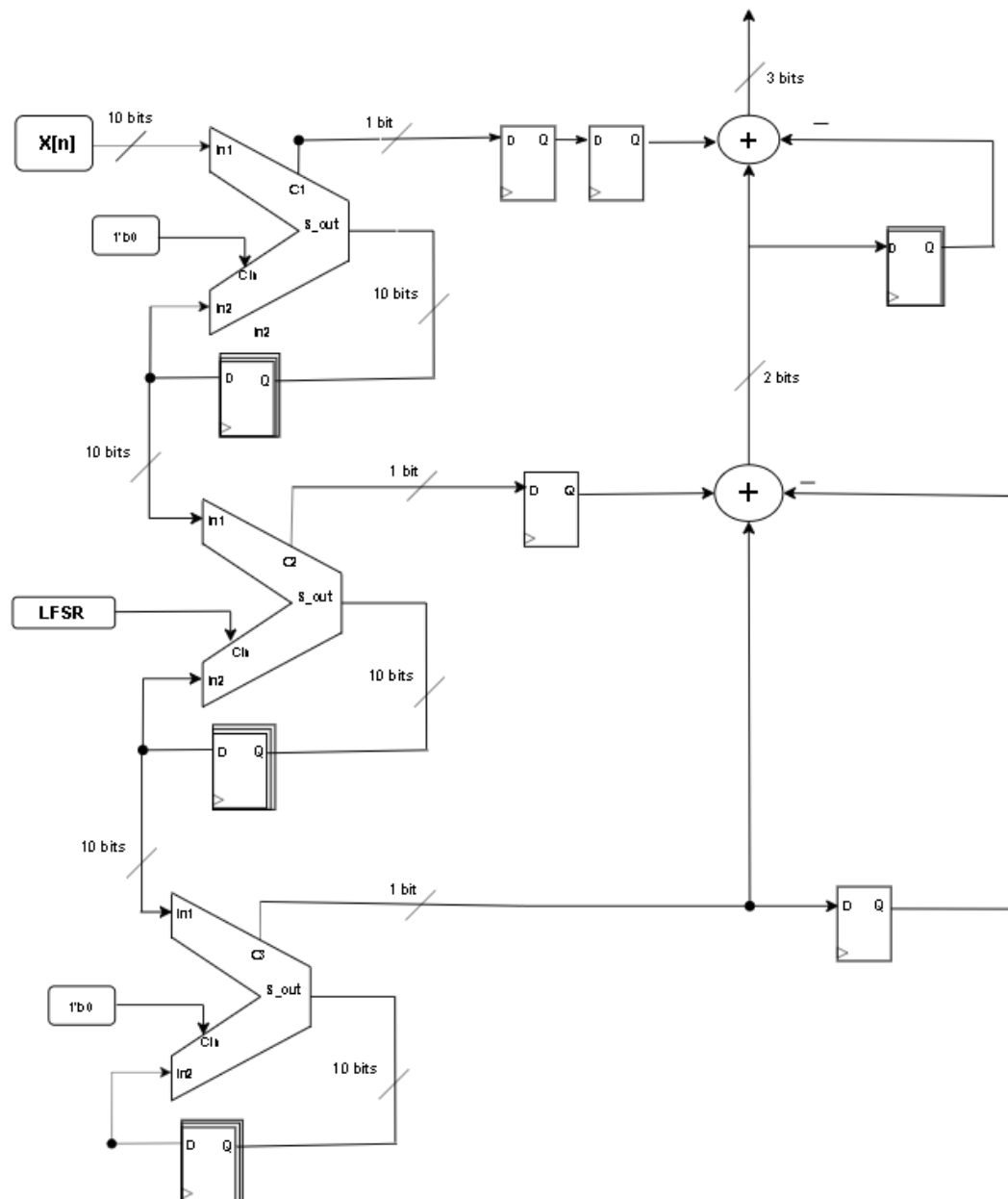


Figure 84: Block diagram

## DCO and Gain Estimation

### Introduction

A Digitally Controlled Oscillator (DCO) is an oscillator whose output frequency is controlled by a digital control word instead of an analog tuning voltage (VCO). In practice, this is achieved by digitally selecting discrete tuning elements such as capacitor units in LC-tank oscillators or by changing the drive strength in ring oscillators.

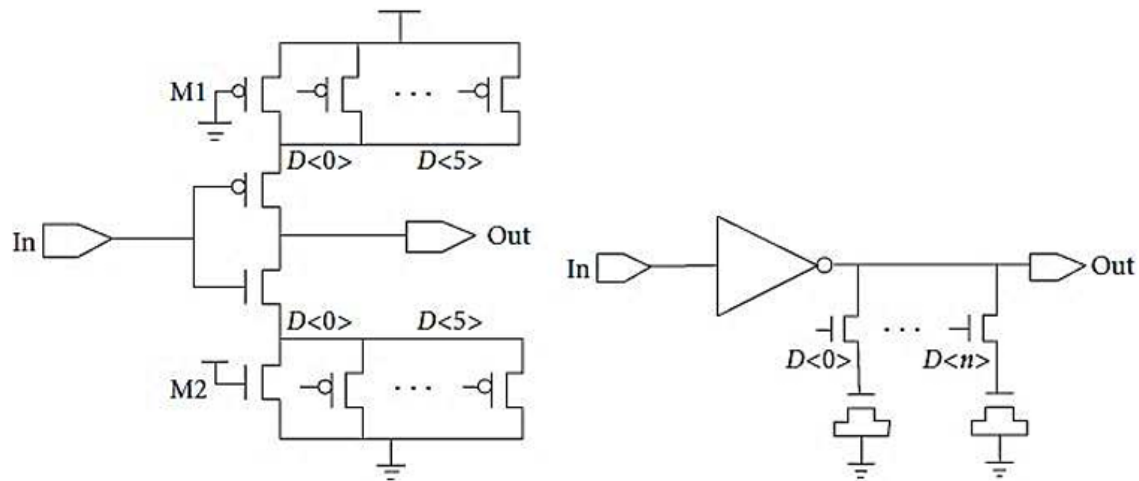


Figure 85: Drive strength control and shunt capacitance controlled standard cells used in DCO

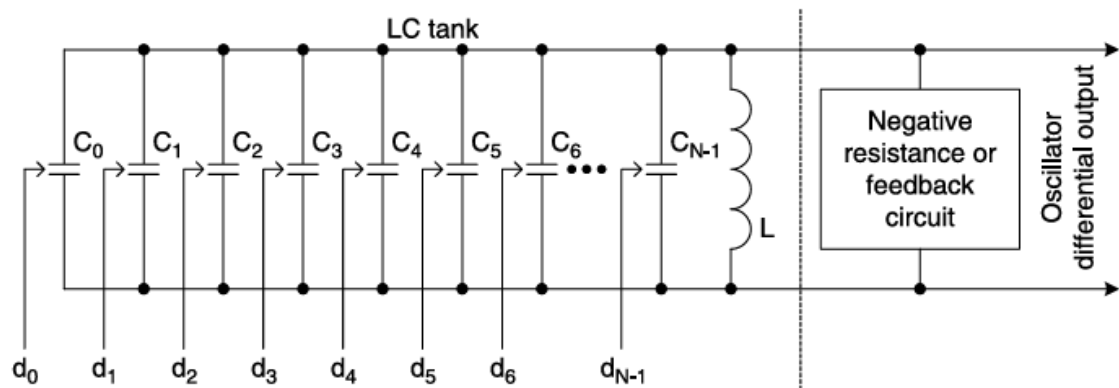


Figure 86: LC tank-based oscillator with switchable capacitors

In a DCO, there is a fundamental trade-off between **operating frequency**, **tuning range**, and **power consumption**. Increasing the tuning range typically requires adding more load capacitance to the LC tank, allowing coverage of more channels and wider PVT variations at the cost of higher power consumption.

## Frequency Control of Digital Oscillators

For an LC tank oscillator, the oscillation frequency is determined by:

$$f = \frac{1}{2\pi\sqrt{LC_k}} \quad (71)$$

The frequency can be controlled digitally by adjusting either  $L$  or  $C$  using a digital control word. In practice, controlling the capacitance is more convenient for 2.4 GHz operation at a 32 nm technology node.

For  $f = 2.4$  GHz, assuming  $L = 1$  nH and  $C = 4.4$  pF, the effective capacitance is expressed as:

$$C_k = C_0 + \sum_{k=0}^{N-1} d_k \cdot \Delta C_k \quad (72)$$

where:

- $C_0$  is the nominal capacitance that ideally resonates at 2.4 GHz when approximately half of the capacitor bank units are enabled, providing symmetric tuning margin around the center frequency,
- $d_k \in \{0, 1\}$  is the digital control bit selecting each capacitor unit,
- $\Delta C_k$  is the capacitance increment associated with the  $k$ -th unit, such that the sum  $\sum d_k \Delta C_k$  determines the target frequency within the tuning range.

The switched capacitor bank can be modeled using varactors, where  $C_0$  represents the total capacitance in the off state and  $\Delta C_k$  denotes the incremental capacitance between the on and off states of the  $k$ -th unit cell, as shown in Fig. 87.

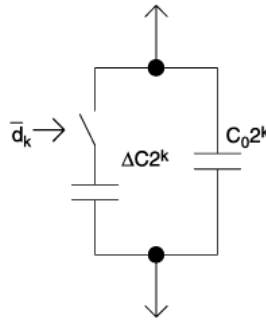


Figure 87: Modeling a binary-weighted switchable capacitor of index  $k$

## DCO Operational Modes

For small capacitance variations, a linear relationship can be approximated between the capacitance change and the resulting frequency change:

$$f_0 + \Delta f = \frac{1}{2\pi\sqrt{LC_0(1 + \Delta C/C_0)}} \approx f_0 \left(1 - \frac{\Delta C}{2C_0}\right) \Rightarrow \frac{\Delta f}{f_0} = -\frac{\Delta C}{2C_0} \quad (73)$$

At  $f_0 = 2.44$  GHz, with  $L = 1$  nH and  $C_0 = 4.26$  pF, achieving a frequency step of 1 kHz requires:

$$\frac{\Delta C}{C_0} = 8.3 \times 10^{-7} \Rightarrow \Delta C \approx 3.5 \text{ aF}$$

Achieving such matching accuracy in fabrication is extremely challenging and would require approximately

$$\log_2 \left( \frac{2.4 \times 10^9}{10^3} \right) \approx 22$$

bits of resolution. A practical approach is therefore to divide the ADPLL operation into multiple modes: a coarse-resolution mode for PVT calibration with large frequency steps, a medium-resolution mode for acquisition (BLE channel selection), and a fine-resolution tracking mode for modulation.

### PVT Calibration Mode

PVT variations can cause frequency deviation of approximately  $\pm 10\%$  around the nominal frequency.

#### Calculation for BLE

The frequency range due to PVT variation spans from 2.16 GHz to 2.64 GHz, corresponding to a range of approximately 500 MHz.

Using a frequency step of 2 MHz requires a **coarse control word of 8 bits**. Since nonlinearity in this mode is less critical, a binary-weighted capacitor bank can be used. This corresponds to a capacitance step of approximately  $\Delta C \approx 7.2$  fF.

### Acquisition Mode

Following PVT calibration, frequency uncertainty may still reach hundreds of MHz when selecting different BLE channels (40 channels spanning 2.4–2.480 GHz). The acquisition mode uses intermediate-resolution frequency steps to rapidly tune toward the target channel frequency.

A dynamic range of at least 84 MHz is required. Using a step size of 0.5 MHz with an 8-bit control word provides a dynamic range of 128 MHz, ensuring sufficient overlap between the PVT calibration mode and the acquisition mode. This corresponds to a capacitance step of approximately  $\Delta C \approx 1.8$  fF.

### Tracking Mode

Once the acquisition mode brings the DCO frequency within the lock range, the tracking mode is used to precisely align the carrier to the selected BLE channel and to apply the fine frequency deviations required for GFSK data modulation. In this mode, a unity-weight (equal-unit) capacitor bank is employed to achieve a highly linear frequency–code characteristic. Any nonlinearity in this region directly distorts the effective modulation index and increases spectral noise.

According to the BLE specification for the 2 Mb/s PHY, GFSK modulation requires a frequency deviation of approximately  $\pm 370$  kHz around the carrier. To compensate for the residual frequency error from the acquisition mode (approximately  $\pm 0.5$  MHz), the tracking mode must provide a usable dynamic range of about 1.74 MHz centered on the target channel frequency.

The tracking mode is divided into two regions. The integer part is directly connected to a capacitor bank, while the fractional part provides finer resolution using a sigma–delta modulator. The integer part uses a 7-bit control word with a frequency step of 25 kHz to cover the 1.74 MHz range. The fractional part uses 3 bits to cover an additional range of approximately 200 kHz.

This corresponds to a capacitance step of approximately  $\Delta C \approx 90.16$  aF for a 25 kHz frequency step.

## DCO Architecture and Implementation

### Normalized DCO (nDCO)

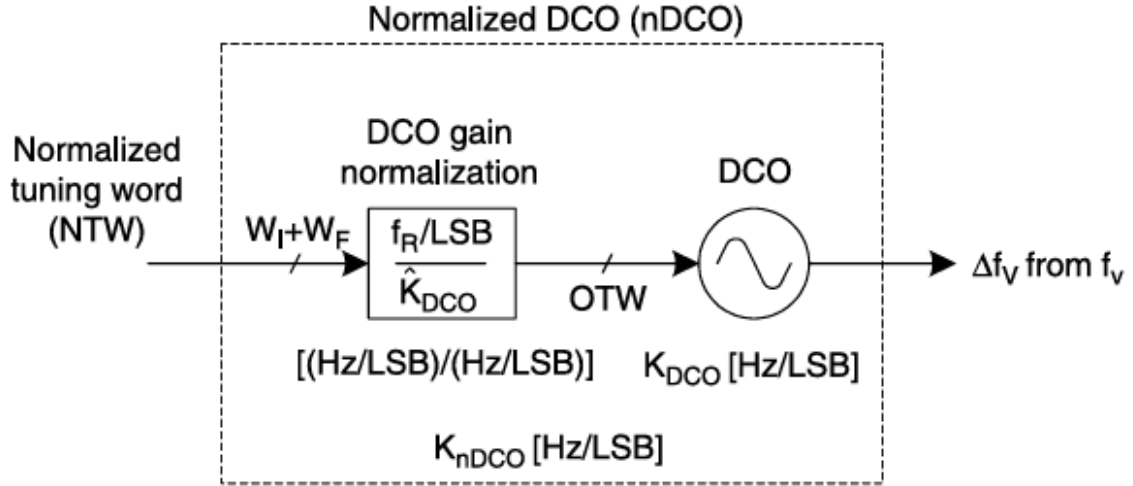


Figure 88: Normalized DCO hardware abstraction layer

$$f_v = f_0 + \Delta f_v = f_0 + K_{\text{DCO}} \cdot \text{OTW} \quad (74)$$

$K_{\text{DCO}}$  represents the frequency deviation corresponding to a one-LSB change in the oscillator tuning word.

Due to its analog nature, the gain  $K_{\text{DCO}}$  is subject to process, voltage, and temperature variations that cannot be precisely predicted. DCO gain normalization decouples the phase and frequency information from these variations by normalizing the phase error to the oscillator clock period.

The total normalized DCO gain is given by:

$$K_{\text{nDCO}} = \frac{F_R \cdot K_{\text{DCO}}}{\widehat{K_{\text{DCO}}}} = F_R \cdot r \quad (75)$$

For perfect gain estimation:

$$r = \frac{K_{\text{DCO}}}{\widehat{K_{\text{DCO}}}} = 1 \quad (76)$$

Sign-sign LMS is used for DCO gain calibration.

## DCO Interface

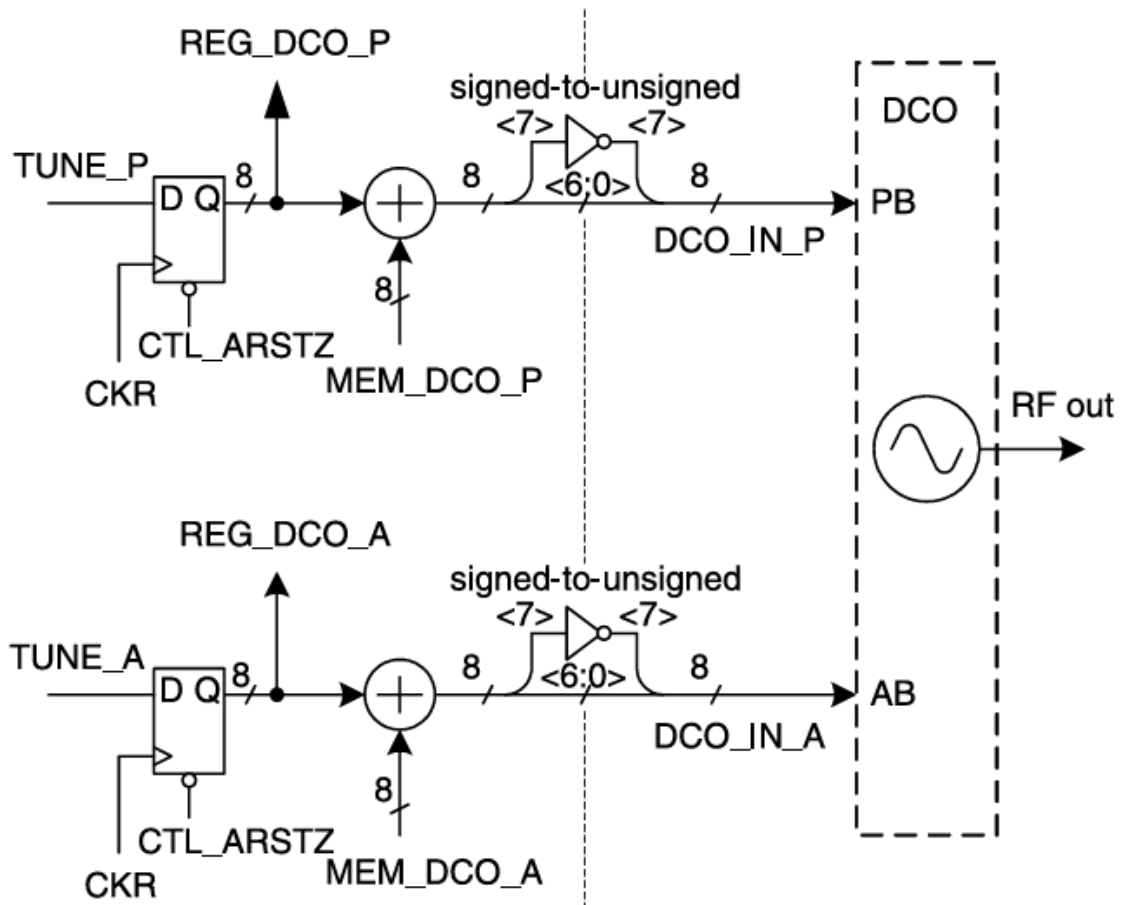


Figure 89: Control circuit of oscillator acquisition bits (OA)

**CTL\_PLL\_A:** signal indicating whether the acquisition mode is active.

**OA\_ZPR** (Zero Phase Restart): clears the accumulated phase error at each mode transition so that the frozen DCO tuning word does not introduce phase discontinuities. Each new mode operates only on the residual phase error around the frequency established by the previous mode.

**MEM\_DCO\_P:** stores the last frequency estimate from the controller look-up table to accelerate convergence.

**REG\_DCO\_P:** reports the frequency offset status words back to the controller.

**The same block is repeated for other modes:**

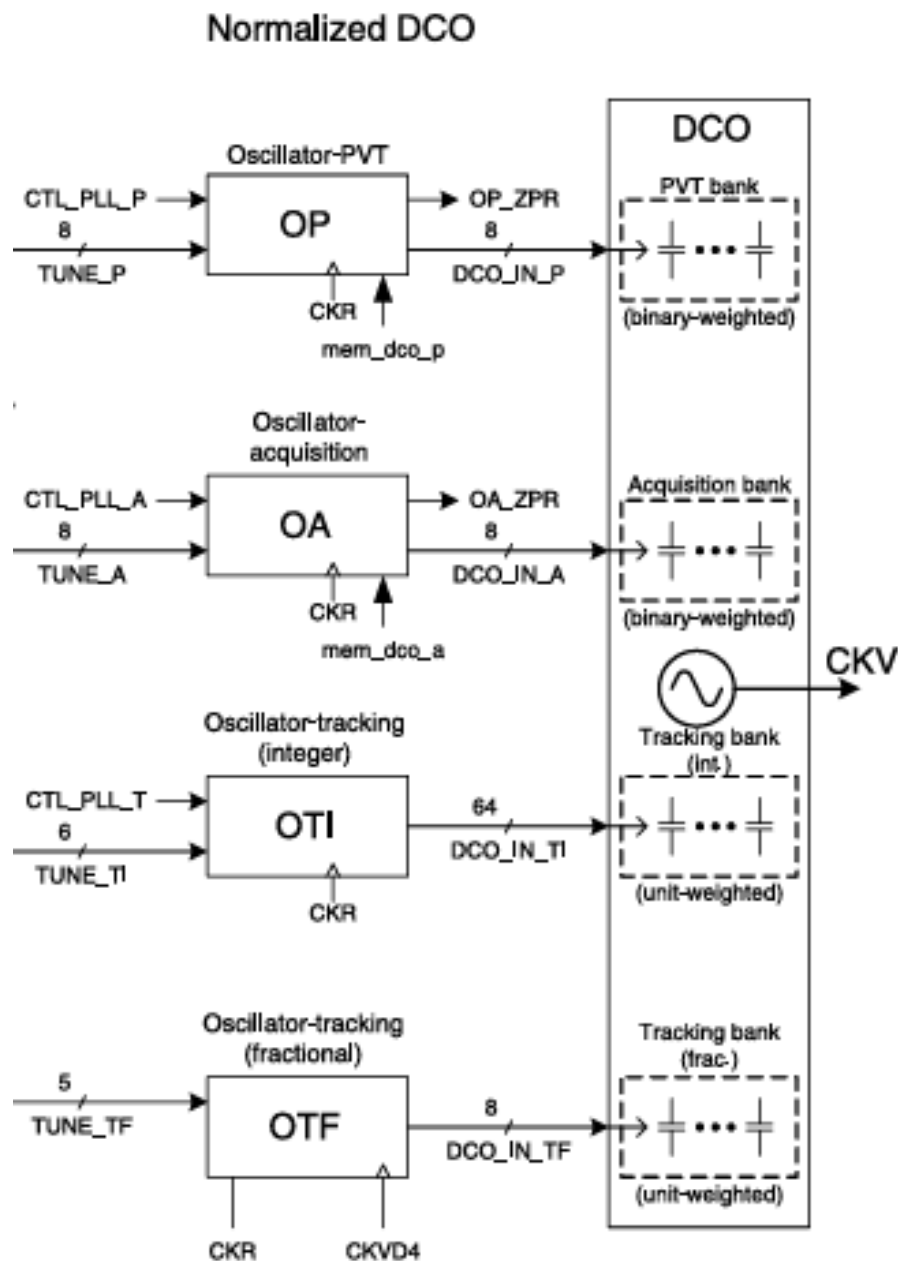


Figure 90: DCO gain paths: implementation block diagram

### Gain Calibration Using LMS

LMS is an adaptive filter that adjusts its coefficients to minimize the mean-square error between a desired signal and the filter output. LMS implements a stochastic gradient-descent update on the filter weights using the current input and error sample, as described by the following equations.

### Filter Output

$$y(n) = \mathbf{w}^T(n) \mathbf{x}(n) \quad (77)$$

### Error Signal

$$e(n) = d(n) - y(n) \quad (78)$$

### Weight Update

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu \mathbf{x}(n) e(n) \quad (79)$$

In this work:

- $y(n)$  is the estimated gain multiplied by the modulation control word,
- $\mathbf{x}(n)$  is the modulation control word,
- $e(n)$  is the frequency error given by  $\phi_e(n) - \phi_e(n-1)$ ,
- $\mathbf{w}$  is the estimated gain  $\frac{F_R}{\widehat{K_{\text{DCO}}}}$ ,
- $\mu$  is the learning rate (step size).

Conventional LMS requires a multiplier, and variants such as NLMS require additional circuitry to compute the optimum learning rate. The simplest LMS variant in terms of low power and low complexity is the sign-sign LMS.

### Sign-Sign LMS (SS-LMS)

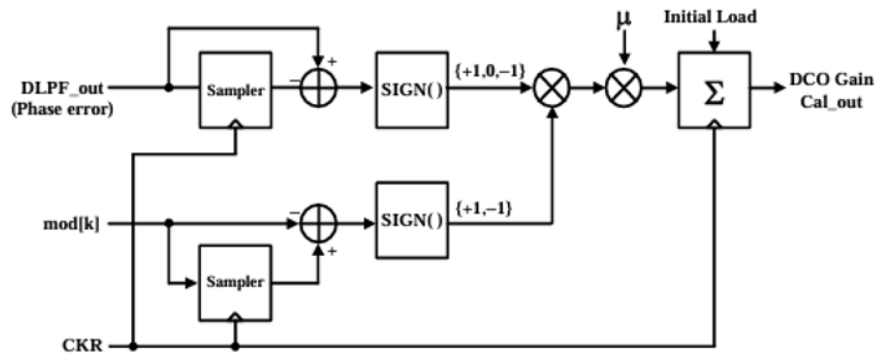


Figure 91: Sign-sign LMS adaptation algorithm for DCO gain calibration

$$\frac{F_R}{\widehat{K_{\text{DCO}}}}(n) = \frac{F_R}{\widehat{K_{\text{DCO}}}}(n-1) + \mu \text{sgn}(\phi_E(n) - \phi_E(n-1)) \cdot \text{sgn}(\text{mod}(n-1) - \text{mod}(n)) \quad (80)$$

The SS-LMS algorithm uses only the sign information of the phase error and the sign information of the modulation data fluctuations. The estimated gain converges linearly with a predefined adaptation step size  $\mu$ .

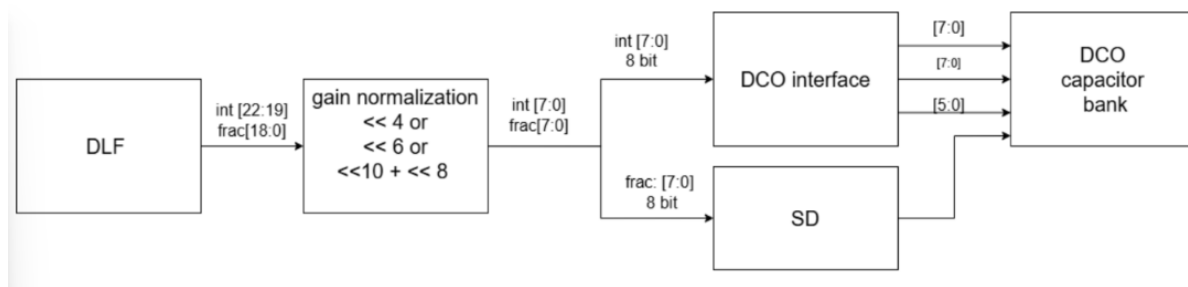
**RTL Implementation****DCO on high level:** High level overview of DCO domain blocks

Figure 92: High level DCO overview

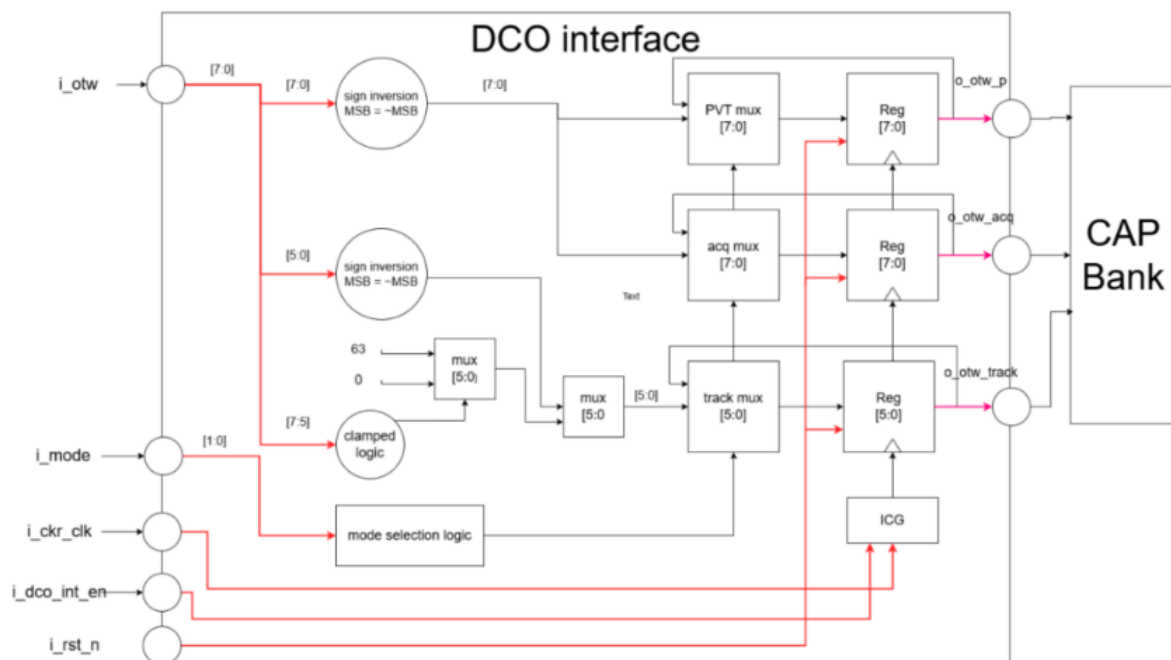
**DCO interface:**

Figure 93: DCO Interface

The port descriptions of DCO interface implementation are listed in Table 12.

| Name         | Direction | Width | Source                            | Destination    | Description                                                                                                                             |
|--------------|-----------|-------|-----------------------------------|----------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| i_clk_ckr    | Input     | 1     | Retimed Reference Clock generator | DCO            | DCO clock synchronized with reference clock with certain delay.                                                                         |
| i_rst_n      | Input     | 1     | Reset Controller                  | DCO            | Asynchronous active-low reset. Initializes all tuning words to mid-scale.                                                               |
| i_dco_int_en | Input     | 1     | Main Control Unit                 | DCO            | Interface enable signal. Acts as a clock gating.                                                                                        |
| i_mode       | Input     | 2     | Main Control Unit / FSM           | DCO            | ADPLL state (dco_mode_t). Used in routing internal signals.                                                                             |
| i_otw        | Input     | 8     | Gain Normalization                | DCO            | Signed integer tuning word applied to the active mode's capacitor bank.                                                                 |
| o_otw_p      | Output    | 8     | DCO                               | DCO Bank PVT   | Unsigned 256-level control word. Converts signed i_otw by MSB inversion. Holds previous state if inactive.                              |
| o_otw_acq    | Output    | 8     | DCO                               | DCO Bank ACQ   | Unsigned 256-level control word. Converts signed i_otw by MSB inversion. Holds previous state if inactive.                              |
| o_otw_track  | Output    | 6     | DCO                               | DCO Bank Track | Unsigned 64-level control word. Saturation guard implemented. Converts signed i_otw by MSB inversion. Holds previous state if inactive. |

Table 12: DCO Interface Ports Description

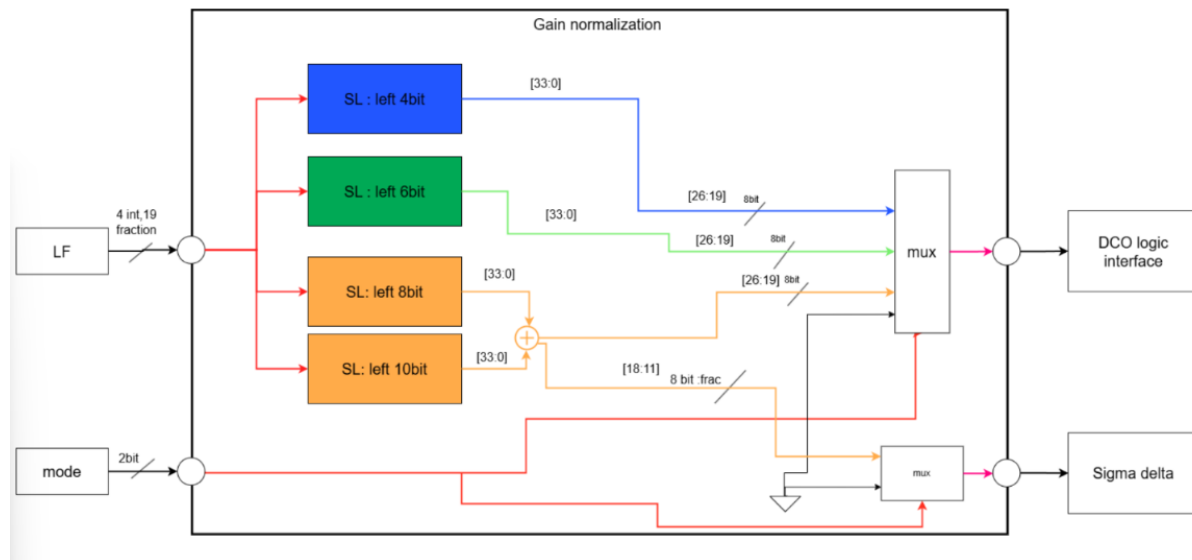
**Gain normalization block:**

Figure 94: Gain normalization block diagram

The port descriptions of the Gain normalization implementation are listed in Table 13.

Table 13: Gain Normalization Block Signal Details

| Signal Name       | Direction | Width              | Source / Destination              | Description                                                     |
|-------------------|-----------|--------------------|-----------------------------------|-----------------------------------------------------------------|
| <b>i_norm</b>     | Input     | (23-bit) Signed    | Source: Digital Loop Filter (DLF) | Normalized phase error from the loop filter.                    |
| <b>i_mode</b>     | Input     | 2-bit (dco_mode.t) | Source: Control Unit              | ADPLL state (PVT, ACQ, or TRACK). Selects multiplication value. |
| <b>o_otw_int</b>  | Output    | 8-bit Signed       | Destination: DCO Logic Interface  | DCO integer bit.                                                |
| <b>o_otw_frac</b> | Output    | 8-bit Unsigned     | Destination: Sigma Delta SD       | DCO fractional bit.                                             |

## LMS

High level overview of interaction of gain estimation block with system block:

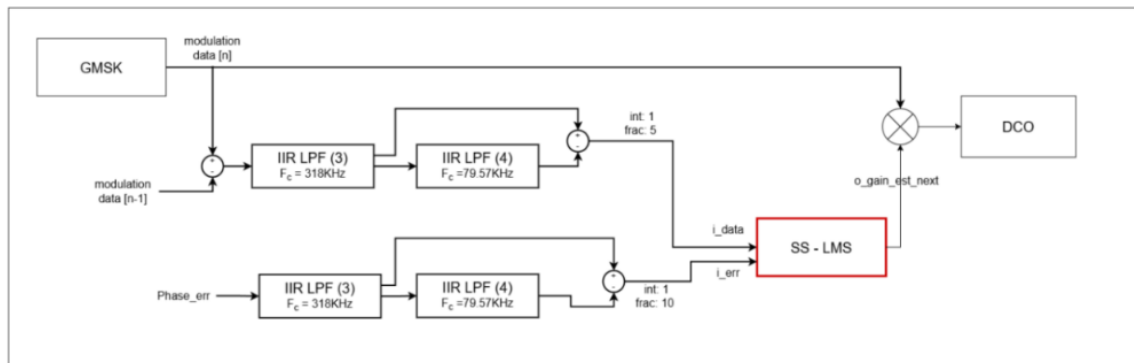


Figure 95: LMS high level interaction

## LMS Block Diagram

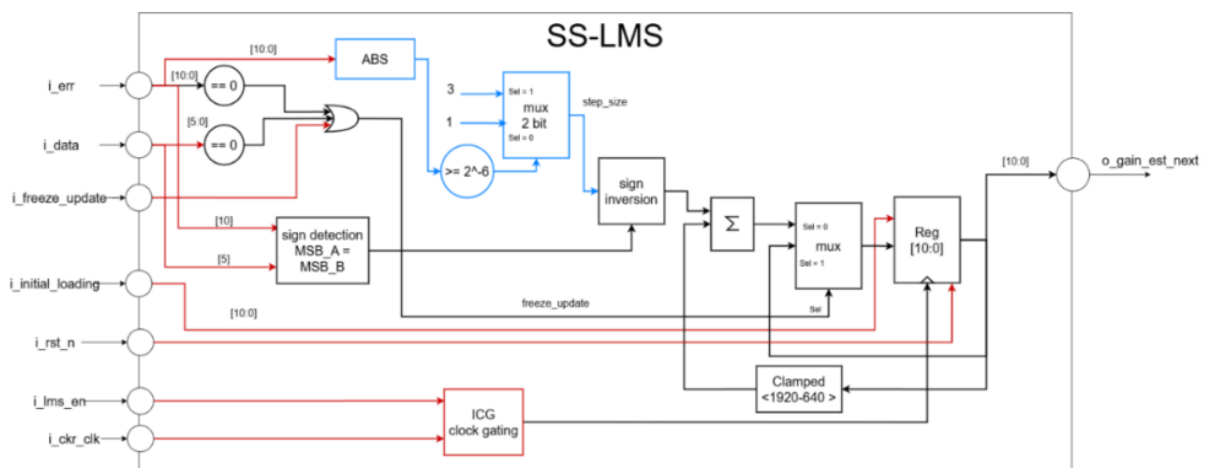


Figure 96: LMS Block diagram

**ABS block:**

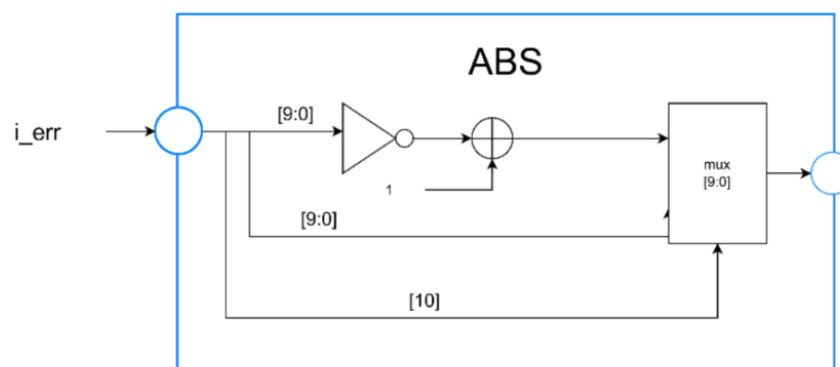


Figure 97: LMS Block diagram

The port descriptions of the LMS implementation are listed in Table 14.

| Signal Name       | Direction | Width | Source / Destination                                   | Description                                                                                      |
|-------------------|-----------|-------|--------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| i_initial_loading | Input     | 11bit | Source: Retention cell                                 | Initial value from LMS previous estimation loaded at reset                                       |
| i_err             | Input     | 11bit | Source: TDC → IIR Filter                               | The signed phase error. The MSB defines the update direction, fraction part determines step size |
| i_data            | Input     | 6bit  | Source: GMSK                                           | Determine LMS update direction                                                                   |
| i_freeze_update   | Input     | 1bit  | Source: Control unit                                   | External command to freeze LMS update, accounting group delay of IIR filter                      |
| i_clk_ckr         | Input     | 1bit  | Source: Retimed Reference Clock generator              | DCO clock synchronized with reference clock with certain delay                                   |
| i_rst_n           | Input     | 1bit  | Source: Control unit                                   | Asynchronous active-low reset. Initialize LMS                                                    |
| i_lms_en          | Input     | 1bit  | Source: Control Unit                                   | clock gate enable                                                                                |
| o_gain_est_next   | Output    | 11bit | Destination: Gain Normalizer in direct modulation path | LMS estimation = $\frac{F_{ref}}{K_{DCO}}$                                                       |

Table 14: SS-LMS Signal List

## IIR Filter

### Introduction

The LMS algorithm estimates DCO gain by measuring the correlation between modulation data and residual phase error. The raw phase error contains several components in addition to the modulation mismatch information, including TDC quantization noise, DCO noise, reference noise, and low-frequency components generated by the ADPLL tracking loop. These components are generally uncorrelated with the modulation data and can degrade LMS convergence. Consequently, both the phase-error path and modulation-data path are filtered before being applied to the LMS estimator.

The filtering structure is implemented using two cascaded first-order IIR filters. The first filter suppresses high-frequency noise while preserving the modulation spectrum. The second filter extracts the slowly varying phase-error components associated with loop dynamics and DC offsets. Subtracting the outputs of the two filters forms an effective band-pass response that isolates the modulation-related information required by the LMS algorithm.

### IIR Filter Analysis

The first-order IIR filter is described by the difference equation:

$$y[n] = (1 - \alpha)y[n - 1] + \alpha x[n] \quad (81)$$

We can derive the filter's z-domain transfer function:

$$H(z) = \frac{\alpha}{1 - (1 - \alpha)z^{-1}} \quad (82)$$

The filter pole is:

$$Z_p = (1 - \alpha) \quad (83)$$

Relating the discrete-time pole location to the equivalent continuous-time cutoff frequency:

$$(1 - \alpha) = e^{-\frac{2\pi f_c}{f_s}} \quad (84)$$

$$\alpha = 1 - e^{-\frac{2\pi f_c}{f_s}} \quad (85)$$

For  $(f_c \ll f_s)$ , the exponential may be approximated as:

$$\alpha \approx \frac{2\pi f_c}{f_s} \quad (86)$$

### First IIR Filter

The first low-pass IIR filter is used to pass the GMSK modulation spectrum while attenuating higher-frequency noise components. Desired cut-off frequency = 600 kHz with sampling frequency = 32 MHz. We get approximately:

$$\alpha = 0.117$$

The nearest power-of-two coefficient is selected for hardware efficiency:

$$\alpha = 2^{-3}$$

Resulting in an individual low-pass 3-dB bandwidth of approximately 680 kHz. Filter response:

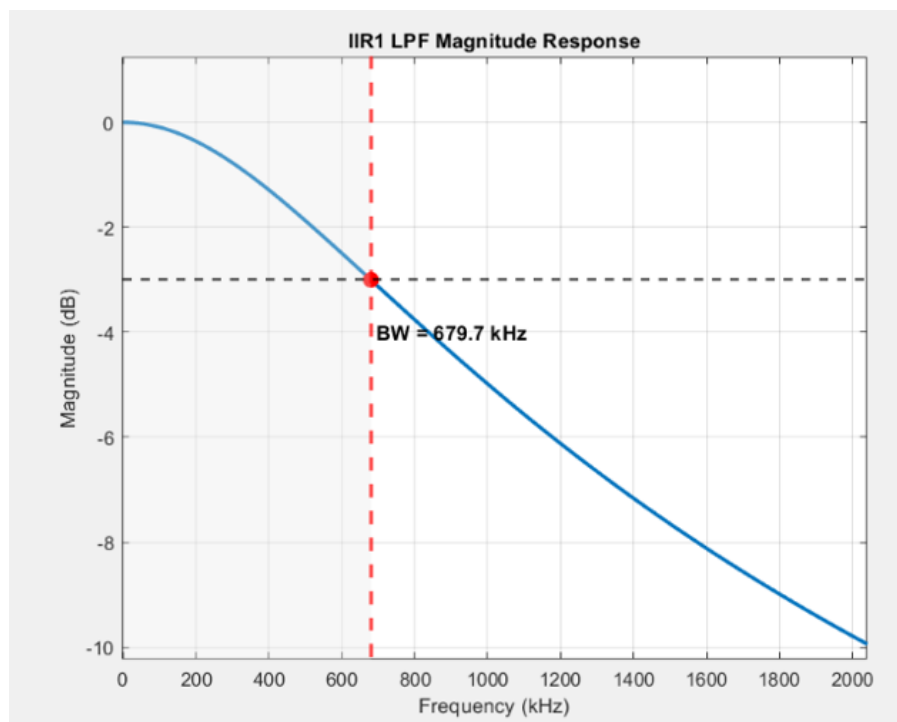


Figure 98: First filter response

## Second IIR Filter

The second low-pass IIR filter is used to extract the low-frequency components associated with the ADPLL loop dynamics and slow DC offsets. For a smaller loop bandwidth, alpha was selected to be:

$$\alpha = 2^{-4}$$

resulting in a low-pass 3-dB bandwidth of approximately 320 kHz.

Filter response:

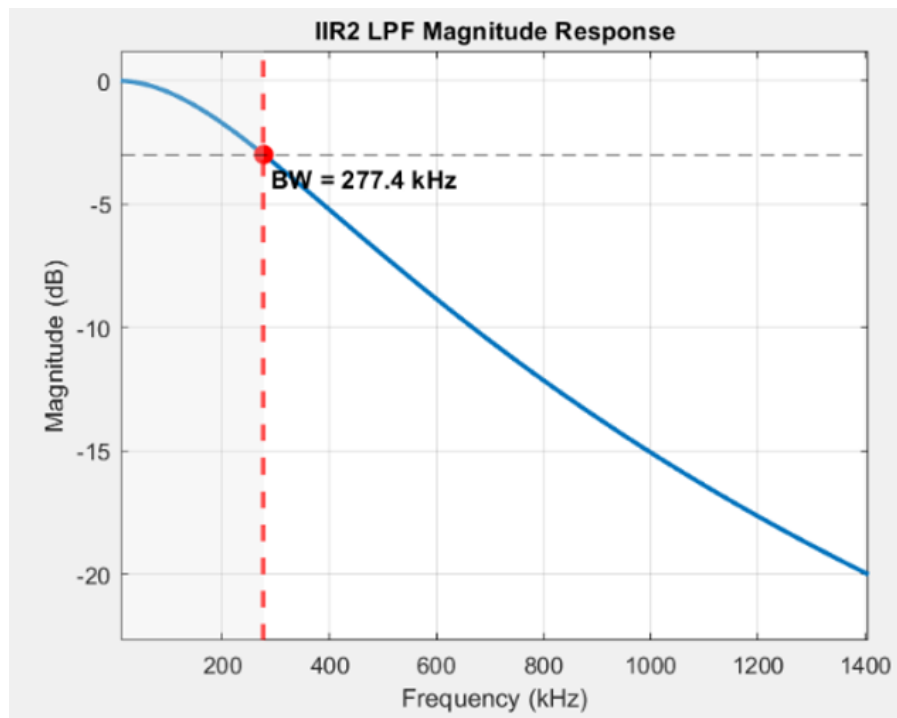


Figure 99: second filter response

### Final Bandpath Response

To isolate the modulation-related information required by the LMS algorithm, the outputs of the two filters are subtracted:

- **Phase error path:**  $i\_phase\_err = o\_IIR1 - o\_IIR2$
- **Modulation path:**  $i\_mod\_data = o\_IIR1 - o\_IIR2$

This operation removes the slow DC-offset and loop-dynamics components while preserving the modulation-related AC components.

The resulting frequency response exhibits a broad band-pass characteristic extending approximately from 200 kHz to 1 MHz.

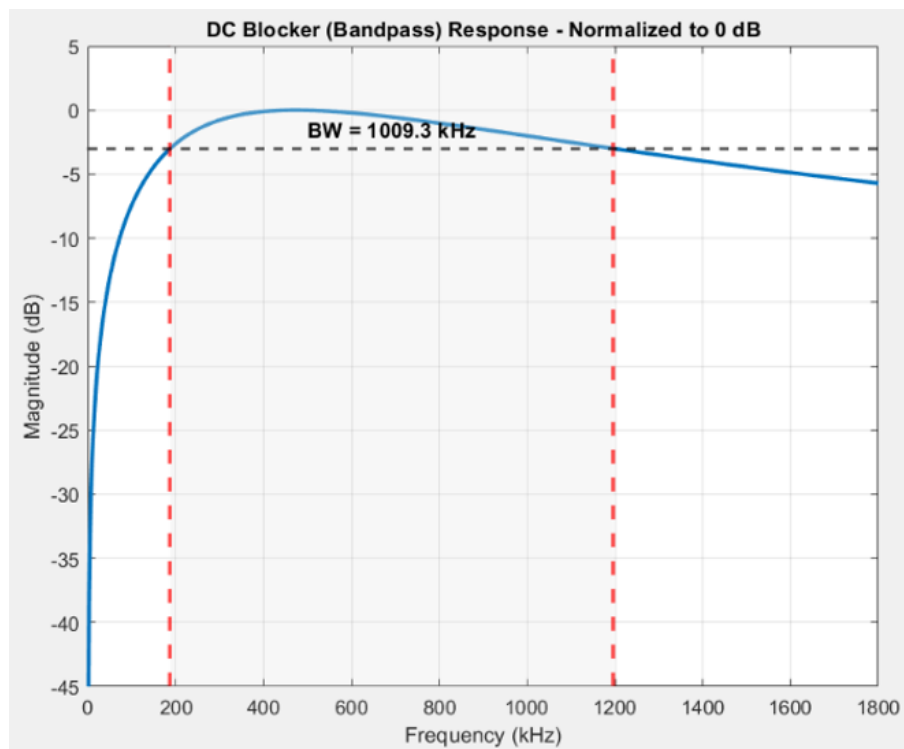


Figure 100: band pass response

Since LMS adaptation is based on the correlation between modulation data and residual phase error, the filter coefficients were further evaluated using correlation analysis. MATLAB simulations showed that the selected coefficients ( $\alpha = 2^{-3}$ ,  $\alpha = 2^{-6}$ ) maximize the correlation between the filtered signals compared with alternative coefficient selections, indicating improved extraction of modulation-related information for DCO gain estimation.

Figure 101 shows correlation between modulation data and phase error at  $\alpha_1 = 2^{-3}$ ,  $\alpha_2 = 2^{-4}$  :

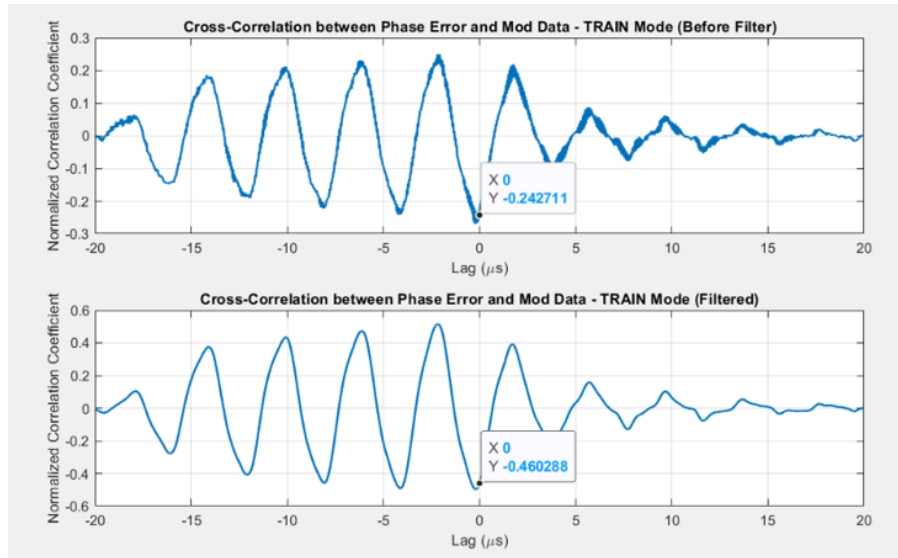


Figure 101: Cross- correlation between modulation data and phase error at  $\alpha_1 = 2^{-3}$ ,  $\alpha_2 = 2^{-4}$

Figure 102 shows correlation between modulation data and phase error at  $\alpha_1 = 2^{-3}$ ,  $\alpha_2 = 2^{-6}$  :

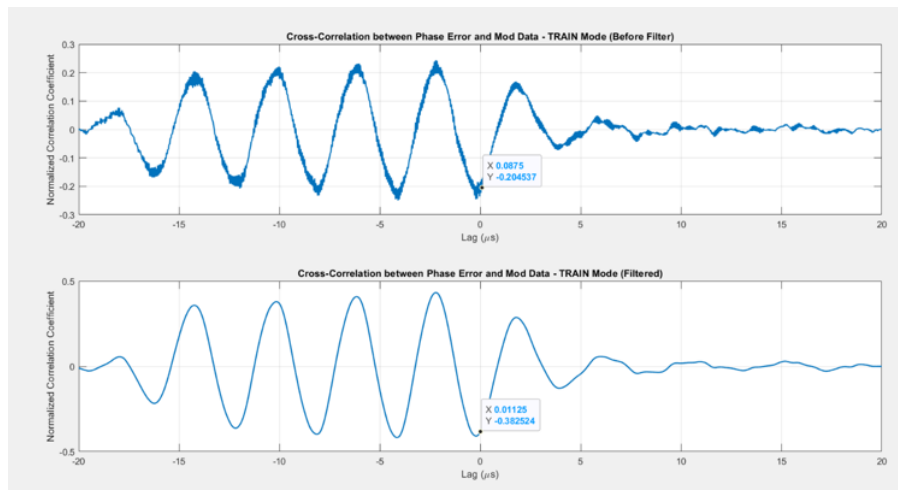


Figure 102: Cross-correlation between modulation data and phase error at  $\alpha_1 = 2^{-3}$ ,  $\alpha_2 = 2^{-6}$

We can see that the selected coefficients give a higher correlation value which will give better performance.

## RTL Block Diagram

The abstract block diagram of the IIR filter is presented in Figure 103.

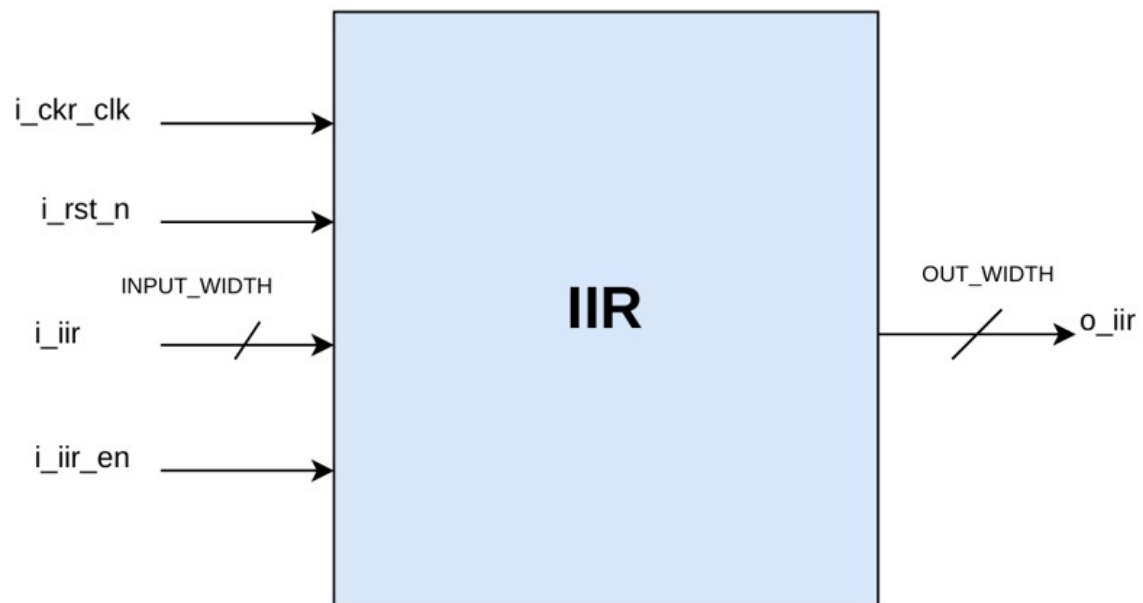


Figure 103: IIR filter abstract diagram

A detailed gate level block diagram of the IIR filter is shown in Figure 104.

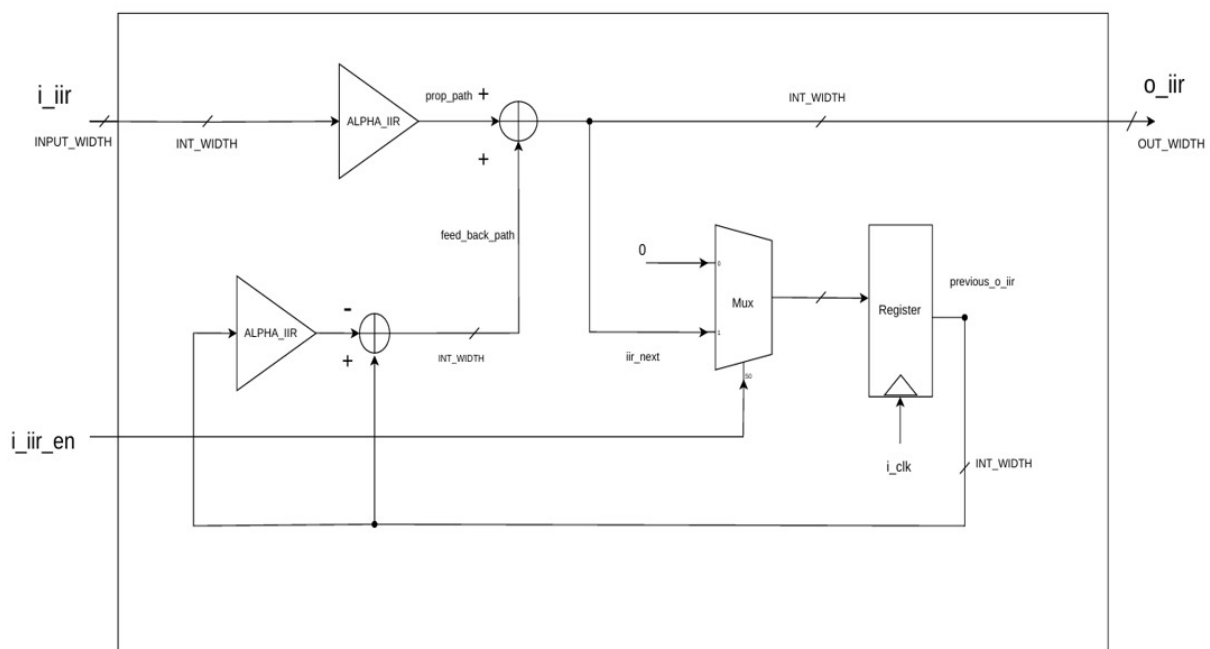


Figure 104: IIR filter block diagram

## RTL Interface

The port descriptions of the IIR filter implementation are listed in Table 15.

| Name      | Direction | Width       | Description                                                                                  |
|-----------|-----------|-------------|----------------------------------------------------------------------------------------------|
| i_ckr_clk | Input     | 1           | Input clock ckr (retimed reference clock).                                                   |
| i_rst_n   | Input     | 1           | Active low asynchronous reset.                                                               |
| i_iir_en  | Input     | 1           | Active high IIR filter enable from control unit.                                             |
| i_iir     | Input     | INPUT_WIDTH | Input to IIR filter (phase error or modulation data according to instance path).             |
| o_iir     | Output    | OUT_WIDTH   | Output from IIR filter (filtered phase error or modulation data according to instance path). |

Table 15: IIR Filter ports description

## Control Unit

Figure 105 shows the overall control architecture of the proposed BLE transmitter. The system is partitioned into two separate clock domains: the crystal clock domain and the CKR clock domain.

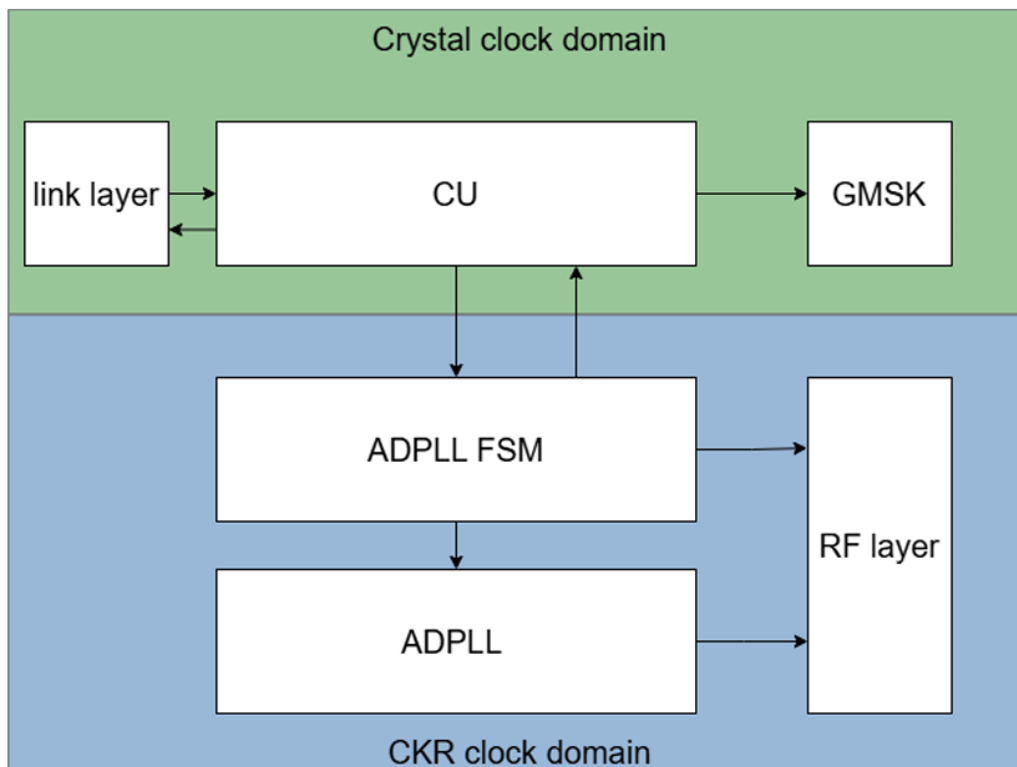


Figure 105: Overall control architecture of the proposed BLE transmitter showing the interaction between the crystal clock domain and the CKR clock domain.

As shown in Fig. 105, the crystal clock domain contains the Link Layer, the Control Unit (CU), and the GMSK modulator. The Link Layer emulates the BLE controller behavior and generates transmission requests together with the binary data stream. These requests are processed by the Control Unit, which supervises the overall transmission sequence and communicates with the lower-level ADPLL controller.

The lower part of the architecture operates in the CKR clock domain and consists of the ADPLL finite-state machine (ADPLL FSM), the ADPLL core, and the RF layer. The ADPLL FSM controls the operating phases of the synthesizer, including calibration, acquisition, tracking, training, and modulation. According to the current state, the FSM activates the required ADPLL blocks and controls the RF front-end.

Bidirectional communication exists between the Control Unit and the ADPLL FSM to exchange control commands and status information across the two clock domains. After the ADPLL reaches the modulation state, the generated frequency-modulated signal is forwarded to the RF layer for transmission.

### CU Architecture

This hierarchical organization separates high-level protocol management from low-level frequency synthesis operations, resulting in improved modularity, simplified control, and enhanced power management.

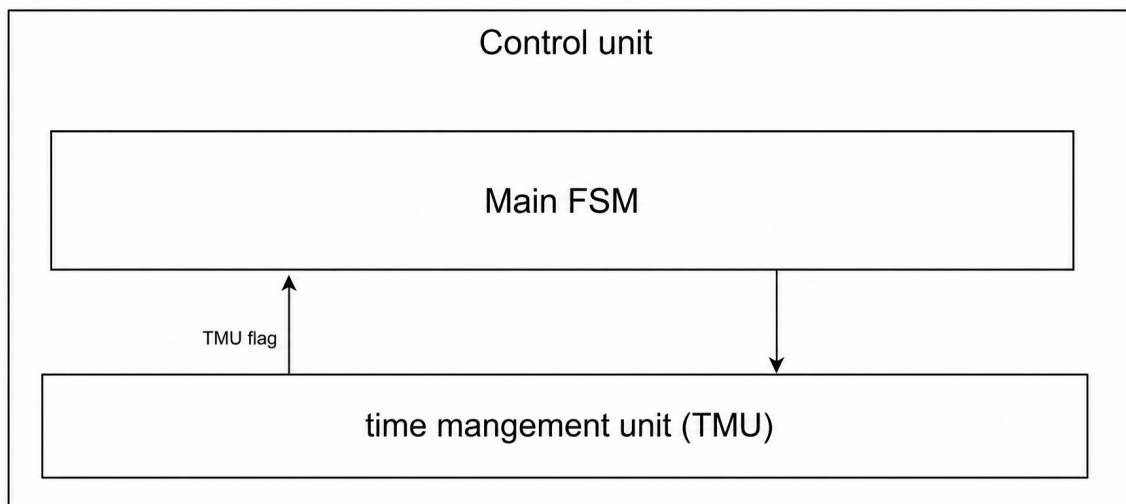


Figure 106: Main control unit Architecture

The Main Control Unit (MCU) is implemented as a finite-state machine (FSM) responsible for coordinating the operation of the proposed ADPLL architecture. It manages the activation and deactivation of the internal blocks according to the current phase of operation. The state transitions are driven by status flags generated by the Time Management Unit (TMU) and by commands received from the Link Layer. Figure 107 illustrates the state transition diagram of the controller.

The controller consists of seven operating states as shown in Figure 107, namely: *IDLE*, *PVT Calibration*, *Acquisition*, *Tracking*, *Training Sequence*, *Modulation*, and *Sleep*.

## CU FSM

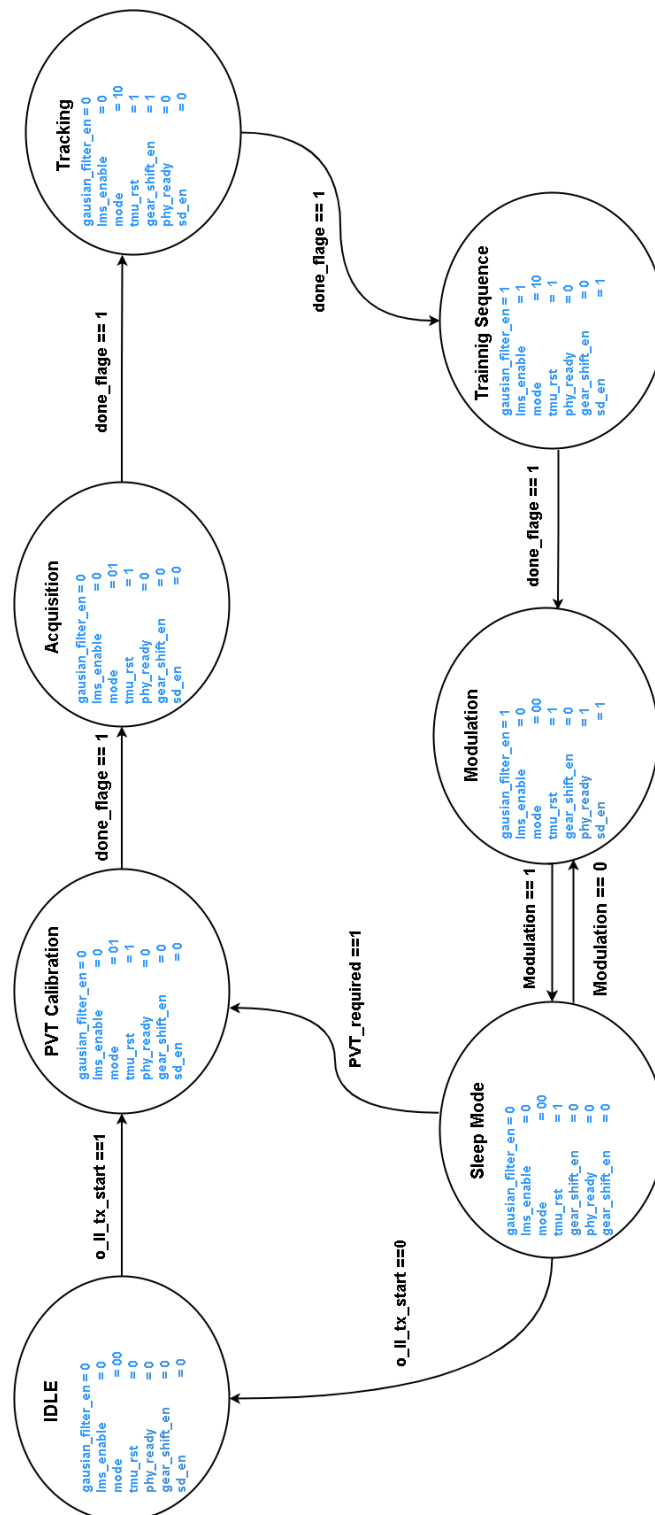


Figure 107: Control Unit FSM

## IDLE State

The IDLE state represents both the initial condition after reset and the inactive condition when no BLE connection is required. In this state, all major blocks are disabled to minimize power consumption.

The Gaussian filter, LMS adaptation block, and GMSK modulator remain inactive, while the TMU (Time Management Unit) is held in reset. In addition, the `phy_ready` signal is deasserted to indicate that the PHY is unavailable.

The controller remains in this state until the Link Layer issues a transmission request (`TX_start = 1`), which initiates the frequency calibration procedure. by the link layer.

**Purpose:** Lowest-power state and system startup point.

## PVT Calibration State

After receiving a transmission request, the controller enters the Process-Voltage-Temperature (PVT) calibration state.

The objective of this stage is to compensate for process variations, supply voltage fluctuations, and temperature changes that affect the characteristics of the digitally controlled oscillator (DCO). During this phase, the TMU performs calibration and generates the required control parameters for the DCO.

The PHY remains unavailable and no data transmission/modulation is allowed.

Once calibration is completed, the TMU asserts `PVT_done_flag`, causing the controller to proceed to the acquisition phase. PVT calibration can be within 1 microsecond

**Purpose:** Establish a correct initial operating point for the oscillator.

## Acquisition State

The acquisition state performs coarse frequency locking. The ADPLL adjusts the DCO frequency until it approaches the desired channel frequency.

During this stage, the loop attempts to reduce the large initial frequency error remaining after calibration.

The PHY remains unavailable because the loop has not yet reached sufficient accuracy for communication.

When the acquisition process converges, the TMU asserts `Acquisition_done_flag`, allowing the FSM to enter the tracking state. Acquisition state can be within 2 microseconds

**Purpose:** Coarse synchronization of the ADPLL.

## Tracking State

In the tracking state, the loop operates in fine-lock mode. Small residual frequency and phase errors are continuously corrected to maintain synchronization.

Unlike the acquisition stage, this mode focuses on improving accuracy and reducing phase noise. The gear-shifting mechanism is enabled to provide smooth transition between loop bandwidths and improve stability.

After the loop reaches the desired locking accuracy, the signal `Tracking_done_flag` becomes active and the controller moves to the training sequence state. Tracking State can be around 15 microseconds so the total lock time is 18 microseconds

**Purpose:** Fine frequency and phase correction.

### Training Sequence State

Before actual payload transmission, a predefined training sequence is generated. This sequence is used by the LMS adaptation block to estimate and compensate for channel impairments.

During this phase, the LMS block is enabled and the modulator operates using internally generated training symbols. Adaptation coefficients are updated during this period.

The PHY is still not declared ready because user data transmission has not started yet.

After completion of the training sequence, `Training_sequence_flag` causes the controller to enter the modulation state.

**Purpose:** Adapt the equalization parameters prior to payload transmission.

### Modulation State

The modulation state corresponds to normal BLE operation.

In this mode, the Gaussian filter and LMS block are enabled, and the PHY is declared ready by asserting `phy_ready`. Input bits are supplied directly from the Link Layer instead of the training sequence generator.

The GMSK modulator converts the incoming bit stream into the modulated signal used by the ADPLL.

The controller remains in this state until packet transmission finishes and `Modulation_end = 1` is asserted.

**Purpose:** Normal payload transmission.

### Sleep State

The sleep state is entered during inter-frame spacing periods or whenever transmission activity temporarily stops.

Most functional blocks are disabled to reduce power consumption while preserving the overall system context. The PHY is marked unavailable and the TMU is reset.

When a new transmission request is issued (`Modulation_start = 1`), the controller wakes up and returns to the PVT calibration state. If no further communication is required, the controller eventually returns to the IDLE state.

**Purpose:** Low-power mode between packet transmissions.

## State Transition Sequence

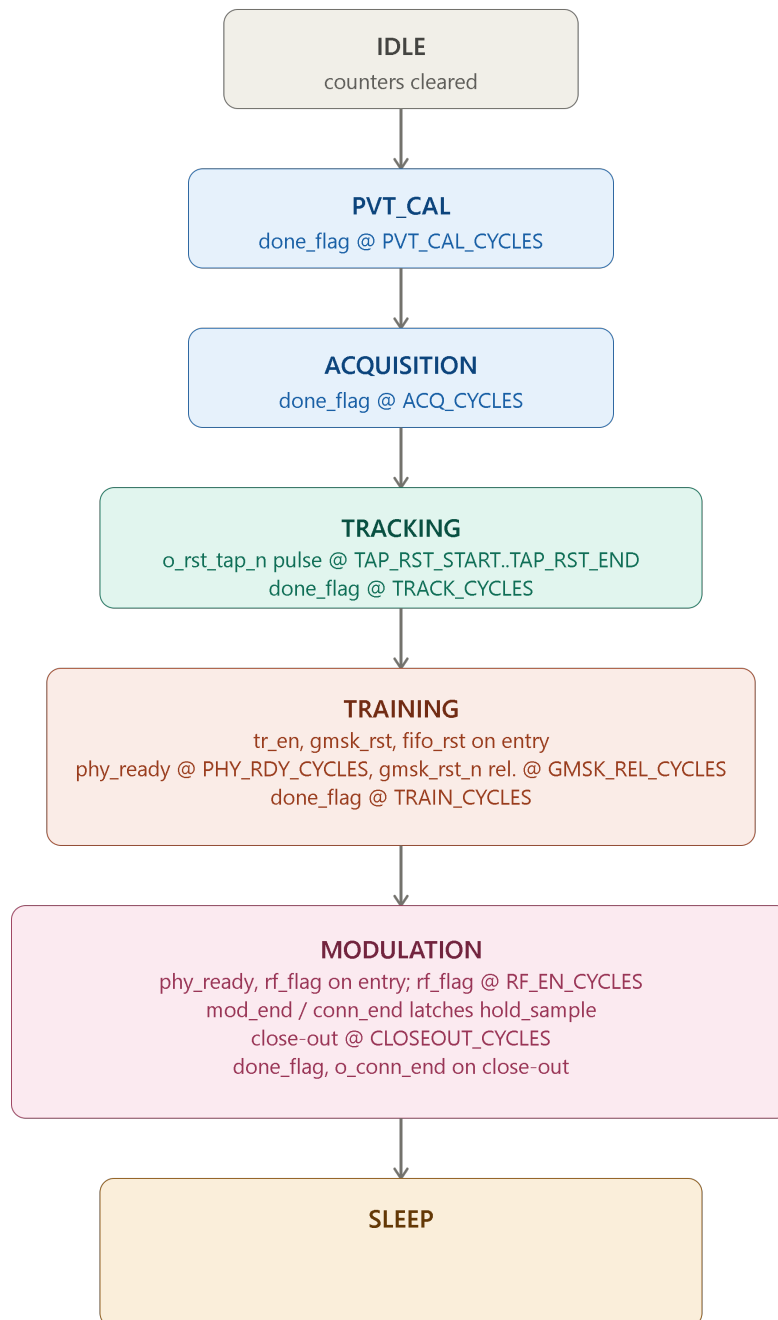
The overall sequence of operation of the proposed controller can be summarized as

$$\text{IDLE} \rightarrow \text{PVT} \rightarrow \text{Acquisition} \rightarrow \text{Tracking} \rightarrow \text{Training} \rightarrow \text{Modulation} \rightarrow \text{Sleep}. \quad (87)$$

This hierarchical operation enables the ADPLL to achieve reliable frequency locking while minimizing power consumption during inactive periods.

## Time Management Unit (TMU)

**Overview** The Time Management Unit (TMU) is a timing and sequencing block that governs the duration of each state of the main Control Unit (CU) and generates the timing-critical control and reset signals required by the PHY/RF datapath. It monitors the current CU state (`i_cu_state`) and, using an internal cycle counter, asserts `o_done_flag` once the required dwell time for that state has elapsed, prompting the CU to advance to the next state. It also produces auxiliary signals such as `o_phy_ready`, `o_pvt_required`, `o_tr_en`, `o_rf_flag`, `o_conn_end`, and the registered resets `o_rst_tap_n`, `o_gmsk_rst_n`, and `o_fifo_rst_n`.

**TMU flow diagram**

The TMU advances through `IDLE` → `PVT_CAL` → `ACQUISITION` → `TRACKING` → `TRAINING` → `MODULATION` → `SLEEP`, with each state transitioning once its internal counter reaches a configurable cycle-count parameter, after which `done_flag` returns control to the main CU. Each state is governed by one or more parameterized timing events and asserts a corresponding set of control outputs. Independently of this sequence, a background PVT watcher increments its own counter whenever enabled and requests recalibration once it reaches a configurable threshold, decoupling long-horizon drift monitoring from the short, per-state timing of the main sequence.

Table 16: Default values of the TMU timing parameters

| Parameter                      | Default value                  |
|--------------------------------|--------------------------------|
| <code>PVT_CAL_CYCLES</code>    | 31                             |
| <code>ACQ_CYCLES</code>        | 127                            |
| <code>TRACK_CYCLES</code>      | 511                            |
| <code>TAP_RST_START</code>     | 477                            |
| <code>TAP_RST_END</code>       | 478                            |
| <code>PHY_RDY_CYCLES</code>    | 691                            |
| <code>GMSK_REL_CYCLES</code>   | 706–707                        |
| <code>TRAIN_CYCLES</code>      | 707                            |
| <code>RF_EN_CYCLES</code>      | 67                             |
| <code>CLOSEOUT_CYCLES</code>   | 84 ( $\approx 2 \mu\text{s}$ ) |
| <code>SLEEP_RDY_CYCLES</code>  | 15                             |
| <code>SLEEP_CYCLES</code>      | 31                             |
| <code>PVT_REQ_THRESHOLD</code> | 0xF9FF                         |

The TMU uses two independent counters and a state-decode block:

- **Main counter** (`counter_reg`, 10-bit): provides timing for per-state dwell times and intra-state events; gated by `i_tmu_en`.
- **Background counter** (`bg_counter_reg`, 16-bit): tracks elapsed time independently of the main counter, used to trigger PVT recalibration; gated by `i_bg_tmu_count_en`.
- **State decoder**: a combinational block on `i_cu_state` that sets counter-clear conditions and internal flags, which are then registered before being driven to the outputs for glitch-free, clock-aligned signaling.

### CU State Behavior

**IDLE** Both counters are held cleared; all flags are inactive.

**PVT\_CAL** Main counter runs; `done_flag` asserts at 31 cycles and the counter clears.

**ACQUISITION** Main counter runs; `done_flag` asserts at 127 cycles.

**TRACKING** Main counter runs; `rst_tap` pulses low for cycles 477–478 (tap-memory reset pulse on `o_rst_tap_n`); `done_flag` asserts at 511 cycles.

**TRAINING** `tr_en`, `gmsk_rst`, and `fifo_rst` assert on entry. At cycle 691, `phy_ready` asserts and `tr_en` deasserts. At cycles 706–707, `gmsk_rst` deasserts. At cycle 707, `fifo_rst` deasserts and `done_flag` asserts.

**MODULATION** On entry, `gmsk_rst`, `fifo_rst`, `phy_ready`, and `rf_flag` assert. `rf_flag` also gates on at cycle 67 during normal operation. Detection of `i_ll_mod_end` or `i_ll_conn_end` latches `int_end_mod_flag_reg` and asserts `hold_sample`, starting a close-out window; once this window reaches 84 cycles ( $\sim 2\ \mu\text{s}$ ), `done_flag` and `o_conn_end` assert and `rf_flag` deasserts.

**SLEEP** `o_conn_end` follows `i_ll_conn_end` directly. `phy_ready` asserts at cycle 15; `done_flag` asserts at cycle 31.

**Default** Counters held cleared; outputs inactive (safe fallback).

### Background Counter and PVT Recalibration

The background counter increments whenever enabled and not cleared (cleared during IDLE and PVT\_CAL). When it reaches `0xF9FF`, `o_pvt_required` asserts and the counter saturates, requesting a PVT recalibration cycle independent of per-state timing.

### Output Signal Summary

Table 17: TMU output signals

| Signal                      | Description                                         |
|-----------------------------|-----------------------------------------------------|
| <code>o_done_flag</code>    | State-completion pulse to the CU                    |
| <code>o_phy_ready</code>    | PHY ready/stable indication                         |
| <code>o_pvt_required</code> | Requests PVT recalibration                          |
| <code>o_conn_end</code>     | Link-layer connection termination indication        |
| <code>o_tr_en</code>        | Training sequence enable                            |
| <code>o_rf_flag</code>      | RF path enable                                      |
| <code>o_rst_tap_n</code>    | Tap-memory reset (active-low, pulsed in TRACKING)   |
| <code>o_gmsk_rst_n</code>   | GMSK chain reset (active-low)                       |
| <code>o_hold_sample</code>  | Holds sample processing during MODULATION close-out |

## Interfaces

### Link Layer Interface

#### Overview

The Link Layer (LL) acts as the top-level controller of the BLE transmitter and serves as the interface between the BLE protocol layer and the underlying hardware blocks. Through a set of control, data, and status signals, the Link Layer coordinates the operation of the Control Unit, the Physical Layer (PHY), and the RF subsystem to ensure a correct and synchronized transmission sequence. This interface effectively decouples the protocol-level decision of *when* to transmit from the hardware-level process of *how* the transmitter chain is prepared and driven, allowing the Link Layer to issue high-level requests while the underlying blocks handle the detailed timing and sequencing internally.

Figure 109 shows the complete set of input and output signals exposed by the Link Layer block. The signals on the right-hand side (`i_RF_ready` and `i_phy_ready`) are inputs to the Link Layer, while `o_phy_bits`, `o_ll_tx_start`, `o_ll_mod`, `o_channel`, and `o_RF_start`

are outputs driven by the Link Layer toward the Control Unit, the Physical Layer, and the RF subsystem.

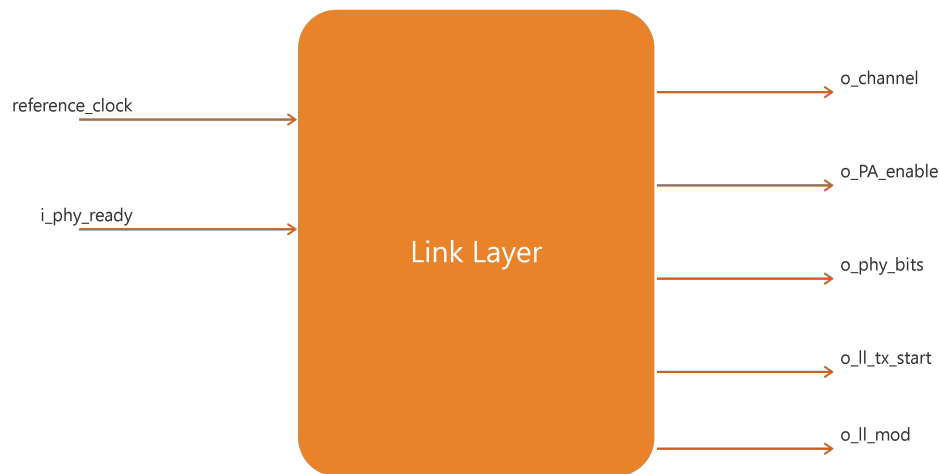


Figure 109: Link Layer interface block diagram showing input and output signals.

### Signal Descriptions

**o\_phy\_bits — Packet Data Output.** This is one of the primary outputs of the Link Layer. It carries the raw packet bits to be transmitted – the complete BLE packet generated by the Link Layer – and forwards them to the Physical Layer for modulation and transmission. The packet content includes the preamble, access address, PDU (header and payload), and CRC, all assembled by the Link Layer prior to transmission. These bits are presented serially and are only sampled by the modulation chain while `o_ll_mod` is asserted, ensuring that data movement on this bus is meaningful only during an active transmission window.

**o\_ll\_tx\_start — Transmission Trigger.** To initiate packet transmission, the Link Layer asserts this signal. It informs the Control Unit that a transmission request has been issued. Upon detecting it, the Control Unit exits the IDLE state and begins the transmitter’s startup sequence, progressing through a fixed series of internal states: **PVT Calibration** → **Acquisition** → **Tracking** → **Training Sequence**.

Each state performs a distinct part of the startup procedure:

- **PVT Calibration** compensates for process-voltage-temperature variations in the timing path, resetting and recalibrating the time-to-digital converter (TMU) so that subsequent timing-dependent operations are accurate.
- **Acquisition** allows the ADPLL to achieve a coarse lock on the selected channel frequency, bringing the loop close to its target frequency quickly but with limited precision.
- **Tracking** refines this lock to steady-state accuracy by enabling gear-shifting of the loop bandwidth, progressively narrowing the loop response as the frequency error decreases.

- **Training Sequence** prepares the adaptive filtering and shaping blocks – the Gaussian filter, LMS adaptation path, and sigma-delta modulator – for live data, ensuring they are not “cold” when modulation begins.

Once this sequence completes, `phy_ready` is asserted. On average, the complete startup sequence – from the assertion of `o_ll_tx_start` to the assertion of `phy_ready` – takes approximately **36  $\mu$ s**. This duration is dominated primarily by the ADPLL’s acquisition and tracking phases, since frequency locking inherently requires multiple reference cycles to converge, whereas PVT calibration and training are comparatively short, fixed-duration operations. This latency is a critical parameter for the Link Layer, as it determines how far in advance a transmission request must be issued relative to the intended on-air transmission time, particularly when operating under BLE’s timing-critical connection event scheduling.

Not every transmission requires the full startup sequence. When the transmitter is in **Sleep Mode** between packets, the Control Unit may either return directly to the Modulation state for the next transmission or re-enter PVT Calibration first, depending on the time elapsed since the last transmission:

- **Short inter-packet interval:** If the time since the last transmission is short enough that timing and frequency drift remain within tolerance, PVT calibration can be skipped entirely. The Control Unit transitions directly from Sleep Mode back into Modulation, since the ADPLL is assumed to still be sufficiently close to its previous lock point and the timing path has not drifted significantly. This shortens the effective startup latency considerably compared to the full 36  $\mu$ s sequence.
- **Long inter-packet interval:** If the elapsed time is longer, voltage and temperature conditions – and consequently circuit timing – may have drifted enough to require correction. In this case, the `PVT_required` condition routes the Control Unit back through the full PVT Calibration → Acquisition → Tracking → Training Sequence path before modulation can resume, incurring the full startup latency again.

In both cases, the Link Layer does not begin sending packet data until `phy_ready` is asserted, regardless of which path the Control Unit takes to reach that state. This means the Link Layer must treat the startup latency as variable rather than fixed, and should not assume a constant delay between issuing `o_ll_tx_start` and being able to transmit – it must instead wait on the readiness signals themselves rather than relying on elapsed time.

**o\_PA\_enable — Power Amplifier Enable.** The Link Layer asserts `o_PA_enable` to enable the Power Amplifier (PA) in the RF front-end through the register file interface. This signal is issued ahead of the actual transmission window because the PA requires a non-negligible settling time before it can deliver a stable and spectrally compliant output. By asserting `o_PA_enable` early — concurrently with or shortly after `o_ll_tx_start` — the PA settling process runs in parallel with the digital PHY startup sequence, ensuring that both the analog and digital chains are ready by the time modulated data needs to be transmitted. The routing of this signal through the register file allows the PA enable timing to be programmable, accommodating variations in PA settling time across process, voltage, and temperature conditions.

**o\_channel — Channel Selection.** The BLE channel number is provided through this signal. The value is stored in the register file and supplied to the ADPLL whenever a new transmission channel is selected, allowing the ADPLL to lock to the corresponding BLE center frequency. Because BLE uses frequency hopping across 40 channels (37 data channels and 3 advertising channels), this value typically changes between consecutive transmissions, meaning the channel update generally coincides with – and helps motivate – the need for a fresh acquisition and tracking cycle in the Control Unit.

**i\_phy\_ready — Digital Readiness.** Once the frequency synthesizer successfully locks to the desired channel and the Control Unit completes the acquisition, tracking, and training sequence described above, the Physical Layer asserts this signal to indicate that the digital transmitter chain is ready. This signal reflects the state of the digital domain only; it carries no information about the analog/RF front-end, which is reported independently through `i_RF_ready`.

**reference\_clock — Reference Clock Input.** The `reference_clock` signal is sourced from the analog portion of the ADPLL and serves as the timing reference for the Link Layer. Throughout this design, it is assumed that both the Link Layer and the Physical Layer operate synchronously on this same reference clock, eliminating any clock domain crossing concerns at their interface. This assumption simplifies the handshaking logic between the two layers, as all control signals — `o_ll_tx_start`, `o_ll_mod`, and `i_phy_ready` — are sampled and driven on the same clock edge, ensuring glitch-free and well-defined timing behavior across the entire transmit chain.

**o\_PA\_enable — Power Amplifier Enable Timing.** In practice, the PA settling time is frequently the bottleneck of the overall transmit-readiness latency. Several analog sub-blocks contribute to this delay:

- **Power amplifier (PA) settling.** The PA typically dominates the RF readiness latency. Its bias network and output matching stage require time to settle to a stable operating point before the PA can deliver linear, spectrally compliant output power. Ramping the PA too quickly can cause overshoot, spectral splatter outside the BLE channel mask, or AM-to-PM distortion that corrupts the GFSK-modulated signal, so the PA enable sequence is deliberately paced rather than instantaneous. This makes PA settling one of the longest individual contributors to overall transmit latency, and in many implementations it is the limiting factor in moving from a cold start to an active transmission.
- **Bias and reference settling.** Bandgap references, bias current generators, and LDOs supplying the RF front-end require a finite settling time after enable before their outputs reach the required accuracy and noise performance. Any residual settling transient at this stage propagates downstream and can affect both the local oscillator and the PA.
- **Local oscillator buffer and mixer warm-up.** Beyond the ADPLL's frequency lock, the RF-side LO buffers and upconversion mixers driven by the synthesizer output need their own bias and signal-swing settling time before they can faithfully pass the modulated signal to the PA.
- **Matching network and antenna interface transients.** Switches and matching networks at the PA output and antenna interface may introduce additional settling delay as parasitic charge redistributes through the network when the RF path is first enabled.

Because PA settling in particular can be both slow and sensitive to process, voltage, and temperature conditions, `o_PA_enable` is asserted as early as possible — concurrently with `o_ll_tx_start` — through the register file interface, allowing the PA settling process to proceed in parallel with the digital PHY startup sequence and preventing it from becoming the dominant term in the overall transmit-readiness latency.

**o\_ll\_mod — Modulation Enable.** The Link Layer continuously monitors both `i_phy_ready` and `i_RF_ready`. Packet transmission is permitted only once both the digital and analog subsystems are ready. Once satisfied, the Link Layer asserts `o_ll_mod`, enabling modulation. While this signal remains active, the packet bits carried on `o_phy_bits` are sequentially transmitted through the modulation chain at the 1 MHz rate defined by the BLE 1M PHY. After the final packet bit has been sent, the Link Layer deasserts `o_ll_mod`, marking the completion of transmission and signaling the Control Unit to transition into Sleep Mode.

## Summary

Together, these signals show that the Link Layer interface does more than provide packet data — it guarantees proper synchronization among the Control Unit, the Physical Layer, and the RF subsystem, ensuring transmission begins only once the entire transmitter chain is fully operational. By separating the digital trigger (`o_ll_tx_start`), the RF trigger (`o_RF_start`), the readiness reporting (`i_phy_ready`, `i_RF_ready`), and the data-enable signal (`o_ll_mod`), the interface allows the digital and analog startup procedures to proceed in parallel, while still guaranteeing that modulation begins only once both domains — and in particular the slower-settling RF chain — are verifiably ready.

## Design Assumptions

Several assumptions were adopted during the implementation of the Link Layer interface to simplify the system design and facilitate integration between transmitter blocks:

1. **Constant bit rate.** The packet bits generated by the Link Layer and delivered through `o_phy_bits` are assumed to arrive at a constant rate of 1 MHz, corresponding to the BLE 1M PHY data rate.
2. **Independent initialization.** The digital PHY and the RF subsystem are assumed to operate independently during their initialization phase. Consequently, separate readiness signals (`i_phy_ready` and `i_RF_ready`) and separate trigger signals (`o_ll_tx_start` and `o_RF_start`) are used to start and indicate the completion of each startup process.
3. **Dual-readiness gating.** Modulation is not allowed to begin unless both `i_phy_ready` is asserted. This guarantees that the ADPLL has completed its locking procedure before any packet data is transmitted.
4. **Preloaded channel.** The BLE channel number is assumed to be preloaded into the register file before the transmission request is issued. The Link Layer simply retrieves the stored channel information and forwards it to the frequency synthesizer when required.
5. **Modulation duration.** The modulation enable signal remains asserted for the entire packet duration and is deasserted immediately after the transmission of the last packet bit.

6. **Conditional re-calibration.** Following Sleep Mode, PVT calibration is assumed to be re-executed only if the elapsed time since the previous transmission exceeds a tolerance threshold for timing/frequency drift; otherwise, the transmitter resumes directly from Modulation, bypassing the full startup latency.
7. **Average digital startup latency.** The full digital startup sequence (PVT Calibration through Training Sequence) is assumed to require approximately 36  $\mu$ s on average, and the Link Layer is assumed not to depend on this being a fixed value – it must wait for `i_phy_ready` and `i_RF_ready` rather than a timed delay.
8. **RF settling as the latency bottleneck.** The RF subsystem’s settling time, dominated by power amplifier bias and output settling, is assumed to be potentially longer than the digital startup latency. Accordingly, `o_PA_enable` is assumed to be asserted concurrently with `o_ll_tx_start` so that analog settling proceeds in parallel with digital calibration rather than being triggered afterward.

## Analog Interface

### Overview

The Analog RF block implements the frequency synthesizer and clock generation chain of the BLE transmitter, centered around the Digitally Controlled Oscillator (DCO) and its LC tank. It is controlled by a set of digitally generated capacitor-bank tuning words coming from the DCO interface and the sigma-delta modulator, along with an enable signal from the Control Unit and a crystal reference clock. In return, it provides the clock signals that drive the rest of the digital transmit chain, including the modulated DCO output, a divided sigma-delta clock, a sampled reference clock, and a cycle counter used for digital frequency/phase measurement.

Critically, the capacitor banks are not simply “coarse/medium/fine” by bit-width alone – each of the three DCO-interface banks corresponds to a specific ADPLL operating mode (PVT, Acquisition, or Tracking), and only the bank associated with the currently active mode is updated; the other banks hold their previous state. This ties the Analog RF interface directly to the Control Unit’s state machine described earlier.

Figure 110 shows the complete set of input and output signals exposed by the Analog RF block. The capacitor-bank state signals, `i_RF_en`, and `i_32_clk` are inputs to the block, while `o_dco_clk`, `o_sd_clk`, `o_ckr_clk`, and `o_dco_cycle_count` are outputs.

### Signal Descriptions

**i\_cap\_state\_m1 — PVT Capacitor Bank Control.** This 8-bit signal carries the tuning word for the PVT capacitor bank (`o_otw_p` at the DCO interface output), which is active and updated only while the Control Unit is in PVT Calibration mode. It performs the coarsest frequency steering, compensating for process-voltage-temperature variation before any closed-loop acquisition begins. The DCO interface converts the signed tuning word to an unsigned 256-level control word (via MSB inversion) before driving the physical capacitors, and holds its previous value whenever PVT mode is not active.

**i\_cap\_state\_m2 — Acquisition Capacitor Bank Control.** This 8-bit signal carries the tuning word for the ACQ capacitor bank (`o_otw_acq`), active and updated only while the Control Unit is in Acquisition mode. It performs the coarse-to-medium frequency steering



Figure 110: Analog RF interface block diagram showing input and output signals.

used to bring the DCO output close to the target channel frequency during the loop's initial lock-in. Like the PVT bank, it is a 256-level unsigned control word and holds its previous state when ACQ mode is inactive.

**i\_cap\_state\_m3 — Tracking Capacitor Bank Control.** This 6-bit signal carries the tuning word for the TRACK capacitor bank (`o_otw_track`), active and updated only while the Control Unit is in Tracking mode. It provides finer steering than the PVT and ACQ banks (64 levels vs. 256), refining the lock to steady-state accuracy. This bank includes a saturation guard to prevent the tuning word from wrapping or overflowing as the loop converges, and likewise holds its previous state when Track mode is inactive.

**i\_cap\_state\_m4 — Sigma-Delta Capacitor Bank Control.** This 3-bit signal carries the tuning word for the fourth and finest capacitor bank, driven directly by the sigma-delta modulator rather than the DCO interface's mode-multiplexed banks. The gain normalization block splits the loop filter's normalized phase error into an integer part (`o_otw_int`, sent to the DCO interface) and a fractional part (`o_otw_frac`, sent to the sigma-delta block); the sigma-delta modulator then dithers this fractional part at high speed to realize an effective frequency resolution finer than the physical capacitor step size of the other three banks would otherwise allow.

**PA\_enable — Power Amplifier Enable.** This signal is asserted by the Link Layer to enable the Power Amplifier in the RF front-end. It is routed through the register file interface, allowing the PA enable timing to be programmable and accommodating variations in PA settling time across process, voltage, and temperature conditions. The PA requires a non-negligible settling time before it can deliver stable and spectrally compliant output power, so this signal is asserted as early as possible — concurrently with `o_ll_tx_start` — to allow PA settling to proceed in parallel with the digital PHY startup sequence, preventing it from becoming the dominant term in the overall transmit-readiness latency.

**o\_dco\_clk — Modulated DCO Output Clock.** This output carries the DCO's oscillating output, onto which the GFSK modulation is applied. It is generated by the LC tank whose

capacitance is set by the combination of all four capacitor banks, and is the primary RF-rate clock of the transmitter. It is also fed back through a differential-to-single-ended converter and phase detector to close the loop, comparing it against the reference clock to generate the phase error signal. Because the tuning words on the four capacitor banks change as the ADPLL moves through PVT, acquisition, and tracking, the instantaneous frequency of `o_dco_clk` is not fixed during this period – it varies continuously as the loop converges toward the target channel frequency, and is only expected to settle once tracking has converged.

**`o_sd_clk` — Sigma-Delta Clock.** This output provides the clock used by the sigma-delta modulator, derived from the DCO output divided by 6 ( $o\_sd\_clk = o\_dco\_clk / 6$ ). Deriving this clock directly from the DCO output keeps the sigma-delta modulator's timing synchronous with the oscillator it is helping to tune. Since `o_sd_clk` is a direct division of `o_dco_clk`, it inherits the same frequency variation as the DCO output during the startup and locking process – as `o_dco_clk` varies while the loop converges, `o_sd_clk` varies proportionally along with it, rather than being a fixed-frequency clock.

**`o_ckr_clk` — Sampled Reference Clock.** This output is the crystal reference clock re-sampled into the DCO clock domain, produced by the retiming clock block shown in the DCO's high-level architecture. This retiming step is what allows the phase detector to compare the reference clock and the DCO output without encountering metastability issues from comparing two otherwise-asynchronous clock domains directly.

**`o_dco_cycle_count` — DCO Cycle Counter.** This output reports a running count of DCO clock cycles, incrementing on every falling edge of `o_dco_clk`. This count is important for computing the integer part of the variable phase: since the variable (DCO) phase is fundamentally a count of how many DCO periods have elapsed, the cycle counter provides this integer component directly, which is then combined with the fractional phase information obtained from the retimed reference clock sampling to form the complete variable phase estimate used by the digital domain.

**`reference_clock` — Reference Clock Output.** This output carries the crystal reference clock out of the Analog RF block to the digital domain, where it serves as the timing reference for the Link Layer and the broader digital PHY chain. Both the Link Layer and the Physical Layer are assumed to operate synchronously on this clock, eliminating clock domain crossing concerns at their interface and ensuring well-defined timing behavior across all control and data signals exchanged between the two layers.

## Summary

Together, these signals show that the Analog RF interface is tightly coupled to the Control Unit's operating mode: each of the three DCO-interface capacitor banks (PVT, ACQ, Track) is active only during its corresponding state, while the sigma-delta-driven fourth bank continuously provides sub-LSB dithering for finer resolution regardless of mode. The derived clocks (`o_sd_clk`, `o_ckr_clk`) and the cycle counter (`o_dco_cycle_count`) close the loop back to the digital domain, giving it everything needed to track and correct the DCO frequency without requiring any additional external circuitry.

## Design Assumptions

Several assumptions were adopted during the implementation and modeling of the Analog RF interface to simplify the system design and make the digital/analog co-simulation tractable:

1. **Fixed varactor mismatch.** The capacitor banks (PVT, ACQ, Track, and the sigma-delta-driven finest bank) are assumed to have a fixed, known mismatch profile across their unit elements. Random or time-varying mismatch between individual varactor cells is not modeled, meaning the mapping from digital tuning word to physical capacitance is treated as deterministic rather than subject to run-time process variation within a single die.
2. **Synchronous signal arrival at the varactors.** The tuning words driving the four capacitor banks are assumed to arrive at their respective varactor arrays synchronously, with no relative skew between banks. Any timing mismatch in how quickly a PVT, ACQ, Track, or sigma-delta update propagates to its physical capacitors is neglected, so all banks are assumed to update their effective capacitance at the same effective instant relative to the DCO clock edge.
3. **Single active bank per mode.** Only the capacitor bank corresponding to the Control Unit's current operating mode (PVT, ACQ, or Track) is assumed to be updated at any given time; the other two mode-dependent banks are assumed to perfectly hold their last value with no leakage or drift while inactive.
4. **Ideal divide-by-6 for the sigma-delta clock.** The generation of `o_sd_clk` from `o_dco_clk` is assumed to be an ideal integer division by 6, with no duty-cycle distortion or accumulated phase error introduced by the divider itself, beyond the frequency variation it naturally inherits from `o_dco_clk`.
5. **No varactor nonlinearity across the tuning range.** The capacitance-vs-tuning-word relationship for each bank is assumed to be sufficiently linear (or well-characterized) across its operating range that the digital tuning word can be treated as a direct, monotonic proxy for frequency step size, without needing to model nonlinear capacitance compression at the extremes of the tuning range.

## Power Management Unit

### Introduction

#### Why Bluetooth Low Energy (BLE) Needs to Save Power

Bluetooth Low Energy (BLE) is very different from classic Bluetooth. Instead of sending a lot of data quickly, BLE is designed to use as little battery power as possible. This makes it perfect for small, cheap devices like sensors or trackers that run on tiny coin-cell batteries (like the CR2032) shown in Figure 111. To make a battery last for years, BLE uses an unbalanced design. It forces larger devices, like smartphones, to do all the heavy processing. This allows the small BLE sensors to stay simple and save their power.



Figure 111: Small sensor battery

### **”Time is Energy” and Power Leakage**

The golden rule of BLE is ”Time is Energy.” The longer a device stays awake, the more battery it burns. To save power, BLE devices spend almost their entire lives in a deep sleep. They only wake up for a tiny fraction of a second to send a quick message, and then they immediately go back to sleep. Because the chip spends so much time sleeping, the biggest drain on the battery is not the actual transmission. Instead, the biggest drain is leakage current.

### **Power Gating Design**

To reduce leakage power during long sleep periods, we use power gating that acts like a physical switch completely cutting off electricity to sections of the chip that are not being used. Our chip is divided into four separate power blocks, also called ”domains”:

- **Always-On Domain (AON):** This part never sleeps. It contains the **FSM wakeup PMU and Clock management system**. It stays awake to remember configuration and controls the power switches for the rest of the chip.
- **Switch domain :** this part contains **Control unit , ADPLL , Register file**

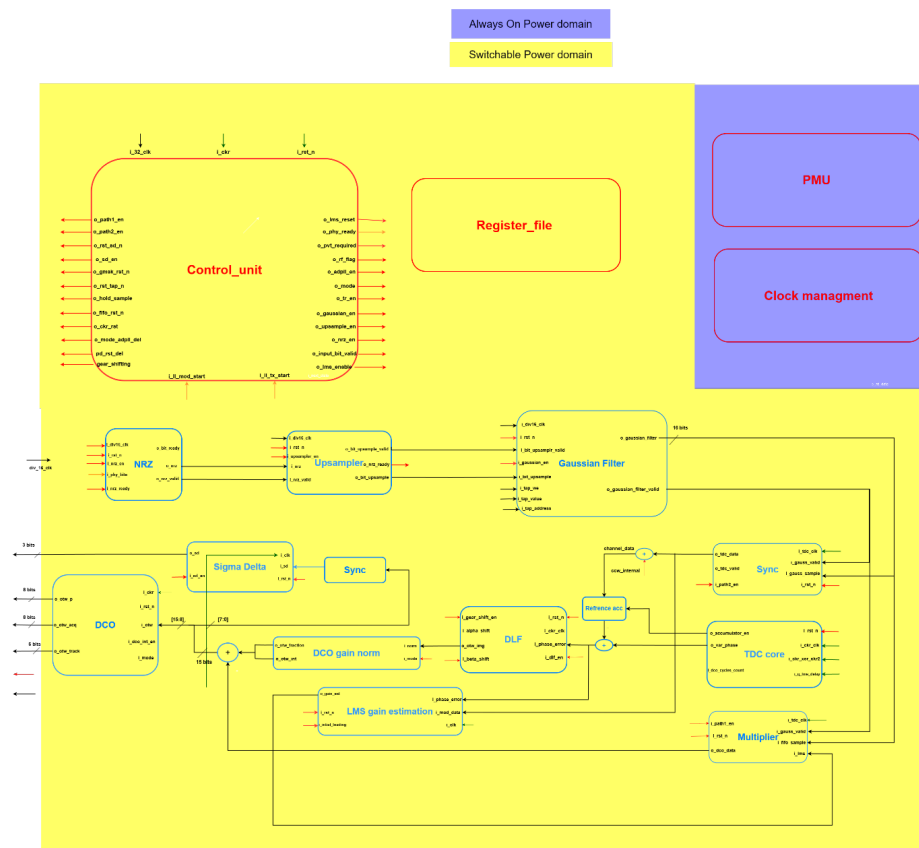


Figure 112: Power domain in system. Yellow indicates the switchable power domain, while blue indicates the always-on power domain.

## Power Management

The PMU FSM is shown in Figure 113 below.

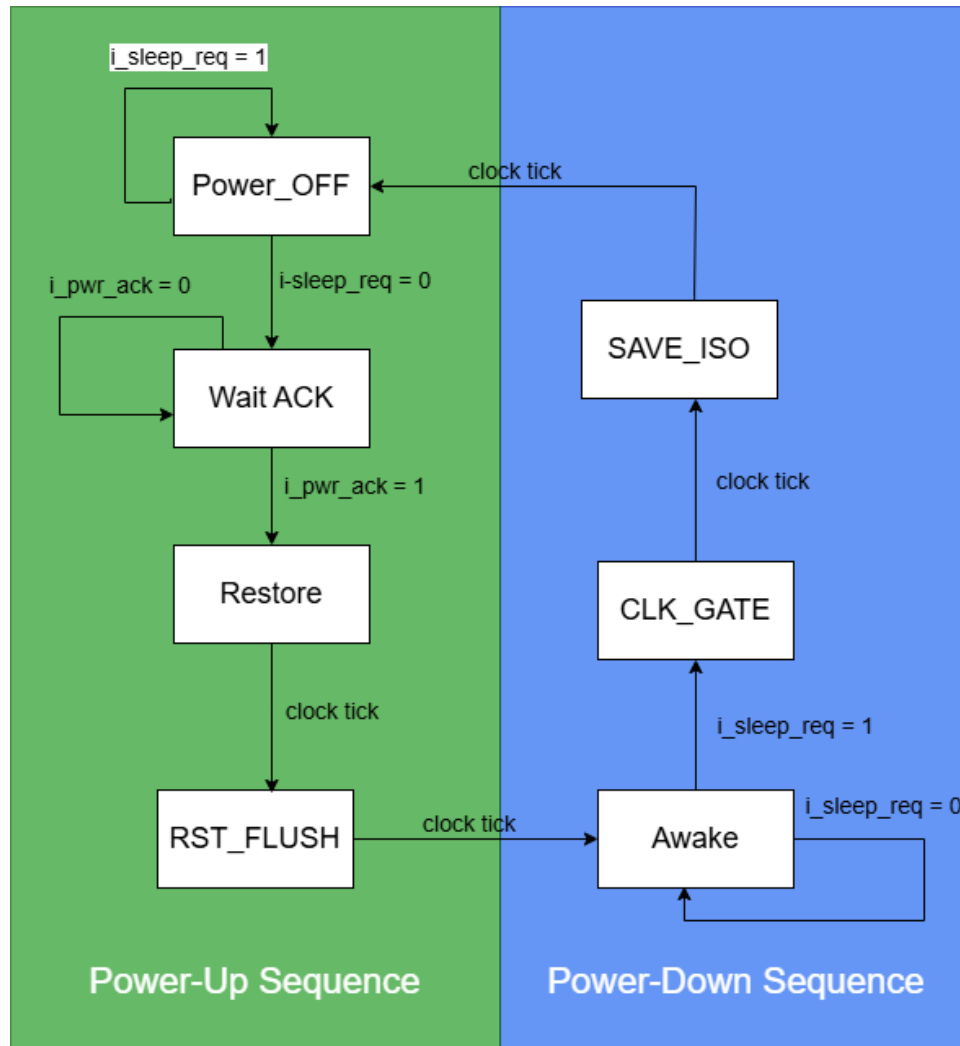


Figure 113: PMU fsm

### 1. The Power-Up Sequence (Waking Up)

When the Link Layer wakes the radio for an upcoming transmission, control unit pulls the sleep request low ( $i\_sleep\_req = 0$ ). The FSM must sequence the power-up carefully to avoid voltage drop and data corruption:

- **Wait ACK State (Power Switch Turn-On):** The FSM enables power switches. Because it takes time for the voltage rails to physically charge and stabilize, the FSM stalls in this state until it receives a power acknowledgment ( $i\_pwr\_ack = 1$ ) from the voltage monitors, guaranteeing the domain is fully energized.
- **Restore State:** Once stable power is confirmed, the FSM asserts the restore signal. To restore all data that safely stored in the AON domain back.
- **RST\_FLUSH State:** The FSM issues a brief, synchronous reset pulse. This flushes out any unpredictable "garbage" states generated inside normal flip-flops, ensuring a clean operational slate.

- **Awake State:** After isolation signals are lifted, the clock gating is disabled, and the system will continue to work normal .

### The Power-Down Sequence (Entering Sleep)

The power-down sequence is initiated when packet end , control unit assert the sleep request signal ,The FSM immediately transitions towards power off state through different state to make sure data we need to save is saved properly without corruption.

- **CLK\_GATE State:** Before altering any power rails, the FSM safely gates the system clocks, to stop all dynamic preventing from saving corrupted data.
- **SAVE\_ISO State (Save & Isolate):** On the next clock tick, the FSM asserts the state retention save signals and enables the domain boundary isolation cells.
- **Power\_OFF State:** on the next clock edge FSK goes to POWER\_OFF State as long as  $i\_sleep\_req = 1$ .

### Configuration Register File

The APB register file enables firmware-level configuration of critical system parameters. This programmability allows the IP to be tuned post-fabrication, actively compensating for hardware non-idealities or process variations to achieve the desired system functionality. Specifically, the register map provides access to the GMSK Gaussian filter coefficients, allowing for the pre-distortion of modulation data if the transmission response violates specifications. Furthermore, it provides configurations for the Timing Management Unit (TMU), the ADPLL Control Unit, the Digital Loop Filter (DLF), and the Least Mean Squares (LMS) calibration engine to ensure optimal performance and precise frequency synthesis.

The register file is implemented as a 16-bit AMBA Advanced Peripheral Bus (APB) v.2 slave interface. The finite state machine (FSM) governing the APB slave operations is illustrated in Figure 114.

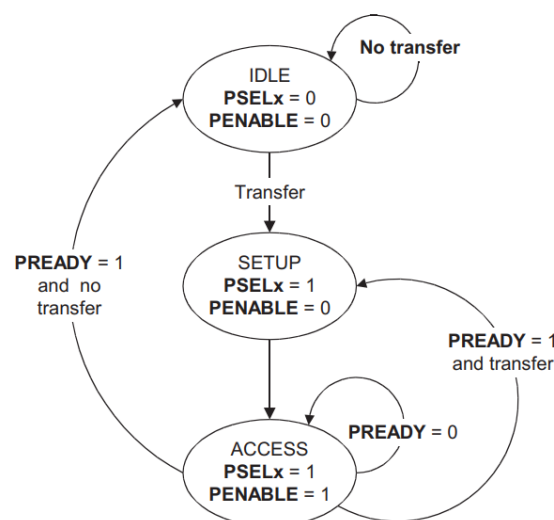


Figure 114: APB v.2 slave FSM [19]

To streamline transactions, this implementation ties the **PSLVERR** signal permanently to 0 (indicating no transfer errors) and the **PREADY** signal to 1, ensuring zero-wait-state access for all read and write operations.

**Register Map**

| Offset      | Name            | Access | Reset       | Description                      |
|-------------|-----------------|--------|-------------|----------------------------------|
| 0x00 - 0x1F | GMSK_TAP_MEM    | R/W    | 0x0000_0000 | Gaussian Filter Coefficients     |
| 0x20        | TMU_CFG_PVT_ACQ | R/W    | 0x0000_0000 | PVT & Acquisition Durations      |
| 0x21        | TMU_CFG_TRACK   | R/W    | 0x0000_0000 | Tracking Duration                |
| 0x22        | TMU_CFG_TRAIN   | R/W    | 0x0000_0000 | Training Sequence Duration       |
| 0x23        | CU_CFG_CHANNEL  | R/W    | 0x0000_0000 | RF Channel Configuration         |
| 0x24        | DLF_CFG_COEF    | R/W    | 0x0000_0000 | Digital Loop Filter Coefficients |
| 0x25        | LMS_CFG_INIT    | R/W    | 0x0000_0000 | LMS Initialization & Trigger     |
| 0x26        | LMS_READ_STORED | R/O    | 0x0000_0000 | LMS Stored Estimation            |

**1. GMSK\_TAP\_MEM (0x00 - 0x1F) - Gaussian Filter Coefficients**

| Bits | Name        | Access | Description                                                                                                          |
|------|-------------|--------|----------------------------------------------------------------------------------------------------------------------|
| 15:0 | Coefficient | R/W    | 32 registers that store the half-taps for the GMSK Gaussian filter. Each register holds a 16-bit signed coefficient. |

**2. TMU\_CFG\_PVT\_ACQ (0x20) - PVT & Acquisition Durations**

| Bits | Name         | Access | Description          |
|------|--------------|--------|----------------------|
| 13:6 | tmu_acq      | R/W    | Acquisition duration |
| 5:0  | tmu_pvt_time | R/W    | PVT duration         |

**3. TMU\_CFG\_TRACK (0x21) - Tracking Duration**

| Bits | Name         | Access | Description       |
|------|--------------|--------|-------------------|
| 9:0  | tmu_tracking | R/W    | Tracking duration |

**4. TMU\_CFG\_TRAIN (0x22) - Training Sequence Duration**

| Bits | Name          | Access | Description       |
|------|---------------|--------|-------------------|
| 9:0  | tmu_train_seq | R/W    | Training duration |

**5. CU\_CFG\_CHANNEL (0x23) - RF Channel Configuration**

| Bits | Name            | Access | Description             |
|------|-----------------|--------|-------------------------|
| 11:7 | cu_channel_frac | R/W    | Channel fractional part |
| 6:0  | cu_channel_int  | R/W    | Channel integer part    |

**6. DLF\_CFG\_COEF (0x24) - Digital Loop Filter Coefficients**

| Bits  | Name            | Access | Description                                           |
|-------|-----------------|--------|-------------------------------------------------------|
| 15:12 | dlf_alpha_pvt   | R/W    | Exponent of proportional path in PVT & acq            |
| 11:8  | dlf_alpha_track | R/W    | Exponent of proportional path in Acquisition/Tracking |
| 7:4   | dlf_beta_pvt    | R/W    | Exponent of integral path in PVT & acq                |
| 3:0   | dlf_beta_track  | R/W    | Exponent of integral path in Tracking                 |

**7. LMS\_CFG\_INIT (0x25) - LMS Initialization & Trigger**

| Bits | Name           | Access | Description                                                           |
|------|----------------|--------|-----------------------------------------------------------------------|
| 11   | lms_load_pulse | R/W    | Load Trigger. Writing a 1 to this bit fires a 1-cycle lms_load_pulse. |
| 10:0 | lms_init_val   | R/W    | The initial LMS value to load                                         |

**8. LMS\_READ\_STORED (0x26) - LMS Stored Estimation**

| Bits | Name            | Access | Description                                                          |
|------|-----------------|--------|----------------------------------------------------------------------|
| 10:0 | LMS Stored Est. | R/O    | Read-only register containing the final/stored LMS estimation value. |

## Time and Power Optimizations

To minimize the time and power overhead of this calibration during future transmission events, the converged LMS gain coefficients are preserved in shadow registers. Because these registers reside within the Always-On (AON) power domain, they survive deep sleep cycles. When the BLE device wakes up for subsequent packets, it seeds the LMS algorithm with these previously estimated values, drastically reducing the required training duration. Furthermore, these hardware registers are fully exposed to the software layers, allowing the firmware to clear, read, or manually override the DCO gain parameters, ensuring the transceiver remains fully configurable.

## Simulation Results

### Gaussian Filter

#### MATLAB Modeling

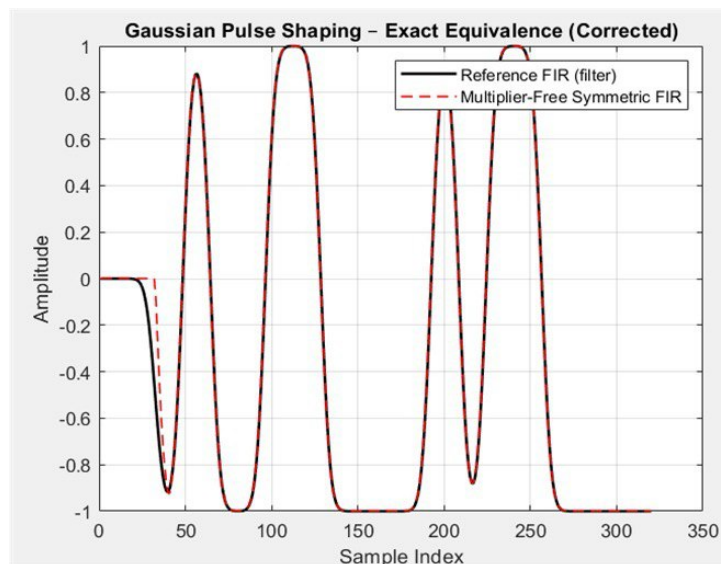


Figure 115: Time-aligned NRZ input and Gaussian filter output after group-delay compensation

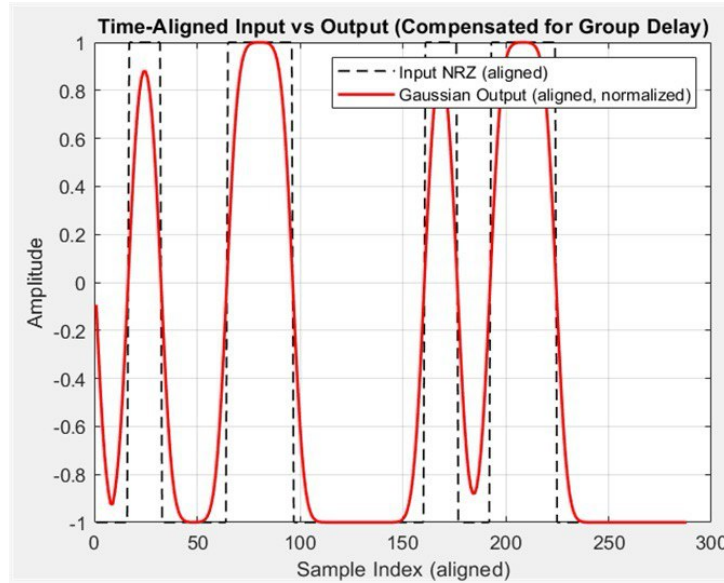


Figure 116: Amplitude comparison between reference FIR and multiplier-free symmetric FIR implementations

The plotted output demonstrates exact equivalence between the reference FIR implementation and the proposed multiplier-free symmetric realization (Gaussian filter). Beyond equivalence, the waveform also illustrates an important property of Gaussian pulse shaping: the output cannot change instantaneously when the input bit changes. Instead, the filter spreads each symbol transition over multiple samples, producing smooth edges.

### Why the Waveform Does Not Immediately Reach +1 or −1 After a Bit Transition

The Gaussian FIR output is a weighted sum (weighted average) of recent input samples:

$$y[n] = \sum_{k=0}^{N-1} h[k] x[n-k]$$

with  $h[k] \geq 0$  and typically normalized such that:

$$\sum_{k=0}^{N-1} h[k] = 1$$

Because of this, immediately after a transition the filter still “remembers” previous samples.

#### Case A: transition from +1 → −1 (often described as 1 → 0 mapped to −1)

Immediately after the bit flips to −1, many samples in the delay line are still +1 from the previous symbol. The output is therefore a mixture:

- some terms contribute  $+h[k]$  from older +1 samples,
- some terms contribute  $-h[k]$  from newer −1 samples.

Since both contributions are present, the sum is not yet equal to −1. The output moves downward gradually and only approaches −1 after the delay line becomes dominated by −1 samples. If another transition occurs before this settling completes, the waveform may never fully reach −1.

**Case B: transition from  $-1 \rightarrow +1$  (0 mapped to  $-1$  then 1)**

The same effect occurs in the opposite direction. After switching to  $+1$ , the delay line still contains some  $-1$  samples, which pull the weighted sum below  $+1$ . The output rises smoothly toward  $+1$  and reaches it only once the filter memory contains mostly  $+1$  samples.

**Key takeaway:** the output level after a transition depends on how long the new symbol remains constant. Longer runs allow the waveform to settle closer to  $\pm 1$ , while frequent transitions keep the waveform between the extremes. This behavior is expected and represents the fundamental mechanism by which Gaussian pulse shaping reduces abrupt spectral components.

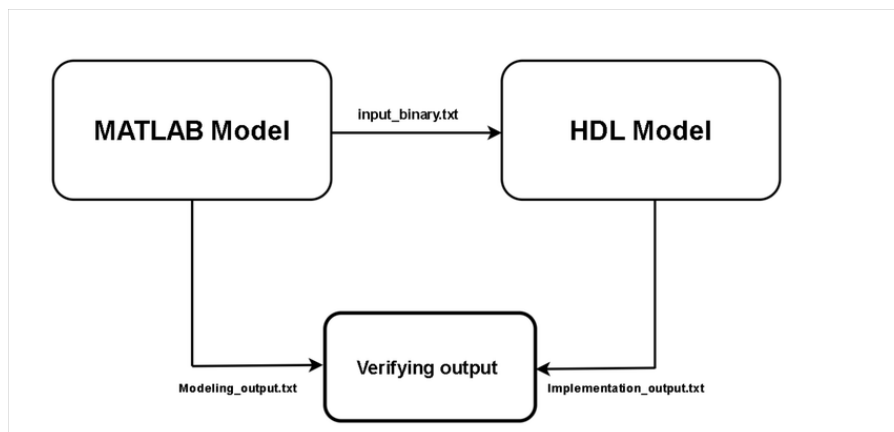
**RTL Verification**

Figure 117: Verification flow comparing the MATLAB bit-accurate model with the HDL model.

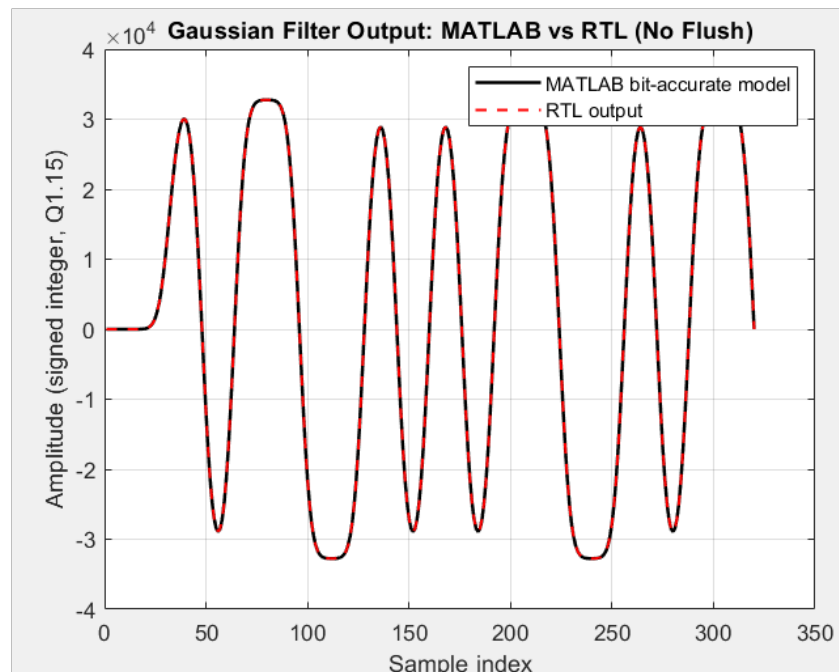


Figure 118: Gaussian filter output: MATLAB bit-accurate model vs. RTL simulation (no flush).

The final verification compares the RTL Gaussian filter output with the MATLAB bit-accurate fixed-point model. The MATLAB model follows the same processing chain used in the RTL: binary input bits are first converted to signed NRZ values, each NRZ symbol is upsampled by a factor of 16, and the resulting sample stream is processed by the Gaussian FIR filter.

The plot shows the Gaussian filter output generated by the MATLAB model and the output generated by the RTL simulation. The MATLAB result is plotted as the reference bit-accurate Q1.15 model, while the RTL output is plotted as the simulated hardware result. The two curves overlap, confirming that the RTL implementation follows the same fixed-point behavior as the MATLAB model.

This agreement verifies several important design points:

- The NRZ mapping and upsampling behavior are consistent between MATLAB and RTL.
- The Gaussian filter coefficients used in the RTL match the Q1.15 coefficients generated in MATLAB.
- The RTL symmetric-pair accumulation and saturation behavior match the MATLAB fixed-point model.

Since the comparison is performed using signed integer Q1.15 values, the result verifies the design at the fixed-point level rather than only at the floating-point algorithm level. The RTL implementation does not use floating-point arithmetic; instead, it uses quantized coefficients, integer accumulation, and saturation. The close overlap between the two outputs confirms that the fixed-point RTL implementation accurately reproduces the expected Gaussian pulse-shaping response.

## Time to Digital Converter

### MATLAB Modeling

A time-domain MATLAB model of the TDC was developed to verify correct functionality and to quantify the measurement error introduced by the TDC delay-line quantization and the finite bit width used to represent the fractional part of the measured phase.

The nominal buffer (tap) delay used in the model is 10 ps, selected as an estimate for a standard buffer implemented in a 32nm process node. The TDC was exercised across the DCO frequency range expected after PVT calibration which is 2.16GHz - 2.64GHz. The simulation steps the DCO period in increments of 2ps (i.e., successive tested CKV periods differ by 2ps) to sample a dense set of operating points within the frequency window.

The fractional part of the TDC output was represented with 10 bits. This choice provides margin relative to the minimum fractional resolution implied by the BLE 5.4 requirement: a frequency deviation step of 250kHz for a reference frequency 32MHz which corresponds to a frequency control word step of :

$$FCW_{\min} = \frac{250k}{32M} = 0.0078125 = \frac{1}{2^7}$$

so at least 7 fractional bits are required to represent the frequency deviation step. The TDC fractional word is chosen to be 10 bits to give extra headroom for quantization.

For each selected DCO frequency the model computed the digital phase for 21 consecutive reference cycles to ensure correct accumulation of integer and fractional parts. At each reference cycle the simulated TDC measurement (integer count plus fractional word) was compared to the expected accumulated phase to yield a per-cycle measurement error. For each CKV period the maximum absolute error across the 21 cycles was recorded. The resulting maximum error across all reference cycles was plotted as a function of the CKV period, which is shown in figure 119 below:

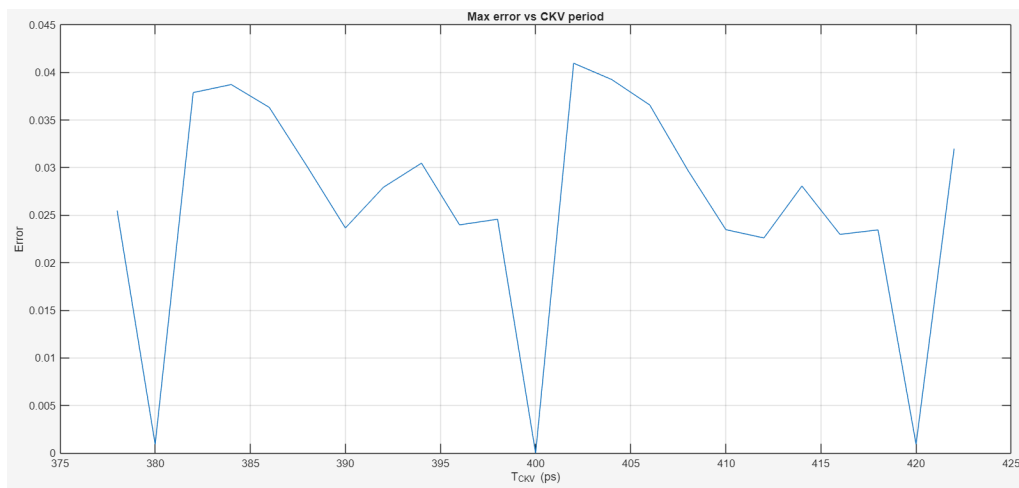


Figure 119: Maximum TDC error against DCO clock period

As shown in figure the maximum error found is **0.041**.

## RTL Verification

### Test Bench Methodology

The testbench evaluates the TDC by selecting a CKV period, simulating 250 reference cycles, and comparing the measured variable phase against an ideal MATLAB model that computes the running sum of  $\frac{f_{DCO}}{f_{REF}}$  without quantization or rounding. The choice of 250 reference cycles increases the coverage of fractional values exercised by the testbench while keeping simulation time manageable. After each run, the TDC is reset, the CKV period is changed, and an additional settling interval is allowed before the next 250-cycle measurement window begins. This procedure stresses the random-phase-start correction logic by exposing the design to different initial phase relationships between CKV and FREF.

The CKV period is swept over the expected operating range of the DCO. In the current testbench, the sweep is performed from 348ps to 520ps in 2ps steps, corresponding to a DCO frequency range of approximately 2.87GHz to 1.92GHz. This range covers the expected operating frequencies of the DCO as well as additional 20% margin for PVT variations. Three additional corner cases at 250ps, 400ps, and 500ps are used as preliminary checks. These corner cases verify that the TDC reports the phase correctly when the fractional part is zero, without introducing spurious cycle errors.

### Test Bench Results

The maximum discrepancy between the ideal MATLAB model and the TDC implementation was 3039 in Q24.16 format, corresponding to an error of approximately **0.0464**. This error is primarily due to quantization in the tapped delay line, since the CKV edge is resolved using discrete tap delays. The effect becomes more pronounced at higher CKV frequencies, where fewer taps span one CKV period and the phase resolution becomes coarser. For example, at a CKV period of 348ps and a tap delay of 10ps, one CKV cycle spans approximately 34.8 taps, however, the rise and fall signals used to calculate the period could be off by 1 leading to number of taps being 30. This corresponds to a resolution of about  $(1/30 \approx 0.0333)$ . Since both the initial fractional phase and the registered fractional phase are quantized before subtraction, this quantization error can propagate into the final phase estimate.

## Output Waveforms

The output waveforms for difference DCO clock periods is presented below to showcase correct functionality of the TDC.

The output waveform for DCO clock frequency equal to 2.5GHz is shown in Figure 120 below:

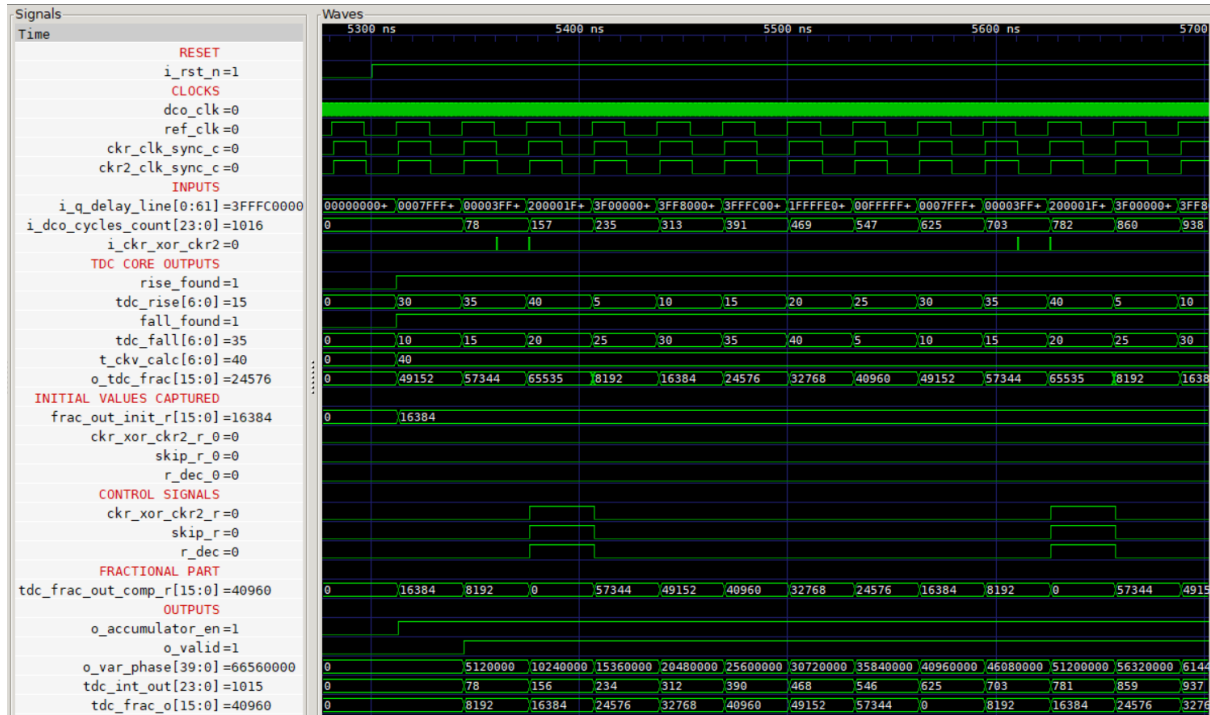


Figure 120: Output waveforms for DCO clock frequency = 2.5GHz

For DCO clock frequency of 2.5GHz the frequency ratio is equal to:

$$\text{Frequency ratio} = \frac{2.5\text{GHz}}{32\text{MHz}} = 78.125$$

So the ratio 78.125 will get accumulated each re-timed reference cycle which is done as shown in Figure 120.

The output waveform for DCO clock frequency equal to 4GHz is shown in Figure 121 below:

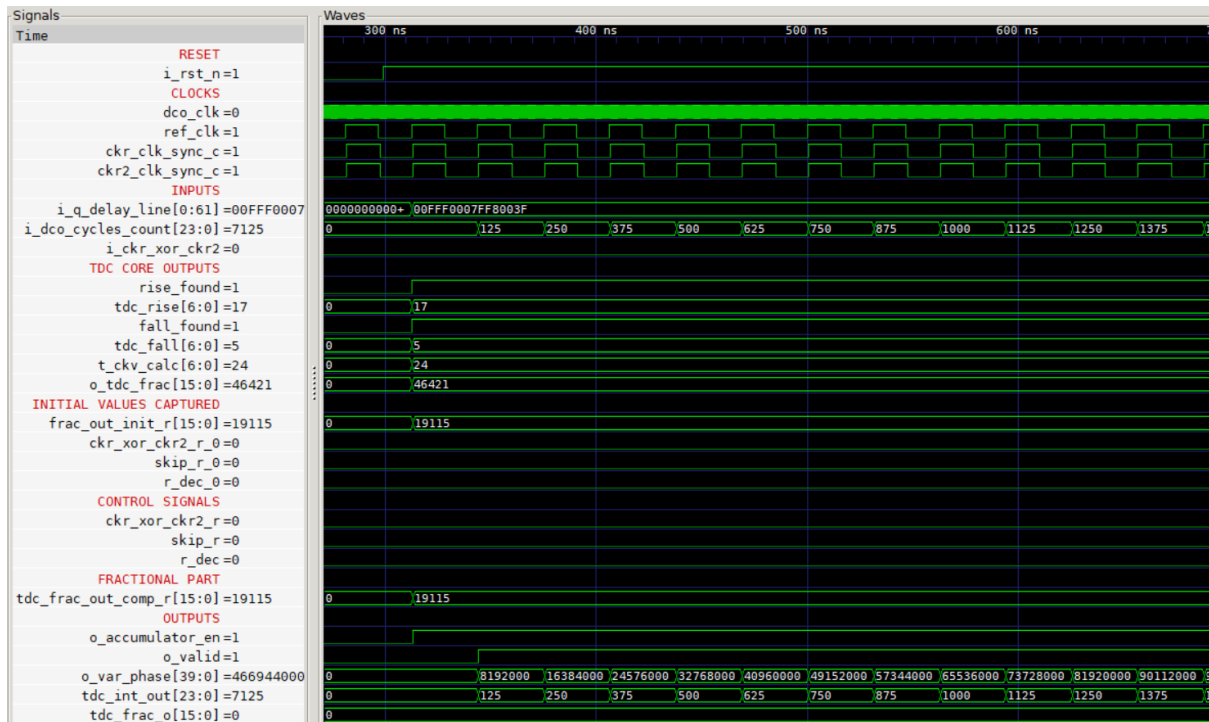


Figure 121: Output waveforms for DCO clock frequency = 4GHz

For DCO clock frequency of 4GHz the frequency ratio is equal to:

$$\text{Frequency ratio} = \frac{4\text{GHz}}{32\text{MHz}} = 125$$

As shown in Figure 121 the ratio 125 gets accumulated correctly each reference cycles with no spurious one cycle errors.

The output waveform for DCO clock period of 364 ps is shown in Figure 122 below:



Figure 122: Output waveforms for DCO clock period = 364 ps

For DCO clock period of 364 ps the frequency ratio is equal to:

$$\text{Frequency ratio} = \frac{31250 \text{ ps}}{364 \text{ ps}} = 85.852$$

As shown in Figure 122 the ratio 85.852 gets accumulated correctly each reference cycles within acceptable error even when `r_dec_0` is asserted. This shows that the random initial phase correction is applied appropriately.

### Code Coverage

To ensure that the testbench exercises the TDC logic, and in particular the error-correction path, code coverage was collected on the `TDC_CORE_TOP` module. The coverage report initially showed holes in toggle coverage because the upper bits of `i_dco_cycles_count` do not toggle within a 250-reference-cycle simulation window as shown in Figure 123. This is expected, since the variable phase does not become large enough to exercise those bits. To avoid unnecessary simulation time, exclusions were added for the unused upper bits of the DCO cycle counter. After applying these exclusions, the coverage report reached 100% as shown in Figure 124, indicating that the testbench fully exercised the intended TDC logic.

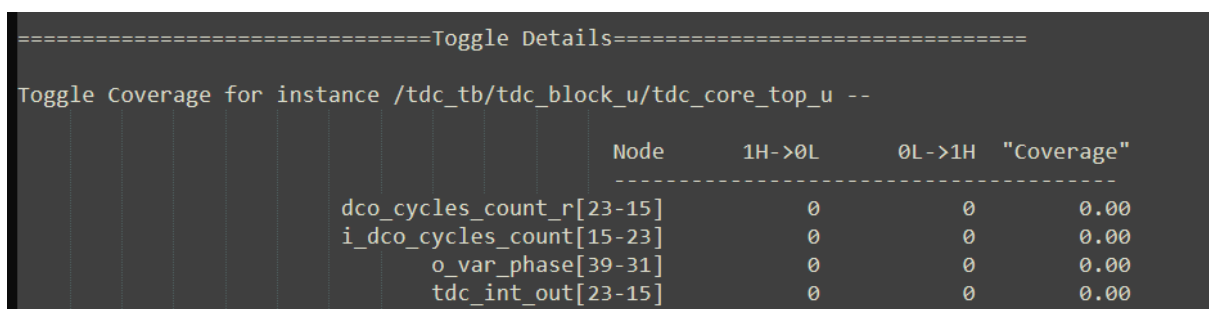


Figure 123: Toggle coverage holes in TDC testbench

| Design Units Coverage Summary ( 100% ) |           |           |           |           |           |
|----------------------------------------|-----------|-----------|-----------|-----------|-----------|
| Coverage Type ↑                        | Bins      | Hits      | Misses    | Coverage  |           |
| Search...                              | Search... | Search... | Search... | Search... | Search... |
| Branches                               | 15        | 15        | 0         | 100%      |           |
| Conditions                             | 2         | 2         | 0         | 100%      |           |
| Expressions                            | 2         | 2         | 0         | 100%      |           |
| Statements                             | 36        | 36        | 0         | 100%      |           |
| Toggles                                | 464       | 464       | 0         | 100%      |           |

Figure 124: Coverage report summary for TDC testbench

The target fractional variable phase space defined during the TDC verification process is presented in the MATLAB plot of Figure 125. To clearly illustrate the coverage density, the plotted data consists of all unique expected fractional parts collected across the measurements and sorted by magnitude, with the horizontal axis serving simply as a sequential index. The dense distribution of these expected values confirms that the deterministic testbench methodology robustly covers the necessary stimulus spectrum, forcing the design into numerous critical phase-alignment scenarios.

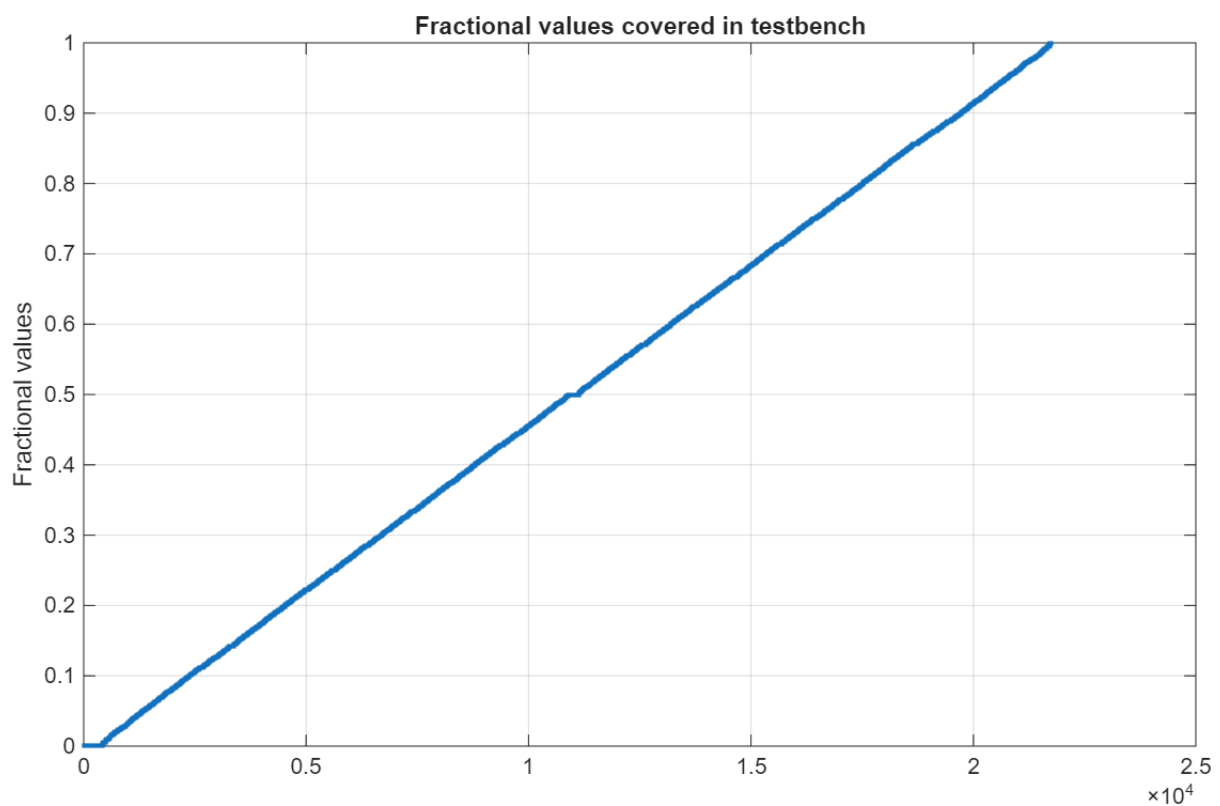


Figure 125: Fractional parts covered in TDC testbench

## Loop Filter

### MATLAB Modeling

To analyze the performance of the proposed type II ADPLL architecture, an ideal sample based ADPLL model was implemented in MATLAB. This simulation focuses on the transient behavior of the Digitally Controlled Oscillator (DCO) as it tracks a desired frequency reference under specific loop filter constraints.

Assuming DCO central frequency = 2.35GHz, target frequency = 2.432GHz and loop filter coefficients are changed to see their effect on performance.

Figures 126, 127 and 128 illustrate the transient response of the system displaying the DCO frequency settling behavior for different  $\alpha$  and  $\rho$  values.

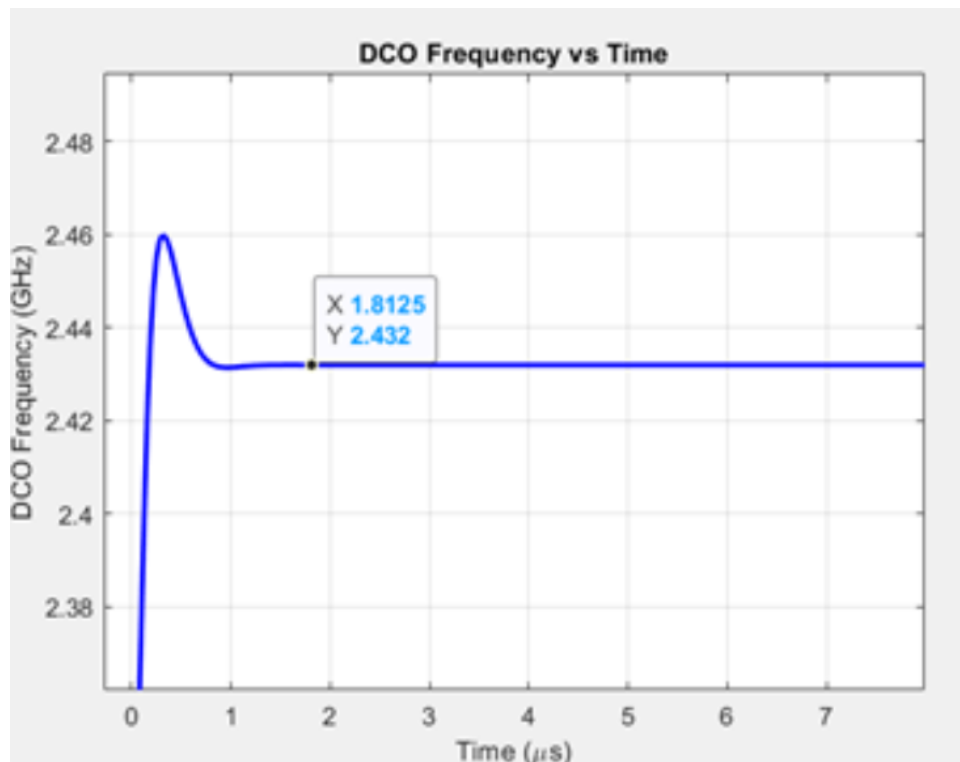


Figure 126: Transient response for  $\alpha = 2^{-2}$ ,  $\rho = 2^{-5}$  (critically damped response)

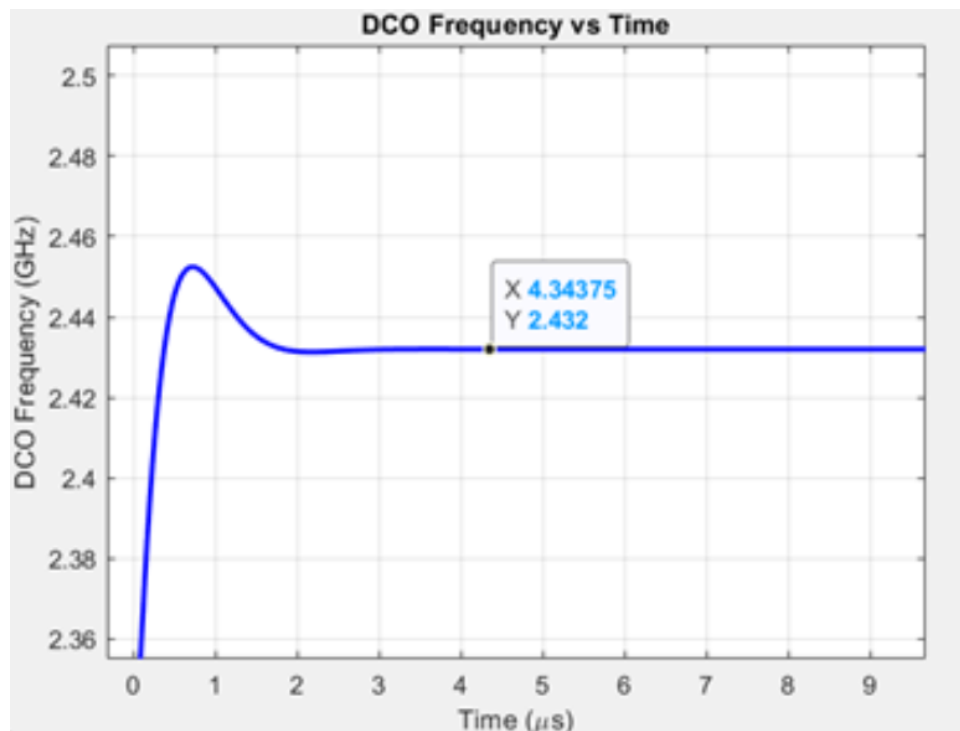


Figure 127: Transient response for  $\alpha = 2^{-3}$ ,  $\rho = 2^{-7}$  (critically damped response)

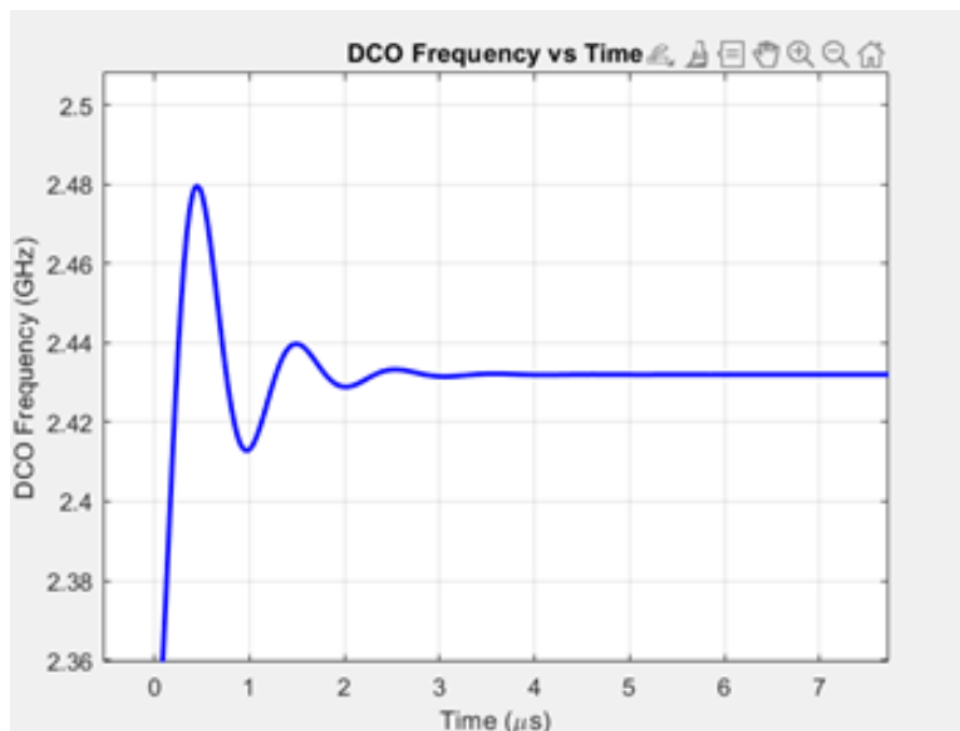


Figure 128: Transient response for  $\alpha = 2^{-3}$ ,  $\rho = 2^{-5}$  (under-damped response)

The simulation results shown above illustrate the critical trade-off between speed and stability in the system which was predicted using the  $s$ -domain approximation.

Sample Based Ideal Matlab model critically damped system response at Type II ADPLL operation with  $\alpha = 2^{-6}$ ,  $\rho = 2^{-13}$

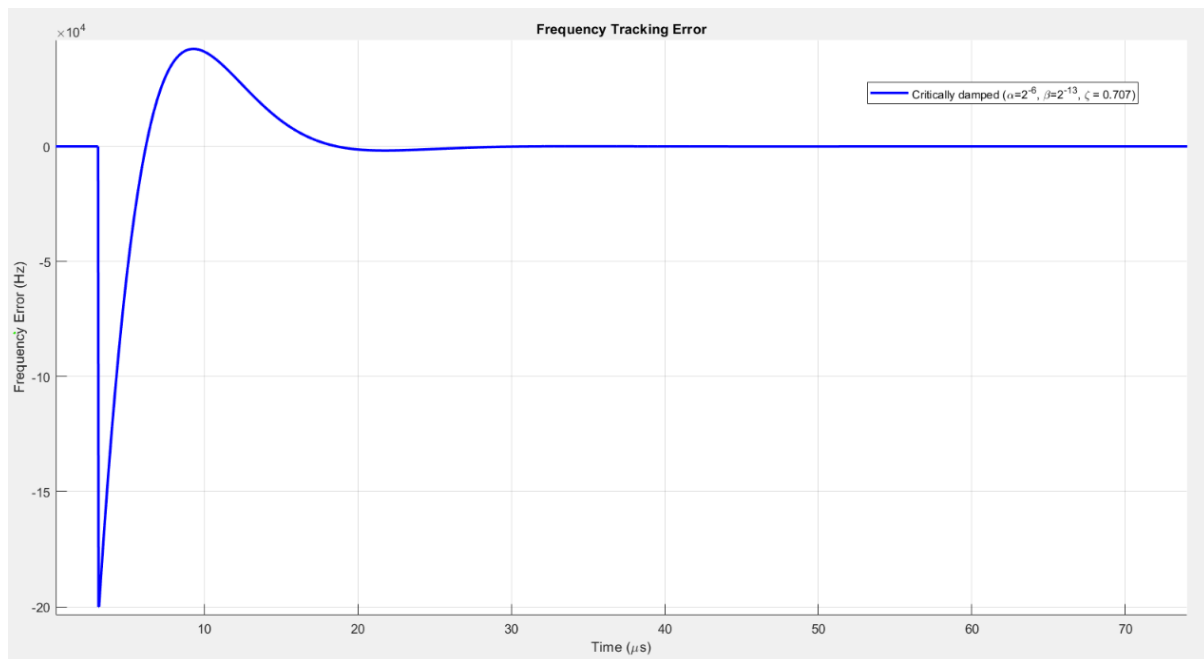


Figure 129: Digital loop filter RTL output vs MATLAB

## RTL Verification

Loop filter verification was performed by testing the loop filter within the ADPLL time-based MATLAB model, applying the same inputs to the RTL implementation, and comparing its outputs against the loop filter in the model. The comparison between RTL and MATLAB model are shown in Figure 130

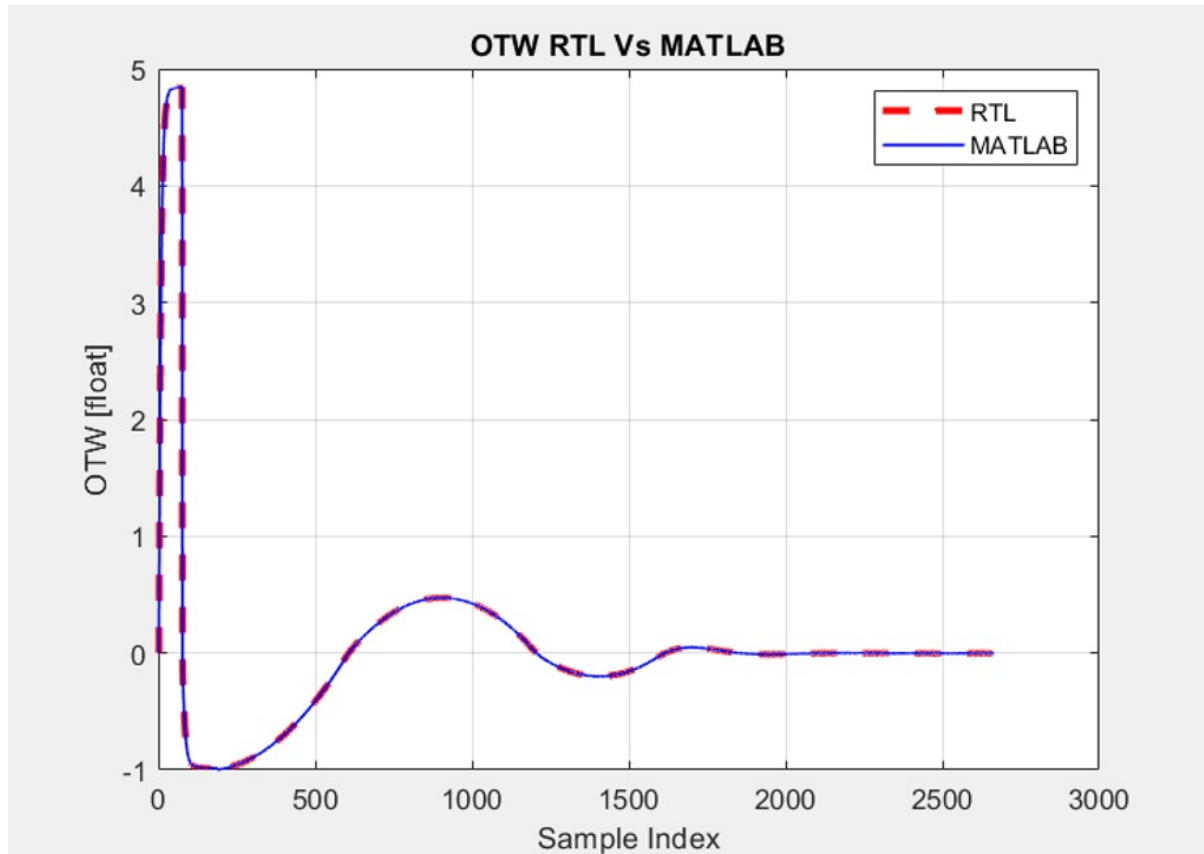


Figure 130: Digital loop filter RTL output vs MATLAB

As shown in Figure 130 the RTL output is identical to that of the MATLAB model confirming the correctness of the implemented loop filter.

## Sigma Delta

### MATLAB Simulation

Table 18: Extended clock-cycle behavior of a MASH 1–1–1 sigma–delta modulator for  $X_{in} = 0.25$

| Clock | $S_{out1}$ | $S_{out2}$ | $S_{out3}$ | $C_{out1}$ | $C_{out2}$ | $C_{out3}$ | $Y_{out}$ |
|-------|------------|------------|------------|------------|------------|------------|-----------|
| 1     | 0.25       | 0.25       | 0.25       | 0          | 0          | 0          | 0         |
| 2     | 0.50       | 0.75       | 0.00       | 0          | 0          | 1          | 1         |
| 3     | 0.75       | 0.50       | 0.50       | 0          | 1          | 0          | -1        |
| 4     | 0.00       | 0.50       | 0.00       | 1          | 0          | 1          | 2         |
| 5     | 0.25       | 0.75       | 0.75       | 0          | 0          | 0          | -2        |
| 6     | 0.50       | 0.25       | 0.00       | 0          | 1          | 1          | 3         |
| 7     | 0.75       | 0.00       | 0.25       | 0          | 1          | 0          | -2        |
| 8     | 0.00       | 0.00       | 0.25       | 1          | 0          | 0          | 1         |
| 9     | 0.25       | 0.25       | 0.50       | 0          | 0          | 0          | 0         |
| 10    | 0.50       | 0.75       | 0.25       | 0          | 0          | 1          | 1         |
| 11    | 0.75       | 0.50       | 0.75       | 0          | 1          | 0          | -1        |
| 12    | 0.00       | 0.50       | 0.25       | 1          | 0          | 1          | 2         |

Table 18 (simulation based) summarizes the extended clock-cycle behavior of the sample-based Original MASH 1–1–1 sigma–delta modulator for a constant input  $X_{in} = 0.25$ . The table lists the internal accumulator states, carry outputs, and the resulting digital output over multiple clock cycles.

The following results are comparing between different types of sigma delta, firstly the original MASH without any dither and XOR based dither in fig 83 Modified Mash that doesnt require dithering in fig 82 and for the normal dither the dither is input of the second accumulator and lastly the modified dither is our proposed MASH in fig. 83

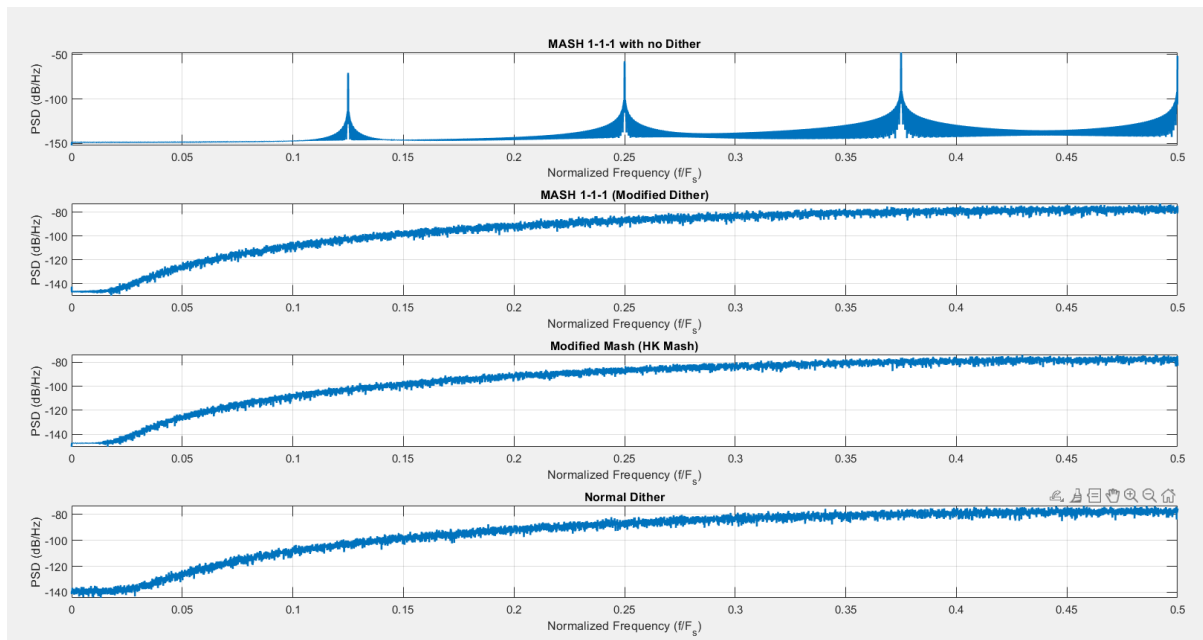


Figure 131: Extended output spectrum of the sample-based MASH 1–1–1  $\Sigma\Delta$  modulator for input 0.25

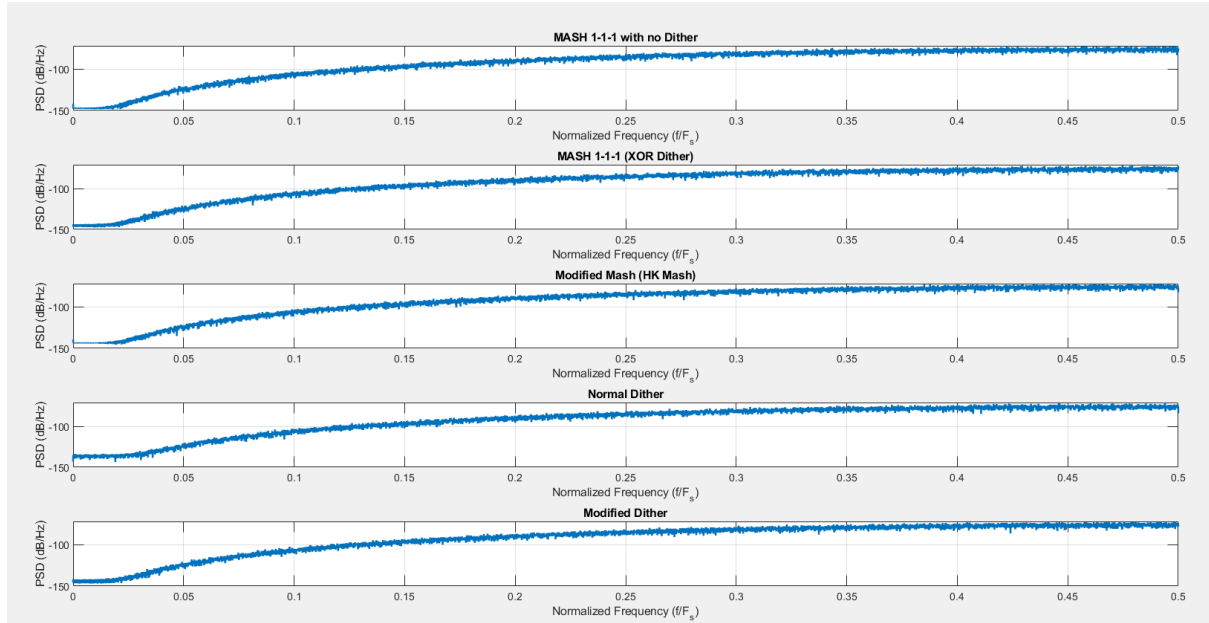


Figure 132: Extended output spectrum of the sample-based MASH 1–1–1  $\Sigma\Delta$  modulator for an irrational input ( $X_{in} = \sqrt{2}/10$ ).

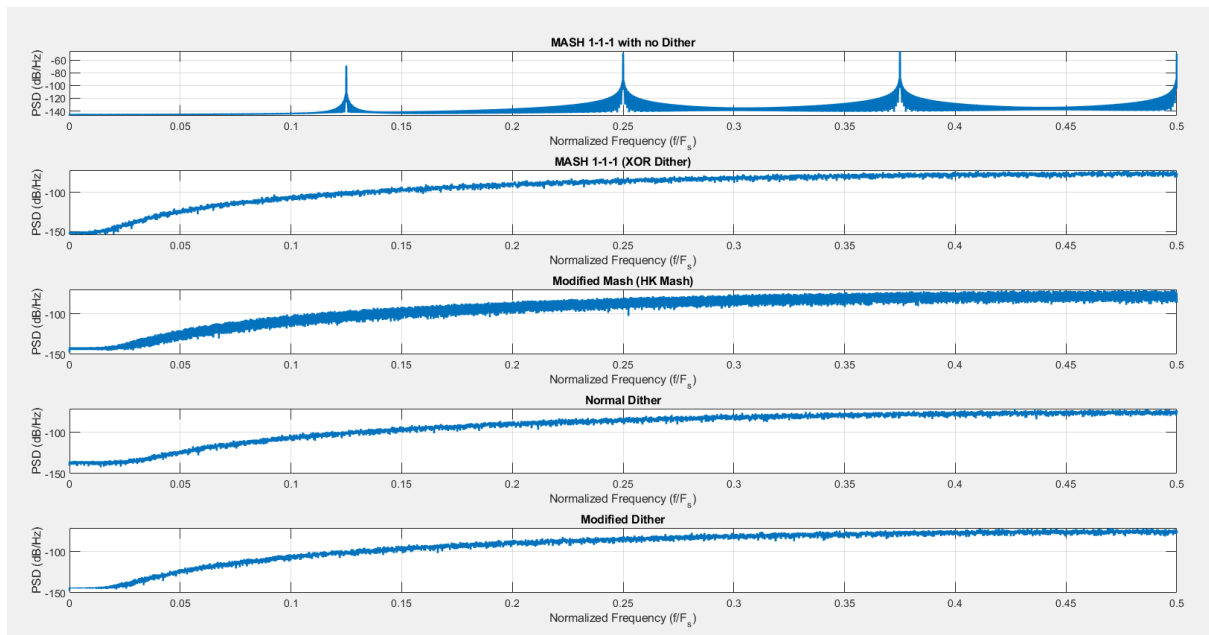


Figure 133: Extended output spectrum of the sample-based MASH 1–1–1  $\Sigma\Delta$  modulator for input 0.75

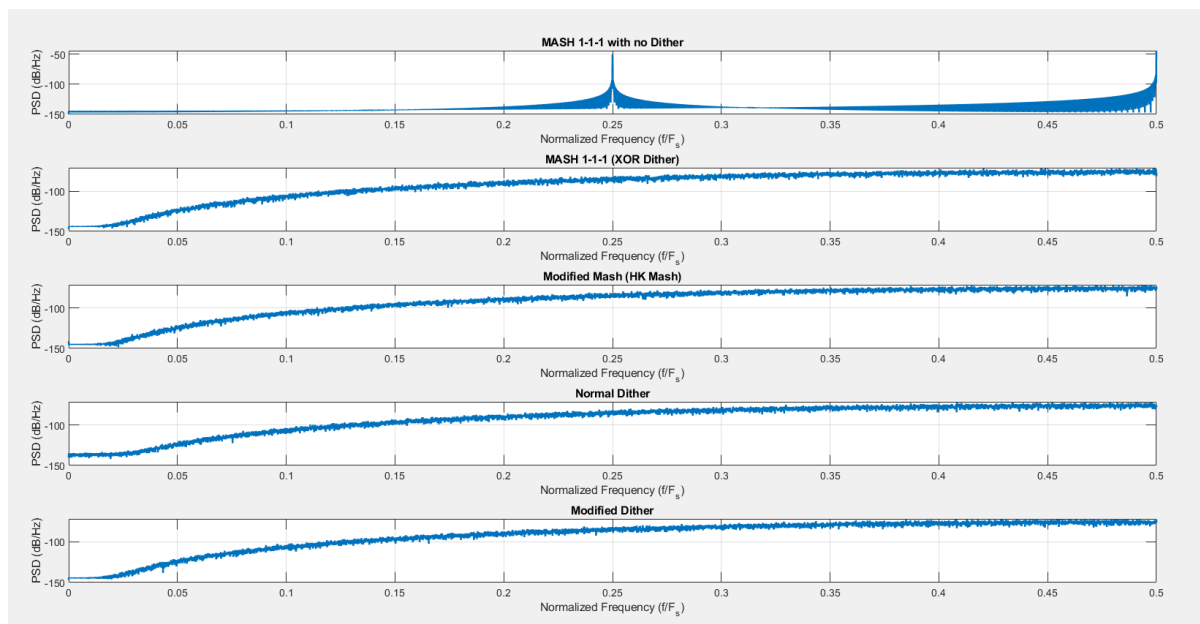


Figure 134: Extended output spectrum of the sample-based MASH 1–1–1  $\Sigma\Delta$  modulator for input 0.5.

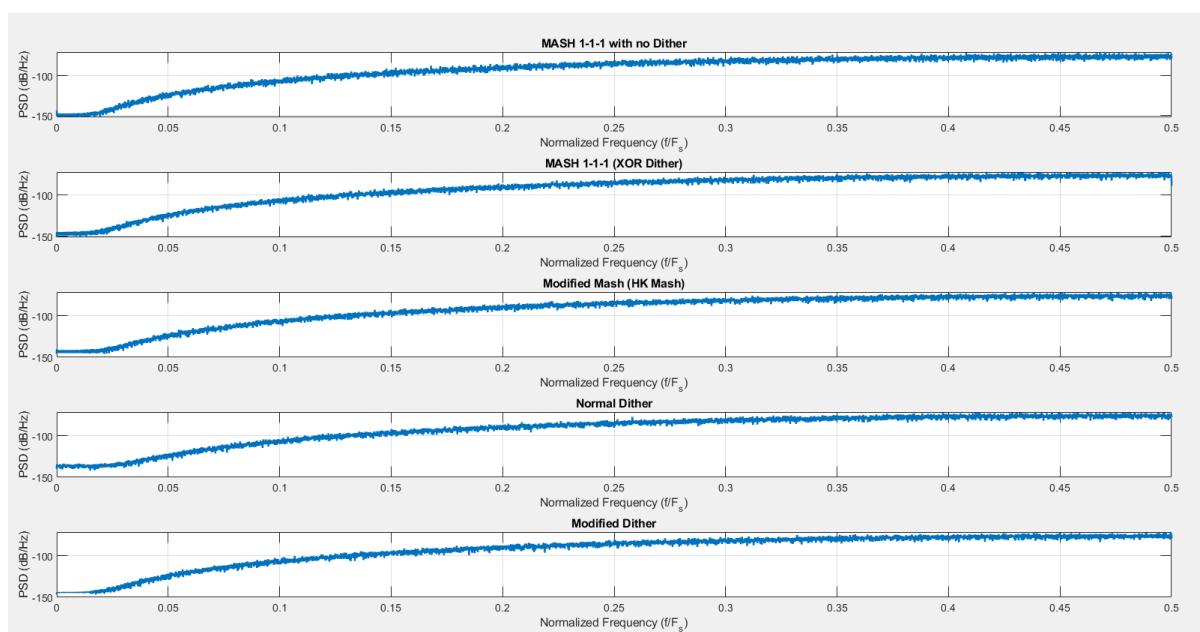


Figure 135: Extended output spectrum of the sample-based MASH 1–1–1  $\Sigma\Delta$  modulator for aninput 0.000052.

Table 19: Average output comparison under no frequency-step mismatch. The error magnitude is defined as  $|\text{Average} - X_{\text{in}}|$ .

| <b>Input</b>  | <b>Architecture</b> | <b>Output</b> | <b>—Error—</b> |
|---------------|---------------------|---------------|----------------|
| 0.000052      | Modified MASH       | 0.000060      | 0.000008       |
|               | Modified Dither     | 0.000040      | 0.000012       |
|               | Normal Dither       | 0.002000      | 0.001948       |
|               | Original MASH       | 0.000060      | 0.000008       |
|               | XOR-based Dither    | 0.000070      | 0.000018       |
| 0.255         | Modified MASH       | 0.256010      | 0.001010       |
|               | Modified Dither     | 0.255000      | 0.000000       |
|               | Normal Dither       | 0.254990      | 0.000010       |
|               | Original MASH       | 0.255000      | 0.000000       |
|               | XOR-based Dither    | 0.254990      | 0.000010       |
| 0.500000      | Modified MASH       | 0.501950      | 0.001950       |
|               | Modified Dither     | 0.500000      | 0.000000       |
|               | Normal Dither       | 0.501950      | 0.001950       |
|               | Original MASH       | 0.500000      | 0.000000       |
|               | XOR-based Dither    | 0.500000      | 0.000000       |
| 0.750         | Modified MASH       | 0.752940      | 0.002940       |
|               | Modified Dither     | 0.750000      | 0.000000       |
|               | Normal Dither       | 0.750010      | 0.000010       |
|               | Original MASH       | 0.750000      | 0.000000       |
|               | XOR-based Dither    | 0.750020      | 0.000020       |
| $\sqrt{3}/10$ | Modified MASH       | 0.173890      | 0.000685       |
|               | Modified Dither     | 0.173210      | 0.000005       |
|               | Normal Dither       | 0.173200      | 0.000005       |
|               | Original MASH       | 0.173190      | 0.000015       |
|               | XOR-based Dither    | 0.173200      | 0.000005       |

## RTL Verification

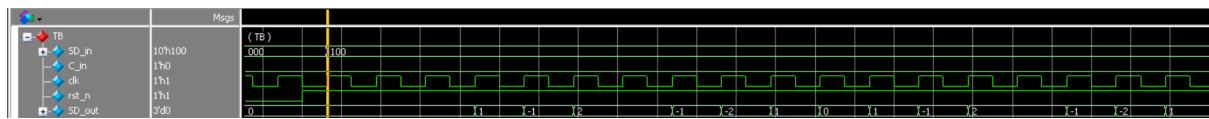


Figure 136: Simulation results for input 0.25

As illustrated in the figure below, the Sigma-Delta modulator processes a constant input of 0.25. The output bit stream exhibits a latency of 3 cycles following the application of the input. Notably, averaging the output over the next 14 cycles yields a value of approximately 0.28571, demonstrating the noise-shaping and conversion principle of the modulator.

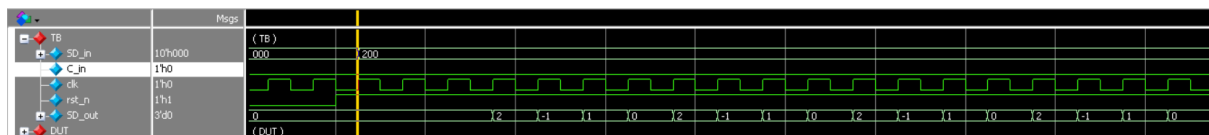


Figure 137: Simulation results for 0.5 input

As illustrated in the figure below, the Sigma-Delta modulator processes a constant input of 0.50. The output bit stream exhibits a latency of 3 cycles following the application of the input. Notably, averaging the output over the next 14 cycles yields a value of approximately 0.4285.

## Implementation Vs Modelling

In this section, a comparison between the MATLAB behavioral model and the Verilog implementation is carried out to verify the correctness of the design. Initially, a MATLAB script is used to generate the input stimulus vector and convert it into a fixed-point binary representation, which is written in a file named *input\_binary.txt*. The MATLAB model is then executed using the same inputs to produce the corresponding output, stored in *Modeling\_output.txt*. Subsequently, the Verilog design is simulated using QuestaSim, where the generated input file is read and applied to the Sigma-Delta modulator. The simulation produces an output file, *Implementation\_output.txt*, which contains the results of the hardware implementation. Finally, a MATLAB verification script is used to read both output files and perform a sample-by-sample comparison. The outputs are visualized by plotting both signals on the same graph, along with the corresponding error signal. The results demonstrate an exact match between the MATLAB model and the Verilog implementation, confirming that the design is both bit accurate and cycle accurate.

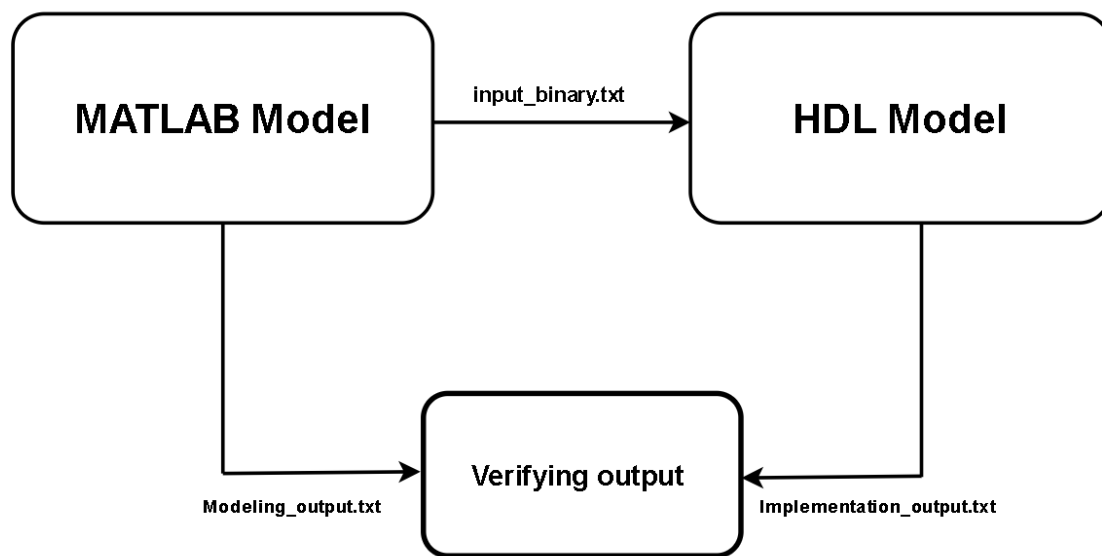


Figure 138: Block diagram of Verification

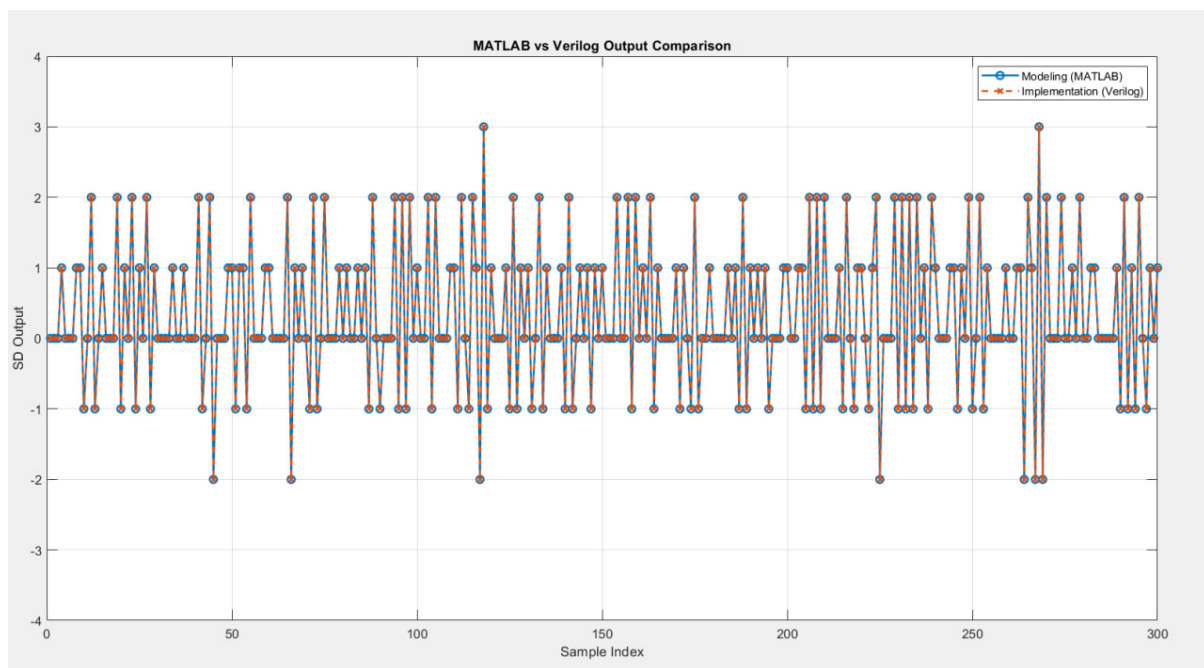


Figure 139: Comparison of matlab and HDL model outputs

Original Input vs SD Average:

| <b>Input</b> | <b>Avg_SD_Output</b> | <b>Error</b> |
|--------------|----------------------|--------------|
| 0.3          | 0.26667              | 0.033333     |
| 0.5          | 0.4                  | 0.1          |
| 0.325        | 0.2                  | 0.125        |
| 0.415        | 0.4                  | 0.015        |
| 0.21         | 0.26667              | -0.056667    |
| 0.39         | 0.26667              | 0.12333      |
| 0.57         | 0.53333              | 0.036667     |
| 0.45         | 0.4                  | 0.05         |
| 0.285        | 0.26667              | 0.018333     |
| 0.256        | 0.26667              | -0.010667    |
| 0.64         | 0.53333              | 0.10667      |
| 0.128        | 0.13333              | -0.0053333   |
| 0.365        | 0.26667              | 0.098333     |
| 0.482        | 0.46667              | 0.015333     |
| 0.297        | 0.2                  | 0.097        |
| 0.553        | 0.53333              | 0.019667     |
| 0.412        | 0.33333              | 0.078667     |
| 0.338        | 0.33333              | 0.0046667    |
| 0.601        | 0.46667              | 0.13433      |
| 0.274        | 0.26667              | 0.0073333    |

This document presented the digital implementation of a Sigma-Delta modulator. The system architecture, mathematical foundation, and fixed-point design considerations were discussed. The implementation achieves proper noise shaping and stable modulation suitable for high-resolution conversion systems.

## IIR Filter

### RTL Verification

The IIR filter verification was performed by testing the IIR filter within the ADPLL time-based MATLAB model, applying the same inputs to the RTL implementation, and comparing its outputs against that of the IIR filter in the model. The comparison between RTL and MATLAB model are shown in Figures 140 and 140.

#### Phase Error Path:

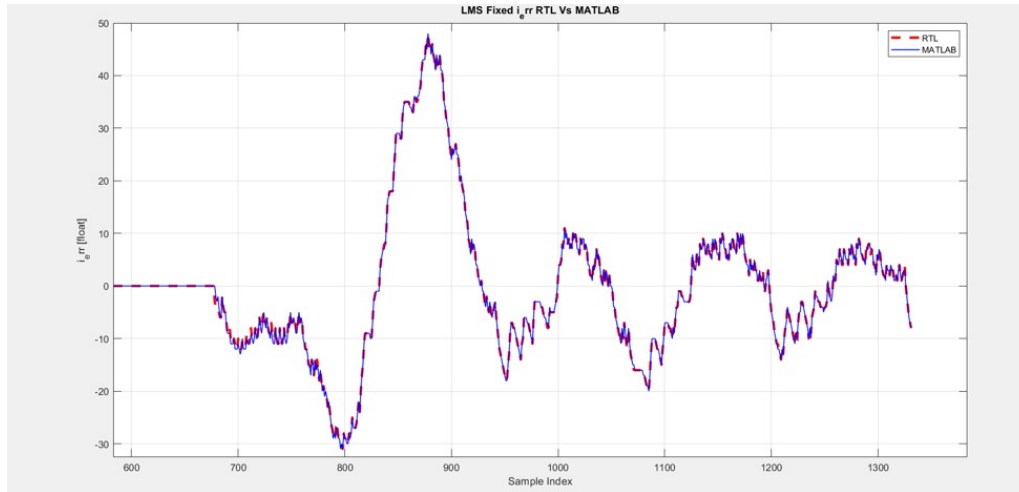


Figure 140: phase error path RTL output vs MATLAB

#### Modulation data Path:

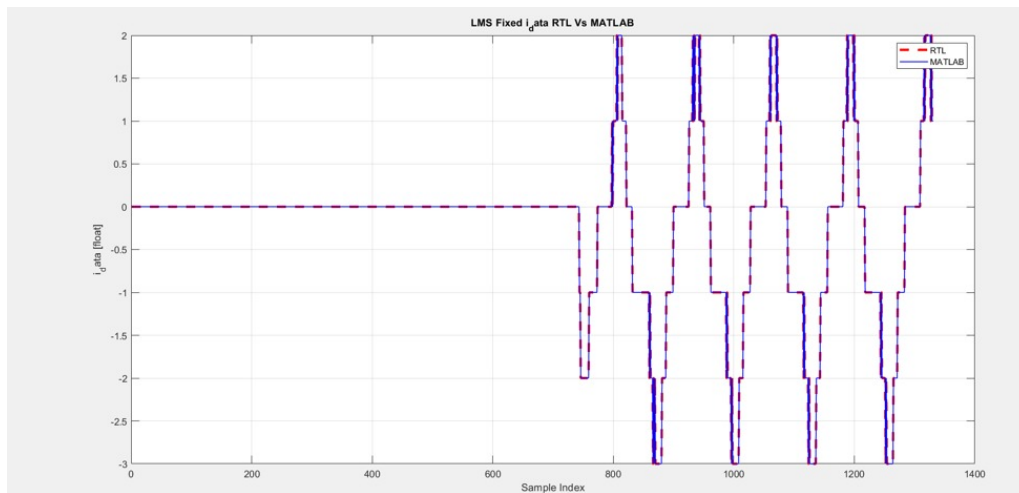


Figure 141: Modulation data path RTL output vs MATLAB

As shown in Figures 140 and 141 the RTL outputs are identical to that of the MATLAB model confirming the correctness of the implemented IIR filter.

## System Intgeration

### MATLAB Simulations

#### Matlab Result:

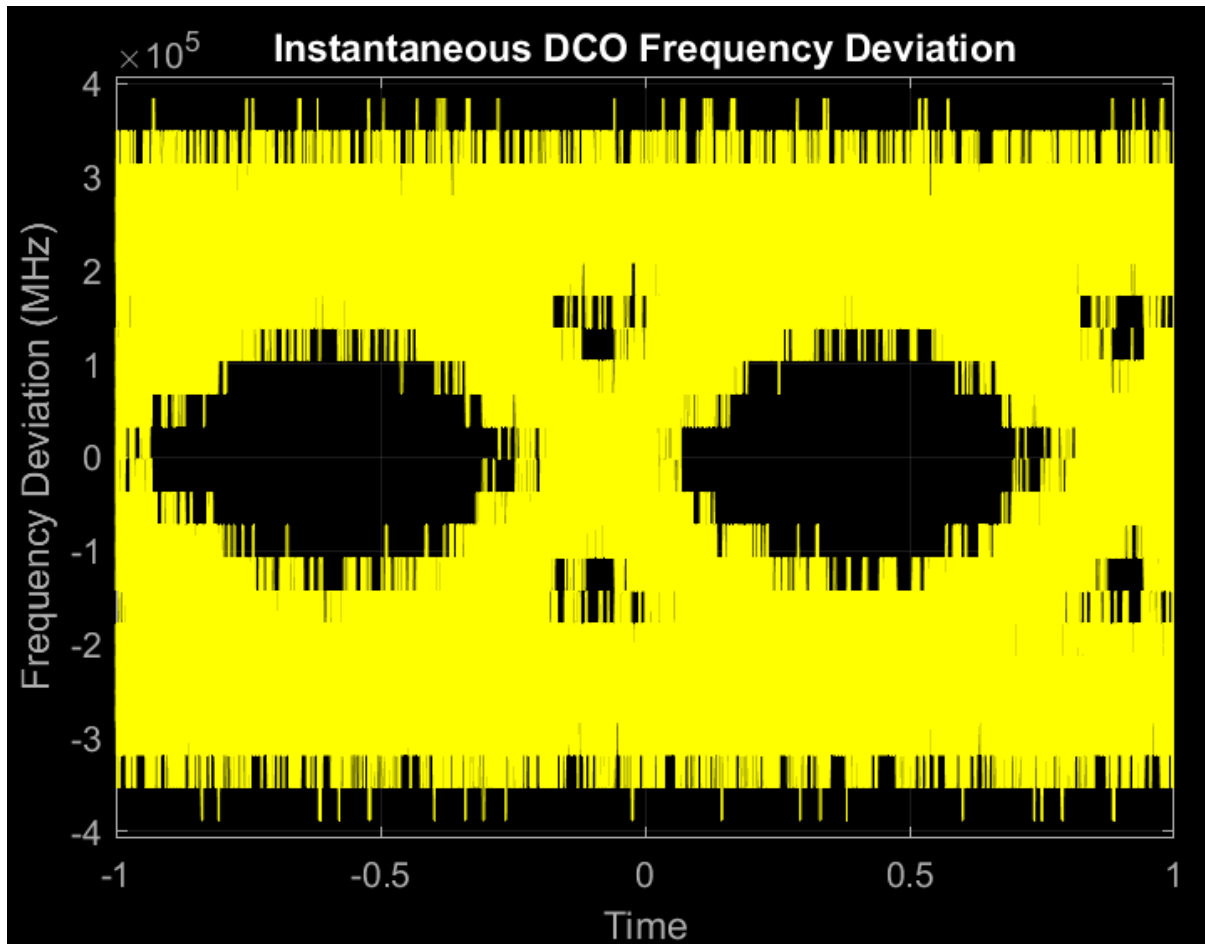


Figure 142: Instantaneous Eye Diagram

The instantaneous frequency deviation at the output of the Digitally Controlled Oscillator (DCO), driven directly by the high-frequency Sigma-Delta ( $\Sigma\Delta$ ) modulator. The dense vertical spread represents the high-frequency quantization noise shaped by the  $\Sigma\Delta$  modulator. Because the DCO has discrete frequency resolution steps, the modulator rapidly transitions between these states to achieve the required fractional frequency values.

In a physical wireless system, the raw digital output must be filtered before it is transmitted over the air. It first passes through the RF front-end filter to reject out-of-band noise. Subsequently, on the receiving side, the RX also applies its own filter on demodulated data before analyzing the baseband data.

To validate the transmitter's performance against real-world conditions, this end-to-end filtering process was modeled. The digital signal was passed through an equivalent system filter with a 2 MHz bandwidth to simulate the combined effects of the RF front-end and the receiver's demodulator.

Figure 144 shows the final eye diagram after the signal has passed through these system filters. The out-of-band quantization noise is successfully removed, leaving a clean, widely open eye with a frequency deviation peaking exactly at the  $\pm 250$  kHz limits required by the Bluetooth Core Specification.



Figure 143: Modulation signal (Instantaneous over filtered)

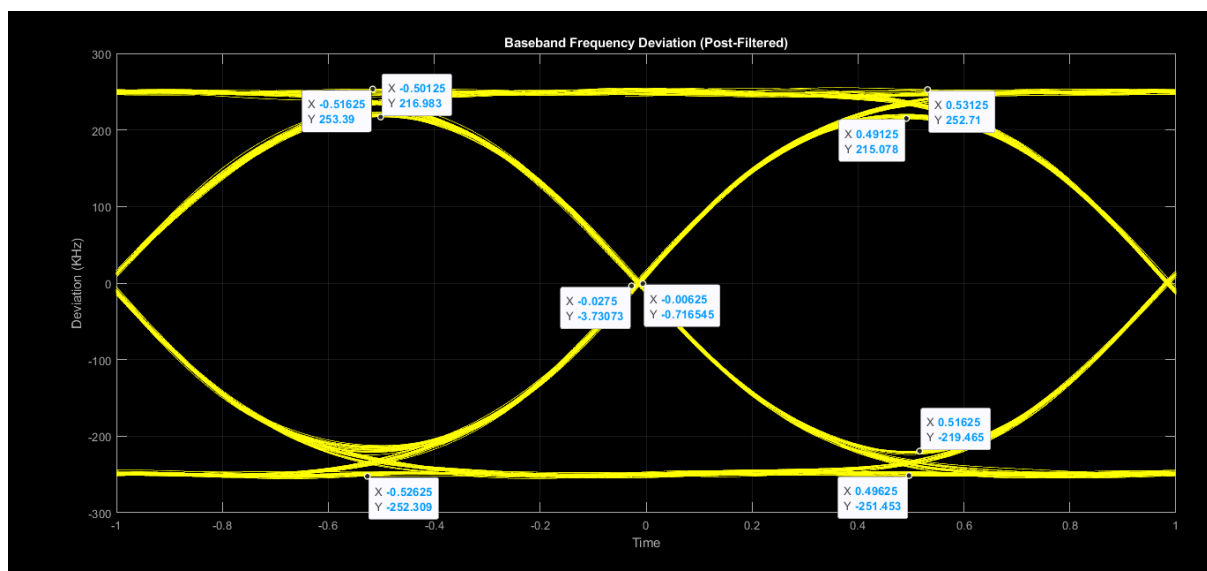


Figure 144: Filtered Eye Diagram

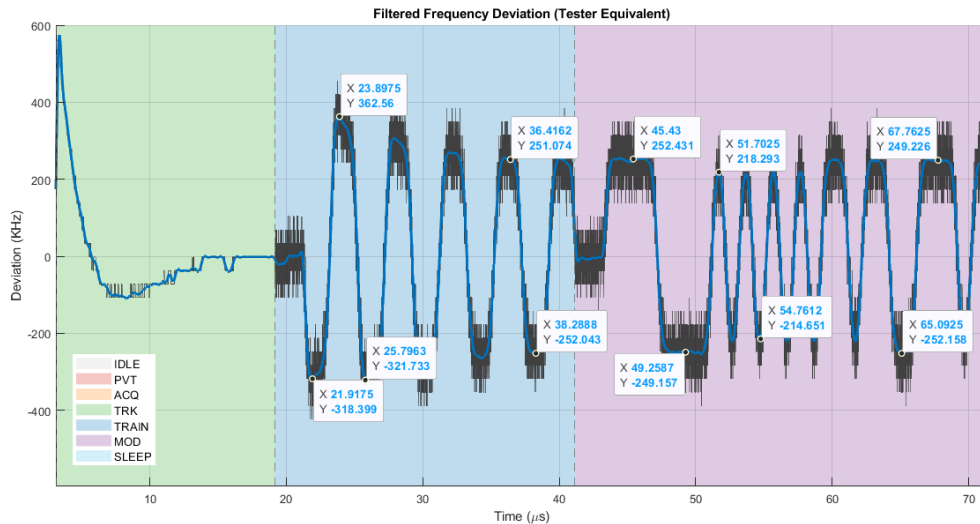


Figure 145: Modulated signal &amp; training sequence

### Gain estimation :

We assume dco analog gain "frequency step vary with PVT" so we need to calibrate DCO gain before sending data on air we use SSLMS to calibrate DCO gain on training sequence , BLE packet are considered to be small and taking into account the device is low power we assume PVT won't vary during packet transmission.

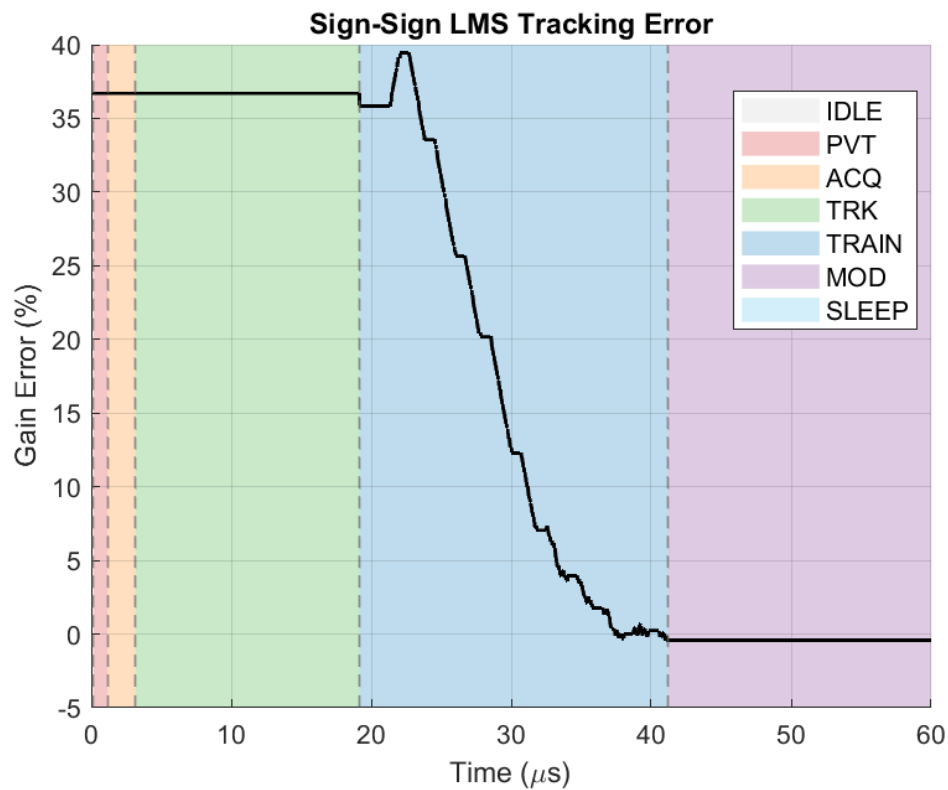


Figure 146: SSLMS coverage

## DCO Gain Calibration and LMS Estimation

### The Impact of PVT Variations

The analog gain of the Digitally Controlled Oscillator (DCO) vary with (PVT) variations. Because Bluetooth Low Energy (BLE) packets are intentionally short to conserve energy, we assumed that environmental PVT conditions remain constant during the transmission. However, the baseline DCO gain must be accurately calibrated before any active payload is transmitted over the air. To achieve this, a Sign-Sign Least Mean Squares (SSLMS) algorithm is employed to dynamically estimate and correct the DCO gain.

### The Training Phase and Initial Distortion

The frequency deviation during the "TRAIN" phase (highlighted in light blue). This signal is for internal calibration and is not transmitted over the air. At the beginning of this phase, the uncalibrated DCO gain results in a significant mismatch between the two paths of the two-point modulation architecture. This mismatch causes severe signal distortion, visibly driving the frequency deviation up to approximately 362 kHz, a value that violates standard BLE specification limits.

### SSLMS Convergence

As the training sequence progresses, the SSLMS algorithm actively minimizes this mismatch. The SSLMS coverage figure plots the corresponding LMS tracking error during this sequence. The initial uncalibrated state exhibits a severe gain error exceeding 30%. However, within approximately 20 symbols ( $22 \mu s$ ), the SSLMS algorithm successfully converges, driving the estimation error down to less than 1%. As the error approaches zero, the SSLMS coverage figure shows the frequency deviation cleanly settling into the specification-compliant  $\pm 250$  kHz range. Once calibration is achieved, the system safely transitions to the "MOD" phase (highlighted in purple) to transmit actual Link Layer data to the antenna.

### Channel lock:

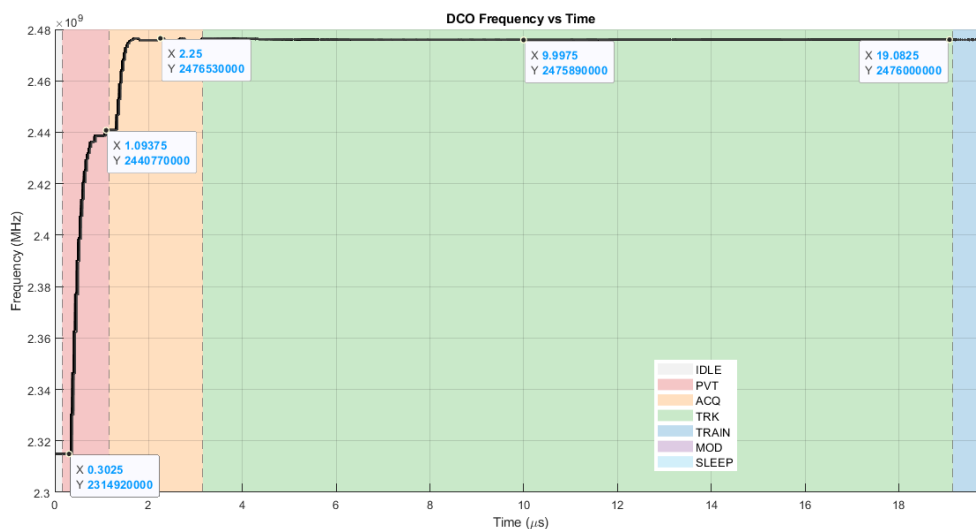


Figure 147: BLE channel lock

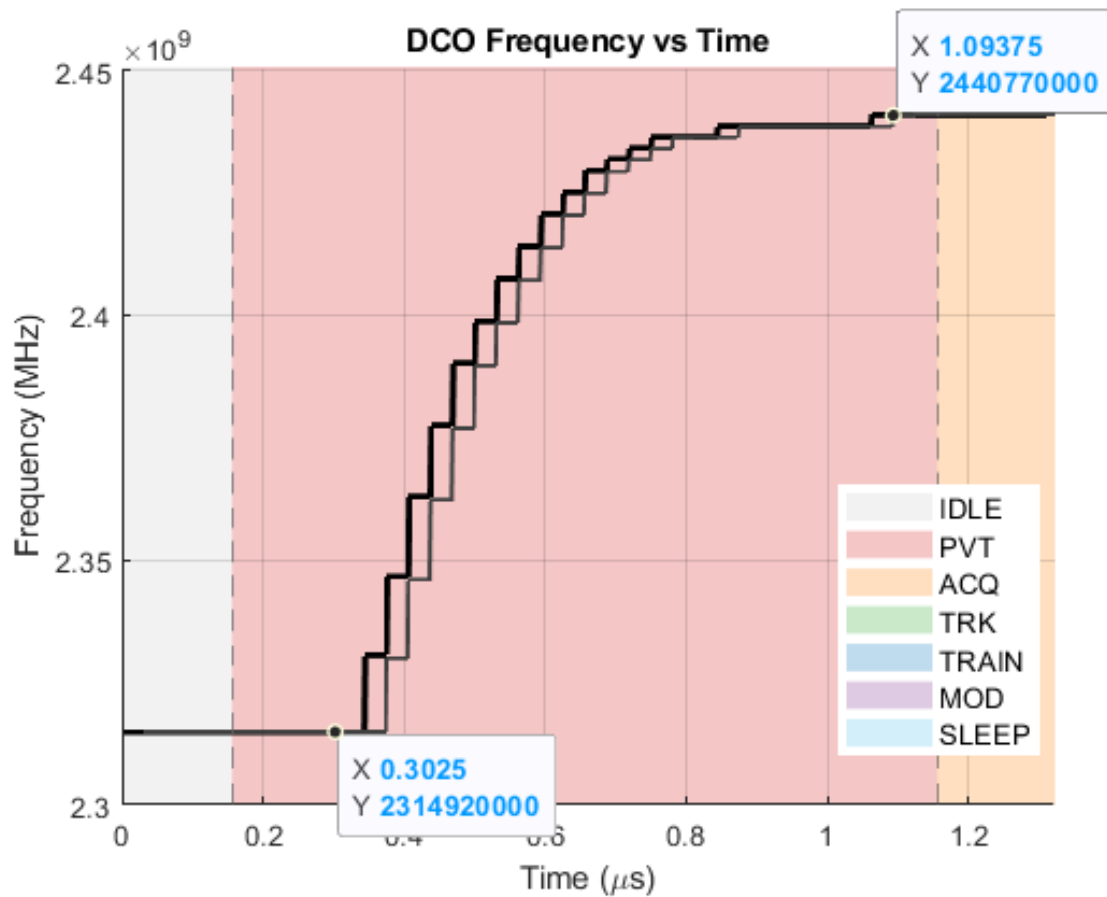


Figure 148: ADPLL lock on channel through three modes

The above figure show ADPLL lock on channel through three modes with different frequency resolution it start at frequency around 2.32GHz. Target channel is 2.476GHz (BLE out of band frequency) due to PVT variation so it BLE start with PVT calibration mode (lock at BLE mid band ) then acquisition mode (lock at channel) and last mode for channel locking is Tracking that has finest frequency step , the three figure below show smooth transition between different mode.

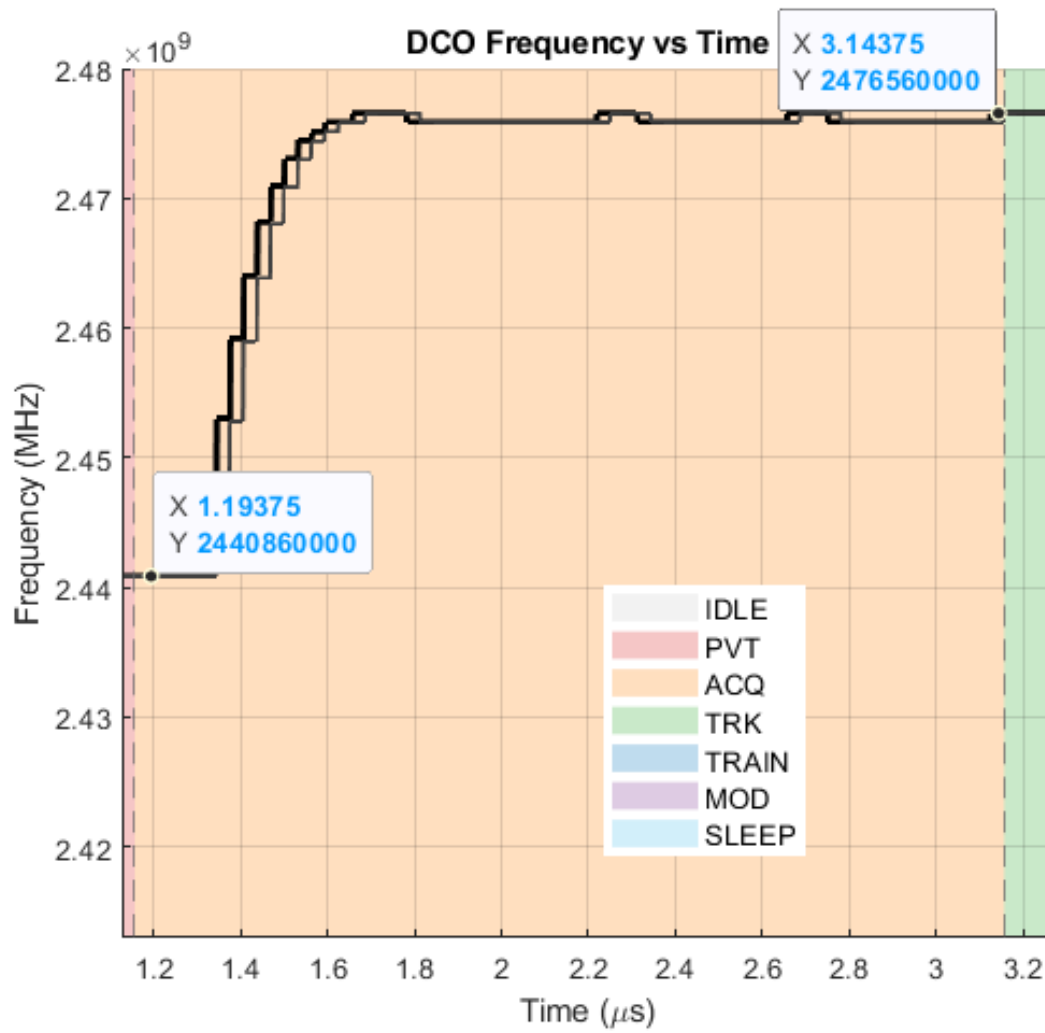


Figure 149: ACQ mode (Zoomed)

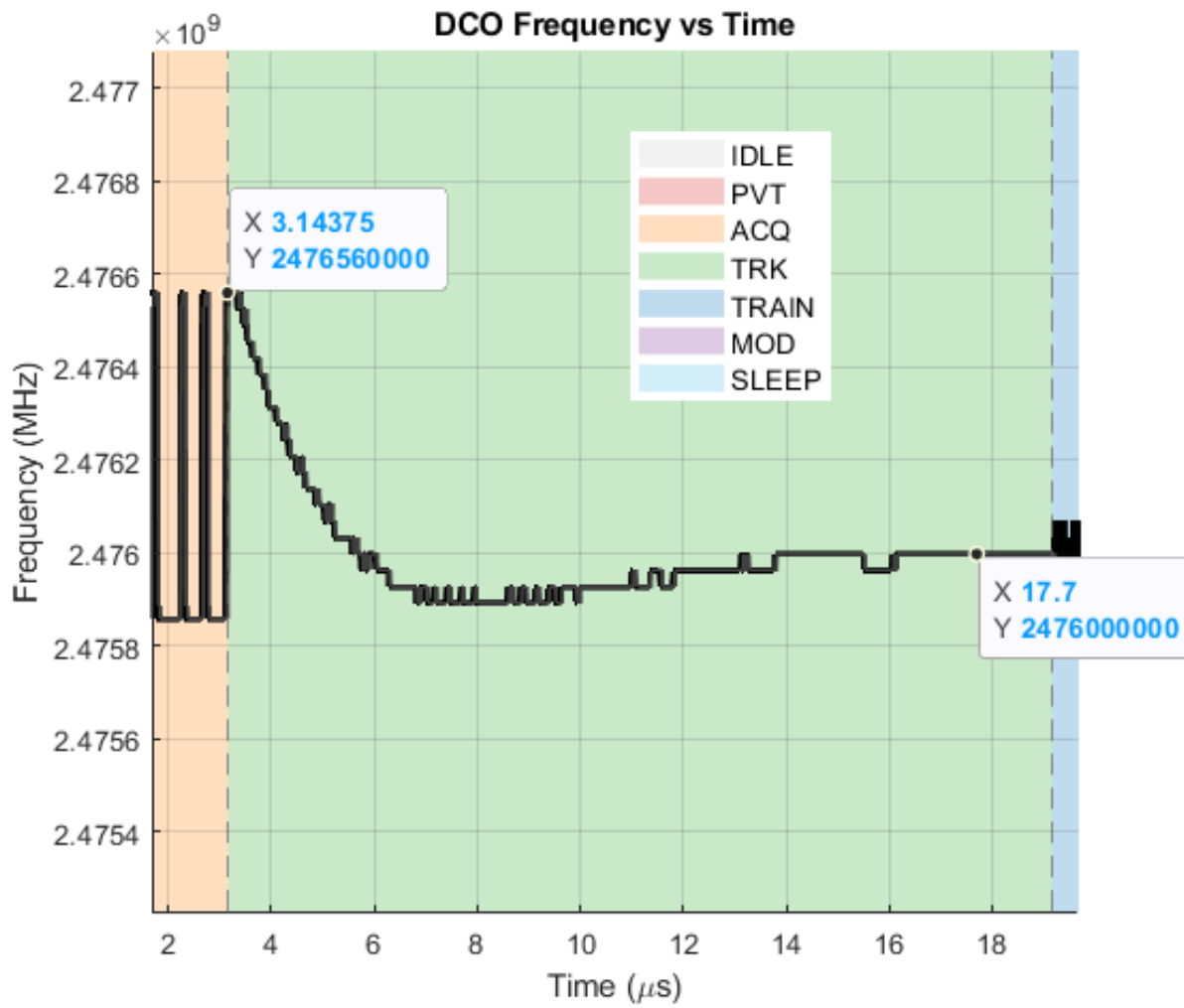


Figure 150: Tracking mode (finest mode)

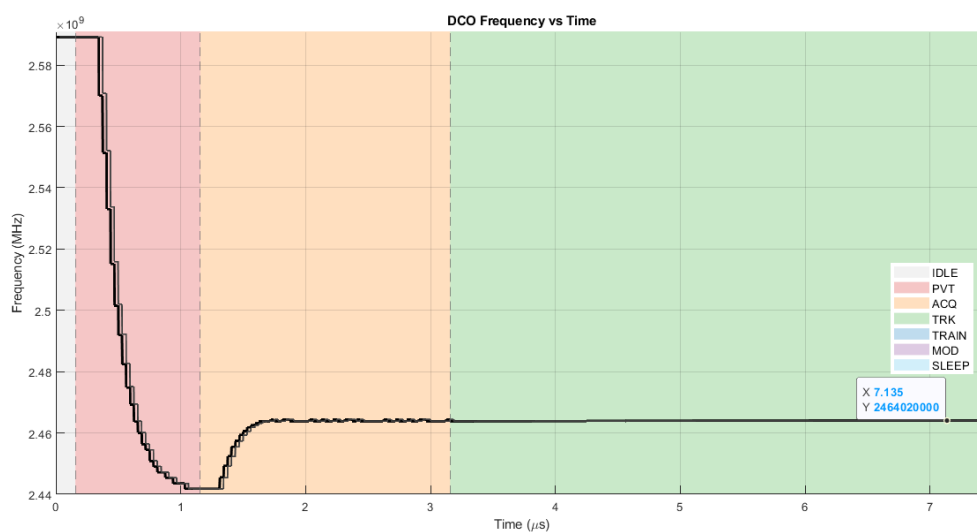


Figure 151: Frequency looked at different channel (Start = 2.580GHz, Target = 2.464GHz)

### Sending many packets during one connection interval:

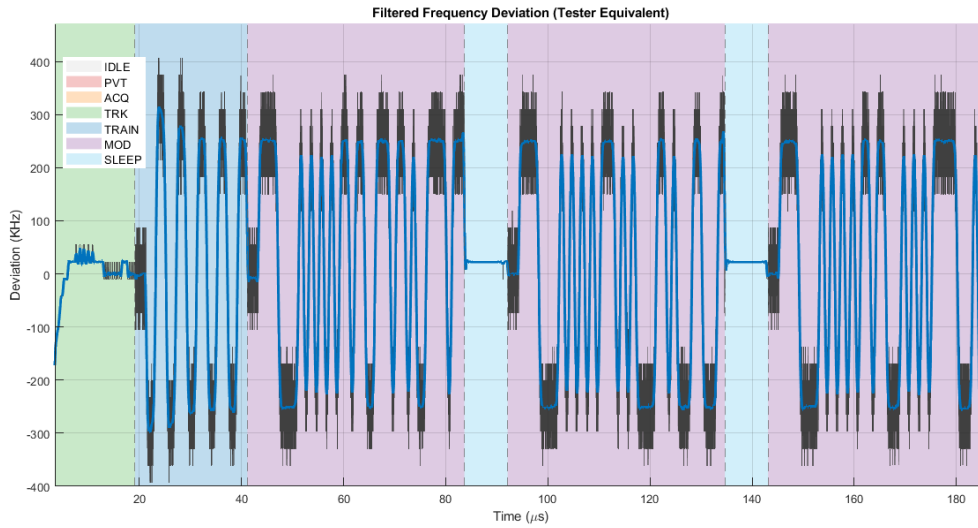


Figure 152: Sending many packets during one connection interval

BLE send one connection interval BLE send and receive many packet -one same channel - between each two successive packet there are interval where BLE receive packet and should response withing  $150\mu s \pm 2\mu s$  after receiving the packet assuming zero peripheral latency.

We made some assumption in above figure for simulation time purpose we take the gap between each packet around  $10\mu s$  BLE will be in sleep mode while the real expected sleep period could exceed  $150\mu s$  and assume packet length around 40 bit while the minimum can exceed 80 bit.

### RTL vs MATLAB

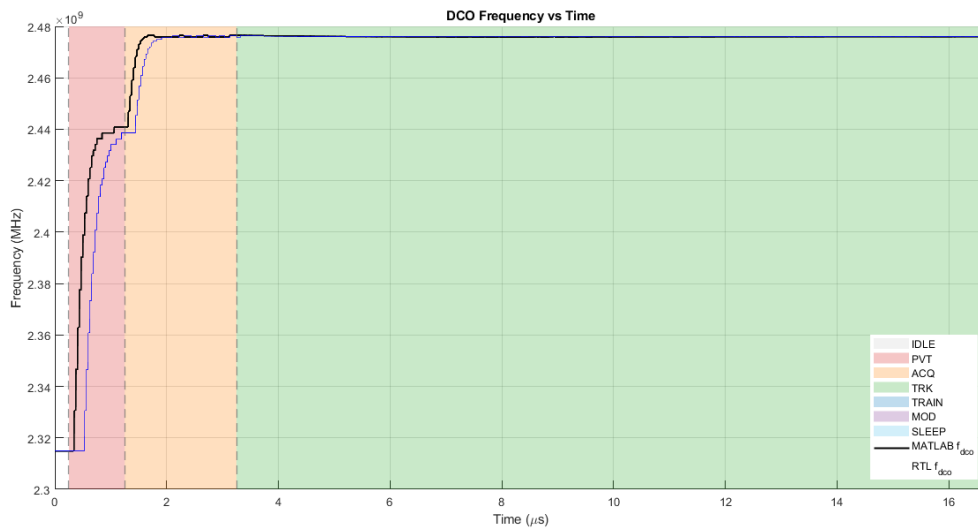


Figure 153: DCO frequency vs time during the startup sequence.

Figure 153 shows the DCO frequency during the startup sequence. The RTL closely follows the MATLAB reference, achieving lock within approximately  $3.5\mu s$ . The locking process progresses through

PVT calibration, followed by the acquisition phase where the frequency ramps up rapidly, before settling into steady-state tracking. The close agreement between the RTL and MATLAB curves validates the correct implementation of the ADPLL locking sequence.

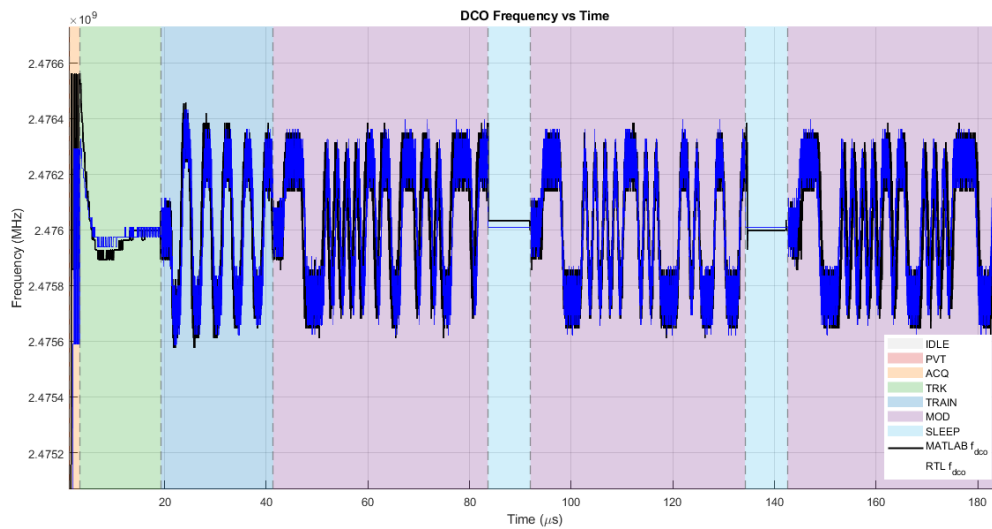


Figure 154: DCO instantaneous frequency across a full multi-packet transmission sequence.

Figure 154 shows the DCO instantaneous frequency across a full multi-packet sequence. The RTL output (blue) tracks the MATLAB reference (black) closely across all operating phases — tracking, training, modulation, and sleep — confirming correct state machine sequencing and frequency modulation behavior. The transmitter correctly cycles through all phases across multiple packets with consistent behavior.

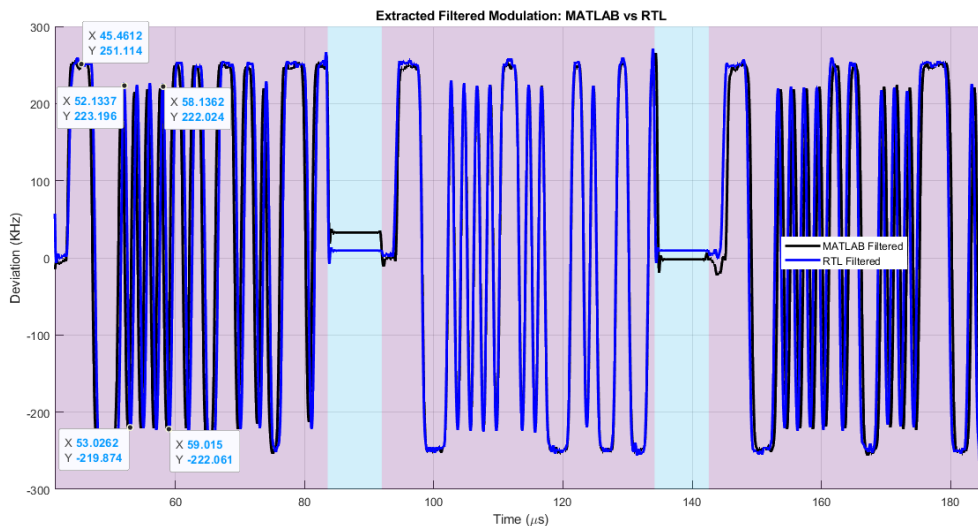


Figure 155: Extracted filtered modulation deviation comparing MATLAB and RTL across multiple packets.

Figure 155 presents the extracted filtered frequency deviation across multiple packets. The RTL output closely tracks the MATLAB reference with a peak deviation of approximately  $\pm 250$  kHz, confirming correct Gaussian filtering and gain calibration behavior. Minor transients are visible at sleep-to-

modulation transitions but settle quickly, consistent with the LMS adaptation converging after the first training sequence.

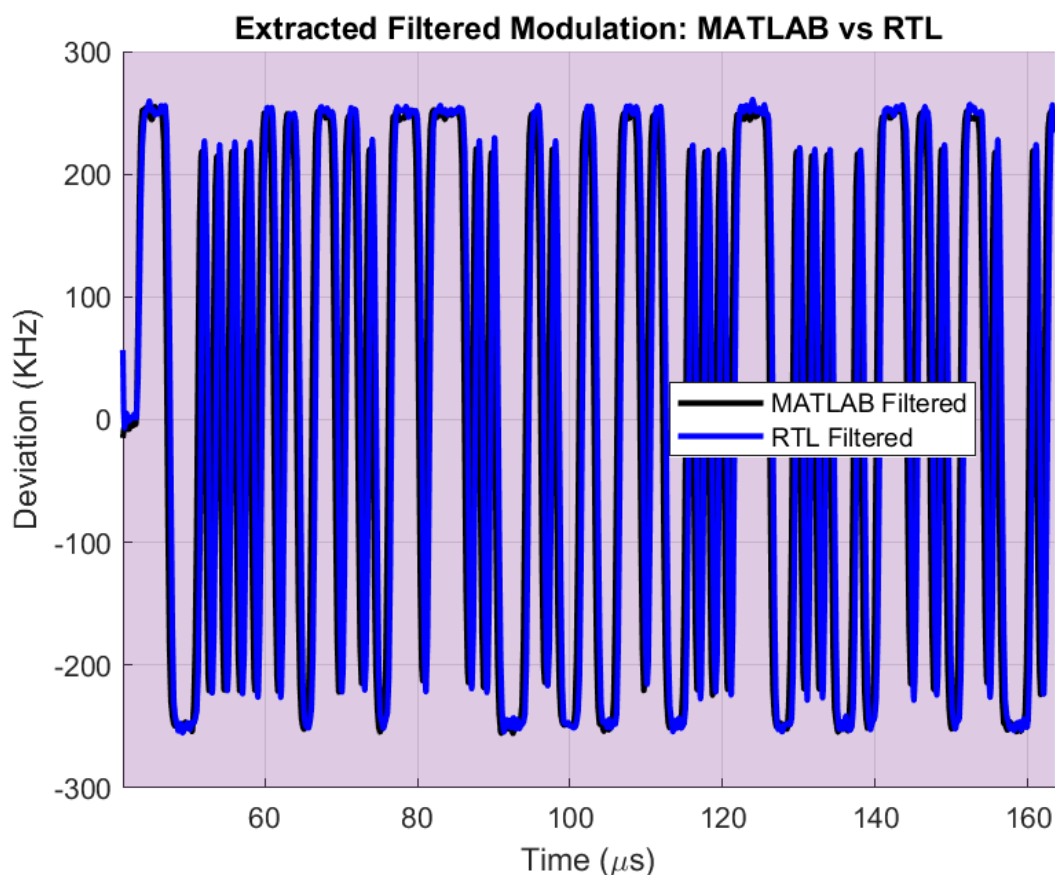


Figure 156: Zoomed view of the filtered modulation deviation during the modulation phase.

Figure 156 provides a zoomed view of the filtered modulation deviation during steady-state modulation. The RTL (blue) and MATLAB (black) waveforms are nearly indistinguishable, with peak deviations of  $\pm 250$  kHz, meeting the BLE specification requirement. This confirms that the Gaussian filter, gain normalizer, and sigma-delta modulator are all correctly implemented and well-calibrated.

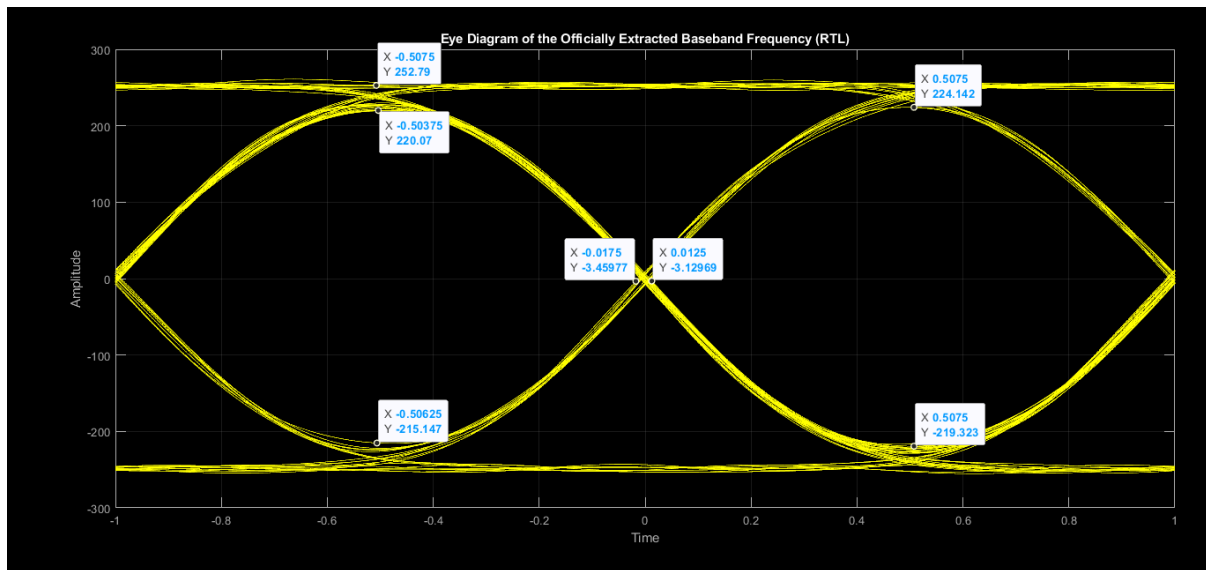


Figure 157: Eye diagram of the RTL extracted baseband frequency deviation.

Figure 157 shows the eye diagram of the RTL extracted baseband frequency deviation. The eye is clearly open with consistent peak levels of approximately  $\pm 250$  kHz and a well-defined zero crossing near 0, indicating low inter-symbol interference and correct modulation shaping. The tight clustering of the waveform traces confirms consistent behavior across all transmitted symbols.

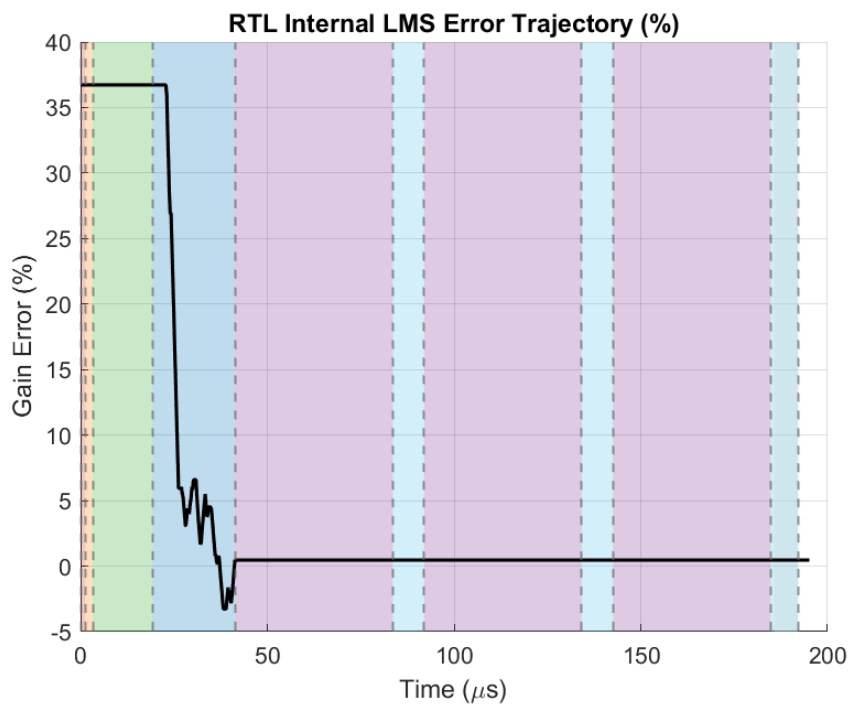


Figure 158: RTL internal LMS gain error trajectory.

Figure 158 shows the LMS gain error trajectory over time. The gain error starts at approximately 37% during the tracking phase and converges to near zero by the end of the first training sequence. The error remains stable at near zero across all subsequent packets, confirming correct LMS adaptation and gain calibration convergence. This validates that the adaptive path successfully compensates for the initial DCO gain mismatch within a single training sequence.

## ASIC Implementation

### Synthesis Summary

Synthesis timing was validated at worst-case PVT corners: **SS (125°C, 0.95V)** for setup and **FF (-40°C, 1.16V)** for hold.

### Quality of Results (QOR) Report

Table 20: Timing Path Group Summary

| Metric                        | ckr2_clk | gmsk_clk | ref_clk | sd_clk |
|-------------------------------|----------|----------|---------|--------|
| Levels of Logic               | 33.00    | 8.00     | 17.00   | 13.00  |
| Critical Path Length (ns)     | 3.24     | 1.20     | 2.05    | 2.07   |
| Critical Path Slack (ns)      | 0.00     | 29.95    | 29.04   | 1.01   |
| Critical Path Clk Period (ns) | 31.25    | 62.50    | 31.25   | 3.22   |
| Total Negative Slack (ns)     | 0.00     | 0.00     | 0.00    | 0.00   |
| No. of Violating Paths        | 0.00     | 0.00     | 0.00    | 0.00   |
| Worst Hold Violation (ns)     | -0.05    | 0.00     | 0.00    | 0.00   |
| Total Hold Violation (ns)     | -0.76    | 0.00     | 0.00    | 0.00   |
| No. of Hold Violations        | 65.00    | 0.00     | 0.00    | 0.00   |

**Comment on result:** Even though there are minor Hold violations in the CKR clock domain, they can easily be solved by inserting buffers in these paths. Hold violations are usually resolved after routing in the backend stage, so we will simply ignore them for the synthesis stage.

### Area Report Summary

- **Combinational area:** 21,972.02
- **Buf/Inv area:** 1,286.73
- **Noncombinational area:** 9,401.80
- **Macro/Black Box area:** 0.00
- **Net Interconnect area:** 5,867.98
- **Total cell area:** 31,373.82
- **Total area:** 37,241.80  $\mu m^2$

**Total equivalent area:** Approximately 18.3K NAND gates on a 32nm process.

## Clock Domain Crossing (CDC)

There are two clock domain crossings in the design between the CKR clock and the SD clock. Even though the control unit controls SD, most of these signals are considered to be static.

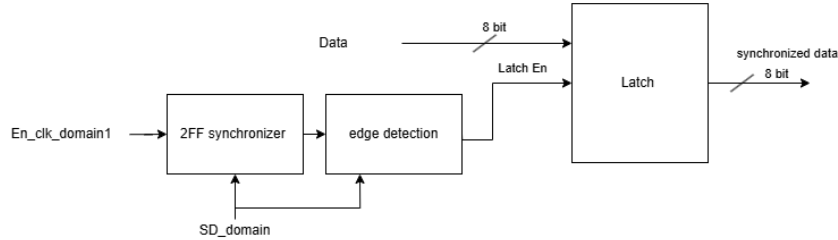


Figure 159: Data Qualifier Synchronizer

We solved the CDC between the SD clock and CKR clock by using a Data Qualifier Synchronizer, and Reset Domain Crossing (RDC) using a reset synchronizer to avoid asynchronous de-assertion.

### CDC Structural Check Results:

- **Fatal Errors:** 0
- **CDC Violations:** 0
- **Status:** Clean. The design handles asynchronous crossings safely with no structural CDC violations detected.

## Equivalence Checking (Formality)

Formal verification was run to ensure the synthesized netlist perfectly matches the functional behavior of the original RTL.

- **Reference design:** r:/WORK/ble\_phy\_top
- **Implementation design:** i:/WORK/ble\_phy\_top

Table 21: Formality Compare Points

| Matched Compare Points   | BBPin | Loop | BBNet | Cut | Port | DFF  | LAT | TOTAL |
|--------------------------|-------|------|-------|-----|------|------|-----|-------|
| Passing (equivalent)     | 0     | 0    | 0     | 0   | 31   | 1198 | 0   | 1229  |
| Failing (not equivalent) | 0     | 0    | 0     | 0   | 0    | 0    | 0   | 0     |

## Specification validation

To verify that the implemented transmitter meets the requirements set by the Bluetooth Low Energy (BLE) physical layer specification, the design was validated through RTL simulation against the relevant compliance metrics. This step confirms that the actual hardware description – rather than a behavioral or algorithmic approximation – produces output consistent with the timing and modulation accuracy required by the BLE standard.

The validation targets two of the key compliance metrics defined by the BLE specification for GFSK-modulated transmitters: the **zero-crossing timing error**, evaluated through the eye diagram of the modulated frequency deviation, and the **frequency deviation ratio**, which compares the minimum and maximum frequency deviations produced by the two-point modulation scheme against the BLE 5.4 requirement that  $|f_{min}| > 0.8 |f_{max}|$ . Together, these checks confirm that the modulation accuracy and timing behavior of the RTL-level transmitter fall within the margins required for spec-compliant operation.

Figure 160 shows the eye diagram of the baseband frequency deviation, extracted from the RTL simulation output and accumulated across all transmitted symbols. The diagram is generated by overlaying the demodulated frequency deviation waveform across consecutive symbol periods, allowing the spread of the zero crossings around the ideal symbol-center transition to be observed directly.

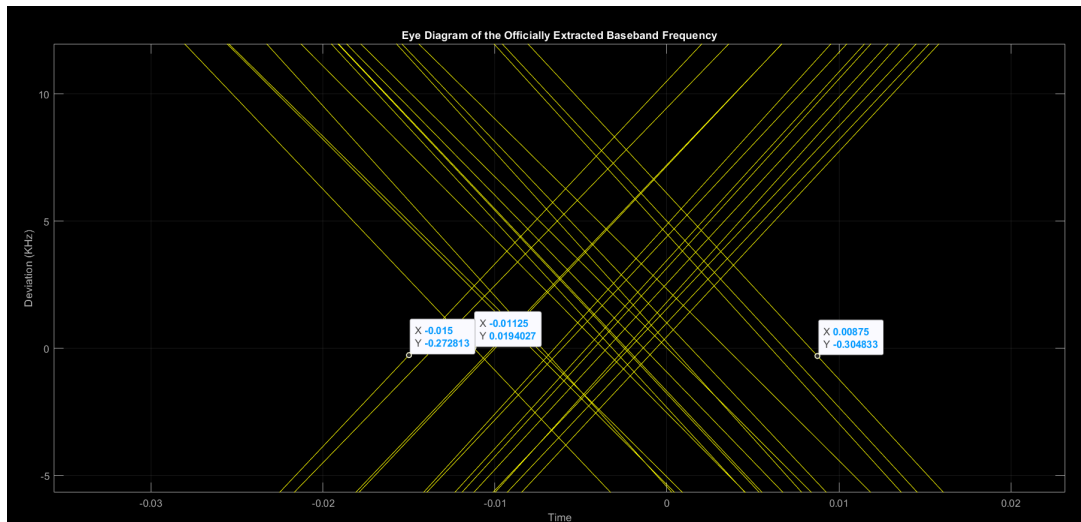


Figure 160: Eye diagram of the officially extracted baseband frequency from RTL simulation, showing the spread of zero crossings around the ideal transition point.

As shown in Figure 160, the zero crossings cluster tightly around  $t = 0$ , with the measured crossing points spanning approximately from  $-0.015 \mu s$  to  $+0.00875 \mu s$ , giving a maximum zero-crossing timing error on the order of **0.02  $\mu s$** . This is well within the BLE specification limit of **0.125  $\mu s$**  for zero-crossing timing error, confirming that the RTL implementation satisfies the timing requirement with a substantial margin – the measured deviation is roughly six times smaller than the allowed limit.

This tight clustering of zero crossings indicates low timing jitter in the modulation path at the RTL level, and reflects the combined accuracy of the Gaussian filtering, two-point modulation gain matching, and DCO frequency control implemented in the digital loop. Unlike the earlier behavioral-model results, this measurement is obtained directly from RTL simulation, so it reflects the actual gate-level timing behavior of the implemented loop filter, modulation path, and DCO control logic rather than an idealized algorithmic approximation.

## Eye Diagram and Frequency Deviation Compliance

Figure 161 shows the full-scale eye diagram of the extracted baseband frequency deviation, complementing the zoomed zero-crossing view presented in Figure 160. While the previous figure focused on the timing accuracy of the zero crossings, this view captures the full vertical opening of the eye, allowing the deviation levels produced by the RTL implementation to be directly read off the plot and compared against the BLE allowed deviation range.

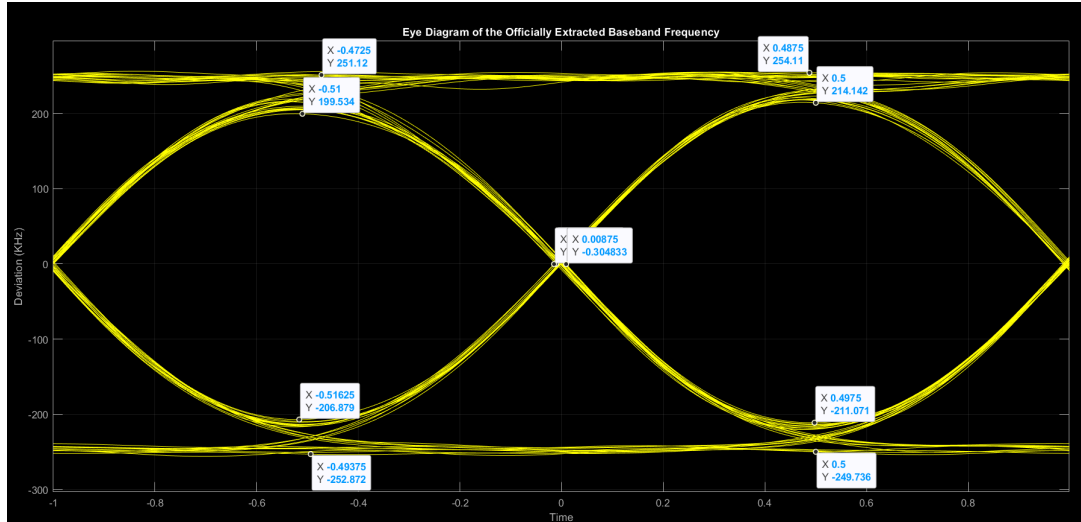


Figure 161: Full-scale eye diagram of the extracted baseband frequency deviation from RTL simulation, showing the upper and lower deviation bands of the two-point modulation.

As shown in Figure 161, the eye exhibits two clearly separated deviation bands, corresponding to the two frequency levels produced by the GFSK two-point modulation scheme. The BLE specification requires the frequency deviation to remain within an allowed range of **185 kHz to 255 kHz** in magnitude. The measured upper deviation band settles at approximately 199–254 kHz, and the measured lower deviation band settles at approximately 207–253 kHz in magnitude. Both bands fall entirely within the 185–275 kHz allowed range across the full eye opening, confirming that the RTL implementation does not violate the BLE frequency deviation specification.

This result indicates that the gain of the two-point modulation path and the DCO frequency control are sufficiently well matched at the RTL level to keep the modulated deviation within the spec-mandated window, without excessive overshoot toward the upper bound or undershoot toward the lower bound.

Figure 162 shows the extracted frequency deviation versus time for the RTL simulation output with input bit pattern 0000111101010101, which combines a sustained run of identical bits with a subsequent alternating-bit sequence. This pattern is used specifically because it produces two distinct deviation regimes: the sustained-bit region allows the Gaussian-filtered modulation to fully settle to its maximum deviation level, while the alternating-bit region forces rapid transitions that do not allow full settling, producing a reduced deviation level. Comparing these two regimes against each other is the basis of the BLE 5.4 minimum-deviation requirement.

As shown in Figure 162, the sustained-bit plateaus (corresponding to the 00001111 portion of the pattern) settle at a maximum deviation of approximately  $f_{max} \approx 251$  kHz. The alternating-bit region (corresponding to the . . . 01010101 portion) reaches a reduced peak deviation of approximately  $f_{min} \approx 215$  kHz, as the rapid bit transitions do not allow the modulated frequency to fully settle before reversing direction.

According to the BLE 5.4 specification, the minimum frequency deviation must satisfy:

$$|f_{min}| > 0.8 |f_{max}|$$

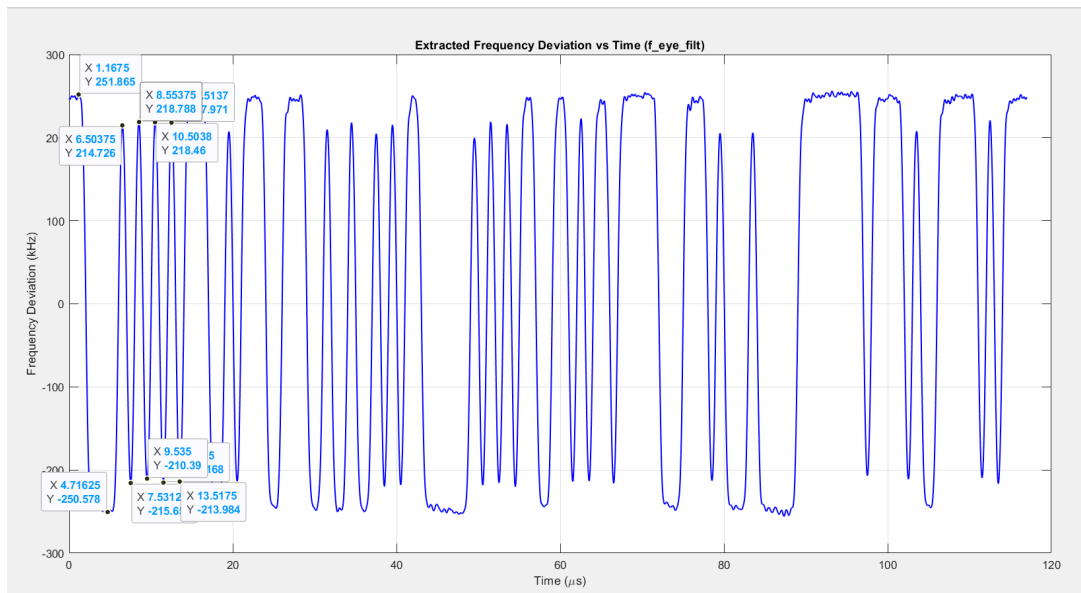


Figure 162: Extracted frequency deviation versus time for the input pattern 0000111101010101, showing the settled plateau deviation from sustained bit runs and the reduced peak deviation from alternating bits.

Substituting the measured values, the required threshold is:

$$0.8 \times 251 \text{ kHz} = 200.8 \text{ kHz}$$

Since the measured minimum deviation of approximately 215 kHz exceeds this threshold (215 kHz > 200.8 kHz), the condition is satisfied. The RTL implementation of the two-point modulation path therefore meets the BLE frequency deviation ratio requirement, confirming that the Gaussian filter and DCO gain are sufficiently matched to preserve adequate frequency separation even under worst-case alternating-bit modulation.

## Conclusion and Future Work

### Summary

This thesis presented the design and analysis of key digital building blocks for a Bluetooth Low Energy (BLE) All-Digital Phase-Locked Loop (ADPLL), with emphasis on hardware-efficient digital signal processing techniques suitable for low-power wireless transmitters. A multiplier-free symmetric Gaussian FIR filter was proposed for BLE pulse shaping, exploiting coefficient symmetry and NRZ input properties to significantly reduce arithmetic complexity while preserving the desired spectral characteristics. The Digitally Controlled Oscillator (DCO) architecture and its operational modes were also analyzed, and a low-complexity sign-sign LMS algorithm was employed for DCO gain calibration to mitigate process, voltage, and temperature (PVT) variations. Furthermore, the integration of a programmable register file enhances the system's overall robustness, providing the necessary firmware-level flexibility to mitigate post-fabrication faults and process variations. Behavioral simulation results demonstrated the correct functionality and robustness of the proposed architecture. In addition, BLE specification compliance was validated through transmitter performance evaluation, including eye diagram analysis, confirming that the generated waveform satisfies the required BLE timing and signal quality requirements. Moreover, synthesis, CDC and formality checks were run on the design and the design did successfully meet timing requirements at worst case PVT corners: SS (125°C, 0.95V) for setup and FF (-40°C, 1.16V) for hold, in addition to reporting no errors in CDC checks nor mismatches in formality reports. These results demonstrate the suitability of the proposed architecture for integration into a fully digital BLE transmitter.

### Future Work

To advance the current verified RTL toward a robust, production-ready silicon implementation, future work will focus on physical realization and system-level validation. This progression will be driven by the following critical phases:

- **Design for Testability (DFT):** Integrating DFT structures, such as scan chains and Automatic Test Pattern Generation (ATPG), is essential to identify physical manufacturing defects (like stuck-at or transition faults) after the chip is fabricated. This guarantees high fault coverage and ensures that only known-good dies are packaged and deployed, ultimately making the final silicon reliable and commercially viable.
- **FPGA Prototyping:** Emulating the architecture on an FPGA is necessary to validate the system at near-real-time speeds under actual environmental conditions. This enables extensive hardware-firmware co-verification and real over-the-air testing that would be far too slow or entirely impossible in standard software simulation, drastically reducing the risk of functional failures prior to a tape-out.
- **Place and Route (PnR):** Translating the synthesized netlist into a physical layout is required to map the logic to the specific foundry technology node. Executing PnR will allow for the accurate extraction of final Power, Performance, and Area (PPA) metrics, ensuring that physical considerations such as Clock Tree Synthesis (CTS) and parasitic delays do not violate strict setup and hold timing constraints before generating the final GDSII file.

## References

- [1] Bluetooth SIG, *Bluetooth Core Specification, Version 5.4*, Bluetooth Special Interest Group (SIG), Dec. 2023.

- [2] F. Xiong, "Digital modulation techniques," Second edition, Boston: Artech House, 2006
- [3] A Review on Micro-Watts All-Digital Frequency Synthesizers by Venkadasamy Navaneethan, Boon Chiat Terence Teo, Annamalai Arasu Muthukumaraswamy, Xian Yang L Lim, and Liter Siek
- [4] An Adaptive Pre-Distortion Technique to Mitigate the DTC Nonlinearity in Digital PLLs by Salvatore Levantino, Member, IEEE, Giovanni Marzin, Member, IEEE, and Carlo Samori, Senior Member, IEEE
- [5] S.Mendel, C.Vogel, A  $z$ -Domain Model and Analysis of Phase-Domain All-Digital Phase-Locked Loops
- [6] A. S. Mokhtara, C. S. Pinga, M. F. Md Dina, N. F. Makmora and M. A. C. Mahadi, Implementation of Booth Multiplier Algorithm using Radix-4 in FPGA
- [7] T. Švedek, M. Herceg, and T. Matic, "A Simple Signal Shaper for GMSK/GFSK and MSK Modulator Based on Sigma-Delta Look-up Table," Dept. of Communications, Faculty of Electrical Engineering, J.J. Strossmayer University of Osijek, Osijek, Croatia.
- [8] MathWorks, "gaussdesign — Gaussian FIR pulse-shaping filter design," MATLAB Signal Processing Toolbox Documentation.
- [9] AMD/Xilinx, "FIR Compiler v7.2 LogiCORE IP Product Guide (PG149)," 2022.
- [10] A. Lopez, "Low-Latency GFSK Demodulation Architecture Comparison and Design for FPGA," IC-Design Group, University of Twente, Enschede, The Netherlands.
- [11] R. Bregović, Y. J. Yu, T. Saramäki, and Y. C. Lim, "Implementation of Linear-Phase FIR Filters for a Rational Sampling Rate Conversion Utilizing the Coefficient Symmetry," IEEE Transactions on Circuits and Systems.
- [12] R. Schiphorst, F. Hoeksema, and K. Slump, "Bluetooth Demodulation Algorithms and Their Performance," Department of Electrical Engineering, Laboratory of Signals and Systems, University of Twente, Enschede, The Netherlands.
- [13] Robert Bogdan Staszewski, Poras T. Balsara, "All -Digital Frequency Synthesizer In Deep - Submicron CMOS" (2006)
- [14] C Priyanka, P. Latha, "Design and Implementation of Time to Digital Converters" (2015)
- [15] K. Hosseini and M. P. Kennedy, "Maximum Sequence Length MASH Digital Delta-Sigma Modulators," *IEEE Journal of Solid-State Circuits*, vol. 43, no. 11, pp. 2627–2638, Nov. 2008.
- [16] K. Hosseini, "Mathematical Analysis of Digital MASH Delta-Sigma Modulators for Fractional- $N$  Frequency Synthesizers," Ph.D. dissertation, Dept. of Microelectronic Engineering, University College Cork, Cork, Ireland, 2008.
- [17] X. Ma, H. Yang, and H. Wang, "Comparison of Sigma-Delta Modulators for Fractional- $N$  PLL Frequency Synthesizers," *Analog Integrated Circuits and Signal Processing*, 2007.
- [18] G. De La Fuente-Cortés, R. Perdomo-Cosme, M. M. Perez-Torres, and V. R. Gonzalez-Diaz, "Theoretical and Hardware-Level Precision in MASH 1–1–1 Modulators with Eliminable Dither," *Analog Integrated Circuits and Signal Processing*, 2025.
- [19] ARM Limited, AMBA Specification (Rev 2.0), ARM IHI 0011A, 1999.